

Essays

Paul Graham

Essays

Collected Writing

By Paul Graham

Table of Contents

Part One — Hackers and Painters (2001–2004)

- Beating the Averages — *April 2001*
- Taste for Makers — *February 2002*
- A Plan for Spam — *August 2002*
- Why Nerds are Unpopular — *February 2003*
- Hackers and Painters — *May 2003*
- What You Can't Say — *January 2004*
- How to Make Wealth — *May 2004*
- Great Hackers — *July 2004*
- The Age of the Essay — *September 2004*

Part Two — The Startup Playbook (2005–2008)

- What You'll Wish You'd Known — *January 2005*
- How to Start a Startup — *March 2005*
- How to Do What You Love — *January 2006*
- The 18 Mistakes That Kill Startups — *October 2006*
- Why to Not Not Start a Startup — *March 2007*
- How to Disagree — *March 2008*

- Be Good — *April 2008*
- Cities and Ambition — *May 2008*
- Lies We Tell Kids — *May 2008*

Part Three — Ideas and Determination (2009–2013)

- Keep Your Identity Small — *February 2009*
- Startups in 13 Sentences — *February 2009*
- Relentlessly Resourceful — *March 2009*
- Maker’s Schedule, Manager’s Schedule — *July 2009*
- The Acceleration of Addictiveness — *July 2010*
- The Top Idea in Your Mind — *July 2010*
- Schlep Blindness — *January 2012*
- Startup = Growth — *September 2012*
- How to Get Startup Ideas — *November 2012*
- Do Things that Don’t Scale — *July 2013*
- How to Raise Money — *September 2013*

Part Four — Work, Wealth, and Life (2014–2022)

- Before the Startup — *October 2014*
- Default Alive or Default Dead? — *October 2015*
- Economic Inequality — *January 2016*
- Life is Short — *January 2016*
- How to Write Usefully — *February 2020*
- The Four Quadrants of Conformism — *July 2020*
- How to Think for Yourself — *November 2020*
- What I Worked On — *February 2021*
- A Project of One’s Own — *June 2021*
- How to Work Hard — *June 2021*
- Putting Ideas into Words — *February 2022*

Part One — Hackers and Painters

2001–2004

Hacking and painting have a lot in common. In fact, of all the different types of people I've known, hackers and painters are among the most alike.

— Paul Graham, “Hackers and Painters”

Beating the Averages

April 2001

April 2001, rev. April 2003

(This article is derived from a talk given at the 2001 Franz Developer Symposium.)

In the summer of 1995, my friend Robert Morris and I started a startup called Viaweb. Our plan was to write software that would let end users build online stores. What was novel about this software, at the time, was that it ran on our server, using ordinary Web pages as the interface.

A lot of people could have been having this idea at the same time, of course, but as far as I know, Viaweb was the first Web-based application. It seemed such a novel idea to us that we named the company after it: Viaweb, because our software worked via the Web, instead of running on your desktop computer.

Another unusual thing about this software was that it was written primarily in a programming language called Lisp. It was one of the first big end-user applications to be written in Lisp, which up till then had been used mostly in universities and research labs. [1]

The Secret Weapon

Eric Raymond has written an essay called “How to Become a Hacker,” and in it, among other things, he tells would-be hackers what languages they should learn. He suggests starting with Python and Java, because they are easy to learn. The serious hacker will also want to learn C, in order to hack Unix, and Perl for system administration and cgi scripts. Finally, the truly serious hacker should consider learning Lisp:

Lisp is worth learning for the profound enlightenment experience you will have when you finally get it; that experience will make you a better programmer for the rest of your days, even if you never actually use Lisp itself a lot.

This is the same argument you tend to hear for learning Latin. It won't get you a job, except perhaps as a classics professor, but it will improve your mind, and make you a better writer in languages you do want to use, like English.

But wait a minute. This metaphor doesn't stretch that far. The reason Latin won't get you a job is that no one speaks it. If you write in Latin, no one can understand you. But Lisp is a computer language, and computers speak whatever language you, the programmer, tell them to.

So if Lisp makes you a better programmer, like he says, why wouldn't you want to use it? If a painter were offered a brush that would make him a better painter, it seems to me that he would want to use it in all his paintings, wouldn't he? I'm not trying to make fun of Eric Raymond here. On the whole, his advice is good. What he says about Lisp is pretty much the conventional wisdom. But there is a contradiction in the conventional wisdom: Lisp will make you a better programmer, and yet you won't use it.

Why not? Programming languages are just tools, after all. If Lisp really does yield better programs, you should use it. And if it doesn't, then who needs it?

This is not just a theoretical question. Software is a very competitive business, prone to natural monopolies. A company that gets software written faster and better will, all other things being equal, put its competitors out of business. And when you're starting a startup, you feel this very keenly. Startups tend to be an all or nothing proposition. You either get rich, or you get nothing. In a startup, if you bet on the wrong technology, your competitors will crush you.

Robert and I both knew Lisp well, and we couldn't see any reason not to trust our instincts and go with Lisp. We knew that everyone else was writing their software in C++ or Perl. But we also knew that that didn't mean anything. If you chose technology that way, you'd be running Windows. When you choose technology, you have to ignore what other people are doing, and consider only what will work the best.

This is especially true in a startup. In a big company, you can do what all the other big companies are doing. But a startup can't do what all the other startups do. I don't think a lot of people realize this, even in startups.

The average big company grows at about ten percent a year. So if you're running a big company and you do everything the way the average big company does it, you can expect to do as well as the average big company— that is, to grow about ten percent a year.

The same thing will happen if you're running a startup, of course. If you do everything the way the average startup does it, you should expect average

performance. The problem here is, average performance means that you'll go out of business. The survival rate for startups is way less than fifty percent. So if you're running a startup, you had better be doing something odd. If not, you're in trouble.

Back in 1995, we knew something that I don't think our competitors understood, and few understand even now: when you're writing software that only has to run on your own servers, you can use any language you want. When you're writing desktop software, there's a strong bias toward writing applications in the same language as the operating system. Ten years ago, writing applications meant writing applications in C. But with Web-based software, especially when you have the source code of both the language and the operating system, you can use whatever language you want.

This new freedom is a double-edged sword, however. Now that you can use any language, you have to think about which one to use. Companies that try to pretend nothing has changed risk finding that their competitors do not.

If you can use any language, which do you use? We chose Lisp. For one thing, it was obvious that rapid development would be important in this market. We were all starting from scratch, so a company that could get new features done before its competitors would have a big advantage. We knew Lisp was a really good language for writing software quickly, and server-based applications magnify the effect of rapid development, because you can release software the minute it's done.

If other companies didn't want to use Lisp, so much the better. It might give us a technological edge, and we needed all the help we could get. When we started Viaweb, we had no experience in business. We didn't know anything about marketing, or hiring people, or raising money, or getting customers. Neither of us had ever even had what you would call a real job. The only thing we were good at was writing software. We hoped that would save us. Any advantage we could get in the software department, we would take.

So you could say that using Lisp was an experiment. Our hypothesis was that if we wrote our software in Lisp, we'd be able to get features done faster than our competitors, and also to do things in our software that they couldn't do. And because Lisp was so high-level, we wouldn't need a big development team, so our costs would be lower. If this were so, we could offer a better product for less money, and still make a profit. We would end up getting all the users, and our competitors would get none, and eventually go out of business. That was what we hoped would happen, anyway.

What were the results of this experiment? Somewhat surprisingly, it worked.

We eventually had many competitors, on the order of twenty to thirty of them, but none of their software could compete with ours. We had a wysiwyg online store builder that ran on the server and yet felt like a desktop application. Our competitors had cgi scripts. And we were always far ahead of them in features. Sometimes, in desperation, competitors would try to introduce features that we didn't have. But with Lisp our development cycle was so fast that we could sometimes duplicate a new feature within a day or two of a competitor announcing it in a press release. By the time journalists covering the press release got round to calling us, we would have the new feature too.

It must have seemed to our competitors that we had some kind of secret weapon—that we were decoding their Enigma traffic or something. In fact we did have a secret weapon, but it was simpler than they realized. No one was leaking news of their features to us. We were just able to develop software faster than anyone thought possible.

When I was about nine I happened to get hold of a copy of *The Day of the Jackal*, by Frederick Forsyth. The main character is an assassin who is hired to kill the president of France. The assassin has to get past the police to get up to an apartment that overlooks the president's route. He walks right by them, dressed up as an old man on crutches, and they never suspect him.

Our secret weapon was similar. We wrote our software in a weird AI language, with a bizarre syntax full of parentheses. For years it had annoyed me to hear Lisp described that way. But now it worked to our advantage. In business, there is nothing more valuable than a technical advantage your competitors don't understand. In business, as in war, surprise is worth as much as force.

And so, I'm a little embarrassed to say, I never said anything publicly about Lisp while we were working on Viaweb. We never mentioned it to the press, and if you searched for Lisp on our Web site, all you'd find were the titles of two books in my bio. This was no accident. A startup should give its competitors as little information as possible. If they didn't know what language our software was written in, or didn't care, I wanted to keep it that way.[2]

The people who understood our technology best were the customers. They didn't care what language Viaweb was written in either, but they noticed that it worked really well. It let them build great looking online stores literally in minutes. And so, by word of mouth mostly, we got more and more users. By the end of 1996 we had about 70 stores online. At the end of 1997 we had 500. Six months later, when Yahoo bought us, we had 1070 users. Today, as Yahoo Store, this software continues to dominate its market. It's one of the more profitable pieces of Yahoo, and the stores built with it are the foundation of Yahoo Shopping. I left Yahoo

in 1999, so I don't know exactly how many users they have now, but the last I heard there were about 20,000.

The Blub Paradox

What's so great about Lisp? And if Lisp is so great, why doesn't everyone use it? These sound like rhetorical questions, but actually they have straightforward answers. Lisp is so great not because of some magic quality visible only to devotees, but because it is simply the most powerful language available. And the reason everyone doesn't use it is that programming languages are not merely technologies, but habits of mind as well, and nothing changes slower. Of course, both these answers need explaining.

I'll begin with a shockingly controversial statement: programming languages vary in power.

Few would dispute, at least, that high level languages are more powerful than machine language. Most programmers today would agree that you do not, ordinarily, want to program in machine language. Instead, you should program in a high-level language, and have a compiler translate it into machine language for you. This idea is even built into the hardware now: since the 1980s, instruction sets have been designed for compilers rather than human programmers.

Everyone knows it's a mistake to write your whole program by hand in machine language. What's less often understood is that there is a more general principle here: that if you have a choice of several languages, it is, all other things being equal, a mistake to program in anything but the most powerful one. [3]

There are many exceptions to this rule. If you're writing a program that has to work very closely with a program written in a certain language, it might be a good idea to write the new program in the same language. If you're writing a program that only has to do something very simple, like number crunching or bit manipulation, you may as well use a less abstract language, especially since it may be slightly faster. And if you're writing a short, throwaway program, you may be better off just using whatever language has the best library functions for the task. But in general, for application software, you want to be using the most powerful (reasonably efficient) language you can get, and using anything else is a mistake, of exactly the same kind, though possibly in a lesser degree, as programming in machine language.

You can see that machine language is very low level. But, at least as a kind of social convention, high-level languages are often all treated as equivalent. They're not. Technically the term "high-level language" doesn't mean anything very definite. There's no dividing line with machine languages on one side and all the high-level languages on the other. Languages fall along a continuum [4] of

abstractness, from the most powerful all the way down to machine languages, which themselves vary in power.

Consider Cobol. Cobol is a high-level language, in the sense that it gets compiled into machine language. Would anyone seriously argue that Cobol is equivalent in power to, say, Python? It's probably closer to machine language than Python.

Or how about Perl 4? Between Perl 4 and Perl 5, lexical closures got added to the language. Most Perl hackers would agree that Perl 5 is more powerful than Perl 4. But once you've admitted that, you've admitted that one high level language can be more powerful than another. And it follows inexorably that, except in special cases, you ought to use the most powerful you can get.

This idea is rarely followed to its conclusion, though. After a certain age, programmers rarely switch languages voluntarily. Whatever language people happen to be used to, they tend to consider just good enough.

Programmers get very attached to their favorite languages, and I don't want to hurt anyone's feelings, so to explain this point I'm going to use a hypothetical language called Blub. Blub falls right in the middle of the abstractness continuum. It is not the most powerful language, but it is more powerful than Cobol or machine language.

And in fact, our hypothetical Blub programmer wouldn't use either of them. Of course he wouldn't program in machine language. That's what compilers are for. And as for Cobol, he doesn't know how anyone can get anything done with it. It doesn't even have `x` (Blub feature of your choice).

As long as our hypothetical Blub programmer is looking down the power continuum, he knows he's looking down. Languages less powerful than Blub are obviously less powerful, because they're missing some feature he's used to. But when our hypothetical Blub programmer looks in the other direction, up the power continuum, he doesn't realize he's looking up. What he sees are merely weird languages. He probably considers them about equivalent in power to Blub, but with all this other hairy stuff thrown in as well. Blub is good enough for him, because he thinks in Blub.

When we switch to the point of view of a programmer using any of the languages higher up the power continuum, however, we find that he in turn looks down upon Blub. How can you get anything done in Blub? It doesn't even have `y`.

By induction, the only programmers in a position to see all the differences in power between the various languages are those who understand the most powerful one. (This is probably what Eric Raymond meant about Lisp making you a better programmer.) You can't trust the opinions of the others, because of the

Blub paradox: they're satisfied with whatever language they happen to use, because it dictates the way they think about programs.

I know this from my own experience, as a high school kid writing programs in Basic. That language didn't even support recursion. It's hard to imagine writing programs without using recursion, but I didn't miss it at the time. I thought in Basic. And I was a whiz at it. Master of all I surveyed.

The five languages that Eric Raymond recommends to hackers fall at various points on the power continuum. Where they fall relative to one another is a sensitive topic. What I will say is that I think Lisp is at the top. And to support this claim I'll tell you about one of the things I find missing when I look at the other four languages. How can you get anything done in them, I think, without macros? [5]

Many languages have something called a macro. But Lisp macros are unique. And believe it or not, what they do is related to the parentheses. The designers of Lisp didn't put all those parentheses in the language just to be different. To the Blub programmer, Lisp code looks weird. But those parentheses are there for a reason. They are the outward evidence of a fundamental difference between Lisp and other languages.

Lisp code is made out of Lisp data objects. And not in the trivial sense that the source files contain characters, and strings are one of the data types supported by the language. Lisp code, after it's read by the parser, is made of data structures that you can traverse.

If you understand how compilers work, what's really going on is not so much that Lisp has a strange syntax as that Lisp has no syntax. You write programs in the parse trees that get generated within the compiler when other languages are parsed. But these parse trees are fully accessible to your programs. You can write programs that manipulate them. In Lisp, these programs are called macros. They are programs that write programs.

Programs that write programs? When would you ever want to do that? Not very often, if you think in Cobol. All the time, if you think in Lisp. It would be convenient here if I could give an example of a powerful macro, and say there! how about that? But if I did, it would just look like gibberish to someone who didn't know Lisp; there isn't room here to explain everything you'd need to know to understand what it meant. In Ansi Common Lisp I tried to move things along as fast as I could, and even so I didn't get to macros until page 160.

But I think I can give a kind of argument that might be convincing. The source code of the Viaweb editor was probably about 20-25% macros. Macros are harder to write than ordinary Lisp functions, and it's considered to be bad style to use

them when they're not necessary. So every macro in that code is there because it has to be. What that means is that at least 20-25% of the code in this program is doing things that you can't easily do in any other language. However skeptical the Blub programmer might be about my claims for the mysterious powers of Lisp, this ought to make him curious. We weren't writing this code for our own amusement. We were a tiny startup, programming as hard as we could in order to put technical barriers between us and our competitors.

A suspicious person might begin to wonder if there was some correlation here. A big chunk of our code was doing things that are very hard to do in other languages. The resulting software did things our competitors' software couldn't do. Maybe there was some kind of connection. I encourage you to follow that thread. There may be more to that old man hobbling along on his crutches than meets the eye.

Aikido for Startups

But I don't expect to convince anyone (over 25) to go out and learn Lisp. The purpose of this article is not to change anyone's mind, but to reassure people already interested in using Lisp— people who know that Lisp is a powerful language, but worry because it isn't widely used. In a competitive situation, that's an advantage. Lisp's power is multiplied by the fact that your competitors don't get it.

If you think of using Lisp in a startup, you shouldn't worry that it isn't widely understood. You should hope that it stays that way. And it's likely to. It's the nature of programming languages to make most people satisfied with whatever they currently use. Computer hardware changes so much faster than personal habits that programming practice is usually ten to twenty years behind the processor. At places like MIT they were writing programs in high-level languages in the early 1960s, but many companies continued to write code in machine language well into the 1980s. I bet a lot of people continued to write machine language until the processor, like a bartender eager to close up and go home, finally kicked them out by switching to a risc instruction set.

Ordinarily technology changes fast. But programming languages are different: programming languages are not just technology, but what programmers think in. They're half technology and half religion.[6] And so the median language, meaning whatever language the median programmer uses, moves as slow as an iceberg. Garbage collection, introduced by Lisp in about 1960, is now widely considered to be a good thing. Runtime typing, ditto, is growing in popularity. Lexical closures, introduced by Lisp in the early 1970s, are now, just barely, on the radar screen. Macros, introduced by Lisp in the mid 1960s, are still terra

incognita.

Obviously, the median language has enormous momentum. I'm not proposing that you can fight this powerful force. What I'm proposing is exactly the opposite: that, like a practitioner of Aikido, you can use it against your opponents.

If you work for a big company, this may not be easy. You will have a hard time convincing the pointy-haired boss to let you build things in Lisp, when he has just read in the paper that some other language is poised, like Ada was twenty years ago, to take over the world. But if you work for a startup that doesn't have pointy-haired bosses yet, you can, like we did, turn the Blub paradox to your advantage: you can use technology that your competitors, glued immovably to the median language, will never be able to match.

If you ever do find yourself working for a startup, here's a handy tip for evaluating competitors. Read their job listings. Everything else on their site may be stock photos or the prose equivalent, but the job listings have to be specific about what they want, or they'll get the wrong candidates.

During the years we worked on Viaweb I read a lot of job descriptions. A new competitor seemed to emerge out of the woodwork every month or so. The first thing I would do, after checking to see if they had a live online demo, was look at their job listings. After a couple years of this I could tell which companies to worry about and which not to. The more of an IT flavor the job descriptions had, the less dangerous the company was. The safest kind were the ones that wanted Oracle experience. You never had to worry about those. You were also safe if they said they wanted C++ or Java developers. If they wanted Perl or Python programmers, that would be a bit frightening— that's starting to sound like a company where the technical side, at least, is run by real hackers. If I had ever seen a job posting looking for Lisp hackers, I would have been really worried.

Notes

[1] Viaweb at first had two parts: the editor, written in Lisp, which people used to build their sites, and the ordering system, written in C, which handled orders. The first version was mostly Lisp, because the ordering system was small. Later we added two more modules, an image generator written in C, and a back-office manager written mostly in Perl.

In January 2003, Yahoo released a new version of the editor written in C++ and Perl. It's hard to say whether the program is no longer written in Lisp, though, because to translate this program into C++ they literally had to write a Lisp interpreter: the source files of all the page-generating templates are still, as far as I know, Lisp code. (See Greenspun's Tenth Rule.)

[2] Robert Morris says that I didn't need to be secretive, because even if our competitors had known we were using Lisp, they wouldn't have understood why: "If they were that smart they'd already be programming in Lisp."

[3] All languages are equally powerful in the sense of being Turing equivalent, but that's not the sense of the word programmers care about. (No one wants to program a Turing machine.) The kind of power programmers care about may not be formally definable, but one way to explain it would be to say that it refers to features you could only get in the less powerful language by writing an interpreter for the more powerful language in it. If language A has an operator for removing spaces from strings and language B doesn't, that probably doesn't make A more powerful, because you can probably write a subroutine to do it in B. But if A supports, say, recursion, and B doesn't, that's not likely to be something you can fix by writing library functions.

[4] Note to nerds: or possibly a lattice, narrowing toward the top; it's not the shape that matters here but the idea that there is at least a partial order.

[5] It is a bit misleading to treat macros as a separate feature. In practice their usefulness is greatly enhanced by other Lisp features like lexical closures and rest parameters.

[6] As a result, comparisons of programming languages either take the form of religious wars or undergraduate textbooks so determinedly neutral that they're really works of anthropology. People who value their peace, or want tenure, avoid the topic. But the question is only half a religious one; there is something there worth studying, especially if you want to design new languages.

— | | More Technical Details

Taste for Makers

February 2002

- Thomas Kuhn, *The Copernican Revolution*

“All of us had been trained by Kelly Johnson and believed fanatically in his insistence that an airplane that looked beautiful would fly the same way.”

- Ben Rich, *Skunk Works*

“Beauty is the first test: there is no permanent place in this world for ugly mathematics.”

- G. H. Hardy, *A Mathematician's Apology* —

I was talking recently to a friend who teaches at MIT. His field is hot now and every year he is inundated by applications from would-be graduate students. “A lot of them seem smart,” he said. “What I can’t tell is whether they have any kind of taste.”

Taste. You don’t hear that word much now. And yet we still need the underlying concept, whatever we call it. What my friend meant was that he wanted students who were not just good technicians, but who could use their technical knowledge to design beautiful things.

Mathematicians call good work “beautiful,” and so, either now or in the past, have scientists, engineers, musicians, architects, designers, writers, and painters. Is it just a coincidence that they used the same word, or is there some overlap in what they meant? If there is an overlap, can we use one field’s discoveries about beauty to help us in another?

For those of us who design things, these are not just theoretical questions. If there is such a thing as beauty, we need to be able to recognize it. We need good taste to make good things. Instead of treating beauty as an airy abstraction, to be either blathered about or avoided depending on how one feels about airy

abstractions, let's try considering it as a practical question: *how do you make good stuff?*

If you mention taste nowadays, a lot of people will tell you that "taste is subjective." They believe this because it really feels that way to them. When they like something, they have no idea why. It could be because it's beautiful, or because their mother had one, or because they saw a movie star with one in a magazine, or because they know it's expensive. Their thoughts are a tangle of unexamined impulses.

Most of us are encouraged, as children, to leave this tangle unexamined. If you make fun of your little brother for coloring people green in his coloring book, your mother is likely to tell you something like "you like to do it your way and he likes to do it his way."

Your mother at this point is not trying to teach you important truths about aesthetics. She's trying to get the two of you to stop bickering.

Like many of the half-truths adults tell us, this one contradicts other things they tell us. After dinning into you that taste is merely a matter of personal preference, they take you to the museum and tell you that you should pay attention because Leonardo is a great artist.

What goes through the kid's head at this point? What does he think "great artist" means? After having been told for years that everyone just likes to do things their own way, he is unlikely to head straight for the conclusion that a great artist is someone whose work is *better* than the others'. A far more likely theory, in his Ptolemaic model of the universe, is that a great artist is something that's good for you, like broccoli, because someone said so in a book.

Saying that taste is just personal preference is a good way to prevent disputes. The trouble is, it's not true. You feel this when you start to design things.

Whatever job people do, they naturally want to do better. Football players like to win games. CEOs like to increase earnings. It's a matter of pride, and a real pleasure, to get better at your job. But if your job is to design things, and there is no such thing as beauty, then there is *no way to get better at your job*. If taste is just personal preference, then everyone's is already perfect: you like whatever you like, and that's it.

As in any job, as you continue to design things, you'll get better at it. Your tastes will change. And, like anyone who gets better at their job, you'll know you're getting better. If so, your old tastes were not merely different, but worse. Poof goes the axiom that taste can't be wrong.

Relativism is fashionable at the moment, and that may hamper you from thinking

about taste, even as yours grows. But if you come out of the closet and admit, at least to yourself, that there is such a thing as good and bad design, then you can start to study good design in detail. How has your taste changed? When you made mistakes, what caused you to make them? What have other people learned about design?

Once you start to examine the question, it's surprising how much different fields' ideas of beauty have in common. The same principles of good design crop up again and again.

Good design is simple. You hear this from math to painting. In math it means that a shorter proof tends to be a better one. Where axioms are concerned, especially, less is more. It means much the same thing in programming. For architects and designers it means that beauty should depend on a few carefully chosen structural elements rather than a profusion of superficial ornament. (Ornament is not in itself bad, only when it's camouflage on insipid form.) Similarly, in painting, a still life of a few carefully observed and solidly modelled objects will tend to be more interesting than a stretch of flashy but mindlessly repetitive painting of, say, a lace collar. In writing it means: say what you mean and say it briefly.

It seems strange to have to emphasize simplicity. You'd think simple would be the default. Ornate is more work. But something seems to come over people when they try to be creative. Beginning writers adopt a pompous tone that doesn't sound anything like the way they speak. Designers trying to be artistic resort to swooshes and curlicues. Painters discover that they're expressionists. It's all evasion. Underneath the long words or the "expressive" brush strokes, there is not much going on, and that's frightening.

When you're forced to be simple, you're forced to face the real problem. When you can't deliver ornament, you have to deliver substance.

Good design is timeless. In math, every proof is timeless unless it contains a mistake. So what does Hardy mean when he says there is no permanent place for ugly mathematics? He means the same thing Kelly Johnson did: if something is ugly, it can't be the best solution. There must be a better one, and eventually someone will discover it.

Aiming at timelessness is a way to make yourself find the best answer: if you can imagine someone surpassing you, you should do it yourself. Some of the greatest masters did this so well that they left little room for those who came after. Every engraver since Durer has had to live in his shadow.

Aiming at timelessness is also a way to evade the grip of fashion. Fashions almost by definition change with time, so if you can make something that will still look

good far into the future, then its appeal must derive more from merit and less from fashion.

Strangely enough, if you want to make something that will appeal to future generations, one way to do it is to try to appeal to past generations. It's hard to guess what the future will be like, but we can be sure it will be like the past in caring nothing for present fashions. So if you can make something that appeals to people today and would also have appealed to people in 1500, there is a good chance it will appeal to people in 2500.

Good design solves the right problem. The typical stove has four burners arranged in a square, and a dial to control each. How do you arrange the dials? The simplest answer is to put them in a row. But this is a simple answer to the wrong question. The dials are for humans to use, and if you put them in a row, the unlucky human will have to stop and think each time about which dial matches which burner. Better to arrange the dials in a square like the burners.

A lot of bad design is industrious, but misguided. In the mid twentieth century there was a vogue for setting text in sans-serif fonts. These fonts *are* closer to the pure, underlying letterforms. But in text that's not the problem you're trying to solve. For legibility it's more important that letters be easy to tell apart. It may look Victorian, but a Times Roman lowercase g is easy to tell from a lowercase y.

Problems can be improved as well as solutions. In software, an intractable problem can usually be replaced by an equivalent one that's easy to solve. Physics progressed faster as the problem became predicting observable behavior, instead of reconciling it with scripture.

Good design is suggestive. Jane Austen's novels contain almost no description; instead of telling you how everything looks, she tells her story so well that you envision the scene for yourself. Likewise, a painting that suggests is usually more engaging than one that tells. Everyone makes up their own story about the Mona Lisa.

In architecture and design, this principle means that a building or object should let you use it how you want: a good building, for example, will serve as a backdrop for whatever life people want to lead in it, instead of making them live as if they were executing a program written by the architect.

In software, it means you should give users a few basic elements that they can combine as they wish, like Lego. In math it means a proof that becomes the basis for a lot of new work is preferable to a proof that was difficult, but doesn't lead to future discoveries; in the sciences generally, citation is considered a rough indicator of merit.

Good design is often slightly funny. This one may not always be true. But Durer's engravings and Saarinen's womb chair and the Pantheon and the original Porsche 911 all seem to me slightly funny. Godel's incompleteness theorem seems like a practical joke.

I think it's because humor is related to strength. To have a sense of humor is to be strong: to keep one's sense of humor is to shrug off misfortunes, and to lose one's sense of humor is to be wounded by them. And so the mark— or at least the prerogative— of strength is not to take oneself too seriously. The confident will often, like swallows, seem to be making fun of the whole process slightly, as Hitchcock does in his films or Bruegel in his paintings— or Shakespeare, for that matter.

Good design may not have to be funny, but it's hard to imagine something that could be called humorless also being good design.

Good design is hard. If you look at the people who've done great work, one thing they all seem to have in common is that they worked very hard. If you're not working hard, you're probably wasting your time.

Hard problems call for great efforts. In math, difficult proofs require ingenious solutions, and those tend to be interesting. Ditto in engineering.

When you have to climb a mountain you toss everything unnecessary out of your pack. And so an architect who has to build on a difficult site, or a small budget, will find that he is forced to produce an elegant design. Fashions and flourishes get knocked aside by the difficult business of solving the problem at all.

Not every kind of hard is good. There is good pain and bad pain. You want the kind of pain you get from going running, not the kind you get from stepping on a nail. A difficult problem could be good for a designer, but a fickle client or unreliable materials would not be.

In art, the highest place has traditionally been given to paintings of people. There is something to this tradition, and not just because pictures of faces get to press buttons in our brains that other pictures don't. We are so good at looking at faces that we force anyone who draws them to work hard to satisfy us. If you draw a tree and you change the angle of a branch five degrees, no one will know. When you change the angle of someone's eye five degrees, people notice.

When Bauhaus designers adopted Sullivan's "form follows function," what they meant was, form *should* follow function. And if function is hard enough, form is forced to follow it, because there is no effort to spare for error. Wild animals are beautiful because they have hard lives.

Good design looks easy. Like great athletes, great designers make it look easy.

Mostly this is an illusion. The easy, conversational tone of good writing comes only on the eighth rewrite.

In science and engineering, some of the greatest discoveries seem so simple that you say to yourself, I could have thought of that. The discoverer is entitled to reply, why didn't you?

Some Leonardo heads are just a few lines. You look at them and you think, all you have to do is get eight or ten lines in the right place and you've made this beautiful portrait. Well, yes, but you have to get them in *exactly* the right place. The slightest error will make the whole thing collapse.

Line drawings are in fact the most difficult visual medium, because they demand near perfection. In math terms, they are a closed-form solution; lesser artists literally solve the same problems by successive approximation. One of the reasons kids give up drawing at ten or so is that they decide to start drawing like grownups, and one of the first things they try is a line drawing of a face. Smack!

In most fields the appearance of ease seems to come with practice. Perhaps what practice does is train your unconscious mind to handle tasks that used to require conscious thought. In some cases you literally train your body. An expert pianist can play notes faster than the brain can send signals to his hand. Likewise an artist, after a while, can make visual perception flow in through his eye and out through his hand as automatically as someone tapping his foot to a beat.

When people talk about being in "the zone," I think what they mean is that the spinal cord has the situation under control. Your spinal cord is less hesitant, and it frees conscious thought for the hard problems.

Good design uses symmetry. I think symmetry may just be one way to achieve simplicity, but it's important enough to be mentioned on its own. Nature uses it a lot, which is a good sign.

There are two kinds of symmetry, repetition and recursion. Recursion means repetition in subelements, like the pattern of veins in a leaf.

Symmetry is unfashionable in some fields now, in reaction to excesses in the past. Architects started consciously making buildings asymmetric in Victorian times and by the 1920s asymmetry was an explicit premise of modernist architecture. Even these buildings only tended to be asymmetric about major axes, though; there were hundreds of minor symmetries.

In writing you find symmetry at every level, from the phrases in a sentence to the plot of a novel. You find the same in music and art. Mosaics (and some Cezannes) get extra visual punch by making the whole picture out of the same atoms. Compositional symmetry yields some of the most memorable paintings,

especially when two halves react to one another, as in the *Creation of Adam* or *American Gothic*.

In math and engineering, recursion, especially, is a big win. Inductive proofs are wonderfully short. In software, a problem that can be solved by recursion is nearly always best solved that way. The Eiffel Tower looks striking partly because it is a recursive solution, a tower on a tower.

The danger of symmetry, and repetition especially, is that it can be used as a substitute for thought.

Good design resembles nature. It's not so much that resembling nature is intrinsically good as that nature has had a long time to work on the problem. It's a good sign when your answer resembles nature's.

It's not cheating to copy. Few would deny that a story should be like life. Working from life is a valuable tool in painting too, though its role has often been misunderstood. The aim is not simply to make a record. The point of painting from life is that it gives your mind something to chew on: when your eyes are looking at something, your hand will do more interesting work.

Imitating nature also works in engineering. Boats have long had spines and ribs like an animal's ribcage. In some cases we may have to wait for better technology: early aircraft designers were mistaken to design aircraft that looked like birds, because they didn't have materials or power sources light enough (the Wrights' engine weighed 152 lbs. and generated only 12 hp.) or control systems sophisticated enough for machines that flew like birds, but I could imagine little unmanned reconnaissance planes flying like birds in fifty years.

Now that we have enough computer power, we can imitate nature's method as well as its results. Genetic algorithms may let us create things too complex to design in the ordinary sense.

Good design is redesign. It's rare to get things right the first time. Experts expect to throw away some early work. They plan for plans to change.

It takes confidence to throw work away. You have to be able to think, *there's more where that came from*. When people first start drawing, for example, they're often reluctant to redo parts that aren't right; they feel they've been lucky to get that far, and if they try to redo something, it will turn out worse. Instead they convince themselves that the drawing is not that bad, really— in fact, maybe they meant it to look that way.

Dangerous territory, that; if anything you should cultivate dissatisfaction. In Leonardo's drawings there are often five or six attempts to get a line right. The distinctive back of the Porsche 911 only appeared in the redesign of an awkward

prototype. In Wright's early plans for the Guggenheim, the right half was a zigurat; he inverted it to get the present shape.

Mistakes are natural. Instead of treating them as disasters, make them easy to acknowledge and easy to fix. Leonardo more or less invented the sketch, as a way to make drawing bear a greater weight of exploration. Open-source software has fewer bugs because it admits the possibility of bugs.

It helps to have a medium that makes change easy. When oil paint replaced tempera in the fifteenth century, it helped painters to deal with difficult subjects like the human figure because, unlike tempera, oil can be blended and overpainted.

Good design can copy. Attitudes to copying often make a round trip. A novice imitates without knowing it; next he tries consciously to be original; finally, he decides it's more important to be right than original.

Unknowing imitation is almost a recipe for bad design. If you don't know where your ideas are coming from, you're probably imitating an imitator. Raphael so pervaded mid-nineteenth century taste that almost anyone who tried to draw was imitating him, often at several removes. It was this, more than Raphael's own work, that bothered the Pre-Raphaelites.

The ambitious are not content to imitate. The second phase in the growth of taste is a conscious attempt at originality.

I think the greatest masters go on to achieve a kind of selflessness. They just want to get the right answer, and if part of the right answer has already been discovered by someone else, that's no reason not to use it. They're confident enough to take from anyone without feeling that their own vision will be lost in the process.

Good design is often strange. Some of the very best work has an uncanny quality: Euler's Formula, Bruegel's *Hunters in the Snow*, the SR-71, Lisp. They're not just beautiful, but strangely beautiful.

I'm not sure why. It may just be my own stupidity. A can-opener must seem miraculous to a dog. Maybe if I were smart enough it would seem the most natural thing in the world that $e^{i\pi} = -1$. It is after all necessarily true.

Most of the qualities I've mentioned are things that can be cultivated, but I don't think it works to cultivate strangeness. The best you can do is not squash it if it starts to appear. Einstein didn't try to make relativity strange. He tried to make it true, and the truth turned out to be strange.

At an art school where I once studied, the students wanted most of all to develop a personal style. But if you just try to make good things, you'll inevitably do it

in a distinctive way, just as each person walks in a distinctive way. Michelangelo was not trying to paint like Michelangelo. He was just trying to paint well; he couldn't help painting like Michelangelo.

The only style worth having is the one you can't help. And this is especially true for strangeness. There is no shortcut to it. The Northwest Passage that the Mannerists, the Romantics, and two generations of American high school students have searched for does not seem to exist. The only way to get there is to go through good and come out the other side.

Good design happens in chunks. The inhabitants of fifteenth century Florence included Brunelleschi, Ghiberti, Donatello, Masaccio, Filippo Lippi, Fra Angelico, Verrocchio, Botticelli, Leonardo, and Michelangelo. Milan at the time was as big as Florence. How many fifteenth century Milanese artists can you name?

Something was happening in Florence in the fifteenth century. And it can't have been heredity, because it isn't happening now. You have to assume that whatever inborn ability Leonardo and Michelangelo had, there were people born in Milan with just as much. What happened to the Milanese Leonardo?

There are roughly a thousand times as many people alive in the US right now as lived in Florence during the fifteenth century. A thousand Leonardos and a thousand Michelangelos walk among us. If DNA ruled, we should be greeted daily by artistic marvels. We aren't, and the reason is that to make Leonardo you need more than his innate ability. You also need Florence in 1450.

Nothing is more powerful than a community of talented people working on related problems. Genes count for little by comparison: being a genetic Leonardo was not enough to compensate for having been born near Milan instead of Florence. Today we move around more, but great work still comes disproportionately from a few hotspots: the Bauhaus, the Manhattan Project, the *New Yorker*, Lockheed's Skunk Works, Xerox Parc.

At any given time there are a few hot topics and a few groups doing great work on them, and it's nearly impossible to do good work yourself if you're too far removed from one of these centers. You can push or pull these trends to some extent, but you can't break away from them. (Maybe *you* can, but the Milanese Leonardo couldn't.)

Good design is often daring. At every period of history, people have believed things that were just ridiculous, and believed them so strongly that you risked ostracism or even violence by saying otherwise.

If our own time were any different, that would be remarkable. As far as I can

tell it isn't.

This problem afflicts not just every era, but in some degree every field. Much Renaissance art was in its time considered shockingly secular: according to Vasari, Botticelli repented and gave up painting, and Fra Bartolommeo and Lorenzo di Credi actually burned some of their work. Einstein's theory of relativity offended many contemporary physicists, and was not fully accepted for decades— in France, not until the 1950s.

Today's experimental error is tomorrow's new theory. If you want to discover great new things, then instead of turning a blind eye to the places where conventional wisdom and truth don't quite meet, you should pay particular attention to them.

As a practical matter, I think it's easier to see ugliness than to imagine beauty. Most of the people who've made beautiful things seem to have done it by fixing something that they thought ugly. Great work usually seems to happen because someone sees something and thinks, *I could do better than that*. Giotto saw traditional Byzantine madonnas painted according to a formula that had satisfied everyone for centuries, and to him they looked wooden and unnatural. Copernicus was so troubled by a hack that all his contemporaries could tolerate that he felt there must be a better solution.

Intolerance for ugliness is not in itself enough. You have to understand a field well before you develop a good nose for what needs fixing. You have to do your homework. But as you become expert in a field, you'll start to hear little voices saying, *What a hack! There must be a better way*. Don't ignore those voices. Cultivate them. The recipe for great work is: very exacting taste, plus the ability to gratify it.

Notes

Sullivan actually said "form ever follows function," but I think the usual misquotation is closer to what modernist architects meant.

Stephen G. Brush, "Why was Relativity Accepted?" *_Phys. Perspect.* 1 (1999) 184-214.

—

A Plan for Spam

August 2002

(This article describes the spam-filtering techniques used in the spamproof web-based mail reader we built to exercise Arc. An improved algorithm is described in Better Bayesian Filtering.)

I think it's possible to stop spam, and that content-based filters are the way to do it. The Achilles heel of the spammers is their message. They can circumvent any other barrier you set up. They have so far, at least. But they have to deliver their message, whatever it is. If we can write software that recognizes their messages, there is no way they can get around that.

To the recipient, spam is easily recognizable. If you hired someone to read your mail and discard the spam, they would have little trouble doing it. How much do we have to do, short of AI, to automate this process?

I think we will be able to solve the problem with fairly simple algorithms. In fact, I've found that you can filter present-day spam acceptably well using nothing more than a Bayesian combination of the spam probabilities of individual words. Using a slightly tweaked (as described below) Bayesian filter, we now miss less than 5 per 1000 spams, with 0 false positives.

The statistical approach is not usually the first one people try when they write spam filters. Most hackers' first instinct is to try to write software that recognizes individual properties of spam. You look at spams and you think, the gall of these guys to try sending me mail that begins "Dear Friend" or has a subject line that's all uppercase and ends in eight exclamation points. I can filter out that stuff with about one line of code.

And so you do, and in the beginning it works. A few simple rules will take a big bite out of your incoming spam. Merely looking for the word "click" will catch 79.7% of the emails in my spam corpus, with only 1.2% false positives.

I spent about six months writing software that looked for individual spam features before I tried the statistical approach. What I found was that recognizing that last few percent of spams got very hard, and that as I made the filters stricter I got more false positives.

False positives are innocent emails that get mistakenly identified as spams. For most users, missing legitimate email is an order of magnitude worse than receiving spam, so a filter that yields false positives is like an acne cure that carries a risk of death to the patient.

The more spam a user gets, the less likely he'll be to notice one innocent mail sitting in his spam folder. And strangely enough, the better your spam filters get, the more dangerous false positives become, because when the filters are really good, users will be more likely to ignore everything they catch.

I don't know why I avoided trying the statistical approach for so long. I think it was because I got addicted to trying to identify spam features myself, as if I were playing some kind of competitive game with the spammers. (Nonhackers don't often realize this, but most hackers are very competitive.) When I did try statistical analysis, I found immediately that it was much cleverer than I had been. It discovered, of course, that terms like "virtumundo" and "teens" were good indicators of spam. But it also discovered that "per" and "FL" and "ff0000" are good indicators of spam. In fact, "ff0000" (html for bright red) turns out to be as good an indicator of spam as any pornographic term.

Here's a sketch of how I do statistical filtering. I start with one corpus of spam and one of nonspam mail. At the moment each one has about 4000 messages in it. I scan the entire text, including headers and embedded html and javascript, of each message in each corpus. I currently consider alphanumeric characters, dashes, apostrophes, and dollar signs to be part of tokens, and everything else to be a token separator. (There is probably room for improvement here.) I ignore tokens that are all digits, and I also ignore html comments, not even considering them as token separators.

I count the number of times each token (ignoring case, currently) occurs in each corpus. At this stage I end up with two large hash tables, one for each corpus, mapping tokens to number of occurrences.

Next I create a third hash table, this time mapping each token to the probability that an email containing it is a spam, which I calculate as follows [1]: (let ((g (* 2 (or (gethash word good) 0))) (b (or (gethash word bad) 0))) (unless (< (+ g b) 5) (max.01 (min.99 (float (/ (min 1 (/ b nbad)) (+ (min 1 (/ g ngood)) (min 1 (/ b nbad)))))))))) where word is the token whose probability we're calculating,

good and bad are the hash tables I created in the first step, and ngood and nbad are the number of nonspam and spam messages respectively.

I explained this as code to show a couple of important details. I want to bias the probabilities slightly to avoid false positives, and by trial and error I've found that a good way to do it is to double all the numbers in good. This helps to distinguish between words that occasionally do occur in legitimate email and words that almost never do. I only consider words that occur more than five times in total (actually, because of the doubling, occurring three times in nonspam mail would be enough). And then there is the question of what probability to assign to words that occur in one corpus but not the other. Again by trial and error I chose .01 and .99. There may be room for tuning here, but as the corpus grows such tuning will happen automatically anyway.

The especially observant will notice that while I consider each corpus to be a single long stream of text for purposes of counting occurrences, I use the number of emails in each, rather than their combined length, as the divisor in calculating spam probabilities. This adds another slight bias to protect against false positives.

When new mail arrives, it is scanned into tokens, and the most interesting fifteen tokens, where interesting is measured by how far their spam probability is from a neutral .5, are used to calculate the probability that the mail is spam. If probs is a list of the fifteen individual probabilities, you calculate the combined probability thus: (let ((prod (apply #'* probs))) (/ prod (+ prod (apply #'* (mapcar #'(lambda (x) (- 1 x)) probs))))) One question that arises in practice is what probability to assign to a word you've never seen, i.e. one that doesn't occur in the hash table of word probabilities. I've found, again by trial and error, that .4 is a good number to use. If you've never seen a word before, it is probably fairly innocent; spam words tend to be all too familiar.

There are examples of this algorithm being applied to actual emails in an appendix at the end.

I treat mail as spam if the algorithm above gives it a probability of more than .9 of being spam. But in practice it would not matter much where I put this threshold, because few probabilities end up in the middle of the range.

One great advantage of the statistical approach is that you don't have to read so many spams. Over the past six months, I've read literally thousands of spams, and it is really kind of demoralizing. Norbert Wiener said if you compete with slaves you become a slave, and there is something similarly degrading about competing with spammers. To recognize individual spam features you have to

try to get into the mind of the spammer, and frankly I want to spend as little time inside the minds of spammers as possible.

But the real advantage of the Bayesian approach, of course, is that you know what you're measuring. Feature-recognizing filters like SpamAssassin assign a spam "score" to email. The Bayesian approach assigns an actual probability. The problem with a "score" is that no one knows what it means. The user doesn't know what it means, but worse still, neither does the developer of the filter. How many *points* should an email get for having the word "sex" in it? A probability can of course be mistaken, but there is little ambiguity about what it means, or how evidence should be combined to calculate it. Based on my corpus, "sex" indicates a .97 probability of the containing email being a spam, whereas "sexy" indicates .99 probability. And Bayes' Rule, equally unambiguous, says that an email containing both words would, in the (unlikely) absence of any other evidence, have a 99.97% chance of being a spam.

Because it is measuring probabilities, the Bayesian approach considers all the evidence in the email, both good and bad. Words that occur disproportionately *rarely* in spam (like "though" or "tonight" or "apparently") contribute as much to decreasing the probability as bad words like "unsubscribe" and "opt-in" do to increasing it. So an otherwise innocent email that happens to include the word "sex" is not going to get tagged as spam.

Ideally, of course, the probabilities should be calculated individually for each user. I get a lot of email containing the word "Lisp", and (so far) no spam that does. So a word like that is effectively a kind of password for sending mail to me. In my earlier spam-filtering software, the user could set up a list of such words and mail containing them would automatically get past the filters. On my list I put words like "Lisp" and also my zipcode, so that (otherwise rather spammy-sounding) receipts from online orders would get through. I thought I was being very clever, but I found that the Bayesian filter did the same thing for me, and moreover discovered a lot of words I hadn't thought of.

When I said at the start that our filters let through less than 5 spams per 1000 with 0 false positives, I'm talking about filtering my mail based on a corpus of my mail. But these numbers are not misleading, because that is the approach I'm advocating: filter each user's mail based on the spam and nonspam mail he receives. Essentially, each user should have two delete buttons, ordinary delete and delete-as-spam. Anything deleted as spam goes into the spam corpus, and everything else goes into the nonspam corpus.

You could start users with a seed filter, but ultimately each user should have his own per-word probabilities based on the actual mail he receives. This (a) makes

the filters more effective, (b) lets each user decide their own precise definition of spam, and (c) perhaps best of all makes it hard for spammers to tune mails to get through the filters. If a lot of the brain of the filter is in the individual databases, then merely tuning spams to get through the seed filters won't guarantee anything about how well they'll get through individual users' varying and much more trained filters.

Content-based spam filtering is often combined with a whitelist, a list of senders whose mail can be accepted with no filtering. One easy way to build such a whitelist is to keep a list of every address the user has ever sent mail to. If a mail reader has a delete-as-spam button then you could also add the from address of every email the user has deleted as ordinary trash.

I'm an advocate of whitelists, but more as a way to save computation than as a way to improve filtering. I used to think that whitelists would make filtering easier, because you'd only have to filter email from people you'd never heard from, and someone sending you mail for the first time is constrained by convention in what they can say to you. Someone you already know might send you an email talking about sex, but someone sending you mail for the first time would not be likely to. The problem is, people can have more than one email address, so a new from-address doesn't guarantee that the sender is writing to you for the first time. It is not unusual for an old friend (especially if he is a hacker) to suddenly send you an email with a new from-address, so you can't risk false positives by filtering mail from unknown addresses especially stringently.

In a sense, though, my filters do themselves embody a kind of whitelist (and blacklist) because they are based on entire messages, including the headers. So to that extent they "know" the email addresses of trusted senders and even the routes by which mail gets from them to me. And they know the same about spam, including the server names, mailer versions, and protocols.

If I thought that I could keep up current rates of spam filtering, I would consider this problem solved. But it doesn't mean much to be able to filter out most present-day spam, because spam evolves. Indeed, most antispam techniques so far have been like pesticides that do nothing more than create a new, resistant strain of bugs.

I'm more hopeful about Bayesian filters, because they evolve with the spam. So as spammers start using "c0ck" instead of "cock" to evade simple-minded spam filters based on individual words, Bayesian filters automatically notice. Indeed, "c0ck" is far more damning evidence than "cock", and Bayesian filters know precisely how much more.

Still, anyone who proposes a plan for spam filtering has to be able to answer the question: if the spammers knew exactly what you were doing, how well could they get past you? For example, I think that if checksum-based spam filtering becomes a serious obstacle, the spammers will just switch to mad-lib techniques for generating message bodies.

To beat Bayesian filters, it would not be enough for spammers to make their emails unique or to stop using individual naughty words. They'd have to make their mails indistinguishable from your ordinary mail. And this I think would severely constrain them. Spam is mostly sales pitches, so unless your regular mail is all sales pitches, spams will inevitably have a different character. And the spammers would also, of course, have to change (and keep changing) their whole infrastructure, because otherwise the headers would look as bad to the Bayesian filters as ever, no matter what they did to the message body. I don't know enough about the infrastructure that spammers use to know how hard it would be to make the headers look innocent, but my guess is that it would be even harder than making the message look innocent.

Assuming they could solve the problem of the headers, the spam of the future will probably look something like this: Hey there. Thought you should check out the following: <http://www.27meg.com/foo> because that is about as much sales pitch as content-based filtering will leave the spammer room to make. (Indeed, it will be hard even to get this past filters, because if everything else in the email is neutral, the spam probability will hinge on the url, and it will take some effort to make that look neutral.)

Spammers range from businesses running so-called opt-in lists who don't even try to conceal their identities, to guys who hijack mail servers to send out spams promoting porn sites. If we use filtering to whittle their options down to mails like the one above, that should pretty much put the spammers on the "legitimate" end of the spectrum out of business; they feel obliged by various state laws to include boilerplate about why their spam is not spam, and how to cancel your "subscription," and that kind of text is easy to recognize.

(I used to think it was naive to believe that stricter laws would decrease spam. Now I think that while stricter laws may not decrease the amount of spam that spammers *send*, they can certainly help filters to decrease the amount of spam that recipients actually see.)

All along the spectrum, if you restrict the sales pitches spammers can make, you will inevitably tend to put them out of business. That word *business* is an important one to remember. The spammers are businessmen. They send spam because it works. It works because although the response rate is abominably low

(at best 15 per million, vs 3000 per million for a catalog mailing), the cost, to them, is practically nothing. The cost is enormous for the recipients, about 5 man-weeks for each million recipients who spend a second to delete the spam, but the spammer doesn't have to pay that.

Sending spam does cost the spammer something, though. [2] So the lower we can get the response rate—whether by filtering, or by using filters to force spammers to dilute their pitches—the fewer businesses will find it worth their while to send spam.

The reason the spammers use the kinds of sales pitches that they do is to increase response rates. This is possibly even more disgusting than getting inside the mind of a spammer, but let's take a quick look inside the mind of someone who *responds* to a spam. This person is either astonishingly credulous or deeply in denial about their sexual interests. In either case, repulsive or idiotic as the spam seems to us, it is exciting to them. The spammers wouldn't say these things if they didn't sound exciting. And “thought you should check out the following” is just not going to have nearly the pull with the spam recipient as the kinds of things that spammers say now. Result: if it can't contain exciting sales pitches, spam becomes less effective as a marketing vehicle, and fewer businesses want to use it.

That is the big win in the end. I started writing spam filtering software because I didn't want have to look at the stuff anymore. But if we get good enough at filtering out spam, it will stop working, and the spammers will actually stop sending it.

Of all the approaches to fighting spam, from software to laws, I believe Bayesian filtering will be the single most effective. But I also think that the more different kinds of antispam efforts we undertake, the better, because any measure that constrains spammers will tend to make filtering easier. And even within the world of content-based filtering, I think it will be a good thing if there are many different kinds of software being used simultaneously. The more different filters there are, the harder it will be for spammers to tune spams to get through them.

Appendix: Examples of Filtering

Here is an example of a spam that arrived while I was writing this article. The fifteen most interesting words in this spam are: qvp0045 indira mx-05 intimail \$7500 freeyankeedom cdo bluefoxmedia jpg unsecured platinum 3d0 qves 7c5 7c266675 The words are a mix of stuff from the headers and from the message body, which is typical of spam. Also typical of spam is that every one of these

words has a spam probability, in my database, of .99. In fact there are more than fifteen words with probabilities of .99, and these are just the first fifteen seen.

Unfortunately that makes this email a boring example of the use of Bayes' Rule. To see an interesting variety of probabilities we have to look at this actually quite atypical spam.

The fifteen most interesting words in this spam, with their probabilities, are: madam 0.99 promotion 0.99 republic 0.99 shortest 0.047225013 mandatory 0.047225013 standardization 0.07347802 sorry 0.08221981 supported 0.09019077 people's 0.09019077 enter 0.9075001 quality 0.8921298 organization 0.12454646 investment 0.8568143 very 0.14758544 valuable 0.82347786 This time the evidence is a mix of good and bad. A word like "shortest" is almost as much evidence for innocence as a word like "madam" or "promotion" is for guilt. But still the case for guilt is stronger. If you combine these numbers according to Bayes' Rule, the resulting probability is .9027.

"Madam" is obviously from spams beginning "Dear Sir or Madam." They're not very common, but the word "madam" *never* occurs in my legitimate email, and it's all about the ratio.

"Republic" scores high because it often shows up in Nigerian scam emails, and also occurs once or twice in spams referring to Korea and South Africa. You might say that it's an accident that it thus helps identify this spam. But I've found when examining spam probabilities that there are a lot of these accidents, and they have an uncanny tendency to push things in the right direction rather than the wrong one. In this case, it is not entirely a coincidence that the word "Republic" occurs in Nigerian scam emails and this spam. There is a whole class of dubious business propositions involving less developed countries, and these in turn are more likely to have names that specify explicitly (because they aren't) that they are republics.[3]

On the other hand, "enter" is a genuine miss. It occurs mostly in unsubscribe instructions, but here is used in a completely innocent way. Fortunately the statistical approach is fairly robust, and can tolerate quite a lot of misses before the results start to be thrown off.

For comparison, here is an example of that rare bird, a spam that gets through the filters. Why? Because by sheer chance it happens to be loaded with words that occur in my actual email: perl 0.01 python 0.01 tcl 0.01 scripting 0.01 morris 0.01 graham 0.01491078 guarantee 0.9762507 cgi 0.9734398 paul 0.027040077 quite 0.030676773 pop3 0.042199217 various 0.06080265 prices 0.9359873 managed 0.06451222 difficult 0.071706355 There are a couple pieces of good news here. First, this mail probably wouldn't get through the filters of someone who didn't

happen to specialize in programming languages and have a good friend called Morris. For the average user, all the top five words here would be neutral and would not contribute to the spam probability.

Second, I think filtering based on word pairs (see below) might well catch this one: “cost effective”, “setup fee”, “money back” – pretty incriminating stuff. And of course if they continued to spam me (or a network I was part of), “Hostex” itself would be recognized as a spam term.

Finally, here is an innocent email. Its fifteen most interesting words are as follows: continuation 0.01 describe 0.01 continuations 0.01 example 0.033600237 programming 0.05214485 i’m 0.055427782 examples 0.07972858 color 0.9189189 localhost 0.09883721 hi 0.116539136 california 0.84421706 same 0.15981844 spot 0.1654587 us-ascii 0.16804294 what 0.19212411 Most of the words here indicate the mail is an innocent one. There are two bad smelling words, “color” (spammers love colored fonts) and “California” (which occurs in testimonials and also in menus in forms), but they are not enough to outweigh obviously innocent words like “continuation” and “example”.

It’s interesting that “describe” rates as so thoroughly innocent. It hasn’t occurred in a single one of my 4000 spams. The data turns out to be full of such surprises. One of the things you learn when you analyze spam texts is how narrow a subset of the language spammers operate in. It’s that fact, together with the equally characteristic vocabulary of any individual user’s mail, that makes Bayesian filtering a good bet.

Appendix: More Ideas

One idea that I haven’t tried yet is to filter based on word pairs, or even triples, rather than individual words. This should yield a much sharper estimate of the probability. For example, in my current database, the word “offers” has a probability of .96. If you based the probabilities on word pairs, you’d end up with “special offers” and “valuable offers” having probabilities of .99 and, say, “approach offers” (as in “this approach offers”) having a probability of .1 or less.

The reason I haven’t done this is that filtering based on individual words already works so well. But it does mean that there is room to tighten the filters if spam gets harder to detect. (Curiously, a filter based on word pairs would be in effect a Markov-chaining text generator running in reverse.)

Specific spam features (e.g. not seeing the recipient’s address in the to: field) do of course have value in recognizing spam. They can be considered in this algorithm by treating them as virtual words. I’ll probably do this in future versions, at least for a handful of the most egregious spam indicators. Feature-recognizing

spam filters are right in many details; what they lack is an overall discipline for combining evidence.

Recognizing nonspam features may be more important than recognizing spam features. False positives are such a worry that they demand extraordinary measures. I will probably in future versions add a second level of testing designed specifically to avoid false positives. If a mail triggers this second level of filters it will be accepted even if its spam probability is above the threshold.

I don't expect this second level of filtering to be Bayesian. It will inevitably be not only ad hoc, but based on guesses, because the number of false positives will not tend to be large enough to notice patterns. (It is just as well, anyway, if a backup system doesn't rely on the same technology as the primary system.)

Another thing I may try in the future is to focus extra attention on specific parts of the email. For example, about 95% of current spam includes the url of a site they want you to visit. (The remaining 5% want you to call a phone number, reply by email or to a US mail address, or in a few cases to buy a certain stock.) The url is in such cases practically enough by itself to determine whether the email is spam.

Domain names differ from the rest of the text in a (non-German) email in that they often consist of several words stuck together. Though computationally expensive in the general case, it might be worth trying to decompose them. If a filter has never seen the token "xxxporn" before it will have an individual spam probability of .4, whereas "xxx" and "porn" individually have probabilities (in my corpus) of .9889 and .99 respectively, and a combined probability of .9998.

I expect decomposing domain names to become more important as spammers are gradually forced to stop using incriminating words in the text of their messages. (A url with an ip address is of course an extremely incriminating sign, except in the mail of a few sysadmins.)

It might be a good idea to have a cooperatively maintained list of urls promoted by spammers. We'd need a trust metric of the type studied by Raph Levien to prevent malicious or incompetent submissions, but if we had such a thing it would provide a boost to any filtering software. It would also be a convenient basis for boycotts.

Another way to test dubious urls would be to send out a crawler to look at the site before the user looked at the email mentioning it. You could use a Bayesian filter to rate the site just as you would an email, and whatever was found on the site could be included in calculating the probability of the email being a spam. A url that led to a redirect would of course be especially suspicious.

One cooperative project that I think really would be a good idea would be to accumulate a giant corpus of spam. A large, clean corpus is the key to making Bayesian filtering work well. Bayesian filters could actually use the corpus as input. But such a corpus would be useful for other kinds of filters too, because it could be used to test them.

Creating such a corpus poses some technical problems. We'd need trust metrics to prevent malicious or incompetent submissions, of course. We'd also need ways of erasing personal information (not just to-addresses and ccs, but also e.g. the arguments to unsubscribe urls, which often encode the to-address) from mails in the corpus. If anyone wants to take on this project, it would be a good thing for the world.

Appendix: Defining Spam

I think there is a rough consensus on what spam is, but it would be useful to have an explicit definition. We'll need to do this if we want to establish a central corpus of spam, or even to compare spam filtering rates meaningfully.

To start with, spam is not unsolicited commercial email. If someone in my neighborhood heard that I was looking for an old Raleigh three-speed in good condition, and sent me an email offering to sell me one, I'd be delighted, and yet this email would be both commercial and unsolicited. The defining feature of spam (in fact, its *raison d'être*) is not that it is unsolicited, but that it is automated.

It is merely incidental, too, that spam is usually commercial. If someone started sending mass email to support some political cause, for example, it would be just as much spam as email promoting a porn site.

I propose we define spam as **unsolicited automated email**. This definition thus includes some email that many legal definitions of spam don't. Legal definitions of spam, influenced presumably by lobbyists, tend to exclude mail sent by companies that have an "existing relationship" with the recipient. But buying something from a company, for example, does not imply that you have solicited ongoing email from them. If I order something from an online store, and they then send me a stream of spam, it's still spam.

Companies sending spam often give you a way to "unsubscribe," or ask you to go to their site and change your "account preferences" if you want to stop getting spam. This is not enough to stop the mail from being spam. Not opting out is not the same as opting in. Unless the recipient explicitly checked a clearly labelled box (whose default was no) asking to receive the email, then it is spam.

In some business relationships, you do implicitly solicit certain kinds of mail.

When you order online, I think you implicitly solicit a receipt, and notification when the order ships. I don't mind when Verisign sends me mail warning that a domain name is about to expire (at least, if they are the actual registrar for it). But when Verisign sends me email offering a FREE Guide to Building My E-Commerce Web Site, that's spam.

Notes:

[1] The examples in this article are translated into Common Lisp for, believe it or not, greater accessibility. The application described here is one that we wrote in order to test a new Lisp dialect called Arc that is not yet released.

[2] Currently the lowest rate seems to be about \$200 to send a million spams. That's very cheap, 1/50th of a cent per spam. But filtering out 95% of spam, for example, would increase the spammers' cost to reach a given audience by a factor of 20. Few can have margins big enough to absorb that.

[3] As a rule of thumb, the more qualifiers there are before the name of a country, the more corrupt the rulers. A country called The Socialist People's Democratic Republic of X is probably the last place in the world you'd want to live.

Thanks to Sarah Harlin for reading drafts of this; Daniel Giffin (who is also writing the production Arc interpreter) for several good ideas about filtering and for creating our mail infrastructure; Robert Morris, Trevor Blackwell and Erann Gat for many discussions about spam; Raph Levien for advice about trust metrics; and Chip Coldwell and Sam Steingold for advice about statistics.

You'll find this essay and 14 others in *Hackers & Painters*.

More Info:

— Plan for Spam FAQ | | Better Bayesian Filtering

Filters that Fight Back | | Will Filters Kill Spam?

Spam is Different | | Filters vs. Blacklists

Trust Metrics | | Filtering Research

Microsoft Patent | | Slashdot Article

The Wrong Way | | LWN: Filter Comparison

CRM114 gets 99.87%

Why Nerds are Unpopular

February 2003

When we were in junior high school, my friend Rich and I made a map of the school lunch tables according to popularity. This was easy to do, because kids only ate lunch with others of about the same popularity. We graded them from A to E. A tables were full of football players and cheerleaders and so on. E tables contained the kids with mild cases of Down's Syndrome, what in the language of the time we called "retards."

We sat at a D table, as low as you could get without looking physically different. We were not being especially candid to grade ourselves as D. It would have taken a deliberate lie to say otherwise. Everyone in the school knew exactly how popular everyone else was, including us.

My stock gradually rose during high school. Puberty finally arrived; I became a decent soccer player; I started a scandalous underground newspaper. So I've seen a good part of the popularity landscape.

I know a lot of people who were nerds in school, and they all tell the same story: there is a strong correlation between being smart and being a nerd, and an even stronger inverse correlation between being a nerd and being popular. Being smart seems to *make* you unpopular.

Why? To someone in school now, that may seem an odd question to ask. The mere fact is so overwhelming that it may seem strange to imagine that it could be any other way. But it could. Being smart doesn't make you an outcast in elementary school. Nor does it harm you in the real world. Nor, as far as I can tell, is the problem so bad in most other countries. But in a typical American secondary school, being smart is likely to make your life difficult. Why?

The key to this mystery is to rephrase the question slightly. Why don't smart kids make themselves popular? If they're so smart, why don't they figure out how popularity works and beat the system, just as they do for standardized

tests?

One argument says that this would be impossible, that the smart kids are unpopular because the other kids envy them for being smart, and nothing they could do could make them popular. I wish. If the other kids in junior high school envied me, they did a great job of concealing it. And in any case, if being smart were really an enviable quality, the girls would have broken ranks. The guys that guys envy, girls like.

In the schools I went to, being smart just didn't matter much. Kids didn't admire it or despise it. All other things being equal, they would have preferred to be on the smart side of average rather than the dumb side, but intelligence counted far less than, say, physical appearance, charisma, or athletic ability.

So if intelligence in itself is not a factor in popularity, why are smart kids so consistently unpopular? The answer, I think, is that they don't really want to be popular.

If someone had told me that at the time, I would have laughed at him. Being unpopular in school makes kids miserable, some of them so miserable that they commit suicide. Telling me that I didn't want to be popular would have seemed like telling someone dying of thirst in a desert that he didn't want a glass of water. Of course I wanted to be popular.

But in fact I didn't, not enough. There was something else I wanted more: to be smart. Not simply to do well in school, though that counted for something, but to design beautiful rockets, or to write well, or to understand how to program computers. In general, to make great things.

At the time I never tried to separate my wants and weigh them against one another. If I had, I would have seen that being smart was more important. If someone had offered me the chance to be the most popular kid in school, but only at the price of being of average intelligence (humor me here), I wouldn't have taken it.

Much as they suffer from their unpopularity, I don't think many nerds would. To them the thought of average intelligence is unbearable. But most kids would take that deal. For half of them, it would be a step up. Even for someone in the eightieth percentile (assuming, as everyone seemed to then, that intelligence is a scalar), who wouldn't drop thirty points in exchange for being loved and admired by everyone?

And that, I think, is the root of the problem. Nerds serve two masters. They want to be popular, certainly, but they want even more to be smart. And popularity is not something you can do in your spare time, not in the fiercely competitive

environment of an American secondary school.

Alberti, arguably the archetype of the Renaissance Man, writes that “no art, however minor, demands less than total dedication if you want to excel in it.” I wonder if anyone in the world works harder at anything than American school kids work at popularity. Navy SEALs and neurosurgery residents seem slackers by comparison. They occasionally take vacations; some even have hobbies. An American teenager may work at being popular every waking hour, 365 days a year.

I don’t mean to suggest they do this consciously. Some of them truly are little Machiavellis, but what I really mean here is that teenagers are always on duty as conformists.

For example, teenage kids pay a great deal of attention to clothes. They don’t consciously dress to be popular. They dress to look good. But to who? To the other kids. Other kids’ opinions become their definition of right, not just for clothes, but for almost everything they do, right down to the way they walk. And so every effort they make to do things “right” is also, consciously or not, an effort to be more popular.

Nerds don’t realize this. They don’t realize that it takes work to be popular. In general, people outside some very demanding field don’t realize the extent to which success depends on constant (though often unconscious) effort. For example, most people seem to consider the ability to draw as some kind of innate quality, like being tall. In fact, most people who “can draw” like drawing, and have spent many hours doing it; that’s why they’re good at it. Likewise, popular isn’t just something you are or you aren’t, but something you make yourself.

The main reason nerds are unpopular is that they have other things to think about. Their attention is drawn to books or the natural world, not fashions and parties. They’re like someone trying to play soccer while balancing a glass of water on his head. Other players who can focus their whole attention on the game beat them effortlessly, and wonder why they seem so incapable.

Even if nerds cared as much as other kids about popularity, being popular would be more work for them. The popular kids learned to be popular, and to want to be popular, the same way the nerds learned to be smart, and to want to be smart: from their parents. While the nerds were being trained to get the right answers, the popular kids were being trained to please.

So far I’ve been finessing the relationship between smart and nerd, using them as if they were interchangeable. In fact it’s only the context that makes them so. A nerd is someone who isn’t socially adept enough. But “enough” depends on where you are. In a typical American school, standards for coolness are so high

(or at least, so specific) that you don't have to be especially awkward to look awkward by comparison.

Few smart kids can spare the attention that popularity requires. Unless they also happen to be good-looking, natural athletes, or siblings of popular kids, they'll tend to become nerds. And that's why smart people's lives are worst between, say, the ages of eleven and seventeen. Life at that age revolves far more around popularity than before or after.

Before that, kids' lives are dominated by their parents, not by other kids. Kids do care what their peers think in elementary school, but this isn't their whole life, as it later becomes.

Around the age of eleven, though, kids seem to start treating their family as a day job. They create a new world among themselves, and standing in this world is what matters, not standing in their family. Indeed, being in trouble in their family can win them points in the world they care about.

The problem is, the world these kids create for themselves is at first a very crude one. If you leave a bunch of eleven-year-olds to their own devices, what you get is *Lord of the Flies*. Like a lot of American kids, I read this book in school. Presumably it was not a coincidence. Presumably someone wanted to point out to us that we were savages, and that we had made ourselves a cruel and stupid world. This was too subtle for me. While the book seemed entirely believable, I didn't get the additional message. I wish they had just told us outright that we were savages and our world was stupid.

Nerds would find their unpopularity more bearable if it merely caused them to be ignored. Unfortunately, to be unpopular in school is to be actively persecuted.

Why? Once again, anyone currently in school might think this a strange question to ask. How could things be any other way? But they could be. Adults don't normally persecute nerds. Why do teenage kids do it?

Partly because teenagers are still half children, and many children are just intrinsically cruel. Some torture nerds for the same reason they pull the legs off spiders. Before you develop a conscience, torture is amusing.

Another reason kids persecute nerds is to make themselves feel better. When you tread water, you lift yourself up by pushing water down. Likewise, in any social hierarchy, people unsure of their own position will try to emphasize it by maltreating those they think rank below. I've read that this is why poor whites in the United States are the group most hostile to blacks.

But I think the main reason other kids persecute nerds is that it's part of the mechanism of popularity. Popularity is only partially about individual

attractiveness. It's much more about alliances. To become more popular, you need to be constantly doing things that bring you close to other popular people, and nothing brings people closer than a common enemy.

Like a politician who wants to distract voters from bad times at home, you can create an enemy if there isn't a real one. By singling out and persecuting a nerd, a group of kids from higher in the hierarchy create bonds between themselves. Attacking an outsider makes them all insiders. This is why the worst cases of bullying happen with groups. Ask any nerd: you get much worse treatment from a group of kids than from any individual bully, however sadistic.

If it's any consolation to the nerds, it's nothing personal. The group of kids who band together to pick on you are doing the same thing, and for the same reason, as a bunch of guys who get together to go hunting. They don't actually hate you. They just need something to chase.

Because they're at the bottom of the scale, nerds are a safe target for the entire school. If I remember correctly, the most popular kids don't persecute nerds; they don't need to stoop to such things. Most of the persecution comes from kids lower down, the nervous middle classes.

The trouble is, there are a lot of them. The distribution of popularity is not a pyramid, but tapers at the bottom like a pear. The least popular group is quite small. (I believe we were the only D table in our cafeteria map.) So there are more people who want to pick on nerds than there are nerds.

As well as gaining points by distancing oneself from unpopular kids, one loses points by being close to them. A woman I know says that in high school she liked nerds, but was afraid to be seen talking to them because the other girls would make fun of her. Unpopularity is a communicable disease; kids too nice to pick on nerds will still ostracize them in self-defense.

It's no wonder, then, that smart kids tend to be unhappy in middle school and high school. Their other interests leave them little attention to spare for popularity, and since popularity resembles a zero-sum game, this in turn makes them targets for the whole school. And the strange thing is, this nightmare scenario happens without any conscious malice, merely because of the shape of the situation.

For me the worst stretch was junior high, when kid culture was new and harsh, and the specialization that would later gradually separate the smarter kids had barely begun. Nearly everyone I've talked to agrees: the nadir is somewhere between eleven and fourteen.

In our school it was eighth grade, which was ages twelve and thirteen for me.

There was a brief sensation that year when one of our teachers overheard a group of girls waiting for the school bus, and was so shocked that the next day she devoted the whole class to an eloquent plea not to be so cruel to one another.

It didn't have any noticeable effect. What struck me at the time was that she was surprised. You mean she doesn't know the kind of things they say to one another? You mean this isn't normal?

It's important to realize that, no, the adults don't know what the kids are doing to one another. They know, in the abstract, that kids are monstrously cruel to one another, just as we know in the abstract that people get tortured in poorer countries. But, like us, they don't like to dwell on this depressing fact, and they don't see evidence of specific abuses unless they go looking for it.

Public school teachers are in much the same position as prison wardens. Wardens' main concern is to keep the prisoners on the premises. They also need to keep them fed, and as far as possible prevent them from killing one another. Beyond that, they want to have as little to do with the prisoners as possible, so they leave them to create whatever social organization they want. From what I've read, the society that the prisoners create is warped, savage, and pervasive, and it is no fun to be at the bottom of it.

In outline, it was the same at the schools I went to. The most important thing was to stay on the premises. While there, the authorities fed you, prevented overt violence, and made some effort to teach you something. But beyond that they didn't want to have too much to do with the kids. Like prison wardens, the teachers mostly left us to ourselves. And, like prisoners, the culture we created was barbaric.

Why is the real world more hospitable to nerds? It might seem that the answer is simply that it's populated by adults, who are too mature to pick on one another. But I don't think this is true. Adults in prison certainly pick on one another. And so, apparently, do society wives; in some parts of Manhattan, life for women sounds like a continuation of high school, with all the same petty intrigues.

I think the important thing about the real world is not that it's populated by adults, but that it's very large, and the things you do have real effects. That's what school, prison, and ladies-who-lunch all lack. The inhabitants of all those worlds are trapped in little bubbles where nothing they do can have more than a local effect. Naturally these societies degenerate into savagery. They have no function for their form to follow.

When the things you do have real effects, it's no longer enough just to be pleasing. It starts to be important to get the right answers, and that's where nerds show to

advantage. Bill Gates will of course come to mind. Though notoriously lacking in social skills, he gets the right answers, at least as measured in revenue.

The other thing that's different about the real world is that it's much larger. In a large enough pool, even the smallest minorities can achieve a critical mass if they clump together. Out in the real world, nerds collect in certain places and form their own societies where intelligence is the most important thing. Sometimes the current even starts to flow in the other direction: sometimes, particularly in university math and science departments, nerds deliberately exaggerate their awkwardness in order to seem smarter. John Nash so admired Norbert Wiener that he adopted his habit of touching the wall as he walked down a corridor.

As a thirteen-year-old kid, I didn't have much more experience of the world than what I saw immediately around me. The warped little world we lived in was, I thought, *the world*. The world seemed cruel and boring, and I'm not sure which was worse.

Because I didn't fit into this world, I thought that something must be wrong with me. I didn't realize that the reason we nerds didn't fit in was that in some ways we were a step ahead. We were already thinking about the kind of things that matter in the real world, instead of spending all our time playing an exacting but mostly pointless game like the others.

We were a bit like an adult would be if he were thrust back into middle school. He wouldn't know the right clothes to wear, the right music to like, the right slang to use. He'd seem to the kids a complete alien. The thing is, he'd know enough not to care what they thought. We had no such confidence.

A lot of people seem to think it's good for smart kids to be thrown together with "normal" kids at this stage of their lives. Perhaps. But in at least some cases the reason the nerds don't fit in really is that everyone else is crazy. I remember sitting in the audience at a "pep rally" at my high school, watching as the cheerleaders threw an effigy of an opposing player into the audience to be torn to pieces. I felt like an explorer witnessing some bizarre tribal ritual.

If I could go back and give my thirteen year old self some advice, the main thing I'd tell him would be to stick his head up and look around. I didn't really grasp it at the time, but the whole world we lived in was as fake as a Twinkie. Not just school, but the entire town. Why do people move to suburbia? To have kids! So no wonder it seemed boring and sterile. The whole place was a giant nursery, an artificial town created explicitly for the purpose of breeding children.

Where I grew up, it felt as if there was nowhere to go, and nothing to do. This was no accident. Suburbs are deliberately designed to exclude the outside world, because it contains things that could endanger children.

And as for the schools, they were just holding pens within this fake world. Officially the purpose of schools is to teach kids. In fact their primary purpose is to keep kids locked up in one place for a big chunk of the day so adults can get things done. And I have no problem with this: in a specialized industrial society, it would be a disaster to have kids running around loose.

What bothers me is not that the kids are kept in prisons, but that (a) they aren't told about it, and (b) the prisons are run mostly by the inmates. Kids are sent off to spend six years memorizing meaningless facts in a world ruled by a caste of giants who run after an oblong brown ball, as if this were the most natural thing in the world. And if they balk at this surreal cocktail, they're called misfits.

Life in this twisted world is stressful for the kids. And not just for the nerds. Like any war, it's damaging even to the winners.

Adults can't avoid seeing that teenage kids are tormented. So why don't they do something about it? Because they blame it on puberty. The reason kids are so unhappy, adults tell themselves, is that monstrous new chemicals, *hormones*, are now coursing through their bloodstream and messing up everything. There's nothing wrong with the system; it's just inevitable that kids will be miserable at that age.

This idea is so pervasive that even the kids believe it, which probably doesn't help. Someone who thinks his feet naturally hurt is not going to stop to consider the possibility that he is wearing the wrong size shoes.

I'm suspicious of this theory that thirteen-year-old kids are intrinsically messed up. If it's physiological, it should be universal. Are Mongol nomads all nihilists at thirteen? I've read a lot of history, and I have not seen a single reference to this supposedly universal fact before the twentieth century. Teenage apprentices in the Renaissance seem to have been cheerful and eager. They got in fights and played tricks on one another of course (Michelangelo had his nose broken by a bully), but they weren't crazy.

As far as I can tell, the concept of the hormone-crazed teenager is coeval with suburbia. I don't think this is a coincidence. I think teenagers are driven crazy by the life they're made to lead. Teenage apprentices in the Renaissance were working dogs. Teenagers now are neurotic lapdogs. Their craziness is the craziness of the idle everywhere.

When I was in school, suicide was a constant topic among the smarter kids. No one I knew did it, but several planned to, and some may have tried. Mostly this was just a pose. Like other teenagers, we loved the dramatic, and suicide seemed very dramatic. But partly it was because our lives were at times genuinely miserable.

Bullying was only part of the problem. Another problem, and possibly an even worse one, was that we never had anything real to work on. Humans like to work; in most of the world, your work is your identity. And all the work we did was pointless, or seemed so at the time.

At best it was practice for real work we might do far in the future, so far that we didn't even know at the time what we were practicing for. More often it was just an arbitrary series of hoops to jump through, words without content designed mainly for testability. (The three main causes of the Civil War were. . . . Test: List the three main causes of the Civil War.)

And there was no way to opt out. The adults had agreed among themselves that this was to be the route to college. The only way to escape this empty life was to submit to it.

Teenage kids used to have a more active role in society. In pre-industrial times, they were all apprentices of one sort or another, whether in shops or on farms or even on warships. They weren't left to create their own societies. They were junior members of adult societies.

Teenagers seem to have respected adults more then, because the adults were the visible experts in the skills they were trying to learn. Now most kids have little idea what their parents do in their distant offices, and see no connection (indeed, there is precious little) between schoolwork and the work they'll do as adults.

And if teenagers respected adults more, adults also had more use for teenagers. After a couple years' training, an apprentice could be a real help. Even the newest apprentice could be made to carry messages or sweep the workshop.

Now adults have no immediate use for teenagers. They would be in the way in an office. So they drop them off at school on their way to work, much as they might drop the dog off at a kennel if they were going away for the weekend.

What happened? We're up against a hard one here. The cause of this problem is the same as the cause of so many present ills: specialization. As jobs become more specialized, we have to train longer for them. Kids in pre-industrial times started working at about 14 at the latest; kids on farms, where most people lived, began far earlier. Now kids who go to college don't start working full-time till 21 or 22. With some degrees, like MDs and PhDs, you may not finish your training till 30.

Teenagers now are useless, except as cheap labor in industries like fast food, which evolved to exploit precisely this fact. In almost any other kind of work, they'd be a net loss. But they're also too young to be left unsupervised. Someone has to watch over them, and the most efficient way to do this is to collect them

together in one place. Then a few adults can watch all of them.

If you stop there, what you're describing is literally a prison, albeit a part-time one. The problem is, many schools practically do stop there. The stated purpose of schools is to educate the kids. But there is no external pressure to do this well. And so most schools do such a bad job of teaching that the kids don't really take it seriously— not even the smart kids. Much of the time we were all, students and teachers both, just going through the motions.

In my high school French class we were supposed to read Hugo's *Les Miserables*. I don't think any of us knew French well enough to make our way through this enormous book. Like the rest of the class, I just skimmed the Cliff's Notes. When we were given a test on the book, I noticed that the questions sounded odd. They were full of long words that our teacher wouldn't have used. Where had these questions come from? From the Cliff's Notes, it turned out. The teacher was using them too. We were all just pretending.

There are certainly great public school teachers. The energy and imagination of my fourth grade teacher, Mr. Mihalko, made that year something his students still talk about, thirty years later. But teachers like him were individuals swimming upstream. They couldn't fix the system.

In almost any group of people you'll find hierarchy. When groups of adults form in the real world, it's generally for some common purpose, and the leaders end up being those who are best at it. The problem with most schools is, they have no purpose. But hierarchy there must be. And so the kids make one out of nothing.

We have a phrase to describe what happens when rankings have to be created without any meaningful criteria. We say that the situation *degenerates into a popularity contest*. And that's exactly what happens in most American schools. Instead of depending on some real test, one's rank depends mostly on one's ability to increase one's rank. It's like the court of Louis XIV. There is no external opponent, so the kids become one another's opponents.

When there is some real external test of skill, it isn't painful to be at the bottom of the hierarchy. A rookie on a football team doesn't resent the skill of the veteran; he hopes to be like him one day and is happy to have the chance to learn from him. The veteran may in turn feel a sense of *noblesse oblige*. And most importantly, their status depends on how well they do against opponents, not on whether they can push the other down.

Court hierarchies are another thing entirely. This type of society debases anyone who enters it. There is neither admiration at the bottom, nor *noblesse oblige* at the top. It's kill or be killed.

This is the sort of society that gets created in American secondary schools. And it happens because these schools have no real purpose beyond keeping the kids all in one place for a certain number of hours each day. What I didn't realize at the time, and in fact didn't realize till very recently, is that the twin horrors of school life, the cruelty and the boredom, both have the same cause.

The mediocrity of American public schools has worse consequences than just making kids unhappy for six years. It breeds a rebelliousness that actively drives kids away from the things they're supposed to be learning.

Like many nerds, probably, it was years after high school before I could bring myself to read anything we'd been assigned then. And I lost more than books. I mistrusted words like "character" and "integrity" because they had been so debased by adults. As they were used then, these words all seemed to mean the same thing: obedience. The kids who got praised for these qualities tended to be at best dull-witted prize bulls, and at worst facile schmoozers. If that was what character and integrity were, I wanted no part of them.

The word I most misunderstood was "tact." As used by adults, it seemed to mean keeping your mouth shut. I assumed it was derived from the same root as "tacit" and "taciturn," and that it literally meant being quiet. I vowed that I would never be tactful; they were never going to shut me up. In fact, it's derived from the same root as "tactile," and what it means is to have a deft touch. Tactful is the opposite of clumsy. I don't think I learned this until college.

Nerds aren't the only losers in the popularity rat race. Nerds are unpopular because they're distracted. There are other kids who deliberately opt out because they're so disgusted with the whole process.

Teenage kids, even rebels, don't like to be alone, so when kids opt out of the system, they tend to do it as a group. At the schools I went to, the focus of rebellion was drug use, specifically marijuana. The kids in this tribe wore black concert t-shirts and were called "freaks."

Freaks and nerds were allies, and there was a good deal of overlap between them. Freaks were on the whole smarter than other kids, though never studying (or at least never appearing to) was an important tribal value. I was more in the nerd camp, but I was friends with a lot of freaks.

They used drugs, at least at first, for the social bonds they created. It was something to do together, and because the drugs were illegal, it was a shared badge of rebellion.

I'm not claiming that bad schools are the whole reason kids get into trouble with drugs. After a while, drugs have their own momentum. No doubt some of the

freaks ultimately used drugs to escape from other problems—trouble at home, for example. But, in my school at least, the reason most kids *started* using drugs was rebellion. Fourteen-year-olds didn't start smoking pot because they'd heard it would help them forget their problems. They started because they wanted to join a different tribe.

Misrule breeds rebellion; this is not a new idea. And yet the authorities still for the most part act as if drugs were themselves the cause of the problem.

The real problem is the emptiness of school life. We won't see solutions till adults realize that. The adults who may realize it first are the ones who were themselves nerds in school. Do you want your kids to be as unhappy in eighth grade as you were? I wouldn't. Well, then, is there anything we can do to fix things? Almost certainly. There is nothing inevitable about the current system. It has come about mostly by default.

Adults, though, are busy. Showing up for school plays is one thing. Taking on the educational bureaucracy is another. Perhaps a few will have the energy to try to change things. I suspect the hardest part is realizing that you can.

Nerds still in school should not hold their breath. Maybe one day a heavily armed force of adults will show up in helicopters to rescue you, but they probably won't be coming this month. Any immediate improvement in nerds' lives is probably going to have to come from the nerds themselves.

Merely understanding the situation they're in should make it less painful. Nerds aren't losers. They're just playing a different game, and a game much closer to the one played in the real world. Adults know this. It's hard to find successful adults now who don't claim to have been nerds in high school.

It's important for nerds to realize, too, that school is not life. School is a strange, artificial thing, half sterile and half feral. It's all-encompassing, like life, but it isn't the real thing. It's only temporary, and if you look, you can see beyond it even while you're still in it.

If life seems awful to kids, it's neither because hormones are turning you all into monsters (as your parents believe), nor because life actually is awful (as you believe). It's because the adults, who no longer have any economic use for you, have abandoned you to spend years cooped up together with nothing real to do. *Any* society of that type is awful to live in. You don't have to look any further to explain why teenage kids are unhappy.

I've said some harsh things in this essay, but really the thesis is an optimistic one—that several problems we take for granted are in fact not insoluble after all. Teenage kids are not inherently unhappy monsters. That should be encouraging

news to kids and adults both.

Thanks to Sarah Harlin, Trevor Blackwell, Robert Morris, Eric Raymond, and Jackie Weicker for reading drafts of this essay, and Maria Daniels for scanning photos.

— | | Re: Why Nerds are Unpopular

Hackers and Painters

May 2003

(This essay is derived from a guest lecture at Harvard, which incorporated an earlier talk at Northeastern.)

When I finished grad school in computer science I went to art school to study painting. A lot of people seemed surprised that someone interested in computers would also be interested in painting. They seemed to think that hacking and painting were very different kinds of work— that hacking was cold, precise, and methodical, and that painting was the frenzied expression of some primal urge.

Both of these images are wrong. Hacking and painting have a lot in common. In fact, of all the different types of people I've known, hackers and painters are among the most alike.

What hackers and painters have in common is that they're both makers. Along with composers, architects, and writers, what hackers and painters are trying to do is make good things. They're not doing research per se, though if in the course of trying to make good things they discover some new technique, so much the better.

I've never liked the term "computer science." The main reason I don't like it is that there's no such thing. Computer science is a grab bag of tenuously related areas thrown together by an accident of history, like Yugoslavia. At one end you have people who are really mathematicians, but call what they're doing computer science so they can get DARPA grants. In the middle you have people working on something like the natural history of computers— studying the behavior of algorithms for routing data through networks, for example. And then at the other extreme you have the hackers, who are trying to write interesting software, and for whom computers are just a medium of expression, as concrete is for architects or paint for painters. It's as if mathematicians, physicists, and architects all had to be in the same department.

Sometimes what the hackers do is called “software engineering,” but this term is just as misleading. Good software designers are no more engineers than architects are. The border between architecture and engineering is not sharply defined, but it’s there. It falls between what and how: architects decide what to do, and engineers figure out how to do it.

What and how should not be kept too separate. You’re asking for trouble if you try to decide what to do without understanding how to do it. But hacking can certainly be more than just deciding how to implement some spec. At its best, it’s creating the spec— though it turns out the best way to do that is to implement it.

Perhaps one day “computer science” will, like Yugoslavia, get broken up into its component parts. That might be a good thing. Especially if it meant independence for my native land, hacking.

Bundling all these different types of work together in one department may be convenient administratively, but it’s confusing intellectually. That’s the other reason I don’t like the name “computer science.” Arguably the people in the middle are doing something like an experimental science. But the people at either end, the hackers and the mathematicians, are not actually doing science.

The mathematicians don’t seem bothered by this. They happily set to work proving theorems like the other mathematicians over in the math department, and probably soon stop noticing that the building they work in says “computer science” on the outside. But for the hackers this label is a problem. If what they’re doing is called science, it makes them feel they ought to be acting scientific. So instead of doing what they really want to do, which is to design beautiful software, hackers in universities and research labs feel they ought to be writing research papers.

In the best case, the papers are just a formality. Hackers write cool software, and then write a paper about it, and the paper becomes a proxy for the achievement represented by the software. But often this mismatch causes problems. It’s easy to drift away from building beautiful things toward building ugly things that make more suitable subjects for research papers.

Unfortunately, beautiful things don’t always make the best subjects for papers. Number one, research must be original— and as anyone who has written a PhD dissertation knows, the way to be sure that you’re exploring virgin territory is to stake out a piece of ground that no one wants. Number two, research must be substantial— and awkward systems yield meatier papers, because you can write about the obstacles you have to overcome in order to get things done. Nothing yields meaty problems like starting with the wrong assumptions. Most of AI is

an example of this rule; if you assume that knowledge can be represented as a list of predicate logic expressions whose arguments represent abstract concepts, you'll have a lot of papers to write about how to make this work. As Ricky Ricardo used to say, "Lucy, you got a lot of explaining to do."

The way to create something beautiful is often to make subtle tweaks to something that already exists, or to combine existing ideas in a slightly new way. This kind of work is hard to convey in a research paper.

So why do universities and research labs continue to judge hackers by publications? For the same reason that "scholastic aptitude" gets measured by simple-minded standardized tests, or the productivity of programmers gets measured in lines of code. These tests are easy to apply, and there is nothing so tempting as an easy test that kind of works.

Measuring what hackers are actually trying to do, designing beautiful software, would be much more difficult. You need a good sense of design to judge good design. And there is no correlation, except possibly a negative one, between people's ability to recognize good design and their confidence that they can.

The only external test is time. Over time, beautiful things tend to thrive, and ugly things tend to get discarded. Unfortunately, the amounts of time involved can be longer than human lifetimes. Samuel Johnson said it took a hundred years for a writer's reputation to converge. You have to wait for the writer's influential friends to die, and then for all their followers to die.

I think hackers just have to resign themselves to having a large random component in their reputations. In this they are no different from other makers. In fact, they're lucky by comparison. The influence of fashion is not nearly so great in hacking as it is in painting.

There are worse things than having people misunderstand your work. A worse danger is that you will yourself misunderstand your work. Related fields are where you go looking for ideas. If you find yourself in the computer science department, there is a natural temptation to believe, for example, that hacking is the applied version of what theoretical computer science is the theory of. All the time I was in graduate school I had an uncomfortable feeling in the back of my mind that I ought to know more theory, and that it was very remiss of me to have forgotten all that stuff within three weeks of the final exam.

Now I realize I was mistaken. Hackers need to understand the theory of computation about as much as painters need to understand paint chemistry. You need to know how to calculate time and space complexity and about Turing completeness. You might also want to remember at least the concept of a state machine, in

case you have to write a parser or a regular expression library. Painters in fact have to remember a good deal more about paint chemistry than that.

I've found that the best sources of ideas are not the other fields that have the word "computer" in their names, but the other fields inhabited by makers. Painting has been a much richer source of ideas than the theory of computation.

For example, I was taught in college that one ought to figure out a program completely on paper before even going near a computer. I found that I did not program this way. I found that I liked to program sitting in front of a computer, not a piece of paper. Worse still, instead of patiently writing out a complete program and assuring myself it was correct, I tended to just spew out code that was hopelessly broken, and gradually beat it into shape. Debugging, I was taught, was a kind of final pass where you caught typos and oversights. The way I worked, it seemed like programming consisted of debugging.

For a long time I felt bad about this, just as I once felt bad that I didn't hold my pencil the way they taught me to in elementary school. If I had only looked over at the other makers, the painters or the architects, I would have realized that there was a name for what I was doing: sketching. As far as I can tell, the way they taught me to program in college was all wrong. You should figure out programs as you're writing them, just as writers and painters and architects do.

Realizing this has real implications for software design. It means that a programming language should, above all, be malleable. A programming language is for thinking of programs, not for expressing programs you've already thought of. It should be a pencil, not a pen. Static typing would be a fine idea if people actually did write programs the way they taught me to in college. But that's not how any of the hackers I know write programs. We need a language that lets us scribble and smudge and smear, not a language where you have to sit with a teacup of types balanced on your knee and make polite conversation with a strict old aunt of a compiler.

While we're on the subject of static typing, identifying with the makers will save us from another problem that afflicts the sciences: math envy. Everyone in the sciences secretly believes that mathematicians are smarter than they are. I think mathematicians also believe this. At any rate, the result is that scientists tend to make their work look as mathematical as possible. In a field like physics this probably doesn't do much harm, but the further you get from the natural sciences, the more of a problem it becomes.

A page of formulas just looks so impressive. (Tip: for extra impressiveness, use Greek variables.) And so there is a great temptation to work on problems you can treat formally, rather than problems that are, say, important.

If hackers identified with other makers, like writers and painters, they wouldn't feel tempted to do this. Writers and painters don't suffer from math envy. They feel as if they're doing something completely unrelated. So are hackers, I think.

If universities and research labs keep hackers from doing the kind of work they want to do, perhaps the place for them is in companies. Unfortunately, most companies won't let hackers do what they want either. Universities and research labs force hackers to be scientists, and companies force them to be engineers.

I only discovered this myself quite recently. When Yahoo bought Viaweb, they asked me what I wanted to do. I had never liked the business side very much, and said that I just wanted to hack. When I got to Yahoo, I found that what hacking meant to them was implementing software, not designing it. Programmers were seen as technicians who translated the visions (if that is the word) of product managers into code.

This seems to be the default plan in big companies. They do it because it decreases the standard deviation of the outcome. Only a small percentage of hackers can actually design software, and it's hard for the people running a company to pick these out. So instead of entrusting the future of the software to one brilliant hacker, most companies set things up so that it is designed by committee, and the hackers merely implement the design.

If you want to make money at some point, remember this, because this is one of the reasons startups win. Big companies want to decrease the standard deviation of design outcomes because they want to avoid disasters. But when you damp oscillations, you lose the high points as well as the low. This is not a problem for big companies, because they don't win by making great products. Big companies win by sucking less than other big companies.

So if you can figure out a way to get in a design war with a company big enough that its software is designed by product managers, they'll never be able to keep up with you. These opportunities are not easy to find, though. It's hard to engage a big company in a design war, just as it's hard to engage an opponent inside a castle in hand to hand combat. It would be pretty easy to write a better word processor than Microsoft Word, for example, but Microsoft, within the castle of their operating system monopoly, probably wouldn't even notice if you did.

The place to fight design wars is in new markets, where no one has yet managed to establish any fortifications. That's where you can win big by taking the bold approach to design, and having the same people both design and implement the product. Microsoft themselves did this at the start. So did Apple. And Hewlett-Packard. I suspect almost every successful startup has.

So one way to build great software is to start your own startup. There are two problems with this, though. One is that in a startup you have to do so much besides write software. At Viaweb I considered myself lucky if I got to hack a quarter of the time. And the things I had to do the other three quarters of the time ranged from tedious to terrifying. I have a benchmark for this, because I once had to leave a board meeting to have some cavities filled. I remember sitting back in the dentist's chair, waiting for the drill, and feeling like I was on vacation.

The other problem with startups is that there is not much overlap between the kind of software that makes money and the kind that's interesting to write. Programming languages are interesting to write, and Microsoft's first product was one, in fact, but no one will pay for programming languages now. If you want to make money, you tend to be forced to work on problems that are too nasty for anyone to solve for free.

All makers face this problem. Prices are determined by supply and demand, and there is just not as much demand for things that are fun to work on as there is for things that solve the mundane problems of individual customers. Acting in off-Broadway plays just doesn't pay as well as wearing a gorilla suit in someone's booth at a trade show. Writing novels doesn't pay as well as writing ad copy for garbage disposals. And hacking programming languages doesn't pay as well as figuring out how to connect some company's legacy database to their Web server.

I think the answer to this problem, in the case of software, is a concept known to nearly all makers: the day job. This phrase began with musicians, who perform at night. More generally, it means that you have one kind of work you do for money, and another for love.

Nearly all makers have day jobs early in their careers. Painters and writers notoriously do. If you're lucky you can get a day job that's closely related to your real work. Musicians often seem to work in record stores. A hacker working on some programming language or operating system might likewise be able to get a day job using it. [1]

When I say that the answer is for hackers to have day jobs, and work on beautiful software on the side, I'm not proposing this as a new idea. This is what open-source hacking is all about. What I'm saying is that open-source is probably the right model, because it has been independently confirmed by all the other makers.

It seems surprising to me that any employer would be reluctant to let hackers work on open-source projects. At Viaweb, we would have been reluctant to hire anyone who didn't. When we interviewed programmers, the main thing we cared

about was what kind of software they wrote in their spare time. You can't do anything really well unless you love it, and if you love to hack you'll inevitably be working on projects of your own. [2]

Because hackers are makers rather than scientists, the right place to look for metaphors is not in the sciences, but among other kinds of makers. What else can painting teach us about hacking?

One thing we can learn, or at least confirm, from the example of painting is how to learn to hack. You learn to paint mostly by doing it. Ditto for hacking. Most hackers don't learn to hack by taking college courses in programming. They learn to hack by writing programs of their own at age thirteen. Even in college classes, you learn to hack mostly by hacking. [3]

Because painters leave a trail of work behind them, you can watch them learn by doing. If you look at the work of a painter in chronological order, you'll find that each painting builds on things that have been learned in previous ones. When there's something in a painting that works very well, you can usually find version 1 of it in a smaller form in some earlier painting.

I think most makers work this way. Writers and architects seem to as well. Maybe it would be good for hackers to act more like painters, and regularly start over from scratch, instead of continuing to work for years on one project, and trying to incorporate all their later ideas as revisions.

The fact that hackers learn to hack by doing it is another sign of how different hacking is from the sciences. Scientists don't learn science by doing it, but by doing labs and problem sets. Scientists start out doing work that's perfect, in the sense that they're just trying to reproduce work someone else has already done for them. Eventually, they get to the point where they can do original work. Whereas hackers, from the start, are doing original work; it's just very bad. So hackers start original, and get good, and scientists start good, and get original.

The other way makers learn is from examples. For a painter, a museum is a reference library of techniques. For hundreds of years it has been part of the traditional education of painters to copy the works of the great masters, because copying forces you to look closely at the way a painting is made.

Writers do this too. Benjamin Franklin learned to write by summarizing the points in the essays of Addison and Steele and then trying to reproduce them. Raymond Chandler did the same thing with detective stories.

Hackers, likewise, can learn to program by looking at good programs— not just at what they do, but the source code too. One of the less publicized benefits of the open-source movement is that it has made it easier to learn to program. When I

learned to program, we had to rely mostly on examples in books. The one big chunk of code available then was Unix, but even this was not open source. Most of the people who read the source read it in illicit photocopies of John Lions' book, which though written in 1977 was not allowed to be published until 1996.

Another example we can take from painting is the way that paintings are created by gradual refinement. Paintings usually begin with a sketch. Gradually the details get filled in. But it is not merely a process of filling in. Sometimes the original plans turn out to be mistaken. Countless paintings, when you look at them in xrays, turn out to have limbs that have been moved or facial features that have been readjusted.

Here's a case where we can learn from painting. I think hacking should work this way too. It's unrealistic to expect that the specifications for a program will be perfect. You're better off if you admit this up front, and write programs in a way that allows specifications to change on the fly.

(The structure of large companies makes this hard for them to do, so here is another place where startups have an advantage.)

Everyone by now presumably knows about the danger of premature optimization. I think we should be just as worried about premature design—deciding too early what a program should do.

The right tools can help us avoid this danger. A good programming language should, like oil paint, make it easy to change your mind. Dynamic typing is a win here because you don't have to commit to specific data representations up front. But the key to flexibility, I think, is to make the language very abstract. The easiest program to change is one that's very short.

This sounds like a paradox, but a great painting has to be better than it has to be. For example, when Leonardo painted the portrait of Ginevra de Benci in the National Gallery, he put a juniper bush behind her head. In it he carefully painted each individual leaf. Many painters might have thought, this is just something to put in the background to frame her head. No one will look that closely at it.

Not Leonardo. How hard he worked on part of a painting didn't depend at all on how closely he expected anyone to look at it. He was like Michael Jordan. Relentless.

Relentlessness wins because, in the aggregate, unseen details become visible. When people walk by the portrait of Ginevra de Benci, their attention is often immediately arrested by it, even before they look at the label and notice that it says Leonardo da Vinci. All those unseen details combine to produce something

that's just stunning, like a thousand barely audible voices all singing in tune.

Great software, likewise, requires a fanatical devotion to beauty. If you look inside good software, you find that parts no one is ever supposed to see are beautiful too. I'm not claiming I write great software, but I know that when it comes to code I behave in a way that would make me eligible for prescription drugs if I approached everyday life the same way. It drives me crazy to see code that's badly indented, or that uses ugly variable names.

If a hacker were a mere implementor, turning a spec into code, then he could just work his way through it from one end to the other like someone digging a ditch. But if the hacker is a creator, we have to take inspiration into account.

In hacking, like painting, work comes in cycles. Sometimes you get excited about some new project and you want to work sixteen hours a day on it. Other times nothing seems interesting.

To do good work you have to take these cycles into account, because they're affected by how you react to them. When you're driving a car with a manual transmission on a hill, you have to back off the clutch sometimes to avoid stalling. Backing off can likewise prevent ambition from stalling. In both painting and hacking there are some tasks that are terrifyingly ambitious, and others that are comfortably routine. It's a good idea to save some easy tasks for moments when you would otherwise stall.

In hacking, this can literally mean saving up bugs. I like debugging: it's the one time that hacking is as straightforward as people think it is. You have a totally constrained problem, and all you have to do is solve it. Your program is supposed to do x. Instead it does y. Where does it go wrong? You know you're going to win in the end. It's as relaxing as painting a wall.

The example of painting can teach us not only how to manage our own work, but how to work together. A lot of the great art of the past is the work of multiple hands, though there may only be one name on the wall next to it in the museum. Leonardo was an apprentice in the workshop of Verrocchio and painted one of the angels in his Baptism of Christ. This sort of thing was the rule, not the exception. Michelangelo was considered especially dedicated for insisting on painting all the figures on the ceiling of the Sistine Chapel himself.

As far as I know, when painters worked together on a painting, they never worked on the same parts. It was common for the master to paint the principal figures and for assistants to paint the others and the background. But you never had one guy painting over the work of another.

I think this is the right model for collaboration in software too. Don't push it

too far. When a piece of code is being hacked by three or four different people, no one of whom really owns it, it will end up being like a common-room. It will tend to feel bleak and abandoned, and accumulate cruft. The right way to collaborate, I think, is to divide projects into sharply defined modules, each with a definite owner, and with interfaces between them that are as carefully designed and, if possible, as articulated as programming languages.

Like painting, most software is intended for a human audience. And so hackers, like painters, must have empathy to do really great work. You have to be able to see things from the user's point of view.

When I was a kid I was always being told to look at things from someone else's point of view. What this always meant in practice was to do what someone else wanted, instead of what I wanted. This of course gave empathy a bad name, and I made a point of not cultivating it.

Boy, was I wrong. It turns out that looking at things from other people's point of view is practically the secret of success. It doesn't necessarily mean being self-sacrificing. Far from it. Understanding how someone else sees things doesn't imply that you'll act in his interest; in some situations— in war, for example— you want to do exactly the opposite. [4]

Most makers make things for a human audience. And to engage an audience you have to understand what they need. Nearly all the greatest paintings are paintings of people, for example, because people are what people are interested in.

Empathy is probably the single most important difference between a good hacker and a great one. Some hackers are quite smart, but when it comes to empathy are practically solipsists. It's hard for such people to design great software [5], because they can't see things from the user's point of view.

One way to tell how good people are at empathy is to watch them explain a technical question to someone without a technical background. We probably all know people who, though otherwise smart, are just comically bad at this. If someone asks them at a dinner party what a programming language is, they'll say something like "Oh, a high-level language is what the compiler uses as input to generate object code." High-level language? Compiler? Object code? Someone who doesn't know what a programming language is obviously doesn't know what these things are, either.

Part of what software has to do is explain itself. So to write good software you have to understand how little users understand. They're going to walk up to the software with no preparation, and it had better do what they guess it will, because they're not going to read the manual. The best system I've ever seen in

this respect was the original Macintosh, in 1985. It did what software almost never does: it just worked. [6]

Source code, too, should explain itself. If I could get people to remember just one quote about programming, it would be the one at the beginning of *Structure and Interpretation of Computer Programs*.

Programs should be written for people to read, and only incidentally for machines to execute.

You need to have empathy not just for your users, but for your readers. It's in your interest, because you'll be one of them. Many a hacker has written a program only to find on returning to it six months later that he has no idea how it works. I know several people who've sworn off Perl after such experiences. [7]

Lack of empathy is associated with intelligence, to the point that there is even something of a fashion for it in some places. But I don't think there's any correlation. You can do well in math and the natural sciences without having to learn empathy, and people in these fields tend to be smart, so the two qualities have come to be associated. But there are plenty of dumb people who are bad at empathy too. Just listen to the people who call in with questions on talk shows. They ask whatever it is they're asking in such a roundabout way that the hosts often have to rephrase the question for them.

So, if hacking works like painting and writing, is it as cool? After all, you only get one life. You might as well spend it working on something great.

Unfortunately, the question is hard to answer. There is always a big time lag in prestige. It's like light from a distant star. Painting has prestige now because of great work people did five hundred years ago. At the time, no one thought these paintings were as important as we do today. It would have seemed very odd to people at the time that Federico da Montefeltro, the Duke of Urbino, would one day be known mostly as the guy with the strange nose in a painting by Piero della Francesca.

So while I admit that hacking doesn't seem as cool as painting now, we should remember that painting itself didn't seem as cool in its glory days as it does now.

What we can say with some confidence is that these are the glory days of hacking. In most fields the great work is done early on. The paintings made between 1430 and 1500 are still unsurpassed. Shakespeare appeared just as professional theater was being born, and pushed the medium so far that every playwright since has had to live in his shadow. Albrecht Durer did the same thing with engraving, and Jane Austen with the novel.

Over and over we see the same pattern. A new medium appears, and people are

so excited about it that they explore most of its possibilities in the first couple generations. Hacking seems to be in this phase now.

Painting was not, in Leonardo's time, as cool as his work helped make it. How cool hacking turns out to be will depend on what we can do with this new medium.

Notes

[1] The greatest damage that photography has done to painting may be the fact that it killed the best day job. Most of the great painters in history supported themselves by painting portraits.

[2] I've been told that Microsoft discourages employees from contributing to open-source projects, even in their spare time. But so many of the best hackers work on open-source projects now that the main effect of this policy may be to ensure that they won't be able to hire any first-rate programmers.

[3] What you learn about programming in college is much like what you learn about books or clothes or dating: what bad taste you had in high school.

[4] Here's an example of applied empathy. At Viaweb, if we couldn't decide between two alternatives, we'd ask, what would our competitors hate most? At one point a competitor added a feature to their software that was basically useless, but since it was one of few they had that we didn't, they made much of it in the trade press. We could have tried to explain that the feature was useless, but we decided it would annoy our competitor more if we just implemented it ourselves, so we hacked together our own version that afternoon.

[5] Except text editors and compilers. Hackers don't need empathy to design these, because they are themselves typical users.

[6] Well, almost. They overshot the available RAM somewhat, causing much inconvenient disk swapping, but this could be fixed within a few months by buying an additional disk drive.

[7] The way to make programs easy to read is not to stuff them with comments. I would take Abelson and Sussman's quote a step further. Programming languages should be designed to express algorithms, and only incidentally to tell computers how to execute them. A good programming language ought to be better for explaining software than English. You should only need comments when there is some kind of kludge you need to warn readers about, just as on a road there are only arrows on parts with unexpectedly sharp curves.

Thanks to Trevor Blackwell, Robert Morris, Dan Giffin, and Lisa Randall for reading drafts of this, and to Henry Leitner and Larry Finkelstein for inviting

me to speak.

What You Can't Say

January 2004

Have you ever seen an old photo of yourself and been embarrassed at the way you looked? *Did we actually dress like that?* We did. And we had no idea how silly we looked. It's the nature of fashion to be invisible, in the same way the movement of the earth is invisible to all of us riding on it.

What scares me is that there are moral fashions too. They're just as arbitrary, and just as invisible to most people. But they're much more dangerous. Fashion is mistaken for good design; moral fashion is mistaken for good. Dressing oddly gets you laughed at. Violating moral fashions can get you fired, ostracized, imprisoned, or even killed.

If you could travel back in a time machine, one thing would be true no matter where you went: you'd have to watch what you said. Opinions we consider harmless could have gotten you in big trouble. I've already said at least one thing that would have gotten me in big trouble in most of Europe in the seventeenth century, and did get Galileo in big trouble when he said it — that the earth moves. [1]

It seems to be a constant throughout history: In every period, people believed things that were just ridiculous, and believed them so strongly that you would have gotten in terrible trouble for saying otherwise.

Is our time any different? To anyone who has read any amount of history, the answer is almost certainly no. It would be a remarkable coincidence if ours were the first era to get everything just right.

It's tantalizing to think we believe things that people in the future will find ridiculous. What *would* someone coming back to visit us in a time machine have to be careful not to say? That's what I want to study here. But I want to do more than just shock everyone with the heresy du jour. I want to find general recipes for discovering what you can't say, in any era.

The Conformist Test

Let's start with a test: Do you have any opinions that you would be reluctant to express in front of a group of your peers?

If the answer is no, you might want to stop and think about that. If everything you believe is something you're supposed to believe, could that possibly be a coincidence? Odds are it isn't. Odds are you just think what you're told.

The other alternative would be that you independently considered every question and came up with the exact same answers that are now considered acceptable. That seems unlikely, because you'd also have to make the same mistakes. Map-makers deliberately put slight mistakes in their maps so they can tell when someone copies them. If another map has the same mistake, that's very convincing evidence.

Like every other era in history, our moral map almost certainly contains a few mistakes. And anyone who makes the same mistakes probably didn't do it by accident. It would be like someone claiming they had independently decided in 1972 that bell-bottom jeans were a good idea.

If you believe everything you're supposed to now, how can you be sure you wouldn't also have believed everything you were supposed to if you had grown up among the plantation owners of the pre-Civil War South, or in Germany in the 1930s — or among the Mongols in 1200, for that matter? Odds are you would have.

Back in the era of terms like “well-adjusted,” the idea seemed to be that there was something wrong with you if you thought things you didn't dare say out loud. This seems backward. Almost certainly, there is something wrong with you if you *don't* think things you don't dare say out loud.

Trouble

What can't we say? One way to find these ideas is simply to look at things people do say, and get in trouble for. [2]

Of course, we're not just looking for things we can't say. We're looking for things we can't say that are true, or at least have enough chance of being true that the question should remain open. But many of the things people get in trouble for saying probably do make it over this second, lower threshold. No one gets in trouble for saying that $2 + 2$ is 5, or that people in Pittsburgh are ten feet tall. Such obviously false statements might be treated as jokes, or at worst as evidence of insanity, but they are not likely to make anyone mad. The statements that make people mad are the ones they worry might be believed. I suspect the statements that make people maddest are those they worry might be true.

If Galileo had said that people in Padua were ten feet tall, he would have been regarded as a harmless eccentric. Saying the earth orbited the sun was another matter. The church knew this would set people thinking.

Certainly, as we look back on the past, this rule of thumb works well. A lot of the statements people got in trouble for seem harmless now. So it's likely that visitors from the future would agree with at least some of the statements that get people in trouble today. Do we have no Galileos? Not likely.

To find them, keep track of opinions that get people in trouble, and start asking, could this be true? Ok, it may be heretical (or whatever modern equivalent), but might it also be true?

Heresy

This won't get us all the answers, though. What if no one happens to have gotten in trouble for a particular idea yet? What if some idea would be so radioactively controversial that no one would dare express it in public? How can we find these too?

Another approach is to follow that word, heresy. In every period of history, there seem to have been labels that got applied to statements to shoot them down before anyone had a chance to ask if they were true or not. "Blasphemy", "sacrilege", and "heresy" were such labels for a good part of western history, as in more recent times "indecent", "improper", and "unamerican" have been. By now these labels have lost their sting. They always do. By now they're mostly used ironically. But in their time, they had real force.

The word "defeatist", for example, has no particular political connotations now. But in Germany in 1917 it was a weapon, used by Ludendorff in a purge of those who favored a negotiated peace. At the start of World War II it was used extensively by Churchill and his supporters to silence their opponents. In 1940, any argument against Churchill's aggressive policy was "defeatist". Was it right or wrong? Ideally, no one got far enough to ask that.

We have such labels today, of course, quite a lot of them, from the all-purpose "inappropriate" to the dreaded "divisive." In any period, it should be easy to figure out what such labels are, simply by looking at what people call ideas they disagree with besides untrue. When a politician says his opponent is mistaken, that's a straightforward criticism, but when he attacks a statement as "divisive" or "racially insensitive" instead of arguing that it's false, we should start paying attention.

So another way to figure out which of our taboos future generations will laugh at is to start with the labels. Take a label — "sexist", for example — and try to

think of some ideas that would be called that. Then for each ask, might this be true?

Just start listing ideas at random? Yes, because they won't really be random. The ideas that come to mind first will be the most plausible ones. They'll be things you've already noticed but didn't let yourself think.

In 1989 some clever researchers tracked the eye movements of radiologists as they scanned chest images for signs of lung cancer. [3] They found that even when the radiologists missed a cancerous lesion, their eyes had usually paused at the site of it. Part of their brain knew there was something there; it just didn't percolate all the way up into conscious knowledge. I think many interesting heretical thoughts are already mostly formed in our minds. If we turn off our self-censorship temporarily, those will be the first to emerge.

Time and Space

If we could look into the future it would be obvious which of our taboos they'd laugh at. We can't do that, but we can do something almost as good: we can look into the past. Another way to figure out what we're getting wrong is to look at what used to be acceptable and is now unthinkable.

Changes between the past and the present sometimes do represent progress. In a field like physics, if we disagree with past generations it's because we're right and they're wrong. But this becomes rapidly less true as you move away from the certainty of the hard sciences. By the time you get to social questions, many changes are just fashion. The age of consent fluctuates like hemlines.

We may imagine that we are a great deal smarter and more virtuous than past generations, but the more history you read, the less likely this seems. People in past times were much like us. Not heroes, not barbarians. Whatever their ideas were, they were ideas reasonable people could believe.

So here is another source of interesting heresies. Diff present ideas against those of various past cultures, and see what you get. [4] Some will be shocking by present standards. Ok, fine; but which might also be true?

You don't have to look into the past to find big differences. In our own time, different societies have wildly varying ideas of what's ok and what isn't. So you can try diffing other cultures' ideas against ours as well. (The best way to do that is to visit them.) Any idea that's considered harmless in a significant percentage of times and places, and yet is taboo in ours, is a candidate for something we're mistaken about.

For example, at the high water mark of political correctness in the early 1990s, Harvard distributed to its faculty and staff a brochure saying, among other things,

that it was inappropriate to compliment a colleague or student's clothes. No more "nice shirt." I think this principle is rare among the world's cultures, past or present. There are probably more where it's considered especially polite to compliment someone's clothing than where it's considered improper. Odds are this is, in a mild form, an example of one of the taboos a visitor from the future would have to be careful to avoid if he happened to set his time machine for Cambridge, Massachusetts, 1992. [5]

Prigs

Of course, if they have time machines in the future they'll probably have a separate reference manual just for Cambridge. This has always been a fussy place, a town of i dotters and t crossers, where you're liable to get both your grammar and your ideas corrected in the same conversation. And that suggests another way to find taboos. Look for prigs, and see what's inside their heads.

Kids' heads are repositories of all our taboos. It seems fitting to us that kids' ideas should be bright and clean. The picture we give them of the world is not merely simplified, to suit their developing minds, but sanitized as well, to suit our ideas of what kids ought to think. [6]

You can see this on a small scale in the matter of dirty words. A lot of my friends are starting to have children now, and they're all trying not to use words like "fuck" and "shit" within baby's hearing, lest baby start using these words too. But these words are part of the language, and adults use them all the time. So parents are giving their kids an inaccurate idea of the language by not using them. Why do they do this? Because they don't think it's fitting that kids should use the whole language. We like children to seem innocent. [7]

Most adults, likewise, deliberately give kids a misleading view of the world. One of the most obvious examples is Santa Claus. We think it's cute for little kids to believe in Santa Claus. I myself think it's cute for little kids to believe in Santa Claus. But one wonders, do we tell them this stuff for their sake, or for ours?

I'm not arguing for or against this idea here. It is probably inevitable that parents should want to dress up their kids' minds in cute little baby outfits. I'll probably do it myself. The important thing for our purposes is that, as a result, a well brought-up teenage kid's brain is a more or less complete collection of all our taboos — and in mint condition, because they're untainted by experience. Whatever we think that will later turn out to be ridiculous, it's almost certainly inside that head.

How do we get at these ideas? By the following thought experiment. Imagine a kind of latter-day Conrad character who has worked for a time as a mercenary in Africa, for a time as a doctor in Nepal, for a time as the manager of a nightclub

in Miami. The specifics don't matter — just someone who has seen a lot. Now imagine comparing what's inside this guy's head with what's inside the head of a well-behaved sixteen year old girl from the suburbs. What does he think that would shock her? He knows the world; she knows, or at least embodies, present taboos. Subtract one from the other, and the result is what we can't say.

Mechanism

I can think of one more way to figure out what we can't say: to look at how taboos are created. How do moral fashions arise, and why are they adopted? If we can understand this mechanism, we may be able to see it at work in our own time.

Moral fashions don't seem to be created the way ordinary fashions are. Ordinary fashions seem to arise by accident when everyone imitates the whim of some influential person. The fashion for broad-toed shoes in late fifteenth century Europe began because Charles VIII of France had six toes on one foot. The fashion for the name Gary began when the actor Frank Cooper adopted the name of a tough mill town in Indiana. Moral fashions more often seem to be created deliberately. When there's something we can't say, it's often because some group doesn't want us to.

The prohibition will be strongest when the group is nervous. The irony of Galileo's situation was that he got in trouble for repeating Copernicus's ideas. Copernicus himself didn't. In fact, Copernicus was a canon of a cathedral, and dedicated his book to the pope. But by Galileo's time the church was in the throes of the Counter-Reformation and was much more worried about unorthodox ideas.

To launch a taboo, a group has to be poised halfway between weakness and power. A confident group doesn't need taboos to protect it. It's not considered improper to make disparaging remarks about Americans, or the English. And yet a group has to be powerful enough to enforce a taboo. Coprophiles, as of this writing, don't seem to be numerous or energetic enough to have had their interests promoted to a lifestyle.

I suspect the biggest source of moral taboos will turn out to be power struggles in which one side only barely has the upper hand. That's where you'll find a group powerful enough to enforce taboos, but weak enough to need them.

Most struggles, whatever they're really about, will be cast as struggles between competing ideas. The English Reformation was at bottom a struggle for wealth and power, but it ended up being cast as a struggle to preserve the souls of Englishmen from the corrupting influence of Rome. It's easier to get people to fight for an idea. And whichever side wins, their ideas will also be considered to

have triumphed, as if God wanted to signal his agreement by selecting that side as the victor.

We often like to think of World War II as a triumph of freedom over totalitarianism. We conveniently forget that the Soviet Union was also one of the winners.

I'm not saying that struggles are never about ideas, just that they will always be made to seem to be about ideas, whether they are or not. And just as there is nothing so unfashionable as the last, discarded fashion, there is nothing so wrong as the principles of the most recently defeated opponent. Representational art is only now recovering from the approval of both Hitler and Stalin. [8]

Although moral fashions tend to arise from different sources than fashions in clothing, the mechanism of their adoption seems much the same. The early adopters will be driven by ambition: self-consciously cool people who want to distinguish themselves from the common herd. As the fashion becomes established they'll be joined by a second, much larger group, driven by fear. [9] This second group adopt the fashion not because they want to stand out but because they are afraid of standing out.

So if you want to figure out what we can't say, look at the machinery of fashion and try to predict what it would make unsayable. What groups are powerful but nervous, and what ideas would they like to suppress? What ideas were tarnished by association when they ended up on the losing side of a recent struggle? If a self-consciously cool person wanted to differentiate himself from preceding fashions (e.g. from his parents), which of their ideas would he tend to reject? What are conventional-minded people afraid of saying?

This technique won't find us all the things we can't say. I can think of some that aren't the result of any recent struggle. Many of our taboos are rooted deep in the past. But this approach, combined with the preceding four, will turn up a good number of unthinkable ideas.

Why

Some would ask, why would one want to do this? Why deliberately go poking around among nasty, disreputable ideas? Why look under rocks?

I do it, first of all, for the same reason I did look under rocks as a kid: plain curiosity. And I'm especially curious about anything that's forbidden. Let me see and decide for myself.

Second, I do it because I don't like the idea of being mistaken. If, like other eras, we believe things that will later seem ridiculous, I want to know what they are so that I, at least, can avoid believing them.

Third, I do it because it's good for the brain. To do good work you need a brain that can go anywhere. And you especially need a brain that's in the habit of going where it's not supposed to.

Great work tends to grow out of ideas that others have overlooked, and no idea is so overlooked as one that's unthinkable. Natural selection, for example. It's so simple. Why didn't anyone think of it before? Well, that is all too obvious. Darwin himself was careful to tiptoe around the implications of his theory. He wanted to spend his time thinking about biology, not arguing with people who accused him of being an atheist.

In the sciences, especially, it's a great advantage to be able to question assumptions. The m.o. of scientists, or at least of the good ones, is precisely that: look for places where conventional wisdom is broken, and then try to pry apart the cracks and see what's underneath. That's where new theories come from.

A good scientist, in other words, does not merely ignore conventional wisdom, but makes a special effort to break it. Scientists go looking for trouble. This should be the m.o. of any scholar, but scientists seem much more willing to look under rocks. [10]

Why? It could be that the scientists are simply smarter; most physicists could, if necessary, make it through a PhD program in French literature, but few professors of French literature could make it through a PhD program in physics. Or it could be because it's clearer in the sciences whether theories are true or false, and this makes scientists bolder. (Or it could be that, because it's clearer in the sciences whether theories are true or false, you have to be smart to get jobs as a scientist, rather than just a good politician.)

Whatever the reason, there seems a clear correlation between intelligence and willingness to consider shocking ideas. This isn't just because smart people actively work to find holes in conventional thinking. I think conventions also have less hold over them to start with. You can see that in the way they dress.

It's not only in the sciences that heresy pays off. In any competitive field, you can win big by seeing things that others daren't. And in every field there are probably heresies few dare utter. Within the US car industry there is a lot of hand-wringing now about declining market share. Yet the cause is so obvious that any observant outsider could explain it in a second: they make bad cars. And they have for so long that by now the US car brands are antibrands — something you'd buy a car despite, not because of. Cadillac stopped being the Cadillac of cars in about 1970. And yet I suspect no one dares say this. [11] Otherwise these companies would have tried to fix the problem.

Training yourself to think unthinkable thoughts has advantages beyond the

thoughts themselves. It's like stretching. When you stretch before running, you put your body into positions much more extreme than any it will assume during the run. If you can think things so outside the box that they'd make people's hair stand on end, you'll have no trouble with the small trips outside the box that people call innovative.

*** Pensieri Stretti***

When you find something you can't say, what do you do with it? My advice is, don't say it. Or at least, pick your battles.

Suppose in the future there is a movement to ban the color yellow. Proposals to paint anything yellow are denounced as "yellowist", as is anyone suspected of liking the color. People who like orange are tolerated but viewed with suspicion. Suppose you realize there is nothing wrong with yellow. If you go around saying this, you'll be denounced as a yellowist too, and you'll find yourself having a lot of arguments with anti-yellowists. If your aim in life is to rehabilitate the color yellow, that may be what you want. But if you're mostly interested in other questions, being labelled as a yellowist will just be a distraction. Argue with idiots, and you become an idiot.

The most important thing is to be able to think what you want, not to say what you want. And if you feel you have to say everything you think, it may inhibit you from thinking improper thoughts. I think it's better to follow the opposite policy. Draw a sharp line between your thoughts and your speech. Inside your head, anything is allowed. Within my head I make a point of encouraging the most outrageous thoughts I can imagine. But, as in a secret society, nothing that happens within the building should be told to outsiders. The first rule of Fight Club is, you do not talk about Fight Club.

When Milton was going to visit Italy in the 1630s, Sir Henry Wootton, who had been ambassador to Venice, told him his motto should be "*i pensieri stretti & il viso sciolto*." Closed thoughts and an open face. Smile at everyone, and don't tell them what you're thinking. This was wise advice. Milton was an argumentative fellow, and the Inquisition was a bit restive at that time. But I think the difference between Milton's situation and ours is only a matter of degree. Every era has its heresies, and if you don't get imprisoned for them you will at least get in enough trouble that it becomes a complete distraction.

I admit it seems cowardly to keep quiet. When I read about the harassment to which the Scientologists subject their critics [12], or that pro-Israel groups are "compiling dossiers" on those who speak out against Israeli human rights abuses [13], or about people being sued for violating the DMCA [14], part of me wants to say, "All right, you bastards, bring it on." The problem is, there are so many

things you can't say. If you said them all you'd have no time left for your real work. You'd have to turn into Noam Chomsky. [15]

The trouble with keeping your thoughts secret, though, is that you lose the advantages of discussion. Talking about an idea leads to more ideas. So the optimal plan, if you can manage it, is to have a few trusted friends you can speak openly to. This is not just a way to develop ideas; it's also a good rule of thumb for choosing friends. The people you can say heretical things to without getting jumped on are also the most interesting to know.

** *Viso Sciolto*? **

I don't think we need the *viso sciolto* so much as the *pensieri stretti*. Perhaps the best policy is to make it plain that you don't agree with whatever zealotry is current in your time, but not to be too specific about what you disagree with. Zealots will try to draw you out, but you don't have to answer them. If they try to force you to treat a question on their terms by asking "are you with us or against us?" you can always just answer "neither".

Better still, answer "I haven't decided." That's what Larry Summers did when a group tried to put him in this position. Explaining himself later, he said "I don't do litmus tests." [16] A lot of the questions people get hot about are actually quite complicated. There is no prize for getting the answer quickly.

If the anti-yellowists seem to be getting out of hand and you want to fight back, there are ways to do it without getting yourself accused of being a yellowist. Like skirmishers in an ancient army, you want to avoid directly engaging the main body of the enemy's troops. Better to harass them with arrows from a distance.

One way to do this is to ratchet the debate up one level of abstraction. If you argue against censorship in general, you can avoid being accused of whatever heresy is contained in the book or film that someone is trying to censor. You can attack labels with meta-labels: labels that refer to the use of labels to prevent discussion. The spread of the term "political correctness" meant the beginning of the end of political correctness, because it enabled one to attack the phenomenon as a whole without being accused of any of the specific heresies it sought to suppress.

Another way to counterattack is with metaphor. Arthur Miller undermined the House Un-American Activities Committee by writing a play, "The Crucible," about the Salem witch trials. He never referred directly to the committee and so gave them no way to reply. What could HUAC do, defend the Salem witch trials? And yet Miller's metaphor stuck so well that to this day the activities of the committee are often described as a "witch-hunt."

Best of all, probably, is humor. Zealots, whatever their cause, invariably lack a sense of humor. They can't reply in kind to jokes. They're as unhappy on the territory of humor as a mounted knight on a skating rink. Victorian prudishness, for example, seems to have been defeated mainly by treating it as a joke. Likewise its reincarnation as political correctness. "I am glad that I managed to write 'The Crucible,'" Arthur Miller wrote, "but looking back I have often wished I'd had the temperament to do an absurd comedy, which is what the situation deserved." [17]

ABQ

A Dutch friend says I should use Holland as an example of a tolerant society. It's true they have a long tradition of comparative open-mindedness. For centuries the low countries were the place to go to say things you couldn't say anywhere else, and this helped to make the region a center of scholarship and industry (which have been closely tied for longer than most people realize). Descartes, though claimed by the French, did much of his thinking in Holland.

And yet, I wonder. The Dutch seem to live their lives up to their necks in rules and regulations. There's so much you can't do there; is there really nothing you can't say?

Certainly the fact that they value open-mindedness is no guarantee. Who thinks they're not open-minded? Our hypothetical prim miss from the suburbs thinks she's open-minded. Hasn't she been taught to be? Ask anyone, and they'll say the same thing: they're pretty open-minded, though they draw the line at things that are really wrong. (Some tribes may avoid "wrong" as judgemental, and may instead use a more neutral sounding euphemism like "negative" or "destructive".)

When people are bad at math, they know it, because they get the wrong answers on tests. But when people are bad at open-mindedness they don't know it. In fact they tend to think the opposite. Remember, it's the nature of fashion to be invisible. It wouldn't work otherwise. Fashion doesn't seem like fashion to someone in the grip of it. It just seems like the right thing to do. It's only by looking from a distance that we see oscillations in people's idea of the right thing to do, and can identify them as fashions.

Time gives us such distance for free. Indeed, the arrival of new fashions makes old fashions easy to see, because they seem so ridiculous by contrast. From one end of a pendulum's swing, the other end seems especially far away.

To see fashion in your own time, though, requires a conscious effort. Without time to give you distance, you have to create distance yourself. Instead of being part of the mob, stand as far away from it as you can and watch what it's doing. And pay especially close attention whenever an idea is being suppressed. Web

filters for children and employees often ban sites containing pornography, violence, and hate speech. What counts as pornography and violence? And what, exactly, is “hate speech?” This sounds like a phrase out of *1984*.

Labels like that are probably the biggest external clue. If a statement is false, that’s the worst thing you can say about it. You don’t need to say that it’s heretical. And if it isn’t false, it shouldn’t be suppressed. So when you see statements being attacked as x-ist or y-ic (substitute your current values of x and y), whether in 1630 or 2030, that’s a sure sign that something is wrong. When you hear such labels being used, ask why.

Especially if you hear yourself using them. It’s not just the mob you need to learn to watch from a distance. You need to be able to watch your own thoughts from a distance. That’s not a radical idea, by the way; it’s the main difference between children and adults. When a child gets angry because he’s tired, he doesn’t know what’s happening. An adult can distance himself enough from the situation to say “never mind, I’m just tired.” I don’t see why one couldn’t, by a similar process, learn to recognize and discount the effects of moral fashions.

You have to take that extra step if you want to think clearly. But it’s harder, because now you’re working against social customs instead of with them. Everyone encourages you to grow up to the point where you can discount your own bad moods. Few encourage you to continue to the point where you can discount society’s bad moods.

How can you see the wave, when you’re the water? Always be questioning. That’s the only defence. What can’t you say? And why?

Notes

Thanks to Sarah Harlin, Trevor Blackwell, Jessica Livingston, Robert Morris, Eric Raymond and Bob van der Zwaan for reading drafts of this essay, and to Lisa Randall, Jackie McDonough, Ryan Stanley and Joel Rainey for conversations about heresy. Needless to say they bear no blame for opinions expressed in it, and especially for opinions *not* expressed in it.

— || Re: What You Can’t Say

How to Make Wealth

May 2004

(This essay was originally published in Hackers & Painters.)

If you wanted to get rich, how would you do it? I think your best bet would be to start or join a startup. That's been a reliable way to get rich for hundreds of years. The word "startup" dates from the 1960s, but what happens in one is very similar to the venture-backed trading voyages of the Middle Ages.

Startups usually involve technology, so much so that the phrase "high-tech startup" is almost redundant. A startup is a small company that takes on a hard technical problem.

Lots of people get rich knowing nothing more than that. You don't have to know physics to be a good pitcher. But I think it could give you an edge to understand the underlying principles. Why do startups have to be small? Will a startup inevitably stop being a startup as it grows larger? And why do they so often work on developing new technology? Why are there so many startups selling new drugs or computer software, and none selling corn oil or laundry detergent?

The Proposition

Economically, you can think of a startup as a way to compress your whole working life into a few years. Instead of working at a low intensity for forty years, you work as hard as you possibly can for four. This pays especially well in technology, where you earn a premium for working fast.

Here is a brief sketch of the economic proposition. If you're a good hacker in your mid twenties, you can get a job paying about \$80,000 per year. So on average such a hacker must be able to do at least \$80,000 worth of work per year for the company just to break even. You could probably work twice as many hours as a corporate employee, and if you focus you can probably get three times as much done in an hour. [1] You should get another multiple of two, at least, by eliminating the drag of the pointy-haired middle manager who would be your

boss in a big company. Then there is one more multiple: how much smarter are you than your job description expects you to be? Suppose another multiple of three. Combine all these multipliers, and I'm claiming you could be 36 times more productive than you're expected to be in a random corporate job. [2] If a fairly good hacker is worth \$80,000 a year at a big company, then a smart hacker working very hard without any corporate bullshit to slow him down should be able to do work worth about \$3 million a year.

Like all back-of-the-envelope calculations, this one has a lot of wiggle room. I wouldn't try to defend the actual numbers. But I stand by the structure of the calculation. I'm not claiming the multiplier is precisely 36, but it is certainly more than 10, and probably rarely as high as 100.

If \$3 million a year seems high, remember that we're talking about the limit case: the case where you not only have zero leisure time but indeed work so hard that you endanger your health.

Startups are not magic. They don't change the laws of wealth creation. They just represent a point at the far end of the curve. There is a conservation law at work here: if you want to make a million dollars, you have to endure a million dollars' worth of pain. For example, one way to make a million dollars would be to work for the Post Office your whole life, and save every penny of your salary. Imagine the stress of working for the Post Office for fifty years. In a startup you compress all this stress into three or four years. You do tend to get a certain bulk discount if you buy the economy-size pain, but you can't evade the fundamental conservation law. If starting a startup were easy, everyone would do it.

Millions, not Billions

If \$3 million a year seems high to some people, it will seem low to others. Three *million*? How do I get to be a billionaire, like Bill Gates?

So let's get Bill Gates out of the way right now. It's not a good idea to use famous rich people as examples, because the press only write about the very richest, and these tend to be outliers. Bill Gates is a smart, determined, and hardworking man, but you need more than that to make as much money as he has. You also need to be very lucky.

There is a large random factor in the success of any company. So the guys you end up reading about in the papers are the ones who are very smart, totally dedicated, *and* win the lottery. Certainly Bill is smart and dedicated, but Microsoft also happens to have been the beneficiary of one of the most spectacular blunders in the history of business: the licensing deal for DOS. No doubt Bill did everything he could to steer IBM into making that blunder, and he has done an excellent job of exploiting it, but if there had been one person with a brain on IBM's side,

Microsoft's future would have been very different. Microsoft at that stage had little leverage over IBM. They were effectively a component supplier. If IBM had required an exclusive license, as they should have, Microsoft would still have signed the deal. It would still have meant a lot of money for them, and IBM could easily have gotten an operating system elsewhere.

Instead IBM ended up using all its power in the market to give Microsoft control of the PC standard. From that point, all Microsoft had to do was execute. They never had to bet the company on a bold decision. All they had to do was play hardball with licensees and copy more innovative products reasonably promptly.

If IBM hadn't made this mistake, Microsoft would still have been a successful company, but it could not have grown so big so fast. Bill Gates would be rich, but he'd be somewhere near the bottom of the Forbes 400 with the other guys his age.

There are a lot of ways to get rich, and this essay is about only one of them. This essay is about how to make money by creating wealth and getting paid for it. There are plenty of other ways to get money, including chance, speculation, marriage, inheritance, theft, extortion, fraud, monopoly, graft, lobbying, counterfeiting, and prospecting. Most of the greatest fortunes have probably involved several of these.

The advantage of creating wealth, as a way to get rich, is not just that it's more legitimate (many of the other methods are now illegal) but that it's more *straightforward*. You just have to do something people want.

Money Is Not Wealth

If you want to create wealth, it will help to understand what it is. Wealth is not the same thing as money. [3] Wealth is as old as human history. Far older, in fact; ants have wealth. Money is a comparatively recent invention.

Wealth is the fundamental thing. Wealth is stuff we want: food, clothes, houses, cars, gadgets, travel to interesting places, and so on. You can have wealth without having money. If you had a magic machine that could on command make you a car or cook you dinner or do your laundry, or do anything else you wanted, you wouldn't need money. Whereas if you were in the middle of Antarctica, where there is nothing to buy, it wouldn't matter how much money you had.

Wealth is what you want, not money. But if wealth is the important thing, why does everyone talk about making money? It is a kind of shorthand: money is a way of moving wealth, and in practice they are usually interchangeable. But they are not the same thing, and unless you plan to get rich by counterfeiting, talking about *making money* can make it harder to understand how to make money.

Money is a side effect of specialization. In a specialized society, most of the things you need, you can't make for yourself. If you want a potato or a pencil or a place to live, you have to get it from someone else.

How do you get the person who grows the potatoes to give you some? By giving him something he wants in return. But you can't get very far by trading things directly with the people who need them. If you make violins, and none of the local farmers wants one, how will you eat?

The solution societies find, as they get more specialized, is to make the trade into a two-step process. Instead of trading violins directly for potatoes, you trade violins for, say, silver, which you can then trade again for anything else you need. The intermediate stuff— the *medium of exchange* -- can be anything that's rare and portable. Historically metals have been the most common, but recently we've been using a medium of exchange, called the *dollar*, that doesn't physically exist. It works as a medium of exchange, however, because its rarity is guaranteed by the U.S. Government.

The advantage of a medium of exchange is that it makes trade work. The disadvantage is that it tends to obscure what trade really means. People think that what a business does is make money. But money is just the intermediate stage— just a shorthand— for whatever people want. What most businesses really do is make wealth. They do something people want. [4]

The Pie Fallacy

A surprising number of people retain from childhood the idea that there is a fixed amount of wealth in the world. There is, in any normal family, a fixed amount of *money* at any moment. But that's not the same thing.

When wealth is talked about in this context, it is often described as a pie. "You can't make the pie larger," say politicians. When you're talking about the amount of money in one family's bank account, or the amount available to a government from one year's tax revenue, this is true. If one person gets more, someone else has to get less.

I can remember believing, as a child, that if a few rich people had all the money, it left less for everyone else. Many people seem to continue to believe something like this well into adulthood. This fallacy is usually there in the background when you hear someone talking about how x percent of the population have y percent of the wealth. If you plan to start a startup, then whether you realize it or not, you're planning to disprove the Pie Fallacy.

What leads people astray here is the abstraction of money. Money is not wealth. It's just something we use to move wealth around. So although there may be, in

certain specific moments (like your family, this month) a fixed amount of money available to trade with other people for things you want, there is not a fixed amount of wealth in the world. *You can make more wealth.* Wealth has been getting created and destroyed (but on balance, created) for all of human history.

Suppose you own a beat-up old car. Instead of sitting on your butt next summer, you could spend the time restoring your car to pristine condition. In doing so you create wealth. The world is— and you specifically are— one pristine old car the richer. And not just in some metaphorical way. If you sell your car, you'll get more for it.

In restoring your old car you have made yourself richer. You haven't made anyone else poorer. So there is obviously not a fixed pie. And in fact, when you look at it this way, you wonder why anyone would think there was. [5]

Kids know, without knowing they know, that they can create wealth. If you need to give someone a present and don't have any money, you make one. But kids are so bad at making things that they consider home-made presents to be a distinct, inferior, sort of thing to store-bought ones— a mere expression of the proverbial thought that counts. And indeed, the lumpy ashtrays we made for our parents did not have much of a resale market.

Craftsmen

The people most likely to grasp that wealth can be created are the ones who are good at making things, the craftsmen. Their hand-made objects become store-bought ones. But with the rise of industrialization there are fewer and fewer craftsmen. One of the biggest remaining groups is computer programmers.

A programmer can sit down in front of a computer and *create wealth*. A good piece of software is, in itself, a valuable thing. There is no manufacturing to confuse the issue. Those characters you type are a complete, finished product. If someone sat down and wrote a web browser that didn't suck (a fine idea, by the way), the world would be that much richer. [5b]

Everyone in a company works together to create wealth, in the sense of making more things people want. Many of the employees (e.g. the people in the mailroom or the personnel department) work at one remove from the actual making of stuff. Not the programmers. They literally think the product, one line at a time. And so it's clearer to programmers that wealth is something that's made, rather than being distributed, like slices of a pie, by some imaginary Daddy.

It's also obvious to programmers that there are huge variations in the rate at which wealth is created. At Viaweb we had one programmer who was a sort of monster of productivity. I remember watching what he did one long day and

estimating that he had added several hundred thousand dollars to the market value of the company. A great programmer, on a roll, could create a million dollars worth of wealth in a couple weeks. A mediocre programmer over the same period will generate zero or even negative wealth (e.g. by introducing bugs).

This is why so many of the best programmers are libertarians. In our world, you sink or swim, and there are no excuses. When those far removed from the creation of wealth—undergraduates, reporters, politicians—hear that the richest 5% of the people have half the total wealth, they tend to think *injustice!* An experienced programmer would be more likely to think *is that all?* The top 5% of programmers probably write 99% of the good software.

Wealth can be created without being sold. Scientists, till recently at least, effectively donated the wealth they created. We are all richer for knowing about penicillin, because we're less likely to die from infections. Wealth is whatever people want, and not dying is certainly something we want. Hackers often donate their work by writing open source software that anyone can use for free. I am much the richer for the operating system FreeBSD, which I'm running on the computer I'm using now, and so is Yahoo, which runs it on all their servers.

What a Job Is

In industrialized countries, people belong to one institution or another at least until their twenties. After all those years you get used to the idea of belonging to a group of people who all get up in the morning, go to some set of buildings, and do things that they do not, ordinarily, enjoy doing. Belonging to such a group becomes part of your identity: name, age, role, institution. If you have to introduce yourself, or someone else describes you, it will be as something like, John Smith, age 10, a student at such and such elementary school, or John Smith, age 20, a student at such and such college.

When John Smith finishes school he is expected to get a job. And what getting a job seems to mean is joining another institution. Superficially it's a lot like college. You pick the companies you want to work for and apply to join them. If one likes you, you become a member of this new group. You get up in the morning and go to a new set of buildings, and do things that you do not, ordinarily, enjoy doing. There are a few differences: life is not as much fun, and you get paid, instead of paying, as you did in college. But the similarities feel greater than the differences. John Smith is now John Smith, 22, a software developer at such and such corporation.

In fact John Smith's life has changed more than he realizes. Socially, a company looks much like college, but the deeper you go into the underlying reality, the more different it gets.

What a company does, and has to do if it wants to continue to exist, is earn money. And the way most companies make money is by creating wealth. Companies can be so specialized that this similarity is concealed, but it is not only manufacturing companies that create wealth. A big component of wealth is location. Remember that magic machine that could make you cars and cook you dinner and so on? It would not be so useful if it delivered your dinner to a random location in central Asia. If wealth means what people want, companies that move things also create wealth. Ditto for many other kinds of companies that don't make anything physical. Nearly all companies exist to do something people want.

And that's what you do, as well, when you go to work for a company. But here there is another layer that tends to obscure the underlying reality. In a company, the work you do is averaged together with a lot of other people's. You may not even be aware you're doing something people want. Your contribution may be indirect. But the company as a whole must be giving people something they want, or they won't make any money. And if they are paying you x dollars a year, then on average you must be contributing at least x dollars a year worth of work, or the company will be spending more than it makes, and will go out of business.

Someone graduating from college thinks, and is told, that he needs to get a job, as if the important thing were becoming a member of an institution. A more direct way to put it would be: you need to start doing something people want. You don't need to join a company to do that. All a company is is a group of people working together to do something people want. It's doing something people want that matters, not joining the group. [6]

For most people the best plan probably is to go to work for some existing company. But it is a good idea to understand what's happening when you do this. A job means doing something people want, averaged together with everyone else in that company.

Working Harder

That averaging gets to be a problem. I think the single biggest problem afflicting large companies is the difficulty of assigning a value to each person's work. For the most part they punt. In a big company you get paid a fairly predictable salary for working fairly hard. You're expected not to be obviously incompetent or lazy, but you're not expected to devote your whole life to your work.

It turns out, though, that there are economies of scale in how much of your life you devote to your work. In the right kind of business, someone who really devoted himself to work could generate ten or even a hundred times as much wealth as an average employee. A programmer, for example, instead of chugging

along maintaining and updating an existing piece of software, could write a whole new piece of software, and with it create a new source of revenue.

Companies are not set up to reward people who want to do this. You can't go to your boss and say, I'd like to start working ten times as hard, so will you please pay me ten times as much? For one thing, the official fiction is that you are already working as hard as you can. But a more serious problem is that the company has no way of measuring the value of your work.

Salesmen are an exception. It's easy to measure how much revenue they generate, and they're usually paid a percentage of it. If a salesman wants to work harder, he can just start doing it, and he will automatically get paid proportionally more.

There is one other job besides sales where big companies can hire first-rate people: in the top management jobs. And for the same reason: their performance can be measured. The top managers are held responsible for the performance of the entire company. Because an ordinary employee's performance can't usually be measured, he is not expected to do more than put in a solid effort. Whereas top management, like salespeople, have to actually come up with the numbers. The CEO of a company that tanks cannot plead that he put in a solid effort. If the company does badly, he's done badly.

A company that could pay all its employees so straightforwardly would be enormously successful. Many employees would work harder if they could get paid for it. More importantly, such a company would attract people who wanted to work especially hard. It would crush its competitors.

Unfortunately, companies can't pay everyone like salesmen. Salesmen work alone. Most employees' work is tangled together. Suppose a company makes some kind of consumer gadget. The engineers build a reliable gadget with all kinds of new features; the industrial designers design a beautiful case for it; and then the marketing people convince everyone that it's something they've got to have. How do you know how much of the gadget's sales are due to each group's efforts? Or, for that matter, how much is due to the creators of past gadgets that gave the company a reputation for quality? There's no way to untangle all their contributions. Even if you could read the minds of the consumers, you'd find these factors were all blurred together.

If you want to go faster, it's a problem to have your work tangled together with a large number of other people's. In a large group, your performance is not separately measurable— and the rest of the group slows you down.

Measurement and Leverage

To get rich you need to get yourself in a situation with two things, measurement

and leverage. You need to be in a position where your performance can be measured, or there is no way to get paid more by doing more. And you have to have leverage, in the sense that the decisions you make have a big effect.

Measurement alone is not enough. An example of a job with measurement but not leverage is doing piecework in a sweatshop. Your performance is measured and you get paid accordingly, but you have no scope for decisions. The only decision you get to make is how fast you work, and that can probably only increase your earnings by a factor of two or three.

An example of a job with both measurement and leverage would be lead actor in a movie. Your performance can be measured in the gross of the movie. And you have leverage in the sense that your performance can make or break it.

CEOs also have both measurement and leverage. They're measured, in that the performance of the company is their performance. And they have leverage in that their decisions set the whole company moving in one direction or another.

I think everyone who gets rich by their own efforts will be found to be in a situation with measurement and leverage. Everyone I can think of does: CEOs, movie stars, hedge fund managers, professional athletes. A good hint to the presence of leverage is the possibility of failure. Upside must be balanced by downside, so if there is big potential for gain there must also be a terrifying possibility of loss. CEOs, stars, fund managers, and athletes all live with the sword hanging over their heads; the moment they start to suck, they're out. If you're in a job that feels safe, you are not going to get rich, because if there is no danger there is almost certainly no leverage.

But you don't have to become a CEO or a movie star to be in a situation with measurement and leverage. All you need to do is be part of a small group working on a hard problem.

Smallness = Measurement

If you can't measure the value of the work done by individual employees, you can get close. You can measure the value of the work done by small groups.

One level at which you can accurately measure the revenue generated by employees is at the level of the whole company. When the company is small, you are thereby fairly close to measuring the contributions of individual employees. A viable startup might only have ten employees, which puts you within a factor of ten of measuring individual effort.

Starting or joining a startup is thus as close as most people can get to saying to one's boss, I want to work ten times as hard, so please pay me ten times as much. There are two differences: you're not saying it to your boss, but directly

to the customers (for whom your boss is only a proxy after all), and you're not doing it individually, but along with a small group of other ambitious people.

It will, ordinarily, be a group. Except in a few unusual kinds of work, like acting or writing books, you can't be a company of one person. And the people you work with had better be good, because it's their work that yours is going to be averaged with.

A big company is like a giant galley driven by a thousand rowers. Two things keep the speed of the galley down. One is that individual rowers don't see any result from working harder. The other is that, in a group of a thousand people, the average rower is likely to be pretty average.

If you took ten people at random out of the big galley and put them in a boat by themselves, they could probably go faster. They would have both carrot and stick to motivate them. An energetic rower would be encouraged by the thought that he could have a visible effect on the speed of the boat. And if someone was lazy, the others would be more likely to notice and complain.

But the real advantage of the ten-man boat shows when you take the ten *best* rowers out of the big galley and put them in a boat together. They will have all the extra motivation that comes from being in a small group. But more importantly, by selecting that small a group you can get the best rowers. Each one will be in the top 1%. It's a much better deal for them to average their work together with a small group of their peers than to average it with everyone.

That's the real point of startups. Ideally, you are getting together with a group of other people who also want to work a lot harder, and get paid a lot more, than they would in a big company. And because startups tend to get founded by self-selecting groups of ambitious people who already know one another (at least by reputation), the level of measurement is more precise than you get from smallness alone. A startup is not merely ten people, but ten people like you.

Steve Jobs once said that the success or failure of a startup depends on the first ten employees. I agree. If anything, it's more like the first five. Being small is not, in itself, what makes startups kick butt, but rather that small groups can be select. You don't want small in the sense of a village, but small in the sense of an all-star team.

The larger a group, the closer its average member will be to the average for the population as a whole. So all other things being equal, a very able person in a big company is probably getting a bad deal, because his performance is dragged down by the overall lower performance of the others. Of course, all other things often are not equal: the able person may not care about money, or may prefer

the stability of a large company. But a very able person who does care about money will ordinarily do better to go off and work with a small group of peers.

Technology = Leverage

Startups offer anyone a way to be in a situation with measurement and leverage. They allow measurement because they're small, and they offer leverage because they make money by inventing new technology.

What is technology? It's *technique*. It's the way we all do things. And when you discover a new way to do things, its value is multiplied by all the people who use it. It is the proverbial fishing rod, rather than the fish. That's the difference between a startup and a restaurant or a barber shop. You fry eggs or cut hair one customer at a time. Whereas if you solve a technical problem that a lot of people care about, you help everyone who uses your solution. That's leverage.

If you look at history, it seems that most people who got rich by creating wealth did it by developing new technology. You just can't fry eggs or cut hair fast enough. What made the Florentines rich in 1200 was the discovery of new techniques for making the high-tech product of the time, fine woven cloth. What made the Dutch rich in 1600 was the discovery of shipbuilding and navigation techniques that enabled them to dominate the seas of the Far East.

Fortunately there is a natural fit between smallness and solving hard problems. The leading edge of technology moves fast. Technology that's valuable today could be worthless in a couple years. Small companies are more at home in this world, because they don't have layers of bureaucracy to slow them down. Also, technical advances tend to come from unorthodox approaches, and small companies are less constrained by convention.

Big companies can develop technology. They just can't do it quickly. Their size makes them slow and prevents them from rewarding employees for the extraordinary effort required. So in practice big companies only get to develop technology in fields where large capital requirements prevent startups from competing with them, like microprocessors, power plants, or passenger aircraft. And even in those fields they depend heavily on startups for components and ideas.

It's obvious that biotech or software startups exist to solve hard technical problems, but I think it will also be found to be true in businesses that don't seem to be about technology. McDonald's, for example, grew big by designing a system, the McDonald's franchise, that could then be reproduced at will all over the face of the earth. A McDonald's franchise is controlled by rules so precise that it is practically a piece of software. Write once, run everywhere. Ditto for

Wal-Mart. Sam Walton got rich not by being a retailer, but by designing a new kind of store.

Use difficulty as a guide not just in selecting the overall aim of your company, but also at decision points along the way. At Viaweb one of our rules of thumb was *run upstairs*. Suppose you are a little, nimble guy being chased by a big, fat, bully. You open a door and find yourself in a staircase. Do you go up or down? I say up. The bully can probably run downstairs as fast as you can. Going upstairs his bulk will be more of a disadvantage. Running upstairs is hard for you but even harder for him.

What this meant in practice was that we deliberately sought hard problems. If there were two features we could add to our software, both equally valuable in proportion to their difficulty, we'd always take the harder one. Not just because it was more valuable, but *because it was harder*. We delighted in forcing bigger, slower competitors to follow us over difficult ground. Like guerillas, startups prefer the difficult terrain of the mountains, where the troops of the central government can't follow. I can remember times when we were just exhausted after wrestling all day with some horrible technical problem. And I'd be delighted, because something that was hard for us would be impossible for our competitors.

This is not just a good way to run a startup. It's what a startup is. Venture capitalists know about this and have a phrase for it: *barriers to entry*. If you go to a VC with a new idea and ask him to invest in it, one of the first things he'll ask is, how hard would this be for someone else to develop? That is, how much difficult ground have you put between yourself and potential pursuers? [7] And you had better have a convincing explanation of why your technology would be hard to duplicate. Otherwise as soon as some big company becomes aware of it, they'll make their own, and with their brand name, capital, and distribution clout, they'll take away your market overnight. You'd be like guerillas caught in the open field by regular army forces.

One way to put up barriers to entry is through patents. But patents may not provide much protection. Competitors commonly find ways to work around a patent. And if they can't, they may simply violate it and invite you to sue them. A big company is not afraid to be sued; it's an everyday thing for them. They'll make sure that suing them is expensive and takes a long time. Ever heard of Philo Farnsworth? He invented television. The reason you've never heard of him is that his company was not the one to make money from it. [8] The company that did was RCA, and Farnsworth's reward for his efforts was a decade of patent litigation.

Here, as so often, the best defense is a good offense. If you can develop technology

that's simply too hard for competitors to duplicate, you don't need to rely on other defenses. Start by picking a hard problem, and then at every decision point, take the harder choice. [9]

The Catch(es)

If it were simply a matter of working harder than an ordinary employee and getting paid proportionately, it would obviously be a good deal to start a startup. Up to a point it would be more fun. I don't think many people like the slow pace of big companies, the interminable meetings, the water-cooler conversations, the clueless middle managers, and so on.

Unfortunately there are a couple catches. One is that you can't choose the point on the curve that you want to inhabit. You can't decide, for example, that you'd like to work just two or three times as hard, and get paid that much more. When you're running a startup, your competitors decide how hard you work. And they pretty much all make the same decision: as hard as you possibly can.

The other catch is that the payoff is only on average proportionate to your productivity. There is, as I said before, a large random multiplier in the success of any company. So in practice the deal is not that you're 30 times as productive and get paid 30 times as much. It is that you're 30 times as productive, and get paid between zero and a thousand times as much. If the mean is 30x, the median is probably zero. Most startups tank, and not just the dogfood portals we all heard about during the Internet Bubble. It's common for a startup to be developing a genuinely good product, take slightly too long to do it, run out of money, and have to shut down.

A startup is like a mosquito. A bear can absorb a hit and a crab is armored against one, but a mosquito is designed for one thing: to score. No energy is wasted on defense. The defense of mosquitos, as a species, is that there are a lot of them, but this is little consolation to the individual mosquito.

Startups, like mosquitos, tend to be an all-or-nothing proposition. And you don't generally know which of the two you're going to get till the last minute. Viaweb came close to tanking several times. Our trajectory was like a sine wave. Fortunately we got bought at the top of the cycle, but it was damned close. While we were visiting Yahoo in California to talk about selling the company to them, we had to borrow a conference room to reassure an investor who was about to back out of a new round of funding that we needed to stay alive.

The all-or-nothing aspect of startups was not something we wanted. Viaweb's hackers were all extremely risk-averse. If there had been some way just to work super hard and get paid for it, without having a lottery mixed in, we would have been delighted. We would have much preferred a 100% chance of \$1 million to a

20% chance of \$10 million, even though theoretically the second is worth twice as much. Unfortunately, there is not currently any space in the business world where you can get the first deal.

The closest you can get is by selling your startup in the early stages, giving up upside (and risk) for a smaller but guaranteed payoff. We had a chance to do this, and stupidly, as we then thought, let it slip by. After that we became comically eager to sell. For the next year or so, if anyone expressed the slightest curiosity about Viaweb we would try to sell them the company. But there were no takers, so we had to keep going.

It would have been a bargain to buy us at an early stage, but companies doing acquisitions are not looking for bargains. A company big enough to acquire startups will be big enough to be fairly conservative, and within the company the people in charge of acquisitions will be among the more conservative, because they are likely to be business school types who joined the company late. They would rather overpay for a safe choice. So it is easier to sell an established startup, even at a large premium, than an early-stage one.

Get Users

I think it's a good idea to get bought, if you can. Running a business is different from growing one. It is just as well to let a big company take over once you reach cruising altitude. It's also financially wiser, because selling allows you to diversify. What would you think of a financial advisor who put all his client's assets into one volatile stock?

How do you get bought? Mostly by doing the same things you'd do if you didn't intend to sell the company. Being profitable, for example. But getting bought is also an art in its own right, and one that we spent a lot of time trying to master.

Potential buyers will always delay if they can. The hard part about getting bought is getting them to act. For most people, the most powerful motivator is not the hope of gain, but the fear of loss. For potential acquirers, the most powerful motivator is the prospect that one of their competitors will buy you. This, as we found, causes CEOs to take red-eyes. The second biggest is the worry that, if they don't buy you now, you'll continue to grow rapidly and will cost more to acquire later, or even become a competitor.

In both cases, what it all comes down to is users. You'd think that a company about to buy you would do a lot of research and decide for themselves how valuable your technology was. Not at all. What they go by is the number of users you have.

In effect, acquirers assume the customers know who has the best technology. And

this is not as stupid as it sounds. Users are the only real proof that you've created wealth. Wealth is what people want, and if people aren't using your software, maybe it's not just because you're bad at marketing. Maybe it's because you haven't made what they want.

Venture capitalists have a list of danger signs to watch out for. Near the top is the company run by techno-weenies who are obsessed with solving interesting technical problems, instead of making users happy. In a startup, you're not just trying to solve problems. You're trying to solve problems *that users care about*.

So I think you should make users the test, just as acquirers do. Treat a startup as an optimization problem in which performance is measured by number of users. As anyone who has tried to optimize software knows, the key is measurement. When you try to guess where your program is slow, and what would make it faster, you almost always guess wrong.

Number of users may not be the perfect test, but it will be very close. It's what acquirers care about. It's what revenues depend on. It's what makes competitors unhappy. It's what impresses reporters, and potential new users. Certainly it's a better test than your a priori notions of what problems are important to solve, no matter how technically adept you are.

Among other things, treating a startup as an optimization problem will help you avoid another pitfall that VCs worry about, and rightly—taking a long time to develop a product. Now we can recognize this as something hackers already know to avoid: premature optimization. Get a version 1.0 out there as soon as you can. Until you have some users to measure, you're optimizing based on guesses.

The ball you need to keep your eye on here is the underlying principle that wealth is what people want. If you plan to get rich by creating wealth, you have to know what people want. So few businesses really pay attention to making customers happy. How often do you walk into a store, or call a company on the phone, with a feeling of dread in the back of your mind? When you hear “your call is important to us, please stay on the line,” do you think, oh good, now everything will be all right?

A restaurant can afford to serve the occasional burnt dinner. But in technology, you cook one thing and that's what everyone eats. So any difference between what people want and what you deliver is multiplied. You please or annoy customers wholesale. The closer you can get to what they want, the more wealth you generate.

Wealth and Power

Making wealth is not the only way to get rich. For most of human history it has

not even been the most common. Until a few centuries ago, the main sources of wealth were mines, slaves and serfs, land, and cattle, and the only ways to acquire these rapidly were by inheritance, marriage, conquest, or confiscation. Naturally wealth had a bad reputation.

Two things changed. The first was the rule of law. For most of the world's history, if you did somehow accumulate a fortune, the ruler or his henchmen would find a way to steal it. But in medieval Europe something new happened. A new class of merchants and manufacturers began to collect in towns. [10] Together they were able to withstand the local feudal lord. So for the first time in our history, the bullies stopped stealing the nerds' lunch money. This was naturally a great incentive, and possibly indeed the main cause of the second big change, industrialization.

A great deal has been written about the causes of the Industrial Revolution. But surely a necessary, if not sufficient, condition was that people who made fortunes be able to enjoy them in peace. [11] One piece of evidence is what happened to countries that tried to return to the old model, like the Soviet Union, and to a lesser extent Britain under the labor governments of the 1960s and early 1970s. Take away the incentive of wealth, and technical innovation grinds to a halt.

Remember what a startup is, economically: a way of saying, I want to work faster. Instead of accumulating money slowly by being paid a regular wage for fifty years, I want to get it over with as soon as possible. So governments that forbid you to accumulate wealth are in effect decreeing that you work slowly. They're willing to let you earn \$3 million over fifty years, but they're not willing to let you work so hard that you can do it in two. They are like the corporate boss that you can't go to and say, I want to work ten times as hard, so please pay me ten times as much. Except this is not a boss you can escape by starting your own company.

The problem with working slowly is not just that technical innovation happens slowly. It's that it tends not to happen at all. It's only when you're deliberately looking for hard problems, as a way to use speed to the greatest advantage, that you take on this kind of project. Developing new technology is a pain in the ass. It is, as Edison said, one percent inspiration and ninety-nine percent perspiration. Without the incentive of wealth, no one wants to do it. Engineers will work on sexy projects like fighter planes and moon rockets for ordinary salaries, but more mundane technologies like light bulbs or semiconductors have to be developed by entrepreneurs.

Startups are not just something that happened in Silicon Valley in the last couple decades. Since it became possible to get rich by creating wealth, everyone who

has done it has used essentially the same recipe: measurement and leverage, where measurement comes from working with a small group, and leverage from developing new techniques. The recipe was the same in Florence in 1200 as it is in Santa Clara today.

Understanding this may help to answer an important question: why Europe grew so powerful. Was it something about the geography of Europe? Was it that Europeans are somehow racially superior? Was it their religion? The answer (or at least the proximate cause) may be that the Europeans rode on the crest of a powerful new idea: allowing those who made a lot of money to keep it.

Once you're allowed to do that, people who want to get rich can do it by generating wealth instead of stealing it. The resulting technological growth translates not only into wealth but into military power. The theory that led to the stealth plane was developed by a Soviet mathematician. But because the Soviet Union didn't have a computer industry, it remained for them a theory; they didn't have hardware capable of executing the calculations fast enough to design an actual airplane.

In that respect the Cold War teaches the same lesson as World War II and, for that matter, most wars in recent history. Don't let a ruling class of warriors and politicians squash the entrepreneurs. The same recipe that makes individuals rich makes countries powerful. Let the nerds keep their lunch money, and you rule the world.

Notes

[1] One valuable thing you tend to get only in startups is *uninterruptability*. Different kinds of work have different time quanta. Someone proofreading a manuscript could probably be interrupted every fifteen minutes with little loss of productivity. But the time quantum for hacking is very long: it might take an hour just to load a problem into your head. So the cost of having someone from personnel call you about a form you forgot to fill out can be huge.

This is why hackers give you such a baleful stare as they turn from their screen to answer your question. Inside their heads a giant house of cards is tottering.

The mere possibility of being interrupted deters hackers from starting hard projects. This is why they tend to work late at night, and why it's next to impossible to write great software in a cubicle (except late at night).

One great advantage of startups is that they don't yet have any of the people who interrupt you. There is no personnel department, and thus no form nor anyone to call you about it.

[2] Faced with the idea that people working for startups might be 20 or 30 times

as productive as those working for large companies, executives at large companies will naturally wonder, how could I get the people working for me to do that? The answer is simple: pay them to.

Internally most companies are run like Communist states. If you believe in free markets, why not turn your company into one?

Hypothesis: A company will be maximally profitable when each employee is paid in proportion to the wealth they generate.

[3] Until recently even governments sometimes didn't grasp the distinction between money and wealth. Adam Smith (*Wealth of Nations*, v:i) mentions several that tried to preserve their "wealth" by forbidding the export of gold or silver. But having more of the medium of exchange would not make a country richer; if you have more money chasing the same amount of material wealth, the only result is higher prices.

[4] There are many senses of the word "wealth," not all of them material. I'm not trying to make a deep philosophical point here about which is the true kind. I'm writing about one specific, rather technical sense of the word "wealth." What people will give you money for. This is an interesting sort of wealth to study, because it is the kind that prevents you from starving. And what people will give you money for depends on them, not you.

When you're starting a business, it's easy to slide into thinking that customers want what you do. During the Internet Bubble I talked to a woman who, because she liked the outdoors, was starting an "outdoor portal." You know what kind of business you should start if you like the outdoors? One to recover data from crashed hard disks.

What's the connection? None at all. Which is precisely my point. If you want to create wealth (in the narrow technical sense of not starving) then you should be especially skeptical about any plan that centers on things you like doing. That is where your idea of what's valuable is least likely to coincide with other people's.

[5] In the average car restoration you probably do make everyone else microscopically poorer, by doing a small amount of damage to the environment. While environmental costs should be taken into account, they don't make wealth a zero-sum game. For example, if you repair a machine that's broken because a part has come unscrewed, you create wealth with no environmental cost.

[5b] This essay was written before Firefox.

[6] Many people feel confused and depressed in their early twenties. Life seemed so much more fun in college. Well, of course it was. Don't be fooled by the surface similarities. You've gone from guest to servant. It's possible to have fun

in this new world. Among other things, you now get to go behind the doors that say “authorized personnel only.” But the change is a shock at first, and all the worse if you’re not consciously aware of it.

[7] When VCs asked us how long it would take another startup to duplicate our software, we used to reply that they probably wouldn’t be able to at all. I think this made us seem naive, or liars.

[8] Few technologies have one clear inventor. So as a rule, if you know the “inventor” of something (the telephone, the assembly line, the airplane, the light bulb, the transistor) it is because their company made money from it, and the company’s PR people worked hard to spread the story. If you don’t know who invented something (the automobile, the television, the computer, the jet engine, the laser), it’s because other companies made all the money.

[9] This is a good plan for life in general. If you have two choices, choose the harder. If you’re trying to decide whether to go out running or sit home and watch TV, go running. Probably the reason this trick works so well is that when you have two choices and one is harder, the only reason you’re even considering the other is laziness. You know in the back of your mind what’s the right thing to do, and this trick merely forces you to acknowledge it.

[10] It is probably no accident that the middle class first appeared in northern Italy and the low countries, where there were no strong central governments. These two regions were the richest of their time and became the twin centers from which Renaissance civilization radiated. If they no longer play that role, it is because other places, like the United States, have been truer to the principles they discovered.

[11] It may indeed be a sufficient condition. But if so, why didn’t the Industrial Revolution happen earlier? Two possible (and not incompatible) answers: (a) It did. The Industrial Revolution was one in a series. (b) Because in medieval towns, monopolies and guild regulations initially slowed the development of new means of production.

Comment on this essay.

Great Hackers

July 2004

(This essay is derived from a talk at Oscon 2004.)

A few months ago I finished a new book, and in reviews I keep noticing words like “provocative” and “controversial.” To say nothing of “idiotic.”

I didn’t mean to make the book controversial. I was trying to make it efficient. I didn’t want to waste people’s time telling them things they already knew. It’s more efficient just to give them the diffs. But I suppose that’s bound to yield an alarming book.

Edisons

There’s no controversy about which idea is most controversial: the suggestion that variation in wealth might not be as big a problem as we think.

I didn’t say in the book that variation in wealth was in itself a good thing. I said in some situations it might be a sign of good things. A throbbing headache is not a good thing, but it can be a sign of a good thing— for example, that you’re recovering consciousness after being hit on the head.

Variation in wealth can be a sign of variation in productivity. (In a society of one, they’re identical.) And *that* is almost certainly a good thing: if your society has no variation in productivity, it’s probably not because everyone is Thomas Edison. It’s probably because you have no Thomas Edisons.

In a low-tech society you don’t see much variation in productivity. If you have a tribe of nomads collecting sticks for a fire, how much more productive is the best stick gatherer going to be than the worst? A factor of two? Whereas when you hand people a complex tool like a computer, the variation in what they can do with it is enormous.

That’s not a new idea. Fred Brooks wrote about it in 1974, and the study he quoted was published in 1968. But I think he underestimated the variation

between programmers. He wrote about productivity in lines of code: the best programmers can solve a given problem in a tenth the time. But what if the problem isn't given? In programming, as in many fields, the hard part isn't solving problems, but deciding what problems to solve. Imagination is hard to measure, but in practice it dominates the kind of productivity that's measured in lines of code.

Productivity varies in any field, but there are few in which it varies so much. The variation between programmers is so great that it becomes a difference in kind. I don't think this is something intrinsic to programming, though. In every field, technology magnifies differences in productivity. I think what's happening in programming is just that we have a lot of technological leverage. But in every field the lever is getting longer, so the variation we see is something that more and more fields will see as time goes on. And the success of companies, and countries, will depend increasingly on how they deal with it.

If variation in productivity increases with technology, then the contribution of the most productive individuals will not only be disproportionately large, but will actually grow with time. When you reach the point where 90% of a group's output is created by 1% of its members, you lose big if something (whether Viking raids, or central planning) drags their productivity down to the average.

If we want to get the most out of them, we need to understand these especially productive people. What motivates them? What do they need to do their jobs? How do you recognize them? How do you get them to come and work for you? And then of course there's the question, how do you become one?

More than Money

I know a handful of super-hackers, so I sat down and thought about what they have in common. Their defining quality is probably that they really love to program. Ordinary programmers write code to pay the bills. Great hackers think of it as something they do for fun, and which they're delighted to find people will pay them for.

Great programmers are sometimes said to be indifferent to money. This isn't quite true. It is true that all they really care about is doing interesting work. But if you make enough money, you get to work on whatever you want, and for that reason hackers *are* attracted by the idea of making really large amounts of money. But as long as they still have to show up for work every day, they care more about what they do there than how much they get paid for it.

Economically, this is a fact of the greatest importance, because it means you don't have to pay great hackers anything like what they're worth. A great programmer might be ten or a hundred times as productive as an ordinary one, but he'll

consider himself lucky to get paid three times as much. As I'll explain later, this is partly because great hackers don't know how good they are. But it's also because money is not the main thing they want.

What do hackers want? Like all craftsmen, hackers like good tools. In fact, that's an understatement. Good hackers find it unbearable to use bad tools. They'll simply refuse to work on projects with the wrong infrastructure.

At a startup I once worked for, one of the things pinned up on our bulletin board was an ad from IBM. It was a picture of an AS400, and the headline read, I think, "hackers despise it." [1]

When you decide what infrastructure to use for a project, you're not just making a technical decision. You're also making a social decision, and this may be the more important of the two. For example, if your company wants to write some software, it might seem a prudent choice to write it in Java. But when you choose a language, you're also choosing a community. The programmers you'll be able to hire to work on a Java project won't be as smart as the ones you could get to work on a project written in Python. And the quality of your hackers probably matters more than the language you choose. Though, frankly, the fact that good hackers prefer Python to Java should tell you something about the relative merits of those languages.

Business types prefer the most popular languages because they view languages as standards. They don't want to bet the company on Betamax. The thing about languages, though, is that they're not just standards. If you have to move bits over a network, by all means use TCP/IP. But a programming language isn't just a format. A programming language is a medium of expression.

I've read that Java has just overtaken Cobol as the most popular language. As a standard, you couldn't wish for more. But as a medium of expression, you could do a lot better. Of all the great programmers I can think of, I know of only one who would voluntarily program in Java. And of all the great programmers I can think of who don't work for Sun, on Java, I know of zero.

Great hackers also generally insist on using open source software. Not just because it's better, but because it gives them more control. Good hackers insist on control. This is part of what makes them good hackers: when something's broken, they need to fix it. You want them to feel this way about the software they're writing for you. You shouldn't be surprised when they feel the same way about the operating system.

A couple years ago a venture capitalist friend told me about a new startup he was involved with. It sounded promising. But the next time I talked to him, he said they'd decided to build their software on Windows NT, and had just hired a

very experienced NT developer to be their chief technical officer. When I heard this, I thought, these guys are doomed. One, the CTO couldn't be a first rate hacker, because to become an eminent NT developer he would have had to use NT voluntarily, multiple times, and I couldn't imagine a great hacker doing that; and two, even if he was good, he'd have a hard time hiring anyone good to work for him if the project had to be built on NT. [2]

The Final Frontier

After software, the most important tool to a hacker is probably his office. Big companies think the function of office space is to express rank. But hackers use their offices for more than that: they use their office as a place to think in. And if you're a technology company, their thoughts are your product. So making hackers work in a noisy, distracting environment is like having a paint factory where the air is full of soot.

The cartoon strip Dilbert has a lot to say about cubicles, and with good reason. All the hackers I know despise them. The mere prospect of being interrupted is enough to prevent hackers from working on hard problems. If you want to get real work done in an office with cubicles, you have two options: work at home, or come in early or late or on a weekend, when no one else is there. Don't companies realize this is a sign that something is broken? An office environment is supposed to be something that *helps* you work, not something you work despite.

Companies like Cisco are proud that everyone there has a cubicle, even the CEO. But they're not so advanced as they think; obviously they still view office space as a badge of rank. Note too that Cisco is famous for doing very little product development in house. They get new technology by buying the startups that created it— where presumably the hackers did have somewhere quiet to work.

One big company that understands what hackers need is Microsoft. I once saw a recruiting ad for Microsoft with a big picture of a door. Work for us, the premise was, and we'll give you a place to work where you can actually get work done. And you know, Microsoft is remarkable among big companies in that they are able to develop software in house. Not well, perhaps, but well enough.

If companies want hackers to be productive, they should look at what they do at home. At home, hackers can arrange things themselves so they can get the most done. And when they work at home, hackers don't work in noisy, open spaces; they work in rooms with doors. They work in cosy, neighborhooody places with people around and somewhere to walk when they need to mull something over, instead of in glass boxes set in acres of parking lots. They have a sofa they can take a nap on when they feel tired, instead of sitting in a coma at their desk, pretending to work. There's no crew of people with vacuum cleaners that roars

through every evening during the prime hacking hours. There are no meetings or, God forbid, corporate retreats or team-building exercises. And when you look at what they're doing on that computer, you'll find it reinforces what I said earlier about tools. They may have to use Java and Windows at work, but at home, where they can choose for themselves, you're more likely to find them using Perl and Linux.

Indeed, these statistics about Cobol or Java being the most popular language can be misleading. What we ought to look at, if we want to know what tools are best, is what hackers choose when they can choose freely— that is, in projects of their own. When you ask that question, you find that open source operating systems already have a dominant market share, and the number one language is probably Perl.

Interesting

Along with good tools, hackers want interesting projects. What makes a project interesting? Well, obviously overtly sexy applications like stealth planes or special effects software would be interesting to work on. But any application can be interesting if it poses novel technical challenges. So it's hard to predict which problems hackers will like, because some become interesting only when the people working on them discover a new kind of solution. Before ITA (who wrote the software inside Orbitz), the people working on airline fare searches probably thought it was one of the most boring applications imaginable. But ITA made it interesting by redefining the problem in a more ambitious way.

I think the same thing happened at Google. When Google was founded, the conventional wisdom among the so-called portals was that search was boring and unimportant. But the guys at Google didn't think search was boring, and that's why they do it so well.

This is an area where managers can make a difference. Like a parent saying to a child, I bet you can't clean up your whole room in ten minutes, a good manager can sometimes redefine a problem as a more interesting one. Steve Jobs seems to be particularly good at this, in part simply by having high standards. There were a lot of small, inexpensive computers before the Mac. He redefined the problem as: make one that's beautiful. And that probably drove the developers harder than any carrot or stick could.

They certainly delivered. When the Mac first appeared, you didn't even have to turn it on to know it would be good; you could tell from the case. A few weeks ago I was walking along the street in Cambridge, and in someone's trash I saw what appeared to be a Mac carrying case. I looked inside, and there was a Mac SE. I carried it home and plugged it in, and it booted. The happy Macintosh

face, and then the finder. My God, it was so simple. It was just like... Google.

Hackers like to work for people with high standards. But it's not enough just to be exacting. You have to insist on the right things. Which usually means that you have to be a hacker yourself. I've seen occasional articles about how to manage programmers. Really there should be two articles: one about what to do if you are yourself a programmer, and one about what to do if you're not. And the second could probably be condensed into two words: give up.

The problem is not so much the day to day management. Really good hackers are practically self-managing. The problem is, if you're not a hacker, you can't tell who the good hackers are. A similar problem explains why American cars are so ugly. I call it the *design paradox*. You might think that you could make your products beautiful just by hiring a great designer to design them. But if you yourself don't have good taste, how are you going to recognize a good designer? By definition you can't tell from his portfolio. And you can't go by the awards he's won or the jobs he's had, because in design, as in most fields, those tend to be driven by fashion and schmoozing, with actual ability a distant third. There's no way around it: you can't manage a process intended to produce beautiful things without knowing what beautiful is. American cars are ugly because American car companies are run by people with bad taste.

Many people in this country think of taste as something elusive, or even frivolous. It is neither. To drive design, a manager must be the most demanding user of a company's products. And if you have really good taste, you can, as Steve Jobs does, make satisfying you the kind of problem that good people like to work on.

Nasty Little Problems

It's pretty easy to say what kinds of problems are not interesting: those where instead of solving a few big, clear, problems, you have to solve a lot of nasty little ones. One of the worst kinds of projects is writing an interface to a piece of software that's full of bugs. Another is when you have to customize something for an individual client's complex and ill-defined needs. To hackers these kinds of projects are the death of a thousand cuts.

The distinguishing feature of nasty little problems is that you don't learn anything from them. Writing a compiler is interesting because it teaches you what a compiler is. But writing an interface to a buggy piece of software doesn't teach you anything, because the bugs are random. [3] So it's not just fastidiousness that makes good hackers avoid nasty little problems. It's more a question of self-preservation. Working on nasty little problems makes you stupid. Good hackers avoid it for the same reason models avoid cheeseburgers.

Of course some problems inherently have this character. And because of supply

and demand, they pay especially well. So a company that found a way to get great hackers to work on tedious problems would be very successful. How would you do it?

One place this happens is in startups. At our startup we had Robert Morris working as a system administrator. That's like having the Rolling Stones play at a bar mitzvah. You can't hire that kind of talent. But people will do any amount of drudgery for companies of which they're the founders. [4]

Bigger companies solve the problem by partitioning the company. They get smart people to work for them by establishing a separate R&D department where employees don't have to work directly on customers' nasty little problems. [5] In this model, the research department functions like a mine. They produce new ideas; maybe the rest of the company will be able to use them.

You may not have to go to this extreme. Bottom-up programming suggests another way to partition the company: have the smart people work as toolmakers. If your company makes software to do x, have one group that builds tools for writing software of that type, and another that uses these tools to write the applications. This way you might be able to get smart people to write 99% of your code, but still keep them almost as insulated from users as they would be in a traditional research department. The toolmakers would have users, but they'd only be the company's own developers. [6]

If Microsoft used this approach, their software wouldn't be so full of security holes, because the less smart people writing the actual applications wouldn't be doing low-level stuff like allocating memory. Instead of writing Word directly in C, they'd be plugging together big Lego blocks of Word-language. (Duplo, I believe, is the technical term.)

Clumping

Along with interesting problems, what good hackers like is other good hackers. Great hackers tend to clump together—sometimes spectacularly so, as at Xerox Parc. So you won't attract good hackers in linear proportion to how good an environment you create for them. The tendency to clump means it's more like the square of the environment. So it's winner take all. At any given time, there are only about ten or twenty places where hackers most want to work, and if you aren't one of them, you won't just have fewer great hackers, you'll have zero.

Having great hackers is not, by itself, enough to make a company successful. It works well for Google and ITA, which are two of the hot spots right now, but it didn't help Thinking Machines or Xerox. Sun had a good run for a while, but their business model is a down elevator. In that situation, even the best hackers can't save you.

I think, though, that all other things being equal, a company that can attract great hackers will have a huge advantage. There are people who would disagree with this. When we were making the rounds of venture capital firms in the 1990s, several told us that software companies didn't win by writing great software, but through brand, and dominating channels, and doing the right deals.

They really seemed to believe this, and I think I know why. I think what a lot of VCs are looking for, at least unconsciously, is the next Microsoft. And of course if Microsoft is your model, you shouldn't be looking for companies that hope to win by writing great software. But VCs are mistaken to look for the next Microsoft, because no startup can be the next Microsoft unless some other company is prepared to bend over at just the right moment and be the next IBM.

It's a mistake to use Microsoft as a model, because their whole culture derives from that one lucky break. Microsoft is a bad data point. If you throw them out, you find that good products do tend to win in the market. What VCs should be looking for is the next Apple, or the next Google.

I think Bill Gates knows this. What worries him about Google is not the power of their brand, but the fact that they have better hackers. [7]

Recognition

So who are the great hackers? How do you know when you meet one? That turns out to be very hard. Even hackers can't tell. I'm pretty sure now that my friend Trevor Blackwell is a great hacker. You may have read on Slashdot how he made his own Segway. The remarkable thing about this project was that he wrote all the software in one day (in Python, incidentally).

For Trevor, that's par for the course. But when I first met him, I thought he was a complete idiot. He was standing in Robert Morris's office babbling at him about something or other, and I remember standing behind him making frantic gestures at Robert to shoo this nut out of his office so we could go to lunch. Robert says he misjudged Trevor at first too. Apparently when Robert first met him, Trevor had just begun a new scheme that involved writing down everything about every aspect of his life on a stack of index cards, which he carried with him everywhere. He'd also just arrived from Canada, and had a strong Canadian accent and a mullet.

The problem is compounded by the fact that hackers, despite their reputation for social obliviousness, sometimes put a good deal of effort into seeming smart. When I was in grad school I used to hang around the MIT AI Lab occasionally. It was kind of intimidating at first. Everyone there spoke so fast. But after a while I learned the trick of speaking fast. You don't have to think any faster; just use twice as many words to say everything.

With this amount of noise in the signal, it's hard to tell good hackers when you meet them. I can't tell, even now. You also can't tell from their resumes. It seems like the only way to judge a hacker is to work with him on something.

And this is the reason that high-tech areas only happen around universities. The active ingredient here is not so much the professors as the students. Startups grow up around universities because universities bring together promising young people and make them work on the same projects. The smart ones learn who the other smart ones are, and together they cook up new projects of their own.

Because you can't tell a great hacker except by working with him, hackers themselves can't tell how good they are. This is true to a degree in most fields. I've found that people who are great at something are not so much convinced of their own greatness as mystified at why everyone else seems so incompetent.

But it's particularly hard for hackers to know how good they are, because it's hard to compare their work. This is easier in most other fields. In the hundred meters, you know in 10 seconds who's fastest. Even in math there seems to be a general consensus about which problems are hard to solve, and what constitutes a good solution. But hacking is like writing. Who can say which of two novels is better? Certainly not the authors.

With hackers, at least, other hackers can tell. That's because, unlike novelists, hackers collaborate on projects. When you get to hit a few difficult problems over the net at someone, you learn pretty quickly how hard they hit them back. But hackers can't watch themselves at work. So if you ask a great hacker how good he is, he's almost certain to reply, I don't know. He's not just being modest. He really doesn't know.

And none of us know, except about people we've actually worked with. Which puts us in a weird situation: we don't know who our heroes should be. The hackers who become famous tend to become famous by random accidents of PR. Occasionally I need to give an example of a great hacker, and I never know who to use. The first names that come to mind always tend to be people I know personally, but it seems lame to use them. So, I think, maybe I should say Richard Stallman, or Linus Torvalds, or Alan Kay, or someone famous like that. But I have no idea if these guys are great hackers. I've never worked with them on anything.

If there is a Michael Jordan of hacking, no one knows, including him.

Cultivation

Finally, the question the hackers have all been wondering about: how do you become a great hacker? I don't know if it's possible to make yourself into one.

But it's certainly possible to do things that make you stupid, and if you can make yourself stupid, you can probably make yourself smart too.

The key to being a good hacker may be to work on what you like. When I think about the great hackers I know, one thing they have in common is the extreme difficulty of making them work on anything they don't want to. I don't know if this is cause or effect; it may be both.

To do something well you have to love it. So to the extent you can preserve hacking as something you love, you're likely to do it well. Try to keep the sense of wonder you had about programming at age 14. If you're worried that your current job is rotting your brain, it probably is.

The best hackers tend to be smart, of course, but that's true in a lot of fields. Is there some quality that's unique to hackers? I asked some friends, and the number one thing they mentioned was curiosity. I'd always supposed that all smart people were curious— that curiosity was simply the first derivative of knowledge. But apparently hackers are particularly curious, especially about how things work. That makes sense, because programs are in effect giant descriptions of how things work.

Several friends mentioned hackers' ability to concentrate— their ability, as one put it, to “tune out everything outside their own heads.” I've certainly noticed this. And I've heard several hackers say that after drinking even half a beer they can't program at all. So maybe hacking does require some special ability to focus. Perhaps great hackers can load a large amount of context into their head, so that when they look at a line of code, they see not just that line but the whole program around it. John McPhee wrote that Bill Bradley's success as a basketball player was due partly to his extraordinary peripheral vision. “Perfect” eyesight means about 47 degrees of vertical peripheral vision. Bill Bradley had 70; he could see the basket when he was looking at the floor. Maybe great hackers have some similar inborn ability. (I cheat by using a very dense language, which shrinks the court.)

This could explain the disconnect over cubicles. Maybe the people in charge of facilities, not having any concentration to shatter, have no idea that working in a cubicle feels to a hacker like having one's brain in a blender. (Whereas Bill, if the rumors of autism are true, knows all too well.)

One difference I've noticed between great hackers and smart people in general is that hackers are more politically incorrect. To the extent there is a secret handshake among good hackers, it's when they know one another well enough to express opinions that would get them stoned to death by the general public. And I can see why political incorrectness would be a useful quality in programming.

Programs are very complex and, at least in the hands of good programmers, very fluid. In such situations it's helpful to have a habit of questioning assumptions.

Can you cultivate these qualities? I don't know. But you can at least not repress them. So here is my best shot at a recipe. If it is possible to make yourself into a great hacker, the way to do it may be to make the following deal with yourself: you never have to work on boring projects (unless your family will starve otherwise), and in return, you'll never allow yourself to do a half-assed job. All the great hackers I know seem to have made that deal, though perhaps none of them had any choice in the matter.

Notes

[1] In fairness, I have to say that IBM makes decent hardware. I wrote this on an IBM laptop.

[2] They did turn out to be doomed. They shut down a few months later.

[3] I think this is what people mean when they talk about the "meaning of life." On the face of it, this seems an odd idea. Life isn't an expression; how could it have meaning? But it can have a quality that feels a lot like meaning. In a project like a compiler, you have to solve a lot of problems, but the problems all fall into a pattern, as in a signal. Whereas when the problems you have to solve are random, they seem like noise.

[4] Einstein at one point worked designing refrigerators. (He had equity.)

[5] It's hard to say exactly what constitutes research in the computer world, but as a first approximation, it's software that doesn't have users.

I don't think it's publication that makes the best hackers want to work in research departments. I think it's mainly not having to have a three hour meeting with a product manager about problems integrating the Korean version of Word 13.27 with the talking paperclip.

[6] Something similar has been happening for a long time in the construction industry. When you had a house built a couple hundred years ago, the local builders built everything in it. But increasingly what builders do is assemble components designed and manufactured by someone else. This has, like the arrival of desktop publishing, given people the freedom to experiment in disastrous ways, but it is certainly more efficient.

[7] Google is much more dangerous to Microsoft than Netscape was. Probably more dangerous than any other company has ever been. Not least because they're determined to fight. On their job listing page, they say that one of their "core values" is "Don't be evil." From a company selling soybean oil or mining

equipment, such a statement would merely be eccentric. But I think all of us in the computer world recognize who that is a declaration of war on.

Thanks to Jessica Livingston, Robert Morris, and Sarah Harlin for reading earlier versions of this talk.

— | | Audio of talk

The Age of the Essay

September 2004

Remember the essays you had to write in high school? Topic sentence, introductory paragraph, supporting paragraphs, conclusion. The conclusion being, say, that Ahab in *Moby Dick* was a Christ-like figure.

Oy. So I'm going to try to give the other side of the story: what an essay really is, and how you write one. Or at least, how I write one.

Mods

The most obvious difference between real essays and the things one has to write in school is that real essays are not exclusively about English literature. Certainly schools should teach students how to write. But due to a series of historical accidents the teaching of writing has gotten mixed together with the study of literature. And so all over the country students are writing not about how a baseball team with a small budget might compete with the Yankees, or the role of color in fashion, or what constitutes a good dessert, but about symbolism in Dickens.

With the result that writing is made to seem boring and pointless. Who cares about symbolism in Dickens? Dickens himself would be more interested in an essay about color or baseball.

How did things get this way? To answer that we have to go back almost a thousand years. Around 1100, Europe at last began to catch its breath after centuries of chaos, and once they had the luxury of curiosity they rediscovered what we call "the classics." The effect was rather as if we were visited by beings from another solar system. These earlier civilizations were so much more sophisticated that for the next several centuries the main work of European scholars, in almost every field, was to assimilate what they knew.

During this period the study of ancient texts acquired great prestige. It seemed the essence of what scholars did. As European scholarship gained momentum

it became less and less important; by 1350 someone who wanted to learn about science could find better teachers than Aristotle in his own era. [1] But schools change slower than scholarship. In the 19th century the study of ancient texts was still the backbone of the curriculum.

The time was then ripe for the question: if the study of ancient texts is a valid field for scholarship, why not modern texts? The answer, of course, is that the original *raison d'être* of classical scholarship was a kind of intellectual archaeology that does not need to be done in the case of contemporary authors. But for obvious reasons no one wanted to give that answer. The archaeological work being mostly done, it implied that those studying the classics were, if not wasting their time, at least working on problems of minor importance.

And so began the study of modern literature. There was a good deal of resistance at first. The first courses in English literature seem to have been offered by the newer colleges, particularly American ones. Dartmouth, the University of Vermont, Amherst, and University College, London taught English literature in the 1820s. But Harvard didn't have a professor of English literature until 1876, and Oxford not till 1885. (Oxford had a chair of Chinese before it had one of English.) [2]

What tipped the scales, at least in the US, seems to have been the idea that professors should do research as well as teach. This idea (along with the PhD, the department, and indeed the whole concept of the modern university) was imported from Germany in the late 19th century. Beginning at Johns Hopkins in 1876, the new model spread rapidly.

Writing was one of the casualties. Colleges had long taught English composition. But how do you do research on composition? The professors who taught math could be required to do original math, the professors who taught history could be required to write scholarly articles about history, but what about the professors who taught rhetoric or composition? What should they do research on? The closest thing seemed to be English literature. [3]

And so in the late 19th century the teaching of writing was inherited by English professors. This had two drawbacks: (a) an expert on literature need not himself be a good writer, any more than an art historian has to be a good painter, and (b) the subject of writing now tends to be literature, since that's what the professor is interested in.

High schools imitate universities. The seeds of our miserable high school experiences were sown in 1892, when the National Education Association "formally recommended that literature and composition be unified in the high school course." [4] The writing component of the 3 Rs then morphed into English, with

the bizarre consequence that high school students now had to write about English literature— to write, without even realizing it, imitations of whatever English professors had been publishing in their journals a few decades before.

It's no wonder if this seems to the student a pointless exercise, because we're now three steps removed from real work: the students are imitating English professors, who are imitating classical scholars, who are merely the inheritors of a tradition growing out of what was, 700 years ago, fascinating and urgently needed work.

No Defense

The other big difference between a real essay and the things they make you write in school is that a real essay doesn't take a position and then defend it. That principle, like the idea that we ought to be writing about literature, turns out to be another intellectual hangover of long forgotten origins.

It's often mistakenly believed that medieval universities were mostly seminaries. In fact they were more law schools. And at least in our tradition lawyers are advocates, trained to take either side of an argument and make as good a case for it as they can. Whether cause or effect, this spirit pervaded early universities. The study of rhetoric, the art of arguing persuasively, was a third of the undergraduate curriculum. [5] And after the lecture the most common form of discussion was the disputation. This is at least nominally preserved in our present-day thesis defense: most people treat the words thesis and dissertation as interchangeable, but originally, at least, a thesis was a position one took and the dissertation was the argument by which one defended it.

Defending a position may be a necessary evil in a legal dispute, but it's not the best way to get at the truth, as I think lawyers would be the first to admit. It's not just that you miss subtleties this way. The real problem is that you can't change the question.

And yet this principle is built into the very structure of the things they teach you to write in high school. The topic sentence is your thesis, chosen in advance, the supporting paragraphs the blows you strike in the conflict, and the conclusion—uh, what is the conclusion? I was never sure about that in high school. It seemed as if we were just supposed to restate what we said in the first paragraph, but in different enough words that no one could tell. Why bother? But when you understand the origins of this sort of "essay," you can see where the conclusion comes from. It's the concluding remarks to the jury.

Good writing should be convincing, certainly, but it should be convincing because you got the right answers, not because you did a good job of arguing. When I give a draft of an essay to friends, there are two things I want to know: which

parts bore them, and which seem unconvincing. The boring bits can usually be fixed by cutting. But I don't try to fix the unconvincing bits by arguing more cleverly. I need to talk the matter over.

At the very least I must have explained something badly. In that case, in the course of the conversation I'll be forced to come up with a clearer explanation, which I can just incorporate in the essay. More often than not I have to change what I was saying as well. But the aim is never to be convincing per se. As the reader gets smarter, convincing and true become identical, so if I can convince smart readers I must be near the truth.

The sort of writing that attempts to persuade may be a valid (or at least inevitable) form, but it's historically inaccurate to call it an essay. An essay is something else.

Trying

To understand what a real essay is, we have to reach back into history again, though this time not so far. To Michel de Montaigne, who in 1580 published a book of what he called "essais." He was doing something quite different from what lawyers do, and the difference is embodied in the name. *Essayer* is the French verb meaning "to try" and an *essai* is an attempt. An essay is something you write to try to figure something out.

Figure out what? You don't know yet. And so you can't begin with a thesis, because you don't have one, and may never have one. An essay doesn't begin with a statement, but with a question. In a real essay, you don't take a position and defend it. You notice a door that's ajar, and you open it and walk in to see what's inside.

If all you want to do is figure things out, why do you need to write anything, though? Why not just sit and think? Well, there precisely is Montaigne's great discovery. Expressing ideas helps to form them. Indeed, helps is far too weak a word. Most of what ends up in my essays I only thought of when I sat down to write them. That's why I write them.

In the things you write in school you are, in theory, merely explaining yourself to the reader. In a real essay you're writing for yourself. You're thinking out loud.

But not quite. Just as inviting people over forces you to clean up your apartment, writing something that other people will read forces you to think well. So it does matter to have an audience. The things I've written just for myself are no good. They tend to peter out. When I run into difficulties, I find I conclude with a few vague questions and then drift off to get a cup of tea.

Many published essays peter out in the same way. Particularly the sort written

by the staff writers of newsmagazines. Outside writers tend to supply editorials of the defend-a-position variety, which make a beeline toward a rousing (and foreordained) conclusion. But the staff writers feel obliged to write something “balanced.” Since they’re writing for a popular magazine, they start with the most radioactively controversial questions, from which—because they’re writing for a popular magazine—they then proceed to recoil in terror. Abortion, for or against? This group says one thing. That group says another. One thing is certain: the question is a complex one. (But don’t get mad at us. We didn’t draw any conclusions.)

The River

Questions aren’t enough. An essay has to come up with answers. They don’t always, of course. Sometimes you start with a promising question and get nowhere. But those you don’t publish. Those are like experiments that get inconclusive results. An essay you publish ought to tell the reader something he didn’t already know.

But *what* you tell him doesn’t matter, so long as it’s interesting. I’m sometimes accused of meandering. In defend-a-position writing that would be a flaw. There you’re not concerned with truth. You already know where you’re going, and you want to go straight there, blustering through obstacles, and hand-waving your way across swampy ground. But that’s not what you’re trying to do in an essay. An essay is supposed to be a search for truth. It would be suspicious if it didn’t meander.

The Meander (aka Menderes) is a river in Turkey. As you might expect, it winds all over the place. But it doesn’t do this out of frivolity. The path it has discovered is the most economical route to the sea. [6]

The river’s algorithm is simple. At each step, flow down. For the essayist this translates to: flow interesting. Of all the places to go next, choose the most interesting. One can’t have quite as little foresight as a river. I always know generally what I want to write about. But not the specific conclusions I want to reach; from paragraph to paragraph I let the ideas take their course.

This doesn’t always work. Sometimes, like a river, one runs up against a wall. Then I do the same thing the river does: backtrack. At one point in this essay I found that after following a certain thread I ran out of ideas. I had to go back seven paragraphs and start over in another direction.

Fundamentally an essay is a train of thought— but a cleaned-up train of thought, as dialogue is cleaned-up conversation. Real thought, like real conversation, is full of false starts. It would be exhausting to read. You need to cut and fill to

emphasize the central thread, like an illustrator inking over a pencil drawing. But don't change so much that you lose the spontaneity of the original.

Err on the side of the river. An essay is not a reference work. It's not something you read looking for a specific answer, and feel cheated if you don't find it. I'd much rather read an essay that went off in an unexpected but interesting direction than one that plodded dutifully along a prescribed course.

Surprise

So what's interesting? For me, interesting means surprise. Interfaces, as Geoffrey James has said, should follow the principle of least astonishment. A button that looks like it will make a machine stop should make it stop, not speed up. Essays should do the opposite. Essays should aim for maximum surprise.

I was afraid of flying for a long time and could only travel vicariously. When friends came back from faraway places, it wasn't just out of politeness that I asked what they saw. I really wanted to know. And I found the best way to get information out of them was to ask what surprised them. How was the place different from what they expected? This is an extremely useful question. You can ask it of the most unobservant people, and it will extract information they didn't even know they were recording.

Surprises are things that you not only didn't know, but that contradict things you thought you knew. And so they're the most valuable sort of fact you can get. They're like a food that's not merely healthy, but counteracts the unhealthy effects of things you've already eaten.

How do you find surprises? Well, therein lies half the work of essay writing. (The other half is expressing yourself well.) The trick is to use yourself as a proxy for the reader. You should only write about things you've thought about a lot. And anything you come across that surprises you, who've thought about the topic a lot, will probably surprise most readers.

For example, in a recent essay I pointed out that because you can only judge computer programmers by working with them, no one knows who the best programmers are overall. I didn't realize this when I began that essay, and even now I find it kind of weird. That's what you're looking for.

So if you want to write essays, you need two ingredients: a few topics you've thought about a lot, and some ability to ferret out the unexpected.

What should you think about? My guess is that it doesn't matter— that anything can be interesting if you get deeply enough into it. One possible exception might be things that have deliberately had all the variation sucked out of them, like working in fast food. In retrospect, was there anything interesting about

working at Baskin-Robbins? Well, it was interesting how important color was to the customers. Kids a certain age would point into the case and say that they wanted yellow. Did they want French Vanilla or Lemon? They would just look at you blankly. They wanted yellow. And then there was the mystery of why the perennial favorite Pralines 'n' Cream was so appealing. (I think now it was the salt.) And the difference in the way fathers and mothers bought ice cream for their kids: the fathers like benevolent kings bestowing largesse, the mothers harried, giving in to pressure. So, yes, there does seem to be some material even in fast food.

I didn't notice those things at the time, though. At sixteen I was about as observant as a lump of rock. I can see more now in the fragments of memory I preserve of that age than I could see at the time from having it all happening live, right in front of me.

Observation

So the ability to ferret out the unexpected must not merely be an inborn one. It must be something you can learn. How do you learn it?

To some extent it's like learning history. When you first read history, it's just a whirl of names and dates. Nothing seems to stick. But the more you learn, the more hooks you have for new facts to stick onto— which means you accumulate knowledge at an exponential rate. Once you remember that Normans conquered England in 1066, it will catch your attention when you hear that other Normans conquered southern Italy at about the same time. Which will make you wonder about Normandy, and take note when a third book mentions that Normans were not, like most of what is now called France, tribes that flowed in as the Roman empire collapsed, but Vikings (norman = north man) who arrived four centuries later in 911. Which makes it easier to remember that Dublin was also established by Vikings in the 840s. Etc, etc squared.

Collecting surprises is a similar process. The more anomalies you've seen, the more easily you'll notice new ones. Which means, oddly enough, that as you grow older, life should become more and more surprising. When I was a kid, I used to think adults had it all figured out. I had it backwards. Kids are the ones who have it all figured out. They're just mistaken.

When it comes to surprises, the rich get richer. But (as with wealth) there may be habits of mind that will help the process along. It's good to have a habit of asking questions, especially questions beginning with Why. But not in the random way that three year olds ask why. There are an infinite number of questions. How do you find the fruitful ones?

I find it especially useful to ask why about things that seem wrong. For example,

why should there be a connection between humor and misfortune? Why do we find it funny when a character, even one we like, slips on a banana peel? There's a whole essay's worth of surprises there for sure.

If you want to notice things that seem wrong, you'll find a degree of skepticism helpful. I take it as an axiom that we're only achieving 1% of what we could. This helps counteract the rule that gets beaten into our heads as children: that things are the way they are because that is how things have to be. For example, everyone I've talked to while writing this essay felt the same about English classes— that the whole process seemed pointless. But none of us had the balls at the time to hypothesize that it was, in fact, all a mistake. We all thought there was just something we weren't getting.

I have a hunch you want to pay attention not just to things that seem wrong, but things that seem wrong in a humorous way. I'm always pleased when I see someone laugh as they read a draft of an essay. But why should I be? I'm aiming for good ideas. Why should good ideas be funny? The connection may be surprise. Surprises make us laugh, and surprises are what one wants to deliver.

I write down things that surprise me in notebooks. I never actually get around to reading them and using what I've written, but I do tend to reproduce the same thoughts later. So the main value of notebooks may be what writing things down leaves in your head.

People trying to be cool will find themselves at a disadvantage when collecting surprises. To be surprised is to be mistaken. And the essence of cool, as any fourteen year old could tell you, is *nil admirari*. When you're mistaken, don't dwell on it; just act like nothing's wrong and maybe no one will notice.

One of the keys to coolness is to avoid situations where inexperience may make you look foolish. If you want to find surprises you should do the opposite. Study lots of different things, because some of the most interesting surprises are unexpected connections between different fields. For example, jam, bacon, pickles, and cheese, which are among the most pleasing of foods, were all originally intended as methods of preservation. And so were books and paintings.

Whatever you study, include history— but social and economic history, not political history. History seems to me so important that it's misleading to treat it as a mere field of study. Another way to describe it is *all the data we have so far*.

Among other things, studying history gives one confidence that there are good ideas waiting to be discovered right under our noses. Swords evolved during the Bronze Age out of daggers, which (like their flint predecessors) had a hilt separate from the blade. Because swords are longer the hilts kept breaking off.

But it took five hundred years before someone thought of casting hilt and blade as one piece.

Disobedience

Above all, make a habit of paying attention to things you're not supposed to, either because they're "inappropriate," or not important, or not what you're supposed to be working on. If you're curious about something, trust your instincts. Follow the threads that attract your attention. If there's something you're really interested in, you'll find they have an uncanny way of leading back to it anyway, just as the conversation of people who are especially proud of something always tends to lead back to it.

For example, I've always been fascinated by comb-overs, especially the extreme sort that make a man look as if he's wearing a beret made of his own hair. Surely this is a lowly sort of thing to be interested in— the sort of superficial quizzing best left to teenage girls. And yet there is something underneath. The key question, I realized, is how does the comber-over not see how odd he looks? And the answer is that he got to look that way *incrementally*. What began as combing his hair a little carefully over a thin patch has gradually, over 20 years, grown into a monstrosity. Gradualness is very powerful. And that power can be used for constructive purposes too: just as you can trick yourself into looking like a freak, you can trick yourself into creating something so grand that you would never have dared to *plan* such a thing. Indeed, this is just how most good software gets created. You start by writing a stripped-down kernel (how hard can it be?) and gradually it grows into a complete operating system. Hence the next leap: could you do the same thing in painting, or in a novel?

See what you can extract from a frivolous question? If there's one piece of advice I would give about writing essays, it would be: don't do as you're told. Don't believe what you're supposed to. Don't write the essay readers expect; one learns nothing from what one expects. And don't write the way they taught you to in school.

The most important sort of disobedience is to write essays at all. Fortunately, this sort of disobedience shows signs of becoming rampant. It used to be that only a tiny number of officially approved writers were allowed to write essays. Magazines published few of them, and judged them less by what they said than who wrote them; a magazine might publish a story by an unknown writer if it was good enough, but if they published an essay on x it had to be by someone who was at least forty and whose job title had x in it. Which is a problem, because there are a lot of things insiders can't say precisely because they're insiders.

The Internet is changing that. Anyone can publish an essay on the Web, and it

gets judged, as any writing should, by what it says, not who wrote it. Who are you to write about x? You are whatever you wrote.

Popular magazines made the period between the spread of literacy and the arrival of TV the golden age of the short story. The Web may well make this the golden age of the essay. And that's certainly not something I realized when I started writing this.

Notes

[1] I'm thinking of Oresme (c. 1323-82). But it's hard to pick a date, because there was a sudden drop-off in scholarship just as Europeans finished assimilating classical science. The cause may have been the plague of 1347; the trend in scientific progress matches the population curve.

[2] Parker, William R. "Where Do College English Departments Come From?" *College English* 28 (1966-67), pp. 339-351. Reprinted in Gray, Donald J. (ed). *The Department of English at Indiana University Bloomington 1868-1970*. Indiana University Publications.

Daniels, Robert V. *The University of Vermont: The First Two Hundred Years*. University of Vermont, 1991.

Mueller, Friedrich M. Letter to the *Pall Mall Gazette*. 1886/87. Reprinted in Bacon, Alan (ed). *The Nineteenth-Century History of English Studies*. Ashgate, 1998.

[3] I'm compressing the story a bit. At first literature took a back seat to philology, which (a) seemed more serious and (b) was popular in Germany, where many of the leading scholars of that generation had been trained.

In some cases the writing teachers were transformed *in situ* into English professors. Francis James Child, who had been Boylston Professor of Rhetoric at Harvard since 1851, became in 1876 the university's first professor of English.

[4] Parker, *op. cit.*, p. 25.

[5] The undergraduate curriculum or *trivium* (whence "trivial") consisted of Latin grammar, rhetoric, and logic. Candidates for masters' degrees went on to study the *quadrivium* of arithmetic, geometry, music, and astronomy. Together these were the seven liberal arts.

The study of rhetoric was inherited directly from Rome, where it was considered the most important subject. It would not be far from the truth to say that education in the classical world meant training landowners' sons to speak well enough to defend their interests in political and legal disputes.

[6] Trevor Blackwell points out that this isn't strictly true, because the outside edges of curves erode faster.

Thanks to Ken Anderson, Trevor Blackwell, Sarah Harlin, Jessica Livingston, Jackie McDonough, and Robert Morris for reading drafts of this.

Part Two — The Startup Playbook

2005–2008

Do what you love doesn't mean, do what you would like to do most this second.

— Paul Graham, “How to Do What You Love”

What You'll Wish You'd Known

January 2005

(I wrote this talk for a high school. I never actually gave it, because the school authorities vetoed the plan to invite me.)

When I said I was speaking at a high school, my friends were curious. What will you say to high school students? So I asked them, what do you wish someone had told you in high school? Their answers were remarkably similar. So I'm going to tell you what we all wish someone had told us.

I'll start by telling you something you don't have to know in high school: what you want to do with your life. People are always asking you this, so you think you're supposed to have an answer. But adults ask this mainly as a conversation starter. They want to know what sort of person you are, and this question is just to get you talking. They ask it the way you might poke a hermit crab in a tide pool, to see what it does.

If I were back in high school and someone asked about my plans, I'd say that my first priority was to learn what the options were. You don't need to be in a rush to choose your life's work. What you need to do is discover what you like. You have to work on stuff you like if you want to be good at what you do.

It might seem that nothing would be easier than deciding what you like, but it turns out to be hard, partly because it's hard to get an accurate picture of most jobs. Being a doctor is not the way it's portrayed on TV. Fortunately you can also watch real doctors, by volunteering in hospitals. [1]

But there are other jobs you can't learn about, because no one is doing them yet. Most of the work I've done in the last ten years didn't exist when I was in high school. The world changes fast, and the rate at which it changes is itself speeding up. In such a world it's not a good idea to have fixed plans.

And yet every May, speakers all over the country fire up the Standard Graduation Speech, the theme of which is: don't give up on your dreams. I know what they mean, but this is a bad way to put it, because it implies you're supposed to be bound by some plan you made early on. The computer world has a name for this: premature optimization. And it is synonymous with disaster. These speakers would do better to say simply, don't give up.

What they really mean is, don't get demoralized. Don't think that you can't do what other people can. And I agree you shouldn't underestimate your potential. People who've done great things tend to seem as if they were a race apart. And most biographies only exaggerate this illusion, partly due to the worshipful attitude biographers inevitably sink into, and partly because, knowing how the story ends, they can't help streamlining the plot till it seems like the subject's life was a matter of destiny, the mere unfolding of some innate genius. In fact I suspect if you had the sixteen year old Shakespeare or Einstein in school with you, they'd seem impressive, but not totally unlike your other friends.

Which is an uncomfortable thought. If they were just like us, then they had to work very hard to do what they did. And that's one reason we like to believe in genius. It gives us an excuse for being lazy. If these guys were able to do what they did only because of some magic Shakespeareanness or Einsteininess, then it's not our fault if we can't do something as good.

I'm not saying there's no such thing as genius. But if you're trying to choose between two theories and one gives you an excuse for being lazy, the other one is probably right.

So far we've cut the Standard Graduation Speech down from "don't give up on your dreams" to "what someone else can do, you can do." But it needs to be cut still further. There is *some* variation in natural ability. Most people overestimate its role, but it does exist. If I were talking to a guy four feet tall whose ambition was to play in the NBA, I'd feel pretty stupid saying, you can do anything if you really try. [2]

We need to cut the Standard Graduation Speech down to, "what someone else with your abilities can do, you can do; and don't underestimate your abilities." But as so often happens, the closer you get to the truth, the messier your sentence gets. We've taken a nice, neat (but wrong) slogan, and churned it up like a mud puddle. It doesn't make a very good speech anymore. But worse still, it doesn't tell you what to do anymore. Someone with your abilities? What are your abilities?

Upwind

I think the solution is to work in the other direction. Instead of working back

from a goal, work forward from promising situations. This is what most successful people actually do anyway.

In the graduation-speech approach, you decide where you want to be in twenty years, and then ask: what should I do now to get there? I propose instead that you don't commit to anything in the future, but just look at the options available now, and choose those that will give you the most promising range of options afterward.

It's not so important what you work on, so long as you're not wasting your time. Work on things that interest you and increase your options, and worry later about which you'll take.

Suppose you're a college freshman deciding whether to major in math or economics. Well, math will give you more options: you can go into almost any field from math. If you major in math it will be easy to get into grad school in economics, but if you major in economics it will be hard to get into grad school in math.

Flying a glider is a good metaphor here. Because a glider doesn't have an engine, you can't fly into the wind without losing a lot of altitude. If you let yourself get far downwind of good places to land, your options narrow uncomfortably. As a rule you want to stay upwind. So I propose that as a replacement for "don't give up on your dreams." Stay upwind.

How do you do that, though? Even if math is upwind of economics, how are you supposed to know that as a high school student?

Well, you don't, and that's what you need to find out. Look for smart people and hard problems. Smart people tend to clump together, and if you can find such a clump, it's probably worthwhile to join it. But it's not straightforward to find these, because there is a lot of faking going on.

To a newly arrived undergraduate, all university departments look much the same. The professors all seem forbiddingly intellectual and publish papers unintelligible to outsiders. But while in some fields the papers are unintelligible because they're full of hard ideas, in others they're deliberately written in an obscure way to seem as if they're saying something important. This may seem a scandalous proposition, but it has been experimentally verified, in the famous *Social Text* affair. Suspecting that the papers published by literary theorists were often just intellectual-sounding nonsense, a physicist deliberately wrote a paper full of intellectual-sounding nonsense, and submitted it to a literary theory journal, which published it.

The best protection is always to be working on hard problems. Writing novels is hard. Reading novels isn't. Hard means worry: if you're not worrying that

something you're making will come out badly, or that you won't be able to understand something you're studying, then it isn't hard enough. There has to be suspense.

Well, this seems a grim view of the world, you may think. What I'm telling you is that you should worry? Yes, but it's not as bad as it sounds. It's exhilarating to overcome worries. You don't see faces much happier than people winning gold medals. And you know why they're so happy? Relief.

I'm not saying this is the only way to be happy. Just that some kinds of worry are not as bad as they sound.

Ambition

In practice, "stay upwind" reduces to "work on hard problems." And you can start today. I wish I'd grasped that in high school.

Most people like to be good at what they do. In the so-called real world this need is a powerful force. But high school students rarely benefit from it, because they're given a fake thing to do. When I was in high school, I let myself believe that my job was to be a high school student. And so I let my need to be good at what I did be satisfied by merely doing well in school.

If you'd asked me in high school what the difference was between high school kids and adults, I'd have said it was that adults had to earn a living. Wrong. It's that adults take responsibility for themselves. Making a living is only a small part of it. Far more important is to take intellectual responsibility for oneself.

If I had to go through high school again, I'd treat it like a day job. I don't mean that I'd slack in school. Working at something as a day job doesn't mean doing it badly. It means not being defined by it. I mean I wouldn't think of myself as a high school student, just as a musician with a day job as a waiter doesn't think of himself as a waiter. [3] And when I wasn't working at my day job I'd start trying to do real work.

When I ask people what they regret most about high school, they nearly all say the same thing: that they wasted so much time. If you're wondering what you're doing now that you'll regret most later, that's probably it. [4]

Some people say this is inevitable — that high school students aren't capable of getting anything done yet. But I don't think this is true. And the proof is that you're bored. You probably weren't bored when you were eight. When you're eight it's called "playing" instead of "hanging out," but it's the same thing. And when I was eight, I was rarely bored. Give me a back yard and a few other kids and I could play all day.

The reason this got stale in middle school and high school, I now realize, is that I was ready for something else. Childhood was getting old.

I'm not saying you shouldn't hang out with your friends — that you should all become humorless little robots who do nothing but work. Hanging out with friends is like chocolate cake. You enjoy it more if you eat it occasionally than if you eat nothing but chocolate cake for every meal. No matter how much you like chocolate cake, you'll be pretty queasy after the third meal of it. And that's what the malaise one feels in high school is: mental queasiness. [5]

You may be thinking, we have to do more than get good grades. We have to have *extracurricular activities*. But you know perfectly well how bogus most of these are. Collecting donations for a charity is an admirable thing to do, but it's not *hard*. It's not getting something done. What I mean by getting something done is learning how to write well, or how to program computers, or what life was really like in preindustrial societies, or how to draw the human face from life. This sort of thing rarely translates into a line item on a college application.

Corruption

It's dangerous to design your life around getting into college, because the people you have to impress to get into college are not a very discerning audience. At most colleges, it's not the professors who decide whether you get in, but admissions officers, and they are nowhere near as smart. They're the NCOs of the intellectual world. They can't tell how smart you are. The mere existence of prep schools is proof of that.

Few parents would pay so much for their kids to go to a school that didn't improve their admissions prospects. Prep schools openly say this is one of their aims. But what that means, if you stop to think about it, is that they can hack the admissions process: that they can take the very same kid and make him seem a more appealing candidate than he would if he went to the local public school. [6]

Right now most of you feel your job in life is to be a promising college applicant. But that means you're designing your life to satisfy a process so mindless that there's a whole industry devoted to subverting it. No wonder you become cynical. The malaise you feel is the same that a producer of reality TV shows or a tobacco industry executive feels. And you don't even get paid a lot.

So what do you do? What you should not do is rebel. That's what I did, and it was a mistake. I didn't realize exactly what was happening to us, but I smelled a major rat. And so I just gave up. Obviously the world sucked, so why bother?

When I discovered that one of our teachers was herself using Cliff's Notes, it

seemed par for the course. Surely it meant nothing to get a good grade in such a class.

In retrospect this was stupid. It was like someone getting fouled in a soccer game and saying, hey, you fouled me, that's against the rules, and walking off the field in indignation. Fouls happen. The thing to do when you get fouled is not to lose your cool. Just keep playing.

By putting you in this situation, society has fouled you. Yes, as you suspect, a lot of the stuff you learn in your classes is crap. And yes, as you suspect, the college admissions process is largely a charade. But like many fouls, this one was unintentional. [7] So just keep playing.

Rebellion is almost as stupid as obedience. In either case you let yourself be defined by what they tell you to do. The best plan, I think, is to step onto an orthogonal vector. Don't just do what they tell you, and don't just refuse to. Instead treat school as a day job. As day jobs go, it's pretty sweet. You're done at 3 o'clock, and you can even work on your own stuff while you're there.

Curiosity

And what's your real job supposed to be? Unless you're Mozart, your first task is to figure that out. What are the great things to work on? Where are the imaginative people? And most importantly, what are you interested in? The word "aptitude" is misleading, because it implies something innate. The most powerful sort of aptitude is a consuming interest in some question, and such interests are often acquired tastes.

A distorted version of this idea has filtered into popular culture under the name "passion." I recently saw an ad for waiters saying they wanted people with a "passion for service." The real thing is not something one could have for waiting on tables. And passion is a bad word for it. A better name would be curiosity.

Kids are curious, but the curiosity I mean has a different shape from kid curiosity. Kid curiosity is broad and shallow; they ask why at random about everything. In most adults this curiosity dries up entirely. It has to: you can't get anything done if you're always asking why about everything. But in ambitious adults, instead of drying up, curiosity becomes narrow and deep. The mud flat morphs into a well.

Curiosity turns work into play. For Einstein, relativity wasn't a book full of hard stuff he had to learn for an exam. It was a mystery he was trying to solve. So it probably felt like less work to him to invent it than it would seem to someone now to learn it in a class.

One of the most dangerous illusions you get from school is the idea that doing

great things requires a lot of discipline. Most subjects are taught in such a boring way that it's only by discipline that you can flog yourself through them. So I was surprised when, early in college, I read a quote by Wittgenstein saying that he had no self-discipline and had never been able to deny himself anything, not even a cup of coffee.

Now I know a number of people who do great work, and it's the same with all of them. They have little discipline. They're all terrible procrastinators and find it almost impossible to make themselves do anything they're not interested in. One still hasn't sent out his half of the thank-you notes from his wedding, four years ago. Another has 26,000 emails in her inbox.

I'm not saying you can get away with zero self-discipline. You probably need about the amount you need to go running. I'm often reluctant to go running, but once I do, I enjoy it. And if I don't run for several days, I feel ill. It's the same with people who do great things. They know they'll feel bad if they don't work, and they have enough discipline to get themselves to their desks to start working. But once they get started, interest takes over, and discipline is no longer necessary.

Do you think Shakespeare was gritting his teeth and diligently trying to write Great Literature? Of course not. He was having fun. That's why he's so good.

If you want to do good work, what you need is a great curiosity about a promising question. The critical moment for Einstein was when he looked at Maxwell's equations and said, what the hell is going on here?

It can take years to zero in on a productive question, because it can take years to figure out what a subject is really about. To take an extreme example, consider math. Most people think they hate math, but the boring stuff you do in school under the name "mathematics" is not at all like what mathematicians do.

The great mathematician G. H. Hardy said he didn't like math in high school either. He only took it up because he was better at it than the other students. Only later did he realize math was interesting — only later did he start to ask questions instead of merely answering them correctly.

When a friend of mine used to grumble because he had to write a paper for school, his mother would tell him: find a way to make it interesting. That's what you need to do: find a question that makes the world interesting. People who do great things look at the same world everyone else does, but notice some odd detail that's compellingly mysterious.

And not only in intellectual matters. Henry Ford's great question was, why do cars have to be a luxury item? What would happen if you treated them as a

commodity? Franz Beckenbauer's was, in effect, why does everyone have to stay in his position? Why can't defenders score goals too?

Now

If it takes years to articulate great questions, what do you do now, at sixteen? Work toward finding one. Great questions don't appear suddenly. They gradually congeal in your head. And what makes them congeal is experience. So the way to find great questions is not to search for them — not to wander about thinking, what great discovery shall I make? You can't answer that; if you could, you'd have made it.

The way to get a big idea to appear in your head is not to hunt for big ideas, but to put in a lot of time on work that interests you, and in the process keep your mind open enough that a big idea can take roost. Einstein, Ford, and Beckenbauer all used this recipe. They all knew their work like a piano player knows the keys. So when something seemed amiss to them, they had the confidence to notice it.

Put in time how and on what? Just pick a project that seems interesting: to master some chunk of material, or to make something, or to answer some question. Choose a project that will take less than a month, and make it something you have the means to finish. Do something hard enough to stretch you, but only just, especially at first. If you're deciding between two projects, choose whichever seems most fun. If one blows up in your face, start another. Repeat till, like an internal combustion engine, the process becomes self-sustaining, and each project generates the next one. (This could take years.)

It may be just as well not to do a project "for school," if that will restrict you or make it seem like work. Involve your friends if you want, but not too many, and only if they're not flakes. Friends offer moral support (few startups are started by one person), but secrecy also has its advantages. There's something pleasing about a secret project. And you can take more risks, because no one will know if you fail.

Don't worry if a project doesn't seem to be on the path to some goal you're supposed to have. Paths can bend a lot more than you think. So let the path grow out the project. The most important thing is to be excited about it, because it's by doing that you learn.

Don't disregard unseemly motivations. One of the most powerful is the desire to be better than other people at something. Hardy said that's what got him started, and I think the only unusual thing about him is that he admitted it. Another powerful motivator is the desire to do, or know, things you're not supposed to. Closely related is the desire to do something audacious. Sixteen year olds aren't

supposed to write novels. So if you try, anything you achieve is on the plus side of the ledger; if you fail utterly, you're doing no worse than expectations. [8]

Beware of bad models. Especially when they excuse laziness. When I was in high school I used to write "existentialist" short stories like ones I'd seen by famous writers. My stories didn't have a lot of plot, but they were very deep. And they were less work to write than entertaining ones would have been. I should have known that was a danger sign. And in fact I found my stories pretty boring; what excited me was the idea of writing serious, intellectual stuff like the famous writers.

Now I have enough experience to realize that those famous writers actually sucked. Plenty of famous people do; in the short term, the quality of one's work is only a small component of fame. I should have been less worried about doing something that seemed cool, and just done something I liked. That's the actual road to coolness anyway.

A key ingredient in many projects, almost a project on its own, is to find good books. Most books are bad. Nearly all textbooks are bad. [9] So don't assume a subject is to be learned from whatever book on it happens to be closest. You have to search actively for the tiny number of good books.

The important thing is to get out there and do stuff. Instead of waiting to be taught, go out and learn.

Your life doesn't have to be shaped by admissions officers. It could be shaped by your own curiosity. It is for all ambitious adults. And you don't have to wait to start. In fact, you don't have to wait to be an adult. There's no switch inside you that magically flips when you turn a certain age or graduate from some institution. You start being an adult when you decide to take responsibility for your life. You can do that at any age. [10]

This may sound like bullshit. I'm just a minor, you may think, I have no money, I have to live at home, I have to do what adults tell me all day long. Well, most adults labor under restrictions just as cumbersome, and they manage to get things done. If you think it's restrictive being a kid, imagine having kids.

The only real difference between adults and high school kids is that adults realize they need to get things done, and high school kids don't. That realization hits most people around 23. But I'm letting you in on the secret early. So get to work. Maybe you can be the first generation whose greatest regret from high school isn't how much time you wasted.

Notes

[1] A doctor friend warns that even this can give an inaccurate picture. "Who

knew how much time it would take up, how little autonomy one would have for endless years of training, and how unbelievably annoying it is to carry a beeper?"

[2] His best bet would probably be to become dictator and intimidate the NBA into letting him play. So far the closest anyone has come is Secretary of Labor.

[3] A day job is one you take to pay the bills so you can do what you really want, like play in a band, or invent relativity.

Treating high school as a day job might actually make it easier for some students to get good grades. If you treat your classes as a game, you won't be demoralized if they seem pointless.

However bad your classes, you need to get good grades in them to get into a decent college. And that *is* worth doing, because universities are where a lot of the clumps of smart people are these days.

[4] The second biggest regret was caring so much about unimportant things. And especially about what other people thought of them.

I think what they really mean, in the latter case, is caring what random people thought of them. Adults care just as much what other people think, but they get to be more selective about the other people.

I have about thirty friends whose opinions I care about, and the opinion of the rest of the world barely affects me. The problem in high school is that your peers are chosen for you by accidents of age and geography, rather than by you based on respect for their judgement.

[5] The key to wasting time is distraction. Without distractions it's too obvious to your brain that you're not doing anything with it, and you start to feel uncomfortable. If you want to measure how dependent you've become on distractions, try this experiment: set aside a chunk of time on a weekend and sit alone and think. You can have a notebook to write your thoughts down in, but nothing else: no friends, TV, music, phone, IM, email, Web, games, books, newspapers, or magazines. Within an hour most people will feel a strong craving for distraction.

[6] I don't mean to imply that the only function of prep schools is to trick admissions officers. They also generally provide a better education. But try this thought experiment: suppose prep schools supplied the same superior education but had a tiny (.001) negative effect on college admissions. How many parents would still send their kids to them?

It might also be argued that kids who went to prep schools, because they've learned more, *are* better college candidates. But this seems empirically false. What you learn in even the best high school is rounding error compared to what

you learn in college. Public school kids arrive at college with a slight disadvantage, but they start to pull ahead in the sophomore year.

(I'm not saying public school kids are smarter than preppies, just that they are within any given college. That follows necessarily if you agree prep schools improve kids' admissions prospects.)

[7] Why does society foul you? Indifference, mainly. There are simply no outside forces pushing high school to be good. The air traffic control system works because planes would crash otherwise. Businesses have to deliver because otherwise competitors would take their customers. But no planes crash if your school sucks, and it has no competitors. High school isn't evil; it's random; but random is pretty bad.

[8] And then of course there is money. It's not a big factor in high school, because you can't do much that anyone wants. But a lot of great things were created mainly to make money. Samuel Johnson said "no man but a blockhead ever wrote except for money." (Many hope he was exaggerating.)

[9] Even college textbooks are bad. When you get to college, you'll find that (with a few stellar exceptions) the textbooks are not written by the leading scholars in the field they describe. Writing college textbooks is unpleasant work, done mostly by people who need the money. It's unpleasant because the publishers exert so much control, and there are few things worse than close supervision by someone who doesn't understand what you're doing. This phenomenon is apparently even worse in the production of high school textbooks.

[10] Your teachers are always telling you to behave like adults. I wonder if they'd like it if you did. You may be loud and disorganized, but you're very docile compared to adults. If you actually started acting like adults, it would be just as if a bunch of adults had been transposed into your bodies. Imagine the reaction of an FBI agent or taxi driver or reporter to being told they had to ask permission to go the bathroom, and only one person could go at a time. To say nothing of the things you're taught. If a bunch of actual adults suddenly found themselves trapped in high school, the first thing they'd do is form a union and renegotiate all the rules with the administration.

Thanks to Ingrid Bassett, Trevor Blackwell, Rich Draves, Dan Giffin, Sarah Harlin, Jessica Livingston, Jackie McDonough, Robert Morris, Mark Nitzberg, Lisa Randall, and Aaron Swartz for reading drafts of this, and to many others for talking to me about high school.

— | | Why Nerds are Unpopular

How to Start a Startup

March 2005

(This essay is derived from a talk at the Harvard Computer Society.)

You need three things to create a successful startup: to start with good people, to make something customers actually want, and to spend as little money as possible. Most startups that fail do it because they fail at one of these. A startup that does all three will probably succeed.

And that's kind of exciting, when you think about it, because all three are doable. Hard, but doable. And since a startup that succeeds ordinarily makes its founders rich, that implies getting rich is doable too. Hard, but doable.

If there is one message I'd like to get across about startups, that's it. There is no magically difficult step that requires brilliance to solve.

The Idea

In particular, you don't need a brilliant idea to start a startup around. The way a startup makes money is to offer people better technology than they have now. But what people have now is often so bad that it doesn't take brilliance to do better.

Google's plan, for example, was simply to create a search site that didn't suck. They had three new ideas: index more of the Web, use links to rank search results, and have clean, simple web pages with unintrusive keyword-based ads. Above all, they were determined to make a site that was good to use. No doubt there are great technical tricks within Google, but the overall plan was straightforward. And while they probably have bigger ambitions now, this alone brings them a billion dollars a year. [1]

There are plenty of other areas that are just as backward as search was before Google. I can think of several heuristics for generating ideas for startups, but most reduce to this: look at something people are trying to do, and figure out

how to do it in a way that doesn't suck.

For example, dating sites currently suck far worse than search did before Google. They all use the same simple-minded model. They seem to have approached the problem by thinking about how to do database matches instead of how dating works in the real world. An undergrad could build something better as a class project. And yet there's a lot of money at stake. Online dating is a valuable business now, and it might be worth a hundred times as much if it worked.

An idea for a startup, however, is only a beginning. A lot of would-be startup founders think the key to the whole process is the initial idea, and from that point all you have to do is execute. Venture capitalists know better. If you go to VC firms with a brilliant idea that you'll tell them about if they sign a nondisclosure agreement, most will tell you to get lost. That shows how much a mere idea is worth. The market price is less than the inconvenience of signing an NDA.

Another sign of how little the initial idea is worth is the number of startups that change their plan en route. Microsoft's original plan was to make money selling programming languages, of all things. Their current business model didn't occur to them until IBM dropped it in their lap five years later.

Ideas for startups are worth something, certainly, but the trouble is, they're not transferrable. They're not something you could hand to someone else to execute. Their value is mainly as starting points: as questions for the people who had them to continue thinking about.

What matters is not ideas, but the people who have them. Good people can fix bad ideas, but good ideas can't save bad people.

People

What do I mean by good people? One of the best tricks I learned during our startup was a rule for deciding who to hire. Could you describe the person as an animal? It might be hard to translate that into another language, but I think everyone in the US knows what it means. It means someone who takes their work a little too seriously; someone who does what they do so well that they pass right through professional and cross over into obsessive.

What it means specifically depends on the job: a salesperson who just won't take no for an answer; a hacker who will stay up till 4:00 AM rather than go to bed leaving code with a bug in it; a PR person who will cold-call *New York Times* reporters on their cell phones; a graphic designer who feels physical pain when something is two millimeters out of place.

Almost everyone who worked for us was an animal at what they did. The woman

in charge of sales was so tenacious that I used to feel sorry for potential customers on the phone with her. You could sense them squirming on the hook, but you knew there would be no rest for them till they'd signed up.

If you think about people you know, you'll find the animal test is easy to apply. Call the person's image to mind and imagine the sentence "so-and-so is an animal." If you laugh, they're not. You don't need or perhaps even want this quality in big companies, but you need it in a startup.

For programmers we had three additional tests. Was the person genuinely smart? If so, could they actually get things done? And finally, since a few good hackers have unbearable personalities, could we stand to have them around?

That last test filters out surprisingly few people. We could bear any amount of nerdiness if someone was truly smart. What we couldn't stand were people with a lot of attitude. But most of those weren't truly smart, so our third test was largely a restatement of the first.

When nerds are unbearable it's usually because they're trying too hard to seem smart. But the smarter they are, the less pressure they feel to act smart. So as a rule you can recognize genuinely smart people by their ability to say things like "I don't know," "Maybe you're right," and "I don't understand x well enough."

This technique doesn't always work, because people can be influenced by their environment. In the MIT CS department, there seems to be a tradition of acting like a brusque know-it-all. I'm told it derives ultimately from Marvin Minsky, in the same way the classic airline pilot manner is said to derive from Chuck Yeager. Even genuinely smart people start to act this way there, so you have to make allowances.

It helped us to have Robert Morris, who is one of the readiest to say "I don't know" of anyone I've met. (At least, he was before he became a professor at MIT.) No one dared put on attitude around Robert, because he was obviously smarter than they were and yet had zero attitude himself.

Like most startups, ours began with a group of friends, and it was through personal contacts that we got most of the people we hired. This is a crucial difference between startups and big companies. Being friends with someone for even a couple days will tell you more than companies could ever learn in interviews. [2]

It's no coincidence that startups start around universities, because that's where smart people meet. It's not what people learn in classes at MIT and Stanford that has made technology companies spring up around them. They could sing campfire songs in the classes so long as admissions worked the same.

If you start a startup, there's a good chance it will be with people you know from college or grad school. So in theory you ought to try to make friends with as many smart people as you can in school, right? Well, no. Don't make a conscious effort to schmooze; that doesn't work well with hackers.

What you should do in college is work on your own projects. Hackers should do this even if they don't plan to start startups, because it's the only real way to learn how to program. In some cases you may collaborate with other students, and this is the best way to get to know good hackers. The project may even grow into a startup. But once again, I wouldn't aim too directly at either target. Don't force things; just work on stuff you like with people you like.

Ideally you want between two and four founders. It would be hard to start with just one. One person would find the moral weight of starting a company hard to bear. Even Bill Gates, who seems to be able to bear a good deal of moral weight, had to have a co-founder. But you don't want so many founders that the company starts to look like a group photo. Partly because you don't need a lot of people at first, but mainly because the more founders you have, the worse disagreements you'll have. When there are just two or three founders, you know you have to resolve disputes immediately or perish. If there are seven or eight, disagreements can linger and harden into factions. You don't want mere voting; you need unanimity.

In a technology startup, which most startups are, the founders should include technical people. During the Internet Bubble there were a number of startups founded by business people who then went looking for hackers to create their product for them. This doesn't work well. Business people are bad at deciding what to do with technology, because they don't know what the options are, or which kinds of problems are hard and which are easy. And when business people try to hire hackers, they can't tell which ones are good. Even other hackers have a hard time doing that. For business people it's roulette.

Do the founders of a startup have to include business people? That depends. We thought so when we started ours, and we asked several people who were said to know about this mysterious thing called "business" if they would be the president. But they all said no, so I had to do it myself. And what I discovered was that business was no great mystery. It's not something like physics or medicine that requires extensive study. You just try to get people to pay you for stuff.

I think the reason I made such a mystery of business was that I was disgusted by the idea of doing it. I wanted to work in the pure, intellectual world of software, not deal with customers' mundane problems. People who don't want to get dragged into some kind of work often develop a protective incompetence

at it. Paul Erdos was particularly good at this. By seeming unable even to cut a grapefruit in half (let alone go to the store and buy one), he forced other people to do such things for him, leaving all his time free for math. Erdos was an extreme case, but most husbands use the same trick to some degree.

Once I was forced to discard my protective incompetence, I found that business was neither so hard nor so boring as I feared. There are esoteric areas of business that are quite hard, like tax law or the pricing of derivatives, but you don't need to know about those in a startup. All you need to know about business to run a startup are commonsense things people knew before there were business schools, or even universities.

If you work your way down the Forbes 400 making an x next to the name of each person with an MBA, you'll learn something important about business school. After Warren Buffett, you don't hit another MBA till number 22, Phil Knight, the CEO of Nike. There are only 5 MBAs in the top 50. What you notice in the Forbes 400 are a lot of people with technical backgrounds. Bill Gates, Steve Jobs, Larry Ellison, Michael Dell, Jeff Bezos, Gordon Moore. The rulers of the technology business tend to come from technology, not business. So if you want to invest two years in something that will help you succeed in business, the evidence suggests you'd do better to learn how to hack than get an MBA. [3]

There is one reason you might want to include business people in a startup, though: because you have to have at least one person willing and able to focus on what customers want. Some believe only business people can do this— that hackers can implement software, but not design it. That's nonsense. There's nothing about knowing how to program that prevents hackers from understanding users, or about not knowing how to program that magically enables business people to understand them.

If you can't understand users, however, you should either learn how or find a co-founder who can. That is the single most important issue for technology startups, and the rock that sinks more of them than anything else.

What Customers Want

It's not just startups that have to worry about this. I think most businesses that fail do it because they don't give customers what they want. Look at restaurants. A large percentage fail, about a quarter in the first year. But can you think of one restaurant that had really good food and went out of business?

Restaurants with great food seem to prosper no matter what. A restaurant with great food can be expensive, crowded, noisy, dingy, out of the way, and even have bad service, and people will keep coming. It's true that a restaurant with

mediocre food can sometimes attract customers through gimmicks. But that approach is very risky. It's more straightforward just to make the food good.

It's the same with technology. You hear all kinds of reasons why startups fail. But can you think of one that had a massively popular product and still failed?

In nearly every failed startup, the real problem was that customers didn't want the product. For most, the cause of death is listed as "ran out of funding," but that's only the immediate cause. Why couldn't they get more funding? Probably because the product was a dog, or never seemed likely to be done, or both.

When I was trying to think of the things every startup needed to do, I almost included a fourth: get a version 1 out as soon as you can. But I decided not to, because that's implicit in making something customers want. The only way to make something customers want is to get a prototype in front of them and refine it based on their reactions.

The other approach is what I call the "Hail Mary" strategy. You make elaborate plans for a product, hire a team of engineers to develop it (people who do this tend to use the term "engineer" for hackers), and then find after a year that you've spent two million dollars to develop something no one wants. This was not uncommon during the Bubble, especially in companies run by business types, who thought of software development as something terrifying that therefore had to be carefully planned.

We never even considered that approach. As a Lisp hacker, I come from the tradition of rapid prototyping. I would not claim (at least, not here) that this is the right way to write every program, but it's certainly the right way to write software for a startup. In a startup, your initial plans are almost certain to be wrong in some way, and your first priority should be to figure out where. The only way to do that is to try implementing them.

Like most startups, we changed our plan on the fly. At first we expected our customers to be Web consultants. But it turned out they didn't like us, because our software was easy to use and we hosted the site. It would be too easy for clients to fire them. We also thought we'd be able to sign up a lot of catalog companies, because selling online was a natural extension of their existing business. But in 1996 that was a hard sell. The middle managers we talked to at catalog companies saw the Web not as an opportunity, but as something that meant more work for them.

We did get a few of the more adventurous catalog companies. Among them was Frederick's of Hollywood, which gave us valuable experience dealing with heavy loads on our servers. But most of our users were small, individual merchants who saw the Web as an opportunity to build a business. Some had retail stores, but

many only existed online. And so we changed direction to focus on these users. Instead of concentrating on the features Web consultants and catalog companies would want, we worked to make the software easy to use.

I learned something valuable from that. It's worth trying very, very hard to make technology easy to use. Hackers are so used to computers that they have no idea how horrifying software seems to normal people. Stephen Hawking's editor told him that every equation he included in his book would cut sales in half. When you work on making technology easier to use, you're riding that curve up instead of down. A 10% improvement in ease of use doesn't just increase your sales 10%. It's more likely to double your sales.

How do you figure out what customers want? Watch them. One of the best places to do this was at trade shows. Trade shows didn't pay as a way of getting new customers, but they were worth it as market research. We didn't just give canned presentations at trade shows. We used to show people how to build real, working stores. Which meant we got to watch as they used our software, and talk to them about what they needed.

No matter what kind of startup you start, it will probably be a stretch for you, the founders, to understand what users want. The only kind of software you can build without studying users is the sort for which you are the typical user. But this is just the kind that tends to be open source: operating systems, programming languages, editors, and so on. So if you're developing technology for money, you're probably not going to be developing it for people like you. Indeed, you can use this as a way to generate ideas for startups: what do people who are not like you want from technology?

When most people think of startups, they think of companies like Apple or Google. Everyone knows these, because they're big consumer brands. But for every startup like that, there are twenty more that operate in niche markets or live quietly down in the infrastructure. So if you start a successful startup, odds are you'll start one of those.

Another way to say that is, if you try to start the kind of startup that has to be a big consumer brand, the odds against succeeding are steeper. The best odds are in niche markets. Since startups make money by offering people something better than they had before, the best opportunities are where things suck most. And it would be hard to find a place where things suck more than in corporate IT departments. You would not believe the amount of money companies spend on software, and the crap they get in return. This imbalance equals opportunity.

If you want ideas for startups, one of the most valuable things you could do is find a middle-sized non-technology company and spend a couple weeks just

watching what they do with computers. Most good hackers have no more idea of the horrors perpetrated in these places than rich Americans do of what goes on in Brazilian slums.

Start by writing software for smaller companies, because it's easier to sell to them. It's worth so much to sell stuff to big companies that the people selling them the crap they currently use spend a lot of time and money to do it. And while you can outhack Oracle with one frontal lobe tied behind your back, you can't outsell an Oracle salesman. So if you want to win through better technology, aim at smaller customers. [4]

They're the more strategically valuable part of the market anyway. In technology, the low end always eats the high end. It's easier to make an inexpensive product more powerful than to make a powerful product cheaper. So the products that start as cheap, simple options tend to gradually grow more powerful till, like water rising in a room, they squash the "high-end" products against the ceiling. Sun did this to mainframes, and Intel is doing it to Sun. Microsoft Word did it to desktop publishing software like Interleaf and Framemaker. Mass-market digital cameras are doing it to the expensive models made for professionals. Avid did it to the manufacturers of specialized video editing systems, and now Apple is doing it to Avid. *Henry Ford* did it to the car makers that preceded him. If you build the simple, inexpensive option, you'll not only find it easier to sell at first, but you'll also be in the best position to conquer the rest of the market.

It's very dangerous to let anyone fly under you. If you have the cheapest, easiest product, you'll own the low end. And if you don't, you're in the crosshairs of whoever does.

Raising Money

To make all this happen, you're going to need money. Some startups have been self-funding—Microsoft for example— but most aren't. I think it's wise to take money from investors. To be self-funding, you have to start as a consulting company, and it's hard to switch from that to a product company.

Financially, a startup is like a pass/fail course. The way to get rich from a startup is to maximize the company's chances of succeeding, not to maximize the amount of stock you retain. So if you can trade stock for something that improves your odds, it's probably a smart move.

To most hackers, getting investors seems like a terrifying and mysterious process. Actually it's merely tedious. I'll try to give an outline of how it works.

The first thing you'll need is a few tens of thousands of dollars to pay your expenses while you develop a prototype. This is called seed capital. Because so

little money is involved, raising seed capital is comparatively easy— at least in the sense of getting a quick yes or no.

Usually you get seed money from individual rich people called “angels.” Often they’re people who themselves got rich from technology. At the seed stage, investors don’t expect you to have an elaborate business plan. Most know that they’re supposed to decide quickly. It’s not unusual to get a check within a week based on a half-page agreement.

We started Viaweb with \$10,000 of seed money from our friend Julian. But he gave us a lot more than money. He’s a former CEO and also a corporate lawyer, so he gave us a lot of valuable advice about business, and also did all the legal work of getting us set up as a company. Plus he introduced us to one of the two angel investors who supplied our next round of funding.

Some angels, especially those with technology backgrounds, may be satisfied with a demo and a verbal description of what you plan to do. But many will want a copy of your business plan, if only to remind themselves what they invested in.

Our angels asked for one, and looking back, I’m amazed how much worry it caused me. “Business plan” has that word “business” in it, so I figured it had to be something I’d have to read a book about business plans to write. Well, it doesn’t. At this stage, all most investors expect is a brief description of what you plan to do and how you’re going to make money from it, and the resumes of the founders. If you just sit down and write out what you’ve been saying to one another, that should be fine. It shouldn’t take more than a couple hours, and you’ll probably find that writing it all down gives you more ideas about what to do.

For the angel to have someone to make the check out to, you’re going to have to have some kind of company. Merely incorporating yourselves isn’t hard. The problem is, for the company to exist, you have to decide who the founders are, and how much stock they each have. If there are two founders with the same qualifications who are both equally committed to the business, that’s easy. But if you have a number of people who are expected to contribute in varying degrees, arranging the proportions of stock can be hard. And once you’ve done it, it tends to be set in stone.

I have no tricks for dealing with this problem. All I can say is, try hard to do it right. I do have a rule of thumb for recognizing when you have, though. When everyone feels they’re getting a slightly bad deal, that they’re doing more than they should for the amount of stock they have, the stock is optimally apportioned.

There is more to setting up a company than incorporating it, of course: insurance, business license, unemployment compensation, various things with the IRS. I’m

not even sure what the list is, because we, ah, skipped all that. When we got real funding near the end of 1996, we hired a great CFO, who fixed everything retroactively. It turns out that no one comes and arrests you if you don't do everything you're supposed to when starting a company. And a good thing too, or a lot of startups would never get started. [5]

It can be dangerous to delay turning yourself into a company, because one or more of the founders might decide to split off and start another company doing the same thing. This does happen. So when you set up the company, as well as as apportioning the stock, you should get all the founders to sign something agreeing that everyone's ideas belong to this company, and that this company is going to be everyone's only job.

[If this were a movie, ominous music would begin here.]

While you're at it, you should ask what else they've signed. One of the worst things that can happen to a startup is to run into intellectual property problems. We did, and it came closer to killing us than any competitor ever did.

As we were in the middle of getting bought, we discovered that one of our people had, early on, been bound by an agreement that said all his ideas belonged to the giant company that was paying for him to go to grad school. In theory, that could have meant someone else owned big chunks of our software. So the acquisition came to a screeching halt while we tried to sort this out. The problem was, since we'd been about to be acquired, we'd allowed ourselves to run low on cash. Now we needed to raise more to keep going. But it's hard to raise money with an IP cloud over your head, because investors can't judge how serious it is.

Our existing investors, knowing that we needed money and had nowhere else to get it, at this point attempted certain gambits which I will not describe in detail, except to remind readers that the word "angel" is a metaphor. The founders thereupon proposed to walk away from the company, after giving the investors a brief tutorial on how to administer the servers themselves. And while this was happening, the acquirers used the delay as an excuse to welch on the deal.

Miraculously it all turned out ok. The investors backed down; we did another round of funding at a reasonable valuation; the giant company finally gave us a piece of paper saying they didn't own our software; and six months later we were bought by Yahoo for much more than the earlier acquirer had agreed to pay. So we were happy in the end, though the experience probably took several years off my life.

Don't do what we did. Before you consummate a startup, ask everyone about their previous IP history.

Once you've got a company set up, it may seem presumptuous to go knocking on the doors of rich people and asking them to invest tens of thousands of dollars in something that is really just a bunch of guys with some ideas. But when you look at it from the rich people's point of view, the picture is more encouraging. Most rich people are looking for good investments. If you really think you have a chance of succeeding, you're doing them a favor by letting them invest. Mixed with any annoyance they might feel about being approached will be the thought: are these guys the next Google?

Usually angels are financially equivalent to founders. They get the same kind of stock and get diluted the same amount in future rounds. How much stock should they get? That depends on how ambitious you feel. When you offer x percent of your company for y dollars, you're implicitly claiming a certain value for the whole company. Venture investments are usually described in terms of that number. If you give an investor new shares equal to 5% of those already outstanding in return for \$100,000, then you've done the deal at a pre-money valuation of \$2 million.

How do you decide what the value of the company should be? There is no rational way. At this stage the company is just a bet. I didn't realize that when we were raising money. Julian thought we ought to value the company at several million dollars. I thought it was preposterous to claim that a couple thousand lines of code, which was all we had at the time, were worth several million dollars. Eventually we settled on one million, because Julian said no one would invest in a company with a valuation any lower. [6]

What I didn't grasp at the time was that the valuation wasn't just the value of the code we'd written so far. It was also the value of our ideas, which turned out to be right, and of all the future work we'd do, which turned out to be a lot.

The next round of funding is the one in which you might deal with actual venture capital firms. But don't wait till you've burned through your last round of funding to start approaching them. VCs are slow to make up their minds. They can take months. You don't want to be running out of money while you're trying to negotiate with them.

Getting money from an actual VC firm is a bigger deal than getting money from angels. The amounts of money involved are larger, millions usually. So the deals take longer, dilute you more, and impose more onerous conditions.

Sometimes the VCs want to install a new CEO of their own choosing. Usually the claim is that you need someone mature and experienced, with a business background. Maybe in some cases this is true. And yet Bill Gates was young and inexperienced and had no business background, and he seems to have

done ok. Steve Jobs got booted out of his own company by someone mature and experienced, with a business background, who then proceeded to ruin the company. So I think people who are mature and experienced, with a business background, may be overrated. We used to call these guys “newscasters,” because they had neat hair and spoke in deep, confident voices, and generally didn’t know much more than they read on the teleprompter.

We talked to a number of VCs, but eventually we ended up financing our startup entirely with angel money. The main reason was that we feared a brand-name VC firm would stick us with a newscaster as part of the deal. That might have been ok if he was content to limit himself to talking to the press, but what if he wanted to have a say in running the company? That would have led to disaster, because our software was so complex. We were a company whose whole m.o. was to win through better technology. The strategic decisions were mostly decisions about technology, and we didn’t need any help with those.

This was also one reason we didn’t go public. Back in 1998 our CFO tried to talk me into it. In those days you could go public as a dogfood portal, so as a company with a real product and real revenues, we might have done well. But I feared it would have meant taking on a newscaster— someone who, as they say, “can talk Wall Street’s language.”

I’m happy to see Google is bucking that trend. They didn’t talk Wall Street’s language when they did their IPO, and Wall Street didn’t buy. And now Wall Street is collectively kicking itself. They’ll pay attention next time. Wall Street learns new languages fast when money is involved.

You have more leverage negotiating with VCs than you realize. The reason is other VCs. I know a number of VCs now, and when you talk to them you realize that it’s a seller’s market. Even now there is too much money chasing too few good deals.

VCs form a pyramid. At the top are famous ones like Sequoia and Kleiner Perkins, but beneath those are a huge number you’ve never heard of. What they all have in common is that a dollar from them is worth one dollar. Most VCs will tell you that they don’t just provide money, but connections and advice. If you’re talking to Vinod Khosla or John Doerr or Mike Moritz, this is true. But such advice and connections can come very expensive. And as you go down the food chain the VCs get rapidly dumber. A few steps down from the top you’re basically talking to bankers who’ve picked up a few new vocabulary words from reading *Wired*. (Does your product use *XML*?) So I’d advise you to be skeptical about claims of experience and connections. Basically, a VC is a source of money. I’d be inclined to go with whoever offered the most money the soonest with the

least strings attached.

You may wonder how much to tell VCs. And you should, because some of them may one day be funding your competitors. I think the best plan is not to be overtly secretive, but not to tell them everything either. After all, as most VCs say, they're more interested in the people than the ideas. The main reason they want to talk about your idea is to judge you, not the idea. So as long as you seem like you know what you're doing, you can probably keep a few things back from them. [7]

Talk to as many VCs as you can, even if you don't want their money, because a) they may be on the board of someone who will buy you, and b) if you seem impressive, they'll be discouraged from investing in your competitors. The most efficient way to reach VCs, especially if you only want them to know about you and don't want their money, is at the conferences that are occasionally organized for startups to present to them.

Not Spending It

When and if you get an infusion of real money from investors, what should you do with it? Not spend it, that's what. In nearly every startup that fails, the proximate cause is running out of money. Usually there is something deeper wrong. But even a proximate cause of death is worth trying hard to avoid.

During the Bubble many startups tried to "get big fast." Ideally this meant getting a lot of customers fast. But it was easy for the meaning to slide over into hiring a lot of people fast.

Of the two versions, the one where you get a lot of customers fast is of course preferable. But even that may be overrated. The idea is to get there first and get all the users, leaving none for competitors. But I think in most businesses the advantages of being first to market are not so overwhelmingly great. Google is again a case in point. When they appeared it seemed as if search was a mature market, dominated by big players who'd spent millions to build their brands: Yahoo, Lycos, Excite, Infoseek, Altavista, Inktomi. Surely 1998 was a little late to arrive at the party.

But as the founders of Google knew, brand is worth next to nothing in the search business. You can come along at any point and make something better, and users will gradually seep over to you. As if to emphasize the point, Google never did any advertising. They're like dealers; they sell the stuff, but they know better than to use it themselves.

The competitors Google buried would have done better to spend those millions improving their software. Future startups should learn from that mistake. Unless

you're in a market where products are as undifferentiated as cigarettes or vodka or laundry detergent, spending a lot on brand advertising is a sign of breakage. And few if any Web businesses are so undifferentiated. The dating sites are running big ad campaigns right now, which is all the more evidence they're ripe for the picking. (Fee, fie, fo, fum, I smell a company run by marketing guys.)

We were compelled by circumstances to grow slowly, and in retrospect it was a good thing. The founders all learned to do every job in the company. As well as writing software, I had to do sales and customer support. At sales I was not very good. I was persistent, but I didn't have the smoothness of a good salesman. My message to potential customers was: you'd be stupid not to sell online, and if you sell online you'd be stupid to use anyone else's software. Both statements were true, but that's not the way to convince people.

I was great at customer support though. Imagine talking to a customer support person who not only knew everything about the product, but would apologize abjectly if there was a bug, and then fix it immediately, while you were on the phone with them. Customers loved us. And we loved them, because when you're growing slow by word of mouth, your first batch of users are the ones who were smart enough to find you by themselves. There is nothing more valuable, in the early stages of a startup, than smart users. If you listen to them, they'll tell you exactly how to make a winning product. And not only will they give you this advice for free, they'll pay you.

We officially launched in early 1996. By the end of that year we had about 70 users. Since this was the era of "get big fast," I worried about how small and obscure we were. But in fact we were doing exactly the right thing. Once you get big (in users or employees) it gets hard to change your product. That year was effectively a laboratory for improving our software. By the end of it, we were so far ahead of our competitors that they never had a hope of catching up. And since all the hackers had spent many hours talking to users, we understood online commerce way better than anyone else.

That's the key to success as a startup. There is nothing more important than understanding your business. You might think that anyone in a business must, *ex officio*, understand it. Far from it. Google's secret weapon was simply that they understood search. I was working for Yahoo when Google appeared, and Yahoo didn't understand search. I know because I once tried to convince the powers that be that we had to make search better, and I got in reply what was then the party line about it: that Yahoo was no longer a mere "search engine." Search was now only a small percentage of our page views, less than one month's growth, and now that we were established as a "media company," or "portal," or whatever we were, search could safely be allowed to wither and drop off, like an

umbilical cord.

Well, a small fraction of page views they may be, but they are an important fraction, because they are the page views that Web sessions start with. I think Yahoo gets that now.

Google understands a few other things most Web companies still don't. The most important is that you should put users before advertisers, even though the advertisers are paying and users aren't. One of my favorite bumper stickers reads "if the people lead, the leaders will follow." Paraphrased for the Web, this becomes "get all the users, and the advertisers will follow." More generally, design your product to please users first, and then think about how to make money from it. If you don't put users first, you leave a gap for competitors who do.

To make something users love, you have to understand them. And the bigger you are, the harder that is. So I say "get big slow." The slower you burn through your funding, the more time you have to learn.

The other reason to spend money slowly is to encourage a culture of cheapness. That's something Yahoo did understand. David Filo's title was "Chief Yahoo," but he was proud that his unofficial title was "Cheap Yahoo." Soon after we arrived at Yahoo, we got an email from Filo, who had been crawling around our directory hierarchy, asking if it was really necessary to store so much of our data on expensive RAID drives. I was impressed by that. Yahoo's market cap then was already in the billions, and they were still worrying about wasting a few gigs of disk space.

When you get a couple million dollars from a VC firm, you tend to feel rich. It's important to realize you're not. A rich company is one with large revenues. This money isn't revenue. It's money investors have given you in the hope you'll be able to generate revenues. So despite those millions in the bank, you're still poor.

For most startups the model should be grad student, not law firm. Aim for cool and cheap, not expensive and impressive. For us the test of whether a startup understood this was whether they had Aeron chairs. The Aeron came out during the Bubble and was very popular with startups. Especially the type, all too common then, that was like a bunch of kids playing house with money supplied by VCs. We had office chairs so cheap that the arms all fell off. This was slightly embarrassing at the time, but in retrospect the grad-studenty atmosphere of our office was another of those things we did right without knowing it.

Our offices were in a wooden triple-decker in Harvard Square. It had been an apartment until about the 1970s, and there was still a claw-footed bathtub in the bathroom. It must once have been inhabited by someone fairly eccentric, because a lot of the chinks in the walls were stuffed with aluminum foil, as if to

protect against cosmic rays. When eminent visitors came to see us, we were a bit sheepish about the low production values. But in fact that place was the perfect space for a startup. We felt like our role was to be impudent underdogs instead of corporate stuffed shirts, and that is exactly the spirit you want.

An apartment is also the right kind of place for developing software. Cube farms suck for that, as you've probably discovered if you've tried it. Ever notice how much easier it is to hack at home than at work? So why not make work more like home?

When you're looking for space for a startup, don't feel that it has to look professional. Professional means doing good work, not elevators and glass walls. I'd advise most startups to avoid corporate space at first and just rent an apartment. You want to live at the office in a startup, so why not have a place designed to be lived in as your office?

Besides being cheaper and better to work in, apartments tend to be in better locations than office buildings. And for a startup location is very important. The key to productivity is for people to come back to work after dinner. Those hours after the phone stops ringing are by far the best for getting work done. Great things happen when a group of employees go out to dinner together, talk over ideas, and then come back to their offices to implement them. So you want to be in a place where there are a lot of restaurants around, not some dreary office park that's a wasteland after 6:00 PM. Once a company shifts over into the model where everyone drives home to the suburbs for dinner, however late, you've lost something extraordinarily valuable. God help you if you actually start in that mode.

If I were going to start a startup today, there are only three places I'd consider doing it: on the Red Line near Central, Harvard, or Davis Squares (Kendall is too sterile); in Palo Alto on University or California Aves; and in Berkeley immediately north or south of campus. These are the only places I know that have the right kind of vibe.

The most important way to not spend money is by not hiring people. I may be an extremist, but I think hiring people is the worst thing a company can do. To start with, people are a recurring expense, which is the worst kind. They also tend to cause you to grow out of your space, and perhaps even move to the sort of uncool office building that will make your software worse. But worst of all, they slow you down: instead of sticking your head in someone's office and checking out an idea with them, eight people have to have a meeting about it. So the fewer people you can hire, the better.

During the Bubble a lot of startups had the opposite policy. They wanted to get

“staffed up” as soon as possible, as if you couldn’t get anything done unless there was someone with the corresponding job title. That’s big company thinking. Don’t hire people to fill the gaps in some a priori org chart. The only reason to hire someone is to do something you’d like to do but can’t.

If hiring unnecessary people is expensive and slows you down, why do nearly all companies do it? I think the main reason is that people like the idea of having a lot of people working for them. This weakness often extends right up to the CEO. If you ever end up running a company, you’ll find the most common question people ask is how many employees you have. This is their way of weighing you. It’s not just random people who ask this; even reporters do. And they’re going to be a lot more impressed if the answer is a thousand than if it’s ten.

This is ridiculous, really. If two companies have the same revenues, it’s the one with fewer employees that’s more impressive. When people used to ask me how many people our startup had, and I answered “twenty,” I could see them thinking that we didn’t count for much. I used to want to add “but our main competitor, whose ass we regularly kick, has a hundred and forty, so can we have credit for the larger of the two numbers?”

As with office space, the number of your employees is a choice between seeming impressive, and being impressive. Any of you who were nerds in high school know about this choice. Keep doing it when you start a company.

Should You?

But should you start a company? Are you the right sort of person to do it? If you are, is it worth it?

More people are the right sort of person to start a startup than realize it. That’s the main reason I wrote this. There could be ten times more startups than there are, and that would probably be a good thing.

I was, I now realize, exactly the right sort of person to start a startup. But the idea terrified me at first. I was forced into it because I was a Lisp hacker. The company I’d been consulting for seemed to be running into trouble, and there were not a lot of other companies using Lisp. Since I couldn’t bear the thought of programming in another language (this was 1995, remember, when “another language” meant C++) the only option seemed to be to start a new company using Lisp.

I realize this sounds far-fetched, but if you’re a Lisp hacker you’ll know what I mean. And if the idea of starting a startup frightened me so much that I only did it out of necessity, there must be a lot of people who would be good at it but who are too intimidated to try.

So who should start a startup? Someone who is a good hacker, between about 23 and 38, and who wants to solve the money problem in one shot instead of getting paid gradually over a conventional working life.

I can't say precisely what a good hacker is. At a first rate university this might include the top half of computer science majors. Though of course you don't have to be a CS major to be a hacker; I was a philosophy major in college.

It's hard to tell whether you're a good hacker, especially when you're young. Fortunately the process of starting startups tends to select them automatically. What drives people to start startups is (or should be) looking at existing technology and thinking, don't these guys realize they should be doing x, y, and z? And that's also a sign that one is a good hacker.

I put the lower bound at 23 not because there's something that doesn't happen to your brain till then, but because you need to see what it's like in an existing business before you try running your own. The business doesn't have to be a startup. I spent a year working for a software company to pay off my college loans. It was the worst year of my adult life, but I learned, without realizing it at the time, a lot of valuable lessons about the software business. In this case they were mostly negative lessons: don't have a lot of meetings; don't have chunks of code that multiple people own; don't have a sales guy running the company; don't make a high-end product; don't let your code get too big; don't leave finding bugs to QA people; don't go too long between releases; don't isolate developers from users; don't move from Cambridge to Route 128; and so on. [8] But negative lessons are just as valuable as positive ones. Perhaps even more valuable: it's hard to repeat a brilliant performance, but it's straightforward to avoid errors. [9]

The other reason it's hard to start a company before 23 is that people won't take you seriously. VCs won't trust you, and will try to reduce you to a mascot as a condition of funding. Customers will worry you're going to flake out and leave them stranded. Even you yourself, unless you're very unusual, will feel your age to some degree; you'll find it awkward to be the boss of someone much older than you, and if you're 21, hiring only people younger rather limits your options.

Some people could probably start a company at 18 if they wanted to. Bill Gates was 19 when he and Paul Allen started Microsoft. (Paul Allen was 22, though, and that probably made a difference.) So if you're thinking, I don't care what he says, I'm going to start a company now, you may be the sort of person who could get away with it.

The other cutoff, 38, has a lot more play in it. One reason I put it there is that I don't think many people have the physical stamina much past that age. I used

to work till 2:00 or 3:00 AM every night, seven days a week. I don't know if I could do that now.

Also, startups are a big risk financially. If you try something that blows up and leaves you broke at 26, big deal; a lot of 26 year olds are broke. By 38 you can't take so many risks— especially if you have kids.

My final test may be the most restrictive. Do you actually want to start a startup? What it amounts to, economically, is compressing your working life into the smallest possible space. Instead of working at an ordinary rate for 40 years, you work like hell for four. And maybe end up with nothing— though in that case it probably won't take four years.

During this time you'll do little but work, because when you're not working, your competitors will be. My only leisure activities were running, which I needed to do to keep working anyway, and about fifteen minutes of reading a night. I had a girlfriend for a total of two months during that three year period. Every couple weeks I would take a few hours off to visit a used bookshop or go to a friend's house for dinner. I went to visit my family twice. Otherwise I just worked.

Working was often fun, because the people I worked with were some of my best friends. Sometimes it was even technically interesting. But only about 10% of the time. The best I can say for the other 90% is that some of it is funnier in hindsight than it seemed then. Like the time the power went off in Cambridge for about six hours, and we made the mistake of trying to start a gasoline powered generator inside our offices. I won't try that again.

I don't think the amount of bullshit you have to deal with in a startup is more than you'd endure in an ordinary working life. It's probably less, in fact; it just seems like a lot because it's compressed into a short period. So mainly what a startup buys you is time. That's the way to think about it if you're trying to decide whether to start one. If you're the sort of person who would like to solve the money problem once and for all instead of working for a salary for 40 years, then a startup makes sense.

For a lot of people the conflict is between startups and graduate school. Grad students are just the age, and just the sort of people, to start software startups. You may worry that if you do you'll blow your chances of an academic career. But it's possible to be part of a startup and stay in grad school, especially at first. Two of our three original hackers were in grad school the whole time, and both got their degrees. There are few sources of energy so powerful as a procrastinating grad student.

If you do have to leave grad school, in the worst case it won't be for too long. If a startup fails, it will probably fail quickly enough that you can return to academic

life. And if it succeeds, you may find you no longer have such a burning desire to be an assistant professor.

If you want to do it, do it. Starting a startup is not the great mystery it seems from outside. It's not something you have to know about "business" to do. Build something users love, and spend less than you make. How hard is that?

Notes

[1] Google's revenues are about two billion a year, but half comes from ads on other sites.

[2] One advantage startups have over established companies is that there are no discrimination laws about starting businesses. For example, I would be reluctant to start a startup with a woman who had small children, or was likely to have them soon. But you're not allowed to ask prospective employees if they plan to have kids soon. Believe it or not, under current US law, you're not even allowed to discriminate on the basis of intelligence. Whereas when you're starting a company, you can discriminate on any basis you want about who you start it with.

[3] Learning to hack is a lot cheaper than business school, because you can do it mostly on your own. For the price of a Linux box, a copy of K&R, and a few hours of advice from your neighbor's fifteen year old son, you'll be well on your way.

[4] Corollary: Avoid starting a startup to sell things to the biggest company of all, the government. Yes, there are lots of opportunities to sell them technology. But let someone else start those startups.

[5] A friend who started a company in Germany told me they do care about the paperwork there, and that there's more of it. Which helps explain why there are not more startups in Germany.

[6] At the seed stage our valuation was in principle \$100,000, because Julian got 10% of the company. But this is a very misleading number, because the money was the least important of the things Julian gave us.

[7] The same goes for companies that seem to want to acquire you. There will be a few that are only pretending to in order to pick your brains. But you can never tell for sure which these are, so the best approach is to seem entirely open, but to fail to mention a few critical technical secrets.

[8] I was as bad an employee as this place was a company. I apologize to anyone who had to work with me there.

[9] You could probably write a book about how to succeed in business by doing everything in exactly the opposite way from the DMV.

Thanks to Trevor Blackwell, Sarah Harlin, Jessica Livingston, and Robert Morris for reading drafts of this essay, and to Steve Melendez and Gregory Price for inviting me to speak.

— | | Domain Name Search

How to Do What You Love

January 2006

To do something well you have to like it. That idea is not exactly novel. We've got it down to four words: "Do what you love." But it's not enough just to tell people that. Doing what you love is complicated.

The very idea is foreign to what most of us learn as kids. When I was a kid, it seemed as if work and fun were opposites by definition. Life had two states: some of the time adults were making you do things, and that was called work; the rest of the time you could do what you wanted, and that was called playing. Occasionally the things adults made you do were fun, just as, occasionally, playing wasn't — for example, if you fell and hurt yourself. But except for these few anomalous cases, work was pretty much defined as not-fun.

And it did not seem to be an accident. School, it was implied, was tedious *because* it was preparation for grownup work.

The world then was divided into two groups, grownups and kids. Grownups, like some kind of cursed race, had to work. Kids didn't, but they did have to go to school, which was a dilute version of work meant to prepare us for the real thing. Much as we disliked school, the grownups all agreed that grownup work was worse, and that we had it easy.

Teachers in particular all seemed to believe implicitly that work was not fun. Which is not surprising: work wasn't fun for most of them. Why did we have to memorize state capitals instead of playing dodgeball? For the same reason they had to watch over a bunch of kids instead of lying on a beach. You couldn't just do what you wanted.

I'm not saying we should let little kids do whatever they want. They may have to be made to work on certain things. But if we make kids work on dull stuff, it might be wise to tell them that tediousness is not the defining quality of work, and indeed that the reason they have to work on dull stuff now is so they can

work on more interesting stuff later. [1]

Once, when I was about 9 or 10, my father told me I could be whatever I wanted when I grew up, so long as I enjoyed it. I remember that precisely because it seemed so anomalous. It was like being told to use dry water. Whatever I thought he meant, I didn't think he meant work could *literally* be fun — fun like playing. It took me years to grasp that.

Jobs

By high school, the prospect of an actual job was on the horizon. Adults would sometimes come to speak to us about their work, or we would go to see them at work. It was always understood that they enjoyed what they did. In retrospect I think one may have: the private jet pilot. But I don't think the bank manager really did.

The main reason they all acted as if they enjoyed their work was presumably the upper-middle class convention that you're supposed to. It would not merely be bad for your career to say that you despised your job, but a social faux-pas.

Why is it conventional to pretend to like what you do? The first sentence of this essay explains that. If you have to like something to do it well, then the most successful people will all like what they do. That's where the upper-middle class tradition comes from. Just as houses all over America are full of chairs that are, without the owners even knowing it, nth-degree imitations of chairs designed 250 years ago for French kings, conventional attitudes about work are, without the owners even knowing it, nth-degree imitations of the attitudes of people who've done great things.

What a recipe for alienation. By the time they reach an age to think about what they'd like to do, most kids have been thoroughly misled about the idea of loving one's work. School has trained them to regard work as an unpleasant duty. Having a job is said to be even more onerous than schoolwork. And yet all the adults claim to like what they do. You can't blame kids for thinking "I am not like these people; I am not suited to this world."

Actually they've been told three lies: the stuff they've been taught to regard as work in school is not real work; grownup work is not (necessarily) worse than schoolwork; and many of the adults around them are lying when they say they like what they do.

The most dangerous liars can be the kids' own parents. If you take a boring job to give your family a high standard of living, as so many people do, you risk infecting your kids with the idea that work is boring. [2] Maybe it would be better for kids in this one case if parents were not so unselfish. A parent who set

an example of loving their work might help their kids more than an expensive house. [3]

It was not till I was in college that the idea of work finally broke free from the idea of making a living. Then the important question became not how to make money, but what to work on. Ideally these coincided, but some spectacular boundary cases (like Einstein in the patent office) proved they weren't identical.

The definition of work was now to make some original contribution to the world, and in the process not to starve. But after the habit of so many years my idea of work still included a large component of pain. Work still seemed to require discipline, because only hard problems yielded grand results, and hard problems couldn't literally be fun. Surely one had to force oneself to work on them.

If you think something's supposed to hurt, you're less likely to notice if you're doing it wrong. That about sums up my experience of graduate school.

Bounds

How much are you supposed to like what you do? Unless you know that, you don't know when to stop searching. And if, like most people, you underestimate it, you'll tend to stop searching too early. You'll end up doing something chosen for you by your parents, or the desire to make money, or prestige — or sheer inertia.

Here's an upper bound: Do what you love doesn't mean, do what you would like to do most *this second*. Even Einstein probably had moments when he wanted to have a cup of coffee, but told himself he ought to finish what he was working on first.

It used to perplex me when I read about people who liked what they did so much that there was nothing they'd rather do. There didn't seem to be any sort of work I liked *that* much. If I had a choice of (a) spending the next hour working on something or (b) be teleported to Rome and spend the next hour wandering about, was there any sort of work I'd prefer? Honestly, no.

But the fact is, almost anyone would rather, at any given moment, float about in the Carribean, or have sex, or eat some delicious food, than work on hard problems. The rule about doing what you love assumes a certain length of time. It doesn't mean, do what will make you happiest *this second*, but what will make you happiest over some longer period, like a week or a month.

Unproductive pleasures pall eventually. After a while you get tired of lying on the beach. If you want to stay happy, you have to do something.

As a lower bound, you have to like your work more than any unproductive

pleasure. You have to like what you do enough that the concept of “spare time” seems mistaken. Which is not to say you have to spend all your time working. You can only work so much before you get tired and start to screw up. Then you want to do something else — even something mindless. But you don’t regard this time as the prize and the time you spend working as the pain you endure to earn it.

I put the lower bound there for practical reasons. If your work is not your favorite thing to do, you’ll have terrible problems with procrastination. You’ll have to force yourself to work, and when you resort to that the results are distinctly inferior.

To be happy I think you have to be doing something you not only enjoy, but admire. You have to be able to say, at the end, wow, that’s pretty cool. This doesn’t mean you have to make something. If you learn how to hang glide, or to speak a foreign language fluently, that will be enough to make you say, for a while at least, wow, that’s pretty cool. What there has to be is a test.

So one thing that falls just short of the standard, I think, is reading books. Except for some books in math and the hard sciences, there’s no test of how well you’ve read a book, and that’s why merely reading books doesn’t quite feel like work. You have to do something with what you’ve read to feel productive.

I think the best test is one Gino Lee taught me: to try to do things that would make your friends say wow. But it probably wouldn’t start to work properly till about age 22, because most people haven’t had a big enough sample to pick friends from before then.

Sirens

What you should not do, I think, is worry about the opinion of anyone beyond your friends. You shouldn’t worry about prestige. Prestige is the opinion of the rest of the world. When you can ask the opinions of people whose judgement you respect, what does it add to consider the opinions of people you don’t even know? [4]

This is easy advice to give. It’s hard to follow, especially when you’re young. [5] Prestige is like a powerful magnet that warps even your beliefs about what you enjoy. It causes you to work not on what you like, but what you’d like to like.

That’s what leads people to try to write novels, for example. They like reading novels. They notice that people who write them win Nobel prizes. What could be more wonderful, they think, than to be a novelist? But liking the idea of being a novelist is not enough; you have to like the actual work of novel-writing if you’re going to be good at it; you have to like making up elaborate lies.

Prestige is just fossilized inspiration. If you do anything well enough, you'll *make* it prestigious. Plenty of things we now consider prestigious were anything but at first. Jazz comes to mind — though almost any established art form would do. So just do what you like, and let prestige take care of itself.

Prestige is especially dangerous to the ambitious. If you want to make ambitious people waste their time on errands, the way to do it is to bait the hook with prestige. That's the recipe for getting people to give talks, write forewords, serve on committees, be department heads, and so on. It might be a good rule simply to avoid any prestigious task. If it didn't suck, they wouldn't have had to make it prestigious.

Similarly, if you admire two kinds of work equally, but one is more prestigious, you should probably choose the other. Your opinions about what's admirable are always going to be slightly influenced by prestige, so if the two seem equal to you, you probably have more genuine admiration for the less prestigious one.

The other big force leading people astray is money. Money by itself is not that dangerous. When something pays well but is regarded with contempt, like telemarketing, or prostitution, or personal injury litigation, ambitious people aren't tempted by it. That kind of work ends up being done by people who are "just trying to make a living." (Tip: avoid any field whose practitioners say this.) The danger is when money is combined with prestige, as in, say, corporate law, or medicine. A comparatively safe and prosperous career with some automatic baseline prestige is dangerously tempting to someone young, who hasn't thought much about what they really like.

The test of whether people love what they do is whether they'd do it even if they weren't paid for it — even if they had to work at another job to make a living. How many corporate lawyers would do their current work if they had to do it for free, in their spare time, and take day jobs as waiters to support themselves?

This test is especially helpful in deciding between different kinds of academic work, because fields vary greatly in this respect. Most good mathematicians would work on math even if there were no jobs as math professors, whereas in the departments at the other end of the spectrum, the availability of teaching jobs is the driver: people would rather be English professors than work in ad agencies, and publishing papers is the way you compete for such jobs. Math would happen without math departments, but it is the existence of English majors, and therefore jobs teaching them, that calls into being all those thousands of dreary papers about gender and identity in the novels of Conrad. No one does that kind of thing for fun.

The advice of parents will tend to err on the side of money. It seems safe to say

there are more undergrads who want to be novelists and whose parents want them to be doctors than who want to be doctors and whose parents want them to be novelists. The kids think their parents are “materialistic.” Not necessarily. All parents tend to be more conservative for their kids than they would for themselves, simply because, as parents, they share risks more than rewards. If your eight year old son decides to climb a tall tree, or your teenage daughter decides to date the local bad boy, you won’t get a share in the excitement, but if your son falls, or your daughter gets pregnant, you’ll have to deal with the consequences.

Discipline

With such powerful forces leading us astray, it’s not surprising we find it so hard to discover what we like to work on. Most people are doomed in childhood by accepting the axiom that work = pain. Those who escape this are nearly all lured onto the rocks by prestige or money. How many even discover something they love to work on? A few hundred thousand, perhaps, out of billions.

It’s hard to find work you love; it must be, if so few do. So don’t underestimate this task. And don’t feel bad if you haven’t succeeded yet. In fact, if you admit to yourself that you’re discontented, you’re a step ahead of most people, who are still in denial. If you’re surrounded by colleagues who claim to enjoy work that you find contemptible, odds are they’re lying to themselves. Not necessarily, but probably.

Although doing great work takes less discipline than people think — because the way to do great work is to find something you like so much that you don’t have to force yourself to do it — *finding* work you love does usually require discipline. Some people are lucky enough to know what they want to do when they’re 12, and just glide along as if they were on railroad tracks. But this seems the exception. More often people who do great things have careers with the trajectory of a ping-pong ball. They go to school to study A, drop out and get a job doing B, and then become famous for C after taking it up on the side.

Sometimes jumping from one sort of work to another is a sign of energy, and sometimes it’s a sign of laziness. Are you dropping out, or boldly carving a new path? You often can’t tell yourself. Plenty of people who will later do great things seem to be disappointments early on, when they’re trying to find their niche.

Is there some test you can use to keep yourself honest? One is to try to do a good job at whatever you’re doing, even if you don’t like it. Then at least you’ll know you’re not using dissatisfaction as an excuse for being lazy. Perhaps more importantly, you’ll get into the habit of doing things well.

Another test you can use is: always produce. For example, if you have a day job you don't take seriously because you plan to be a novelist, are you producing? Are you writing pages of fiction, however bad? As long as you're producing, you'll know you're not merely using the hazy vision of the grand novel you plan to write one day as an opiate. The view of it will be obstructed by the all too palpably flawed one you're actually writing.

"Always produce" is also a heuristic for finding the work you love. If you subject yourself to that constraint, it will automatically push you away from things you think you're supposed to work on, toward things you actually like. "Always produce" will discover your life's work the way water, with the aid of gravity, finds the hole in your roof.

Of course, figuring out what you like to work on doesn't mean you get to work on it. That's a separate question. And if you're ambitious you have to keep them separate: you have to make a conscious effort to keep your ideas about what you want from being contaminated by what seems possible. [6]

It's painful to keep them apart, because it's painful to observe the gap between them. So most people pre-emptively lower their expectations. For example, if you asked random people on the street if they'd like to be able to draw like Leonardo, you'd find most would say something like "Oh, I can't draw." This is more a statement of intention than fact; it means, I'm not going to try. Because the fact is, if you took a random person off the street and somehow got them to work as hard as they possibly could at drawing for the next twenty years, they'd get surprisingly far. But it would require a great moral effort; it would mean staring failure in the eye every day for years. And so to protect themselves people say "I can't."

Another related line you often hear is that not everyone can do work they love — that someone has to do the unpleasant jobs. Really? How do you make them? In the US the only mechanism for forcing people to do unpleasant jobs is the draft, and that hasn't been invoked for over 30 years. All we can do is encourage people to do unpleasant work, with money and prestige.

If there's something people still won't do, it seems as if society just has to make do without. That's what happened with domestic servants. For millennia that was the canonical example of a job "someone had to do." And yet in the mid twentieth century servants practically disappeared in rich countries, and the rich have just had to do without.

So while there may be some things someone has to do, there's a good chance anyone saying that about any particular job is mistaken. Most unpleasant jobs would either get automated or go undone if no one were willing to do them.

Two Routes

There's another sense of "not everyone can do work they love" that's all too true, however. One has to make a living, and it's hard to get paid for doing work you love. There are two routes to that destination:

The organic route: as you become more eminent, gradually to increase the parts of your job that you like at the expense of those you don't.

> > The two-job route: to work at things you don't like to get money to work on things you do.

The organic route is more common. It happens naturally to anyone who does good work. A young architect has to take whatever work he can get, but if he does well he'll gradually be in a position to pick and choose among projects. The disadvantage of this route is that it's slow and uncertain. Even tenure is not real freedom.

The two-job route has several variants depending on how long you work for money at a time. At one extreme is the "day job," where you work regular hours at one job to make money, and work on what you love in your spare time. At the other extreme you work at something till you make enough not to have to work for money again.

The two-job route is less common than the organic route, because it requires a deliberate choice. It's also more dangerous. Life tends to get more expensive as you get older, so it's easy to get sucked into working longer than you expected at the money job. Worse still, anything you work on changes you. If you work too long on tedious stuff, it will rot your brain. And the best paying jobs are most dangerous, because they require your full attention.

The advantage of the two-job route is that it lets you jump over obstacles. The landscape of possible jobs isn't flat; there are walls of varying heights between different kinds of work. [7] The trick of maximizing the parts of your job that you like can get you from architecture to product design, but not, probably, to music. If you make money doing one thing and then work on another, you have more freedom of choice.

Which route should you take? That depends on how sure you are of what you want to do, how good you are at taking orders, how much risk you can stand, and the odds that anyone will pay (in your lifetime) for what you want to do. If you're sure of the general area you want to work in and it's something people are likely to pay you for, then you should probably take the organic route. But if you don't know what you want to work on, or don't like to take orders, you may want to take the two-job route, if you can stand the risk.

Don't decide too soon. Kids who know early what they want to do seem impressive, as if they got the answer to some math question before the other kids. They have an answer, certainly, but odds are it's wrong.

A friend of mine who is a quite successful doctor complains constantly about her job. When people applying to medical school ask her for advice, she wants to shake them and yell "Don't do it!" (But she never does.) How did she get into this fix? In high school she already wanted to be a doctor. And she is so ambitious and determined that she overcame every obstacle along the way — including, unfortunately, not liking it.

Now she has a life chosen for her by a high-school kid.

When you're young, you're given the impression that you'll get enough information to make each choice before you need to make it. But this is certainly not so with work. When you're deciding what to do, you have to operate on ridiculously incomplete information. Even in college you get little idea what various types of work are like. At best you may have a couple internships, but not all jobs offer internships, and those that do don't teach you much more about the work than being a batboy teaches you about playing baseball.

In the design of lives, as in the design of most other things, you get better results if you use flexible media. So unless you're fairly sure what you want to do, your best bet may be to choose a type of work that could turn into either an organic or two-job career. That was probably part of the reason I chose computers. You can be a professor, or make a lot of money, or morph it into any number of other kinds of work.

It's also wise, early on, to seek jobs that let you do many different things, so you can learn faster what various kinds of work are like. Conversely, the extreme version of the two-job route is dangerous because it teaches you so little about what you like. If you work hard at being a bond trader for ten years, thinking that you'll quit and write novels when you have enough money, what happens when you quit and then discover that you don't actually like writing novels?

Most people would say, I'd take that problem. Give me a million dollars and I'll figure out what to do. But it's harder than it looks. Constraints give your life shape. Remove them and most people have no idea what to do: look at what happens to those who win lotteries or inherit money. Much as everyone thinks they want financial security, the happiest people are not those who have it, but those who like what they do. So a plan that promises freedom at the expense of knowing what to do with it may not be as good as it seems.

Whichever route you take, expect a struggle. Finding work you love is very difficult. Most people fail. Even if you succeed, it's rare to be free to work on

what you want till your thirties or forties. But if you have the destination in sight you'll be more likely to arrive at it. If you know you can love work, you're in the home stretch, and if you know what work you love, you're practically there.

Notes

[1] Currently we do the opposite: when we make kids do boring work, like arithmetic drills, instead of admitting frankly that it's boring, we try to disguise it with superficial decorations.

[2] One father told me about a related phenomenon: he found himself concealing from his family how much he liked his work. When he wanted to go to work on a saturday, he found it easier to say that it was because he "had to" for some reason, rather than admitting he preferred to work than stay home with them.

[3] Something similar happens with suburbs. Parents move to suburbs to raise their kids in a safe environment, but suburbs are so dull and artificial that by the time they're fifteen the kids are convinced the whole world is boring.

[4] I'm not saying friends should be the only audience for your work. The more people you can help, the better. But friends should be your compass.

[5] Donald Hall said young would-be poets were mistaken to be so obsessed with being published. But you can imagine what it would do for a 24 year old to get a poem published in *The New Yorker*. Now to people he meets at parties he's a real poet. Actually he's no better or worse than he was before, but to a clueless audience like that, the approval of an official authority makes all the difference. So it's a harder problem than Hall realizes. The reason the young care so much about prestige is that the people they want to impress are not very discerning.

[6] This is isomorphic to the principle that you should prevent your beliefs about how things are from being contaminated by how you wish they were. Most people let them mix pretty promiscuously. The continuing popularity of religion is the most visible index of that.

[7] A more accurate metaphor would be to say that the graph of jobs is not very well connected.

Thanks to Trevor Blackwell, Dan Friedman, Sarah Harlin, Jessica Livingston, Jackie McDonough, Robert Morris, Peter Norvig, David Sloo, and Aaron Swartz for reading drafts of this.

The 18 Mistakes That Kill Startups

October 2006

In the Q & A period after a recent talk, someone asked what made startups fail. After standing there gaping for a few seconds I realized this was kind of a trick question. It's equivalent to asking how to make a startup succeed — if you avoid every cause of failure, you succeed — and that's too big a question to answer on the fly.

Afterwards I realized it could be helpful to look at the problem from this direction. If you have a list of all the things you shouldn't do, you can turn that into a recipe for succeeding just by negating. And this form of list may be more useful in practice. It's easier to catch yourself doing something you shouldn't than always to remember to do something you should. [1]

In a sense there's just one mistake that kills startups: not making something users want. If you make something users want, you'll probably be fine, whatever else you do or don't do. And if you don't make something users want, then you're dead, whatever else you do or don't do. So really this is a list of 18 things that cause startups not to make something users want. Nearly all failure funnels through that.

1. Single Founder

Have you ever noticed how few successful startups were founded by just one person? Even companies you think of as having one founder, like Oracle, usually turn out to have more. It seems unlikely this is a coincidence.

What's wrong with having one founder? To start with, it's a vote of no confidence. It probably means the founder couldn't talk any of his friends into starting the company with him. That's pretty alarming, because his friends are the ones who know him best.

But even if the founder's friends were all wrong and the company is a good bet, he's still at a disadvantage. Starting a startup is too hard for one person. Even if you could do all the work yourself, you need colleagues to brainstorm with, to talk you out of stupid decisions, and to cheer you up when things go wrong.

The last one might be the most important. The low points in a startup are so low that few could bear them alone. When you have multiple founders, *esprit de corps* binds them together in a way that seems to violate conservation laws. Each thinks "I can't let my friends down." This is one of the most powerful forces in human nature, and it's missing when there's just one founder.

2. Bad Location

Startups prosper in some places and not others. Silicon Valley dominates, then Boston, then Seattle, Austin, Denver, and New York. After that there's not much. Even in New York the number of startups per capita is probably a 20th of what it is in Silicon Valley. In towns like Houston and Chicago and Detroit it's too small to measure.

Why is the falloff so sharp? Probably for the same reason it is in other industries. What's the sixth largest fashion center in the US? The sixth largest center for oil, or finance, or publishing? Whatever they are they're probably so far from the top that it would be misleading even to call them centers.

It's an interesting question why cities become startup hubs, but the reason startups prosper in them is probably the same as it is for any industry: that's where the experts are. Standards are higher; people are more sympathetic to what you're doing; the kind of people you want to hire want to live there; supporting industries are there; the people you run into in chance meetings are in the same business. Who knows exactly how these factors combine to boost startups in Silicon Valley and squish them in Detroit, but it's clear they do from the number of startups per capita in each.

3. Marginal Niche

Most of the groups that apply to Y Combinator suffer from a common problem: choosing a small, obscure niche in the hope of avoiding competition.

If you watch little kids playing sports, you notice that below a certain age they're afraid of the ball. When the ball comes near them their instinct is to avoid it. I didn't make a lot of catches as an eight year old outfielder, because whenever a fly ball came my way, I used to close my eyes and hold my glove up more for protection than in the hope of catching it.

Choosing a marginal project is the startup equivalent of my eight year old strategy for dealing with fly balls. If you make anything good, you're going to

have competitors, so you may as well face that. You can only avoid competition by avoiding good ideas.

I think this shrinking from big problems is mostly unconscious. It's not that people think of grand ideas but decide to pursue smaller ones because they seem safer. Your unconscious won't even let you think of grand ideas. So the solution may be to think about ideas without involving yourself. What would be a great idea for *someone else* to do as a startup?

4. Derivative Idea

Many of the applications we get are imitations of some existing company. That's one source of ideas, but not the best. If you look at the origins of successful startups, few were started in imitation of some other startup. Where did they get their ideas? Usually from some specific, unsolved problem the founders identified.

Our startup made software for making online stores. When we started it, there wasn't any; the few sites you could order from were hand-made at great expense by web consultants. We knew that if online shopping ever took off, these sites would have to be generated by software, so we wrote some. Pretty straightforward.

It seems like the best problems to solve are ones that affect you personally. Apple happened because Steve Wozniak wanted a computer, Google because Larry and Sergey couldn't find stuff online, Hotmail because Sabeer Bhatia and Jack Smith couldn't exchange email at work.

So instead of copying the Facebook, with some variation that the Facebook rightly ignored, look for ideas from the other direction. Instead of starting from companies and working back to the problems they solved, look for problems and imagine the company that might solve them. [2] What do people complain about? What do you wish there was?

5. Obstinacy

In some fields the way to succeed is to have a vision of what you want to achieve, and to hold true to it no matter what setbacks you encounter. Starting startups is not one of them. The stick-to-your-vision approach works for something like winning an Olympic gold medal, where the problem is well-defined. Startups are more like science, where you need to follow the trail wherever it leads.

So don't get too attached to your original plan, because it's probably wrong. Most successful startups end up doing something different than they originally intended — often so different that it doesn't even seem like the same company. You have to be prepared to see the better idea when it arrives. And the hardest part of that is often discarding your old idea.

But openness to new ideas has to be tuned just right. Switching to a new idea every week will be equally fatal. Is there some kind of external test you can use? One is to ask whether the ideas represent some kind of progression. If in each new idea you're able to re-use most of what you built for the previous ones, then you're probably in a process that converges. Whereas if you keep restarting from scratch, that's a bad sign.

Fortunately there's someone you can ask for advice: your users. If you're thinking about turning in some new direction and your users seem excited about it, it's probably a good bet.

6. Hiring Bad Programmers

I forgot to include this in the early versions of the list, because nearly all the founders I know are programmers. This is not a serious problem for them. They might accidentally hire someone bad, but it's not going to kill the company. In a pinch they can do whatever's required themselves.

But when I think about what killed most of the startups in the e-commerce business back in the 90s, it was bad programmers. A lot of those companies were started by business guys who thought the way startups worked was that you had some clever idea and then hired programmers to implement it. That's actually much harder than it sounds — almost impossibly hard in fact — because business guys can't tell which are the good programmers. They don't even get a shot at the best ones, because no one really good wants a job implementing the vision of a business guy.

In practice what happens is that the business guys choose people they think are good programmers (it says here on his resume that he's a Microsoft Certified Developer) but who aren't. Then they're mystified to find that their startup lumbers along like a World War II bomber while their competitors scream past like jet fighters. This kind of startup is in the same position as a big company, but without the advantages.

So how do you pick good programmers if you're not a programmer? I don't think there's an answer. I was about to say you'd have to find a good programmer to help you hire people. But if you can't recognize good programmers, how would you even do that?

7. Choosing the Wrong Platform

A related problem (since it tends to be done by bad programmers) is choosing the wrong platform. For example, I think a lot of startups during the Bubble killed themselves by deciding to build server-based applications on Windows. Hotmail was still running on FreeBSD for years after Microsoft bought it, presumably

because Windows couldn't handle the load. If Hotmail's founders had chosen to use Windows, they would have been swamped.

PayPal only just dodged this bullet. After they merged with X.com, the new CEO wanted to switch to Windows — even after PayPal cofounder Max Levchin showed that their software scaled only 1% as well on Windows as Unix. Fortunately for PayPal they switched CEOs instead.

Platform is a vague word. It could mean an operating system, or a programming language, or a “framework” built on top of a programming language. It implies something that both supports and limits, like the foundation of a house.

The scary thing about platforms is that there are always some that seem to outsiders to be fine, responsible choices and yet, like Windows in the 90s, will destroy you if you choose them. Java applets were probably the most spectacular example. This was supposed to be the new way of delivering applications. Presumably it killed just about 100% of the startups who believed that.

How do you pick the right platforms? The usual way is to hire good programmers and let them choose. But there is a trick you could use if you're not a programmer: visit a top computer science department and see what they use in research projects.

8. Slowness in Launching

Companies of all sizes have a hard time getting software done. It's intrinsic to the medium; software is always 85% done. It takes an effort of will to push through this and get something released to users. [3]

Startups make all kinds of excuses for delaying their launch. Most are equivalent to the ones people use for procrastinating in everyday life. There's something that needs to happen first. Maybe. But if the software were 100% finished and ready to launch at the push of a button, would they still be waiting?

One reason to launch quickly is that it forces you to actually *finish* some quantum of work. Nothing is truly finished till it's released; you can see that from the rush of work that's always involved in releasing anything, no matter how finished you thought it was. The other reason you need to launch is that it's only by bouncing your idea off users that you fully understand it.

Several distinct problems manifest themselves as delays in launching: working too slowly; not truly understanding the problem; fear of having to deal with users; fear of being judged; working on too many different things; excessive perfectionism. Fortunately you can combat all of them by the simple expedient of forcing yourself to launch *something* fairly quickly.

9. Launching Too Early

Launching too slowly has probably killed a hundred times more startups than launching too fast, but it is possible to launch too fast. The danger here is that you ruin your reputation. You launch something, the early adopters try it out, and if it's no good they may never come back.

So what's the minimum you need to launch? We suggest startups think about what they plan to do, identify a core that's both (a) useful on its own and (b) something that can be incrementally expanded into the whole project, and then get that done as soon as possible.

This is the same approach I (and many other programmers) use for writing software. Think about the overall goal, then start by writing the smallest subset of it that does anything useful. If it's a subset, you'll have to write it anyway, so in the worst case you won't be wasting your time. But more likely you'll find that implementing a working subset is both good for morale and helps you see more clearly what the rest should do.

The early adopters you need to impress are fairly tolerant. They don't expect a newly launched product to do everything; it just has to do *something*.

10. Having No Specific User in Mind

You can't build things users like without understanding them. I mentioned earlier that the most successful startups seem to have begun by trying to solve a problem their founders had. Perhaps there's a rule here: perhaps you create wealth in proportion to how well you understand the problem you're solving, and the problems you understand best are your own. [4]

That's just a theory. What's not a theory is the converse: if you're trying to solve problems you don't understand, you're hosed.

And yet a surprising number of founders seem willing to assume that someone, they're not sure exactly who, will want what they're building. Do the founders want it? No, they're not the target market. Who is? Teenagers. People interested in local events (that one is a perennial tarpit). Or "business" users. What business users? Gas stations? Movie studios? Defense contractors?

You can of course build something for users other than yourself. We did. But you should realize you're stepping into dangerous territory. You're flying on instruments, in effect, so you should (a) consciously shift gears, instead of assuming you can rely on your intuitions as you ordinarily would, and (b) look at the instruments.

In this case the instruments are the users. When designing for other people you have to be empirical. You can no longer guess what will work; you have to find users and measure their responses. So if you're going to make something for

teenagers or “business” users or some other group that doesn’t include you, you have to be able to talk some specific ones into using what you’re making. If you can’t, you’re on the wrong track.

11. Raising Too Little Money

Most successful startups take funding at some point. Like having more than one founder, it seems a good bet statistically. How much should you take, though?

Startup funding is measured in time. Every startup that isn’t profitable (meaning nearly all of them, initially) has a certain amount of time left before the money runs out and they have to stop. This is sometimes referred to as runway, as in “How much runway do you have left?” It’s a good metaphor because it reminds you that when the money runs out you’re going to be airborne or dead.

Too little money means not enough to get airborne. What airborne means depends on the situation. Usually you have to advance to a visibly higher level: if all you have is an idea, a working prototype; if you have a prototype, launching; if you’re launched, significant growth. It depends on investors, because until you’re profitable that’s who you have to convince.

So if you take money from investors, you have to take enough to get to the next step, whatever that is. [5] Fortunately you have some control over both how much you spend and what the next step is. We advise startups to set both low, initially: spend practically nothing, and make your initial goal simply to build a solid prototype. This gives you maximum flexibility.

12. Spending Too Much

It’s hard to distinguish spending too much from raising too little. If you run out of money, you could say either was the cause. The only way to decide which to call it is by comparison with other startups. If you raised five million and ran out of money, you probably spent too much.

Burning through too much money is not as common as it used to be. Founders seem to have learned that lesson. Plus it keeps getting cheaper to start a startup. So as of this writing few startups spend too much. None of the ones we’ve funded have. (And not just because we make small investments; many have gone on to raise further rounds.)

The classic way to burn through cash is by hiring a lot of people. This bites you twice: in addition to increasing your costs, it slows you down—so money that’s getting consumed faster has to last longer. Most hackers understand why that happens; Fred Brooks explained it in *The Mythical Man-Month*.

We have three general suggestions about hiring: (a) don’t do it if you can avoid

it, (b) pay people with equity rather than salary, not just to save money, but because you want the kind of people who are committed enough to prefer that, and (c) only hire people who are either going to write code or go out and get users, because those are the only things you need at first.

13. Raising Too Much Money

It's obvious how too little money could kill you, but is there such a thing as having too much?

Yes and no. The problem is not so much the money itself as what comes with it. As one VC who spoke at Y Combinator said, "Once you take several million dollars of my money, the clock is ticking." If VCs fund you, they're not going to let you just put the money in the bank and keep operating as two guys living on ramen. They want that money to go to work. [6] At the very least you'll move into proper office space and hire more people. That will change the atmosphere, and not entirely for the better. Now most of your people will be employees rather than founders. They won't be as committed; they'll need to be told what to do; they'll start to engage in office politics.

When you raise a lot of money, your company moves to the suburbs and has kids.

Perhaps more dangerously, once you take a lot of money it gets harder to change direction. Suppose your initial plan was to sell something to companies. After taking VC money you hire a sales force to do that. What happens now if you realize you should be making this for consumers instead of businesses? That's a completely different kind of selling. What happens, in practice, is that you don't realize that. The more people you have, the more you stay pointed in the same direction.

Another drawback of large investments is the time they take. The time required to raise money grows with the amount. [7] When the amount rises into the millions, investors get very cautious. VCs never quite say yes or no; they just engage you in an apparently endless conversation. Raising VC scale investments is thus a huge time sink — more work, probably, than the startup itself. And you don't want to be spending all your time talking to investors while your competitors are spending theirs building things.

We advise founders who go on to seek VC money to take the first reasonable deal they get. If you get an offer from a reputable firm at a reasonable valuation with no unusually onerous terms, just take it and get on with building the company. [8] Who cares if you could get a 30% better deal elsewhere? Economically, startups are an all-or-nothing game. Bargain-hunting among investors is a waste of time.

14. Poor Investor Management

As a founder, you have to manage your investors. You shouldn't ignore them, because they may have useful insights. But neither should you let them run the company. That's supposed to be your job. If investors had sufficient vision to run the companies they fund, why didn't they start them?

Pissing off investors by ignoring them is probably less dangerous than caving in to them. In our startup, we erred on the ignoring side. A lot of our energy got drained away in disputes with investors instead of going into the product. But this was less costly than giving in, which would probably have destroyed the company. If the founders know what they're doing, it's better to have half their attention focused on the product than the full attention of investors who don't.

How hard you have to work on managing investors usually depends on how much money you've taken. When you raise VC-scale money, the investors get a great deal of control. If they have a board majority, they're literally your bosses. In the more common case, where founders and investors are equally represented and the deciding vote is cast by neutral outside directors, all the investors have to do is convince the outside directors and they control the company.

If things go well, this shouldn't matter. So long as you seem to be advancing rapidly, most investors will leave you alone. But things don't always go smoothly in startups. Investors have made trouble even for the most successful companies. One of the most famous examples is Apple, whose board made a nearly fatal blunder in firing Steve Jobs. Apparently even Google got a lot of grief from their investors early on.

15. Sacrificing Users to (Supposed) Profit

When I said at the beginning that if you make something users want, you'll be fine, you may have noticed I didn't mention anything about having the right business model. That's not because making money is unimportant. I'm not suggesting that founders start companies with no chance of making money in the hope of unloading them before they tank. The reason we tell founders not to worry about the business model initially is that making something people want is so much harder.

I don't know why it's so hard to make something people want. It seems like it should be straightforward. But you can tell it must be hard by how few startups do it.

Because making something people want is so much harder than making money from it, you should leave business models for later, just as you'd leave some trivial but messy feature for version 2. In version 1, solve the core problem. And the core problem in a startup is how to create wealth (= how much people

want something x the number who want it), not how to convert that wealth into money.

The companies that win are the ones that put users first. Google, for example. They made search work, then worried about how to make money from it. And yet some startup founders still think it's irresponsible not to focus on the business model from the beginning. They're often encouraged in this by investors whose experience comes from less malleable industries.

It *is* irresponsible not to think about business models. It's just ten times more irresponsible not to think about the product.

16. Not Wanting to Get Your Hands Dirty

Nearly all programmers would rather spend their time writing code and have someone else handle the messy business of extracting money from it. And not just the lazy ones. Larry and Sergey apparently felt this way too at first. After developing their new search algorithm, the first thing they tried was to get some other company to buy it.

Start a company? Yech. Most hackers would rather just have ideas. But as Larry and Sergey found, there's not much of a market for ideas. No one trusts an idea till you embody it in a product and use that to grow a user base. Then they'll pay big time.

Maybe this will change, but I doubt it will change much. There's nothing like users for convincing acquirers. It's not just that the risk is decreased. The acquirers are human, and they have a hard time paying a bunch of young guys millions of dollars just for being clever. When the idea is embodied in a company with a lot of users, they can tell themselves they're buying the users rather than the cleverness, and this is easier for them to swallow. [9]

If you're going to attract users, you'll probably have to get up from your computer and go find some. It's unpleasant work, but if you can make yourself do it you have a much greater chance of succeeding. In the first batch of startups we funded, in the summer of 2005, most of the founders spent all their time building their applications. But there was one who was away half the time talking to executives at cell phone companies, trying to arrange deals. Can you imagine anything more painful for a hacker? [10] But it paid off, because this startup seems the most successful of that group by an order of magnitude.

If you want to start a startup, you have to face the fact that you can't just hack. At least one hacker will have to spend some of the time doing business stuff.

17. Fights Between Founders

Fights between founders are surprisingly common. About 20% of the startups we've funded have had a founder leave. It happens so often that we've reversed our attitude to vesting. We still don't require it, but now we advise founders to vest so there will be an orderly way for people to quit.

A founder leaving doesn't necessarily kill a startup, though. Plenty of successful startups have had that happen. [11] Fortunately it's usually the least committed founder who leaves. If there are three founders and one who was lukewarm leaves, big deal. If you have two and one leaves, or a guy with critical technical skills leaves, that's more of a problem. But even that is survivable. Blogger got down to one person, and they bounced back.

Most of the disputes I've seen between founders could have been avoided if they'd been more careful about who they started a company with. Most disputes are not due to the situation but the people. Which means they're inevitable. And most founders who've been burned by such disputes probably had misgivings, which they suppressed, when they started the company. Don't suppress misgivings. It's much easier to fix problems before the company is started than after. So don't include your housemate in your startup because he'd feel left out otherwise. Don't start a company with someone you dislike because they have some skill you need and you worry you won't find anyone else. The people are the most important ingredient in a startup, so don't compromise there.

18. A Half-Hearted Effort

The failed startups you hear most about are the spectacular flameouts. Those are actually the elite of failures. The most common type is not the one that makes spectacular mistakes, but the one that doesn't do much of anything — the one we never even hear about, because it was some project a couple guys started on the side while working on their day jobs, but which never got anywhere and was gradually abandoned.

Statistically, if you want to avoid failure, it would seem like the most important thing is to quit your day job. Most founders of failed startups don't quit their day jobs, and most founders of successful ones do. If startup failure were a disease, the CDC would be issuing bulletins warning people to avoid day jobs.

Does that mean you should quit your day job? Not necessarily. I'm guessing here, but I'd guess that many of these would-be founders may not have the kind of determination it takes to start a company, and that in the back of their minds, they know it. The reason they don't invest more time in their startup is that they know it's a bad investment. [12]

I'd also guess there's some band of people who could have succeeded if they'd taken the leap and done it full-time, but didn't. I have no idea how wide this band

is, but if the winner/borderline/hopeless progression has the sort of distribution you'd expect, the number of people who could have made it, if they'd quit their day job, is probably an order of magnitude larger than the number who do make it. [13]

If that's true, most startups that could succeed fail because the founders don't devote their whole efforts to them. That certainly accords with what I see out in the world. Most startups fail because they don't make something people want, and the reason most don't is that they don't try hard enough.

In other words, starting startups is just like everything else. The biggest mistake you can make is not to try hard enough. To the extent there's a secret to success, it's not to be in denial about that.

Notes

[1] This is not a complete list of the causes of failure, just those you can control. There are also several you can't, notably ineptitude and bad luck.

[2] Ironically, one variant of the Facebook that might work is a facebook exclusively for college students.

[3] Steve Jobs tried to motivate people by saying "Real artists ship." This is a fine sentence, but unfortunately not true. Many famous works of art are unfinished. It's true in fields that have hard deadlines, like architecture and filmmaking, but even there people tend to be tweaking stuff till it's yanked out of their hands.

[4] There's probably also a second factor: startup founders tend to be at the leading edge of technology, so problems they face are probably especially valuable.

[5] You should take more than you think you'll need, maybe 50% to 100% more, because software takes longer to write and deals longer to close than you expect.

[6] Since people sometimes call us VCs, I should add that we're not. VCs invest large amounts of other people's money. We invest small amounts of our own, like angel investors.

[7] Not linearly of course, or it would take forever to raise five million dollars. In practice it just feels like it takes forever.

Though if you include the cases where VCs don't invest, it would literally take forever in the median case. And maybe we should, because the danger of chasing large investments is not just that they take a long time. That's the *best* case. The real danger is that you'll expend a lot of time and get nothing.

[8] Some VCs will offer you an artificially low valuation to see if you have the balls to ask for more. It's lame that VCs play such games, but some do. If you're

dealing with one of those you should push back on the valuation a bit.

[9] Suppose YouTube’s founders had gone to Google in 2005 and told them “Google Video is badly designed. Give us \$10 million and we’ll tell you all the mistakes you made.” They would have gotten the royal raspberry. Eighteen months later Google paid \$1.6 billion for the same lesson, partly because they could then tell themselves that they were buying a phenomenon, or a community, or some vague thing like that.

I don’t mean to be hard on Google. They did better than their competitors, who may have now missed the video boat entirely.

[10] Yes, actually: dealing with the government. But phone companies are up there.

[11] Many more than most people realize, because companies don’t advertise this. Did you know Apple originally had three founders?

[12] I’m not dissing these people. I don’t have the determination myself. I’ve twice come close to starting startups since Viaweb, and both times I bailed because I realized that without the spur of poverty I just wasn’t willing to endure the stress of a startup.

[13] So how do you know whether you’re in the category of people who should quit their day job, or the presumably larger one who shouldn’t? I got to the point of saying that this was hard to judge for yourself and that you should seek outside advice, before realizing that that’s what we do. We think of ourselves as investors, but viewed from the other direction Y Combinator is a service for advising people whether or not to quit their day job. We could be mistaken, and no doubt often are, but we do at least bet money on our conclusions.

Thanks to Sam Altman, Jessica Livingston, Greg McAdoo, and Robert Morris for reading drafts of this.

Why to Not Not Start a Startup

March 2007

(This essay is derived from talks at the 2007 Startup School and the Berkeley CSUA.)

We've now been doing Y Combinator long enough to have some data about success rates. Our first batch, in the summer of 2005, had eight startups in it. Of those eight, it now looks as if at least four succeeded. Three have been acquired: Reddit was a merger of two, Reddit and Infogami, and a third was acquired that we can't talk about yet. Another from that batch was Loopt, which is doing so well they could probably be acquired in about ten minutes if they wanted to.

So about half the founders from that first summer, less than two years ago, are now rich, at least by their standards. (One thing you learn when you get rich is that there are many degrees of it.)

I'm not ready to predict our success rate will stay as high as 50%. That first batch could have been an anomaly. But we should be able to do better than the oft-quoted (and probably made up) standard figure of 10%. I'd feel safe aiming at 25%.

Even the founders who fail don't seem to have such a bad time. Of those first eight startups, three are now probably dead. In two cases the founders just went on to do other things at the end of the summer. I don't think they were traumatized by the experience. The closest to a traumatic failure was Kiko, whose founders kept working on their startup for a whole year before being squashed by Google Calendar. But they ended up happy. They sold their software on eBay for a quarter of a million dollars. After they paid back their angel investors, they had about a year's salary each. [1] Then they immediately went on to start a new and much more exciting startup, Justin.TV.

So here is an even more striking statistic: 0% of that first batch had a terrible experience. They had ups and downs, like every startup, but I don't think any would have traded it for a job in a cubicle. And that statistic is probably not an anomaly. Whatever our long-term success rate ends up being, I think the rate of people who wish they'd gotten a regular job will stay close to 0%.

The big mystery to me is: why don't more people start startups? If nearly everyone who does it prefers it to a regular job, and a significant percentage get rich, why doesn't everyone want to do this? A lot of people think we get thousands of applications for each funding cycle. In fact we usually only get several hundred. Why don't more people apply? And while it must seem to anyone watching this world that startups are popping up like crazy, the number is small compared to the number of people with the necessary skills. The great majority of programmers still go straight from college to cubicle, and stay there.

It seems like people are not acting in their own interest. What's going on? Well, I can answer that. Because of Y Combinator's position at the very start of the venture funding process, we're probably the world's leading experts on the psychology of people who aren't sure if they want to start a company.

There's nothing wrong with being unsure. If you're a hacker thinking about starting a startup and hesitating before taking the leap, you're part of a grand tradition. Larry and Sergey seem to have felt the same before they started Google, and so did Jerry and Filo before they started Yahoo. In fact, I'd guess the most successful startups are the ones started by uncertain hackers rather than gung-ho business guys.

We have some evidence to support this. Several of the most successful startups we've funded told us later that they only decided to apply at the last moment. Some decided only hours before the deadline.

The way to deal with uncertainty is to analyze it into components. Most people who are reluctant to do something have about eight different reasons mixed together in their heads, and don't know themselves which are biggest. Some will be justified and some bogus, but unless you know the relative proportion of each, you don't know whether your overall uncertainty is mostly justified or mostly bogus.

So I'm going to list all the components of people's reluctance to start startups, and explain which are real. Then would-be founders can use this as a checklist to examine their own feelings.

I admit my goal is to increase your self-confidence. But there are two things different here from the usual confidence-building exercise. One is that I'm motivated to be honest. Most people in the confidence-building business have

already achieved their goal when you buy the book or pay to attend the seminar where they tell you how great you are. Whereas if I encourage people to start startups who shouldn't, I make my own life worse. If I encourage too many people to apply to Y Combinator, it just means more work for me, because I have to read all the applications.

The other thing that's going to be different is my approach. Instead of being positive, I'm going to be negative. Instead of telling you "come on, you can do it" I'm going to consider all the reasons you aren't doing it, and show why most (but not all) should be ignored. We'll start with the one everyone's born with.

1. Too young

A lot of people think they're too young to start a startup. Many are right. The median age worldwide is about 27, so probably a third of the population can truthfully say they're too young.

What's too young? One of our goals with Y Combinator was to discover the lower bound on the age of startup founders. It always seemed to us that investors were too conservative here—that they wanted to fund professors, when really they should be funding grad students or even undergrads.

The main thing we've discovered from pushing the edge of this envelope is not where the edge is, but how fuzzy it is. The outer limit may be as low as 16. We don't look beyond 18 because people younger than that can't legally enter into contracts. But the most successful founder we've funded so far, Sam Altman, was 19 at the time.

Sam Altman, however, is an outlying data point. When he was 19, he seemed like he had a 40 year old inside him. There are other 19 year olds who are 12 inside.

There's a reason we have a distinct word "adult" for people over a certain age. There is a threshold you cross. It's conventionally fixed at 21, but different people cross it at greatly varying ages. You're old enough to start a startup if you've crossed this threshold, whatever your age.

How do you tell? There are a couple tests adults use. I realized these tests existed after meeting Sam Altman, actually. I noticed that I felt like I was talking to someone much older. Afterward I wondered, what am I even measuring? What made him seem older?

One test adults use is whether you still have the kid flake reflex. When you're a little kid and you're asked to do something hard, you can cry and say "I can't do it" and the adults will probably let you off. As a kid there's a magic button you can press by saying "I'm just a kid" that will get you out of most difficult

situations. Whereas adults, by definition, are not allowed to flake. They still do, of course, but when they do they're ruthlessly pruned.

The other way to tell an adult is by how they react to a challenge. Someone who's not yet an adult will tend to respond to a challenge from an adult in a way that acknowledges their dominance. If an adult says "that's a stupid idea," a kid will either crawl away with his tail between his legs, or rebel. But rebelling presumes inferiority as much as submission. The adult response to "that's a stupid idea," is simply to look the other person in the eye and say "Really? Why do you think so?"

There are a lot of adults who still react childishly to challenges, of course. What you don't often find are kids who react to challenges like adults. When you do, you've found an adult, whatever their age.

2. Too inexperienced

I once wrote that startup founders should be at least 23, and that people should work for another company for a few years before starting their own. I no longer believe that, and what changed my mind is the example of the startups we've funded.

I still think 23 is a better age than 21. But the best way to get experience if you're 21 is to start a startup. So, paradoxically, if you're too inexperienced to start a startup, what you should do is start one. That's a way more efficient cure for inexperience than a normal job. In fact, getting a normal job may actually make you less able to start a startup, by turning you into a tame animal who thinks he needs an office to work in and a product manager to tell him what software to write.

What really convinced me of this was the Kikos. They started a startup right out of college. Their inexperience caused them to make a lot of mistakes. But by the time we funded their second startup, a year later, they had become extremely formidable. They were certainly not tame animals. And there is no way they'd have grown so much if they'd spent that year working at Microsoft, or even Google. They'd still have been diffident junior programmers.

So now I'd advise people to go ahead and start startups right out of college. There's no better time to take risks than when you're young. Sure, you'll probably fail. But even failure will get you to the ultimate goal faster than getting a job.

It worries me a bit to be saying this, because in effect we're advising people to educate themselves by failing at our expense, but it's the truth.

3. Not determined enough

You need a lot of determination to succeed as a startup founder. It's probably the single best predictor of success.

Some people may not be determined enough to make it. It's hard for me to say for sure, because I'm so determined that I can't imagine what's going on in the heads of people who aren't. But I know they exist.

Most hackers probably underestimate their determination. I've seen a lot become visibly more determined as they get used to running a startup. I can think of several we've funded who would have been delighted at first to be bought for \$2 million, but are now set on world domination.

How can you tell if you're determined enough, when Larry and Sergey themselves were unsure at first about starting a company? I'm guessing here, but I'd say the test is whether you're sufficiently driven to work on your own projects. Though they may have been unsure whether they wanted to start a company, it doesn't seem as if Larry and Sergey were meek little research assistants, obediently doing their advisors' bidding. They started projects of their own.

4. Not smart enough

You may need to be moderately smart to succeed as a startup founder. But if you're worried about this, you're probably mistaken. If you're smart enough to worry that you might not be smart enough to start a startup, you probably are.

And in any case, starting a startup just doesn't require that much intelligence. Some startups do. You have to be good at math to write Mathematica. But most companies do more mundane stuff where the decisive factor is effort, not brains. Silicon Valley can warp your perspective on this, because there's a cult of smartness here. People who aren't smart at least try to act that way. But if you think it takes a lot of intelligence to get rich, try spending a couple days in some of the fancier bits of New York or LA.

If you don't think you're smart enough to start a startup doing something technically difficult, just write enterprise software. Enterprise software companies aren't technology companies, they're sales companies, and sales depends mostly on effort.

5. Know nothing about business

This is another variable whose coefficient should be zero. You don't need to know anything about business to start a startup. The initial focus should be the product. All you need to know in this phase is how to build things people want. If you succeed, you'll have to think about how to make money from it. But this is so easy you can pick it up on the fly.

I get a fair amount of flak for telling founders just to make something great and not worry too much about making money. And yet all the empirical evidence points that way: pretty much 100% of startups that make something popular manage to make money from it. And acquirers tell me privately that revenue is not what they buy startups for, but their strategic value. Which means, because they made something people want. Acquirers know the rule holds for them too: if users love you, you can always make money from that somehow, and if they don't, the cleverest business model in the world won't save you.

So why do so many people argue with me? I think one reason is that they hate the idea that a bunch of twenty year olds could get rich from building something cool that doesn't make any money. They just don't want that to be possible. But how possible it is doesn't depend on how much they want it to be.

For a while it annoyed me to hear myself described as some kind of irresponsible pied piper, leading impressionable young hackers down the road to ruin. But now I realize this kind of controversy is a sign of a good idea.

The most valuable truths are the ones most people don't believe. They're like undervalued stocks. If you start with them, you'll have the whole field to yourself. So when you find an idea you know is good but most people disagree with, you should not merely ignore their objections, but push aggressively in that direction. In this case, that means you should seek out ideas that would be popular but seem hard to make money from.

We'll bet a seed round you can't make something popular that we can't figure out how to make money from.

6. No cofounder

Not having a cofounder is a real problem. A startup is too much for one person to bear. And though we differ from other investors on a lot of questions, we all agree on this. All investors, without exception, are more likely to fund you with a cofounder than without.

We've funded two single founders, but in both cases we suggested their first priority should be to find a cofounder. Both did. But we'd have preferred them to have cofounders before they applied. It's not super hard to get a cofounder for a project that's just been funded, and we'd rather have cofounders committed enough to sign up for something super hard.

If you don't have a cofounder, what should you do? Get one. It's more important than anything else. If there's no one where you live who wants to start a startup with you, move where there are people who do. If no one wants to work with you on your current idea, switch to an idea people want to work on.

If you're still in school, you're surrounded by potential cofounders. A few years out it gets harder to find them. Not only do you have a smaller pool to draw from, but most already have jobs, and perhaps even families to support. So if you had friends in college you used to scheme about startups with, stay in touch with them as well as you can. That may help keep the dream alive.

It's possible you could meet a cofounder through something like a user's group or a conference. But I wouldn't be too optimistic. You need to work with someone to know whether you want them as a cofounder. [2]

The real lesson to draw from this is not how to find a cofounder, but that you should start startups when you're young and there are lots of them around.

7. No idea

In a sense, it's not a problem if you don't have a good idea, because most startups change their idea anyway. In the average Y Combinator startup, I'd guess 70% of the idea is new at the end of the first three months. Sometimes it's 100%.

In fact, we're so sure the founders are more important than the initial idea that we're going to try something new this funding cycle. We're going to let people apply with no idea at all. If you want, you can answer the question on the application form that asks what you're going to do with "We have no idea." If you seem really good we'll accept you anyway. We're confident we can sit down with you and cook up some promising project.

Really this just codifies what we do already. We put little weight on the idea. We ask mainly out of politeness. The kind of question on the application form that we really care about is the one where we ask what cool things you've made. If what you've made is version one of a promising startup, so much the better, but the main thing we care about is whether you're good at making things. Being lead developer of a popular open source project counts almost as much.

That solves the problem if you get funded by Y Combinator. What about in the general case? Because in another sense, it is a problem if you don't have an idea. If you start a startup with no idea, what do you do next?

So here's the brief recipe for getting startup ideas. Find something that's missing in your own life, and supply that need—no matter how specific to you it seems. Steve Wozniak built himself a computer; who knew so many other people would want them? A need that's narrow but genuine is a better starting point than one that's broad but hypothetical. So even if the problem is simply that you don't have a date on Saturday night, if you can think of a way to fix that by writing software, you're onto something, because a lot of other people have the same problem.

8. No room for more startups

A lot of people look at the ever-increasing number of startups and think “this can’t continue.” Implicit in their thinking is a fallacy: that there is some limit on the number of startups there could be. But this is false. No one claims there’s any limit on the number of people who can work for salary at 1000-person companies. Why should there be any limit on the number who can work for equity at 5-person companies? [3]

Nearly everyone who works is satisfying some kind of need. Breaking up companies into smaller units doesn’t make those needs go away. Existing needs would probably get satisfied more efficiently by a network of startups than by a few giant, hierarchical organizations, but I don’t think that would mean less opportunity, because satisfying current needs would lead to more. Certainly this tends to be the case in individuals. Nor is there anything wrong with that. We take for granted things that medieval kings would have considered effeminate luxuries, like whole buildings heated to spring temperatures year round. And if things go well, our descendants will take for granted things we would consider shockingly luxurious. There is no absolute standard for material wealth. Health care is a component of it, and that alone is a black hole. For the foreseeable future, people will want ever more material wealth, so there is no limit to the amount of work available for companies, and for startups in particular.

Usually the limited-room fallacy is not expressed directly. Usually it’s implicit in statements like “there are only so many startups Google, Microsoft, and Yahoo can buy.” Maybe, though the list of acquirers is a lot longer than that. And whatever you think of other acquirers, Google is not stupid. The reason big companies buy startups is that they’ve created something valuable. And why should there be any limit to the number of valuable startups companies can acquire, any more than there is a limit to the amount of wealth individual people want? Maybe there would be practical limits on the number of startups any one acquirer could assimilate, but if there is value to be had, in the form of upside that founders are willing to forgo in return for an immediate payment, acquirers will evolve to consume it. Markets are pretty smart that way.

9. Family to support

This one is real. I wouldn’t advise anyone with a family to start a startup. I’m not saying it’s a bad idea, just that I don’t want to take responsibility for advising it. I’m willing to take responsibility for telling 22 year olds to start startups. So what if they fail? They’ll learn a lot, and that job at Microsoft will still be waiting for them if they need it. But I’m not prepared to cross moms.

What you can do, if you have a family and want to start a startup, is start

a consulting business you can then gradually turn into a product business. Empirically the chances of pulling that off seem very small. You're never going to produce Google this way. But at least you'll never be without an income.

Another way to decrease the risk is to join an existing startup instead of starting your own. Being one of the first employees of a startup is a lot like being a founder, in both the good ways and the bad. You'll be roughly $1/n^2$ founder, where n is your employee number.

As with the question of cofounders, the real lesson here is to start startups when you're young.

10. Independently wealthy

This is my excuse for not starting a startup. Startups are stressful. Why do it if you don't need the money? For every "serial entrepreneur," there are probably twenty sane ones who think "Start another company? Are you crazy?"

I've come close to starting new startups a couple times, but I always pull back because I don't want four years of my life to be consumed by random schleps. I know this business well enough to know you can't do it half-heartedly. What makes a good startup founder so dangerous is his willingness to endure infinite schleps.

There is a bit of a problem with retirement, though. Like a lot of people, I like to work. And one of the many weird little problems you discover when you get rich is that a lot of the interesting people you'd like to work with are not rich. They need to work at something that pays the bills. Which means if you want to have them as colleagues, you have to work at something that pays the bills too, even though you don't need to. I think this is what drives a lot of serial entrepreneurs, actually.

That's why I love working on Y Combinator so much. It's an excuse to work on something interesting with people I like.

11. Not ready for commitment

This was my reason for not starting a startup for most of my twenties. Like a lot of people that age, I valued freedom most of all. I was reluctant to do anything that required a commitment of more than a few months. Nor would I have wanted to do anything that completely took over my life the way a startup does. And that's fine. If you want to spend your time travelling around, or playing in a band, or whatever, that's a perfectly legitimate reason not to start a company.

If you start a startup that succeeds, it's going to consume at least three or four years. (If it fails, you'll be done a lot quicker.) So you shouldn't do it if you're

not ready for commitments on that scale. Be aware, though, that if you get a regular job, you'll probably end up working there for as long as a startup would take, and you'll find you have much less spare time than you might expect. So if you're ready to clip on that ID badge and go to that orientation session, you may also be ready to start that startup.

12. Need for structure

I'm told there are people who need structure in their lives. This seems to be a nice way of saying they need someone to tell them what to do. I believe such people exist. There's plenty of empirical evidence: armies, religious cults, and so on. They may even be the majority.

If you're one of these people, you probably shouldn't start a startup. In fact, you probably shouldn't even go to work for one. In a good startup, you don't get told what to do very much. There may be one person whose job title is CEO, but till the company has about twelve people no one should be telling anyone what to do. That's too inefficient. Each person should just do what they need to without anyone telling them.

If that sounds like a recipe for chaos, think about a soccer team. Eleven people manage to work together in quite complicated ways, and yet only in occasional emergencies does anyone tell anyone else what to do. A reporter once asked David Beckham if there were any language problems at Real Madrid, since the players were from about eight different countries. He said it was never an issue, because everyone was so good they never had to talk. They all just did the right thing.

How do you tell if you're independent-minded enough to start a startup? If you'd bristle at the suggestion that you aren't, then you probably are.

13. Fear of uncertainty

Perhaps some people are deterred from starting startups because they don't like the uncertainty. If you go to work for Microsoft, you can predict fairly accurately what the next few years will be like—all too accurately, in fact. If you start a startup, anything might happen.

Well, if you're troubled by uncertainty, I can solve that problem for you: if you start a startup, it will probably fail. Seriously, though, this is not a bad way to think about the whole experience. Hope for the best, but expect the worst. In the worst case, it will at least be interesting. In the best case you might get rich.

No one will blame you if the startup tanks, so long as you made a serious effort. There may once have been a time when employers would regard that as a mark against you, but they wouldn't now. I asked managers at big companies, and

they all said they'd prefer to hire someone who'd tried to start a startup and failed over someone who'd spent the same time working at a big company.

Nor will investors hold it against you, as long as you didn't fail out of laziness or incurable stupidity. I'm told there's a lot of stigma attached to failing in other places—in Europe, for example. Not here. In America, companies, like practically everything else, are disposable.

14. Don't realize what you're avoiding

One reason people who've been out in the world for a year or two make better founders than people straight from college is that they know what they're avoiding. If their startup fails, they'll have to get a job, and they know how much jobs suck.

If you've had summer jobs in college, you may think you know what jobs are like, but you probably don't. Summer jobs at technology companies are not real jobs. If you get a summer job as a waiter, that's a real job. Then you have to carry your weight. But software companies don't hire students for the summer as a source of cheap labor. They do it in the hope of recruiting them when they graduate. So while they're happy if you produce, they don't expect you to.

That will change if you get a real job after you graduate. Then you'll have to earn your keep. And since most of what big companies do is boring, you're going to have to work on boring stuff. Easy, compared to college, but boring. At first it may seem cool to get paid for doing easy stuff, after paying to do hard stuff in college. But that wears off after a few months. Eventually it gets demoralizing to work on dumb stuff, even if it's easy and you get paid a lot.

And that's not the worst of it. The thing that really sucks about having a regular job is the expectation that you're supposed to be there at certain times. Even Google is afflicted with this, apparently. And what this means, as everyone who's had a regular job can tell you, is that there are going to be times when you have absolutely no desire to work on anything, and you're going to have to go to work anyway and sit in front of your screen and pretend to. To someone who likes work, as most good hackers do, this is torture.

In a startup, you skip all that. There's no concept of office hours in most startups. Work and life just get mixed together. But the good thing about that is that no one minds if you have a life at work. In a startup you can do whatever you want most of the time. If you're a founder, what you want to do most of the time is work. But you never have to pretend to.

If you took a nap in your office in a big company, it would seem unprofessional. But if you're starting a startup and you fall asleep in the middle of the day, your

cofounders will just assume you were tired.

15. Parents want you to be a doctor

A significant number of would-be startup founders are probably dissuaded from doing it by their parents. I'm not going to say you shouldn't listen to them. Families are entitled to their own traditions, and who am I to argue with them? But I will give you a couple reasons why a safe career might not be what your parents really want for you.

One is that parents tend to be more conservative for their kids than they would be for themselves. This is actually a rational response to their situation. Parents end up sharing more of their kids' ill fortune than good fortune. Most parents don't mind this; it's part of the job; but it does tend to make them excessively conservative. And erring on the side of conservatism is still erring. In almost everything, reward is proportionate to risk. So by protecting their kids from risk, parents are, without realizing it, also protecting them from rewards. If they saw that, they'd want you to take more risks.

The other reason parents may be mistaken is that, like generals, they're always fighting the last war. If they want you to be a doctor, odds are it's not just because they want you to help the sick, but also because it's a prestigious and lucrative career. [4] But not so lucrative or prestigious as it was when their opinions were formed. When I was a kid in the seventies, a doctor was *the* thing to be. There was a sort of golden triangle involving doctors, Mercedes 450SLs, and tennis. All three vertices now seem pretty dated.

The parents who want you to be a doctor may simply not realize how much things have changed. Would they be that unhappy if you were Steve Jobs instead? So I think the way to deal with your parents' opinions about what you should do is to treat them like feature requests. Even if your only goal is to please them, the way to do that is not simply to give them what they ask for. Instead think about why they're asking for something, and see if there's a better way to give them what they need.

16. A job is the default

This leads us to the last and probably most powerful reason people get regular jobs: it's the default thing to do. Defaults are enormously powerful, precisely because they operate without any conscious choice.

To almost everyone except criminals, it seems an axiom that if you need money, you should get a job. Actually this tradition is not much more than a hundred years old. Before that, the default way to make a living was by farming. It's a bad plan to treat something only a hundred years old as an axiom. By historical

standards, that's something that's changing pretty rapidly.

We may be seeing another such change right now. I've read a lot of economic history, and I understand the startup world pretty well, and it now seems to me fairly likely that we're seeing the beginning of a change like the one from farming to manufacturing.

And you know what? If you'd been around when that change began (around 1000 in Europe) it would have seemed to nearly everyone that running off to the city to make your fortune was a crazy thing to do. Though serfs were in principle forbidden to leave their manors, it can't have been that hard to run away to a city. There were no guards patrolling the perimeter of the village. What prevented most serfs from leaving was that it seemed insanely risky. Leave one's plot of land? Leave the people you'd spent your whole life with, to live in a giant city of three or four thousand complete strangers? How would you live? How would you get food, if you didn't grow it?

Frightening as it seemed to them, it's now the default with us to live by our wits. So if it seems risky to you to start a startup, think how risky it once seemed to your ancestors to live as we do now. Oddly enough, the people who know this best are the very ones trying to get you to stick to the old model. How can Larry and Sergey say you should come work as their employee, when they didn't get jobs themselves?

Now we look back on medieval peasants and wonder how they stood it. How grim it must have been to till the same fields your whole life with no hope of anything better, under the thumb of lords and priests you had to give all your surplus to and acknowledge as your masters. I wouldn't be surprised if one day people look back on what we consider a normal job in the same way. How grim it would be to commute every day to a cubicle in some soulless office complex, and be told what to do by someone you had to acknowledge as a boss—someone who could call you into their office and say “take a seat,” and you'd sit! Imagine having to ask *permission* to release software to users. Imagine being sad on Sunday afternoons because the weekend was almost over, and tomorrow you'd have to get up and go to work. How did they stand it?

It's exciting to think we may be on the cusp of another shift like the one from farming to manufacturing. That's why I care about startups. Startups aren't interesting just because they're a way to make a lot of money. I couldn't care less about other ways to do that, like speculating in securities. At most those are interesting the way puzzles are. There's more going on with startups. They may represent one of those rare, historic shifts in the way wealth is created.

That's ultimately what drives us to work on Y Combinator. We want to make

money, if only so we don't have to stop doing it, but that's not the main goal. There have only been a handful of these great economic shifts in human history. It would be an amazing hack to make one happen faster.

Notes

[1] The only people who lost were us. The angels had convertible debt, so they had first claim on the proceeds of the auction. Y Combinator only got 38 cents on the dollar.

[2] The best kind of organization for that might be an open source project, but those don't involve a lot of face to face meetings. Maybe it would be worth starting one that did.

[3] There need to be some number of big companies to acquire the startups, so the number of big companies couldn't decrease to zero.

[4] Thought experiment: If doctors did the same work, but as impoverished outcasts, which parents would still want their kids to be doctors?

Thanks to Trevor Blackwell, Jessica Livingston, and Robert Morris for reading drafts of this, to the founders of Zenter for letting me use their web-based PowerPoint killer even though it isn't launched yet, and to Ming-Hay Luk of the Berkeley CSUA for inviting me to speak.

Comment on this essay.

How to Disagree

March 2008

The web is turning writing into a conversation. Twenty years ago, writers wrote and readers read. The web lets readers respond, and increasingly they do—in comment threads, on forums, and in their own blog posts.

Many who respond to something disagree with it. That's to be expected. Agreeing tends to motivate people less than disagreeing. And when you agree there's less to say. You could expand on something the author said, but he has probably already explored the most interesting implications. When you disagree you're entering territory he may not have explored.

The result is there's a lot more disagreeing going on, especially measured by the word. That doesn't mean people are getting angrier. The structural change in the way we communicate is enough to account for it. But though it's not anger that's driving the increase in disagreement, there's a danger that the increase in disagreement will make people angrier. Particularly online, where it's easy to say things you'd never say face to face.

If we're all going to be disagreeing more, we should be careful to do it well. What does it mean to disagree well? Most readers can tell the difference between mere name-calling and a carefully reasoned refutation, but I think it would help to put names on the intermediate stages. So here's an attempt at a disagreement hierarchy:

DH0. Name-calling.

This is the lowest form of disagreement, and probably also the most common. We've all seen comments like this:

u r a fag!!!!!!!!!!

But it's important to realize that more articulate name-calling has just as little weight. A comment like

The author is a self-important dilettante.

is really nothing more than a pretentious version of “u r a fag.”

DH1. Ad Hominem.

An ad hominem attack is not quite as weak as mere name-calling. It might actually carry some weight. For example, if a senator wrote an article saying senators’ salaries should be increased, one could respond:

Of course he would say that. He’s a senator.

This wouldn’t refute the author’s argument, but it may at least be relevant to the case. It’s still a very weak form of disagreement, though. If there’s something wrong with the senator’s argument, you should say what it is; and if there isn’t, what difference does it make that he’s a senator?

Saying that an author lacks the authority to write about a topic is a variant of ad hominem—and a particularly useless sort, because good ideas often come from outsiders. The question is whether the author is correct or not. If his lack of authority caused him to make mistakes, point those out. And if it didn’t, it’s not a problem.

DH2. Responding to Tone.

The next level up we start to see responses to the writing, rather than the writer. The lowest form of these is to disagree with the author’s tone. E.g.

I can’t believe the author dismisses intelligent design in such a cavalier fashion.

Though better than attacking the author, this is still a weak form of disagreement. It matters much more whether the author is wrong or right than what his tone is. Especially since tone is so hard to judge. Someone who has a chip on their shoulder about some topic might be offended by a tone that to other readers seemed neutral.

So if the worst thing you can say about something is to criticize its tone, you’re not saying much. Is the author flippant, but correct? Better that than grave and wrong. And if the author is incorrect somewhere, say where.

DH3. Contradiction.

In this stage we finally get responses to what was said, rather than how or by whom. The lowest form of response to an argument is simply to state the opposing case, with little or no supporting evidence.

This is often combined with DH2 statements, as in:

I can't believe the author dismisses intelligent design in such a cavalier fashion. Intelligent design is a legitimate scientific theory.

Contradiction can sometimes have some weight. Sometimes merely seeing the opposing case stated explicitly is enough to see that it's right. But usually evidence will help.

DH4. Counterargument.

At level 4 we reach the first form of convincing disagreement: counterargument. Forms up to this point can usually be ignored as proving nothing. Counterargument might prove something. The problem is, it's hard to say exactly what.

Counterargument is contradiction plus reasoning and/or evidence. When aimed squarely at the original argument, it can be convincing. But unfortunately it's common for counterarguments to be aimed at something slightly different. More often than not, two people arguing passionately about something are actually arguing about two different things. Sometimes they even agree with one another, but are so caught up in their squabble they don't realize it.

There could be a legitimate reason for arguing against something slightly different from what the original author said: when you feel they missed the heart of the matter. But when you do that, you should say explicitly you're doing it.

DH5. Refutation.

The most convincing form of disagreement is refutation. It's also the rarest, because it's the most work. Indeed, the disagreement hierarchy forms a kind of pyramid, in the sense that the higher you go the fewer instances you find.

To refute someone you probably have to quote them. You have to find a "smoking gun," a passage in whatever you disagree with that you feel is mistaken, and then explain why it's mistaken. If you can't find an actual quote to disagree with, you may be arguing with a straw man.

While refutation generally entails quoting, quoting doesn't necessarily imply refutation. Some writers quote parts of things they disagree with to give the appearance of legitimate refutation, then follow with a response as low as DH3 or even DH0.

DH6. Refuting the Central Point.

The force of a refutation depends on what you refute. The most powerful form of disagreement is to refute someone's central point.

Even as high as DH5 we still sometimes see deliberate dishonesty, as when

someone picks out minor points of an argument and refutes those. Sometimes the spirit in which this is done makes it more of a sophisticated form of ad hominem than actual refutation. For example, correcting someone's grammar, or harping on minor mistakes in names or numbers. Unless the opposing argument actually depends on such things, the only purpose of correcting them is to discredit one's opponent.

Truly refuting something requires one to refute its central point, or at least one of them. And that means one has to commit explicitly to what the central point is. So a truly effective refutation would look like:

The author's main point seems to be x. As he says: > » > > But this is wrong for the following reasons. . .

The quotation you point out as mistaken need not be the actual statement of the author's main point. It's enough to refute something it depends upon.

What It Means

Now we have a way of classifying forms of disagreement. What good is it? One thing the disagreement hierarchy *doesn't* give us is a way of picking a winner. DH levels merely describe the form of a statement, not whether it's correct. A DH6 response could still be completely mistaken.

But while DH levels don't set a lower bound on the convincingness of a reply, they do set an upper bound. A DH6 response might be unconvincing, but a DH2 or lower response is always unconvincing.

The most obvious advantage of classifying the forms of disagreement is that it will help people to evaluate what they read. In particular, it will help them to see through intellectually dishonest arguments. An eloquent speaker or writer can give the impression of vanquishing an opponent merely by using forceful words. In fact that is probably the defining quality of a demagogue. By giving names to the different forms of disagreement, we give critical readers a pin for popping such balloons.

Such labels may help writers too. Most intellectual dishonesty is unintentional. Someone arguing against the tone of something he disagrees with may believe he's really saying something. Zooming out and seeing his current position on the disagreement hierarchy may inspire him to try moving up to counterargument or refutation.

But the greatest benefit of disagreeing well is not just that it will make conversations better, but that it will make the people who have them happier. If you study conversations, you find there is a lot more meanness down in DH1 than up in DH6. You don't have to be mean when you have a real point to make. In fact,

you don't want to. If you have something real to say, being mean just gets in the way.

If moving up the disagreement hierarchy makes people less mean, that will make most of them happier. Most people don't really enjoy being mean; they do it because they can't help it.

Thanks to Trevor Blackwell and Jessica Livingston for reading drafts of this.

Related:

— What You Can't Say | | The Age of the Essay

Be Good

April 2008

(This essay is derived from a talk at the 2008 Startup School.)

About a month after we started Y Combinator we came up with the phrase that became our motto: Make something people want. We've learned a lot since then, but if I were choosing now that's still the one I'd pick.

Another thing we tell founders is not to worry too much about the business model, at least at first. Not because making money is unimportant, but because it's so much easier than building something great.

A couple weeks ago I realized that if you put those two ideas together, you get something surprising. Make something people want. Don't worry too much about making money. What you've got is a description of a charity.

When you get an unexpected result like this, it could either be a bug or a new discovery. Either businesses aren't supposed to be like charities, and we've proven by *reductio ad absurdum* that one or both of the principles we began with is false. Or we have a new idea.

I suspect it's the latter, because as soon as this thought occurred to me, a whole bunch of other things fell into place.

Examples

For example, Craigslist. It's not a charity, but they run it like one. And they're astoundingly successful. When you scan down the list of most popular web sites, the number of employees at Craigslist looks like a misprint. Their revenues aren't as high as they could be, but most startups would be happy to trade places with them.

In Patrick O'Brian's novels, his captains always try to get upwind of their opponents. If you're upwind, you decide when and if to engage the other ship. Craigslist is effectively upwind of enormous revenues. They'd face some challenges

if they wanted to make more, but not the sort you face when you're tacking upwind, trying to force a crappy product on ambivalent users by spending ten times as much on sales as on development. [1]

I'm not saying startups should aim to end up like Craigslist. They're a product of unusual circumstances. But they're a good model for the early phases.

Google looked a lot like a charity in the beginning. They didn't have ads for over a year. At year 1, Google was indistinguishable from a nonprofit. If a nonprofit or government organization had started a project to index the web, Google at year 1 is the limit of what they'd have produced.

Back when I was working on spam filters I thought it would be a good idea to have a web-based email service with good spam filtering. I wasn't thinking of it as a company. I just wanted to keep people from getting spammed. But as I thought more about this project, I realized it would probably have to be a company. It would cost something to run, and it would be a pain to fund with grants and donations.

That was a surprising realization. Companies often claim to be benevolent, but it was surprising to realize there were purely benevolent projects that had to be embodied as companies to work.

I didn't want to start another company, so I didn't do it. But if someone had, they'd probably be quite rich now. There was a window of about two years when spam was increasing rapidly but all the big email services had terrible filters. If someone had launched a new, spam-free mail service, users would have flocked to it.

Notice the pattern here? From either direction we get to the same spot. If you start from successful startups, you find they often behaved like nonprofits. And if you start from ideas for nonprofits, you find they'd often make good startups.

Power

How wide is this territory? Would all good nonprofits be good companies? Possibly not. What makes Google so valuable is that their users have money. If you make people with money love you, you can probably get some of it. But could you also base a successful startup on behaving like a nonprofit to people who don't have money? Could you, for example, grow a successful startup out of curing an unfashionable but deadly disease like malaria?

I'm not sure, but I suspect that if you pushed this idea, you'd be surprised how far it would go. For example, people who apply to Y Combinator don't generally have much money, and yet we can profit by helping them, because with our help they could make money. Maybe the situation is similar with malaria. Maybe

an organization that helped lift its weight off a country could benefit from the resulting growth.

I'm not proposing this is a serious idea. I don't know anything about malaria. But I've been kicking ideas around long enough to know when I come across a powerful one.

One way to guess how far an idea extends is to ask yourself at what point you'd bet against it. The thought of betting against benevolence is alarming in the same way as saying that something is technically impossible. You're just asking to be made a fool of, because these are such powerful forces. [2]

For example, initially I thought maybe this principle only applied to Internet startups. Obviously it worked for Google, but what about Microsoft? Surely Microsoft isn't benevolent? But when I think back to the beginning, they were. Compared to IBM they were like Robin Hood. When IBM introduced the PC, they thought they were going to make money selling hardware at high prices. But by gaining control of the PC standard, Microsoft opened up the market to any manufacturer. Hardware prices plummeted, and lots of people got to have computers who couldn't otherwise have afforded them. It's the sort of thing you'd expect Google to do.

Microsoft isn't so benevolent now. Now when one thinks of what Microsoft does to users, all the verbs that come to mind begin with F. [3] And yet it doesn't seem to pay. Their stock price has been flat for years. Back when they were Robin Hood, their stock price rose like Google's. Could there be a connection?

You can see how there would be. When you're small, you can't bully customers, so you have to charm them. Whereas when you're big you can maltreat them at will, and you tend to, because it's easier than satisfying them. You grow big by being nice, but you can stay big by being mean.

You get away with it till the underlying conditions change, and then all your victims escape. So "Don't be evil" may be the most valuable thing Paul Buchheit made for Google, because it may turn out to be an elixir of corporate youth. I'm sure they find it constraining, but think how valuable it will be if it saves them from lapsing into the fatal laziness that afflicted Microsoft and IBM.

The curious thing is, this elixir is freely available to any other company. Anyone can adopt "Don't be evil." The catch is that people will hold you to it. So I don't think you're going to see record labels or tobacco companies using this discovery.

Morale

There's a lot of external evidence that benevolence works. But how does it work? One advantage of investing in a large number of startups is that you get a lot

of data about how they work. From what we've seen, being good seems to help startups in three ways: it improves their morale, it makes other people want to help them, and above all, it helps them be decisive.

Morale is tremendously important to a startup—so important that morale alone is almost enough to determine success. Startups are often described as emotional roller-coasters. One minute you're going to take over the world, and the next you're doomed. The problem with feeling you're doomed is not just that it makes you unhappy, but that it makes you *stop working*. So the downhill of the roller-coaster are more of a self fulfilling prophecy than the uphill. If feeling you're going to succeed makes you work harder, that probably improves your chances of succeeding, but if feeling you're going to fail makes you stop working, that practically guarantees you'll fail.

Here's where benevolence comes in. If you feel you're really helping people, you'll keep working even when it seems like your startup is doomed. Most of us have some amount of natural benevolence. The mere fact that someone needs you makes you want to help them. So if you start the kind of startup where users come back each day, you've basically built yourself a giant tamagotchi. You've made something you need to take care of.

Blogger is a famous example of a startup that went through really low lows and survived. At one point they ran out of money and everyone left. Evan Williams came in to work the next day, and there was no one but him. What kept him going? Partly that users needed him. He was hosting thousands of people's blogs. He couldn't just let the site die.

There are many advantages of launching quickly, but the most important may be that once you have users, the tamagotchi effect kicks in. Once you have users to take care of, you're forced to figure out what will make them happy, and that's actually very valuable information.

The added confidence that comes from trying to help people can also help you with investors. One of the founders of Chatterous told me recently that he and his cofounder had decided that this service was something the world needed, so they were going to keep working on it no matter what, even if they had to move back to Canada and live in their parents' basements.

Once they realized this, they stopped caring so much what investors thought about them. They still met with them, but they weren't going to die if they didn't get their money. And you know what? The investors got a lot more interested. They could sense that the Chatterouses were going to do this startup with or without them.

If you're really committed and your startup is cheap to run, you become very

hard to kill. And practically all startups, even the most successful, come close to death at some point. So if doing good for people gives you a sense of mission that makes you harder to kill, that alone more than compensates for whatever you lose by not choosing a more selfish project.

Help

Another advantage of being good is that it makes other people want to help you. This too seems to be an inborn trait in humans.

One of the startups we've funded, Octopart, is currently locked in a classic battle of good versus evil. They're a search site for industrial components. A lot of people need to search for components, and before Octopart there was no good way to do it. That, it turned out, was no coincidence.

Octopart built the right way to search for components. Users like it and they've been growing rapidly. And yet for most of Octopart's life, the biggest distributor, Digi-Key, has been trying to force them take their prices off the site. Octopart is sending them customers for free, and yet Digi-Key is trying to make that traffic stop. Why? Because their current business model depends on overcharging people who have incomplete information about prices. They don't want search to work.

The Octoparts are the nicest guys in the world. They dropped out of the PhD program in physics at Berkeley to do this. They just wanted to fix a problem they encountered in their research. Imagine how much time you could save the world's engineers if they could do searches online. So when I hear that a big, evil company is trying to stop them in order to keep search broken, it makes me really want to help them. It makes me spend more time on the Octoparts than I do with most of the other startups we've funded. It just made me spend several minutes telling you how great they are. Why? Because they're good guys and they're trying to help the world.

If you're benevolent, people will rally around you: investors, customers, other companies, and potential employees. In the long term the most important may be the potential employees. I think everyone knows now that good hackers are much better than mediocre ones. If you can attract the best hackers to work for you, as Google has, you have a big advantage. And the very best hackers tend to be idealistic. They're not desperate for a job. They can work wherever they want. So most want to work on things that will make the world better.

Compass

But the most important advantage of being good is that it acts as a compass. One of the hardest parts of doing a startup is that you have so many choices.

There are just two or three of you, and a thousand things you could do. How do you decide?

Here's the answer: Do whatever's best for your users. You can hold onto this like a rope in a hurricane, and it will save you if anything can. Follow it and it will take you through everything you need to do.

It's even the answer to questions that seem unrelated, like how to convince investors to give you money. If you're a good salesman, you could try to just talk them into it. But the more reliable route is to convince them through your users: if you make something users love enough to tell their friends, you grow exponentially, and that will convince any investor.

Being good is a particularly useful strategy for making decisions in complex situations because it's stateless. It's like telling the truth. The trouble with lying is that you have to remember everything you've said in the past to make sure you don't contradict yourself. If you tell the truth you don't have to remember anything, and that's a really useful property in domains where things happen fast.

For example, Y Combinator has now invested in 80 startups, 57 of which are still alive. (The rest have died or merged or been acquired.) When you're trying to advise 57 startups, it turns out you have to have a stateless algorithm. You can't have ulterior motives when you have 57 things going on at once, because you can't remember them. So our rule is just to do whatever's best for the founders. Not because we're particularly benevolent, but because it's the only algorithm that works on that scale.

When you write something telling people to be good, you seem to be claiming to be good yourself. So I want to say explicitly that I am not a particularly good person. When I was a kid I was firmly in the camp of bad. The way adults used the word good, it seemed to be synonymous with quiet, so I grew up very suspicious of it.

You know how there are some people whose names come up in conversation and everyone says "He's *such* a great guy?" People never say that about me. The best I get is "he means well." I am not claiming to be good. At best I speak good as a second language.

So I'm not suggesting you be good in the usual sanctimonious way. I'm suggesting it because it works. It will work not just as a statement of "values," but as a guide to strategy, and even a design spec for software. Don't just not be evil. Be good.

Notes

[1] Fifty years ago it would have seemed shocking for a public company not to pay dividends. Now many tech companies don't. The markets seem to have figured out how to value potential dividends. Maybe that isn't the last step in this evolution. Maybe markets will eventually get comfortable with potential earnings. (VCs already are, and at least some of them consistently make money.)

I realize this sounds like the stuff one used to hear about the "new economy" during the Bubble. Believe me, I was not drinking that kool-aid at the time. But I'm convinced there were some good ideas buried in Bubble thinking. For example, it's ok to focus on growth instead of profits—but only if the growth is genuine. You can't be buying users; that's a pyramid scheme. But a company with rapid, genuine growth is valuable, and eventually markets learn how to value valuable things.

[2] The idea of starting a company with benevolent aims is currently undervalued, because the kind of people who currently make that their explicit goal don't usually do a very good job.

It's one of the standard career paths of trustafarians to start some vaguely benevolent business. The problem with most of them is that they either have a bogus political agenda or are feebly executed. The trustafarians' ancestors didn't get rich by preserving their traditional culture; maybe people in Bolivia don't want to either. And starting an organic farm, though it's at least straightforwardly benevolent, doesn't help people on the scale that Google does.

Most explicitly benevolent projects don't hold themselves sufficiently accountable. They act as if having good intentions were enough to guarantee good effects.

[3] Users dislike their new operating system so much that they're starting petitions to save the old one. And the old one was nothing special. The hackers within Microsoft must know in their hearts that if the company really cared about users they'd just advise them to switch to OSX.

Thanks to Trevor Blackwell, Paul Buchheit, Jessica Livingston, and Robert Morris for reading drafts of this.

Cities and Ambition

May 2008

Great cities attract ambitious people. You can sense it when you walk around one. In a hundred subtle ways, the city sends you a message: you could do more; you should try harder.

The surprising thing is how different these messages can be. New York tells you, above all: you should make more money. There are other messages too, of course. You should be hipper. You should be better looking. But the clearest message is that you should be richer.

What I like about Boston (or rather Cambridge) is that the message there is: you should be smarter. You really should get around to reading all those books you've been meaning to.

When you ask what message a city sends, you sometimes get surprising answers. As much as they respect brains in Silicon Valley, the message the Valley sends is: you should be more powerful.

That's not quite the same message New York sends. Power matters in New York too of course, but New York is pretty impressed by a billion dollars even if you merely inherited it. In Silicon Valley no one would care except a few real estate agents. What matters in Silicon Valley is how much effect you have on the world. The reason people there care about Larry and Sergey is not their wealth but the fact that they control Google, which affects practically everyone.

How much does it matter what message a city sends? Empirically, the answer seems to be: a lot. You might think that if you had enough strength of mind to do great things, you'd be able to transcend your environment. Where you live should make at most a couple percent difference. But if you look at the historical evidence, it seems to matter more than that. Most people who did great things

were clumped together in a few places where that sort of thing was done at the time.

You can see how powerful cities are from something I wrote about earlier: the case of the Milanese Leonardo. Practically every fifteenth century Italian painter you've heard of was from Florence, even though Milan was just as big. People in Florence weren't genetically different, so you have to assume there was someone born in Milan with as much natural ability as Leonardo. What happened to him?

If even someone with the same natural ability as Leonardo couldn't beat the force of environment, do you suppose you can?

I don't. I'm fairly stubborn, but I wouldn't try to fight this force. I'd rather use it. So I've thought a lot about where to live.

I'd always imagined Berkeley would be the ideal place — that it would basically be Cambridge with good weather. But when I finally tried living there a couple years ago, it turned out not to be. The message Berkeley sends is: you should live better. Life in Berkeley is very civilized. It's probably the place in America where someone from Northern Europe would feel most at home. But it's not humming with ambition.

In retrospect it shouldn't have been surprising that a place so pleasant would attract people interested above all in quality of life. Cambridge with good weather, it turns out, is not Cambridge. The people you find in Cambridge are not there by accident. You have to make sacrifices to live there. It's expensive and somewhat grubby, and the weather's often bad. So the kind of people you find in Cambridge are the kind of people who want to live where the smartest people are, even if that means living in an expensive, grubby place with bad weather.

As of this writing, Cambridge seems to be the intellectual capital of the world. I realize that seems a preposterous claim. What makes it true is that it's more preposterous to claim about anywhere else. American universities currently seem to be the best, judging from the flow of ambitious students. And what US city has a stronger claim? New York? A fair number of smart people, but diluted by a much larger number of neanderthals in suits. The Bay Area has a lot of smart people too, but again, diluted; there are two great universities, but they're far apart. Harvard and MIT are practically adjacent by West Coast standards, and they're surrounded by about 20 other colleges and universities. [1]

Cambridge as a result feels like a town whose main industry is ideas, while New York's is finance and Silicon Valley's is startups.

When you talk about cities in the sense we are, what you're really talking about is collections of people. For a long time cities were the only large collections of people, so you could use the two ideas interchangeably. But we can see how much things are changing from the examples I've mentioned. New York is a classic great city. But Cambridge is just part of a city, and Silicon Valley is not even that. (San Jose is not, as it sometimes claims, the capital of Silicon Valley. It's just 178 square miles at one end of it.)

Maybe the Internet will change things further. Maybe one day the most important community you belong to will be a virtual one, and it won't matter where you live physically. But I wouldn't bet on it. The physical world is very high bandwidth, and some of the ways cities send you messages are quite subtle.

One of the exhilarating things about coming back to Cambridge every spring is walking through the streets at dusk, when you can see into the houses. When you walk through Palo Alto in the evening, you see nothing but the blue glow of TVs. In Cambridge you see shelves full of promising-looking books. Palo Alto was probably much like Cambridge in 1960, but you'd never guess now that there was a university nearby. Now it's just one of the richer neighborhoods in Silicon Valley. [2]

A city speaks to you mostly by accident — in things you see through windows, in conversations you overhear. It's not something you have to seek out, but something you can't turn off. One of the occupational hazards of living in Cambridge is overhearing the conversations of people who use interrogative intonation in declarative sentences. But on average I'll take Cambridge conversations over New York or Silicon Valley ones.

A friend who moved to Silicon Valley in the late 90s said the worst thing about living there was the low quality of the eavesdropping. At the time I thought she was being deliberately eccentric. Sure, it can be interesting to eavesdrop on people, but is good quality eavesdropping so important that it would affect where you chose to live? Now I understand what she meant. The conversations you overhear tell you what sort of people you're among.

No matter how determined you are, it's hard not to be influenced by the people around you. It's not so much that you do whatever a city expects of you, but that you get discouraged when no one around you cares about the same things you do.

There's an imbalance between encouragement and discouragement like that

between gaining and losing money. Most people overvalue negative amounts of money: they'll work much harder to avoid losing a dollar than to gain one. Similarly, although there are plenty of people strong enough to resist doing something just because that's what one is supposed to do where they happen to be, there are few strong enough to keep working on something no one around them cares about.

Because ambitions are to some extent incompatible and admiration is a zero-sum game, each city tends to focus on one type of ambition. The reason Cambridge is the intellectual capital is not just that there's a concentration of smart people there, but that there's nothing *else* people there care about more. Professors in New York and the Bay area are second class citizens — till they start hedge funds or startups respectively.

This suggests an answer to a question people in New York have wondered about since the Bubble: whether New York could grow into a startup hub to rival Silicon Valley. One reason that's unlikely is that someone starting a startup in New York would feel like a second class citizen. [3] There's already something else people in New York admire more.

In the long term, that could be a bad thing for New York. The power of an important new technology does eventually convert to money. So by caring more about money and less about power than Silicon Valley, New York is recognizing the same thing, but slower. [4] And in fact it has been losing to Silicon Valley at its own game: the ratio of New York to California residents in the Forbes 400 has decreased from 1.45 (81:56) when the list was first published in 1982 to .83 (73:88) in 2007.

Not all cities send a message. Only those that are centers for some type of ambition do. And it can be hard to tell exactly what message a city sends without living there. I understand the messages of New York, Cambridge, and Silicon Valley because I've lived for several years in each of them. DC and LA seem to send messages too, but I haven't spent long enough in either to say for sure what they are.

The big thing in LA seems to be fame. There's an A List of people who are most in demand right now, and what's most admired is to be on it, or friends with those who are. Beneath that, the message is much like New York's, though perhaps with more emphasis on physical attractiveness.

In DC the message seems to be that the most important thing is who you know. You want to be an insider. In practice this seems to work much as in LA. There's

an A List and you want to be on it or close to those who are. The only difference is how the A List is selected. And even that is not that different.

At the moment, San Francisco's message seems to be the same as Berkeley's: you should live better. But this will change if enough startups choose SF over the Valley. During the Bubble that was a predictor of failure — a self-indulgent choice, like buying expensive office furniture. Even now I'm suspicious when startups choose SF. But if enough good ones do, it stops being a self-indulgent choice, because the center of gravity of Silicon Valley will shift there.

I haven't found anything like Cambridge for intellectual ambition. Oxford and Cambridge (England) feel like Ithaca or Hanover: the message is there, but not as strong.

Paris was once a great intellectual center. If you went there in 1300, it might have sent the message Cambridge does now. But I tried living there for a bit last year, and the ambitions of the inhabitants are not intellectual ones. The message Paris sends now is: do things with style. I liked that, actually. Paris is the only city I've lived in where people genuinely cared about art. In America only a few rich people buy original art, and even the more sophisticated ones rarely get past judging it by the brand name of the artist. But looking through windows at dusk in Paris you can see that people there actually care what paintings look like. Visually, Paris has the best eavesdropping I know. [5]

There's one more message I've heard from cities: in London you can still (barely) hear the message that one should be more aristocratic. If you listen for it you can also hear it in Paris, New York, and Boston. But this message is everywhere very faint. It would have been strong 100 years ago, but now I probably wouldn't have picked it up at all if I hadn't deliberately tuned in to that wavelength to see if there was any signal left.

So far the complete list of messages I've picked up from cities is: wealth, style, hipness, physical attractiveness, fame, political power, economic power, intelligence, social class, and quality of life.

My immediate reaction to this list is that it makes me slightly queasy. I'd always considered ambition a good thing, but I realize now that was because I'd always implicitly understood it to mean ambition in the areas I cared about. When you list everything ambitious people are ambitious about, it's not so pretty.

On closer examination I see a couple things on the list that are surprising in the light of history. For example, physical attractiveness wouldn't have been there 100 years ago (though it might have been 2400 years ago). It has always

mattered for women, but in the late twentieth century it seems to have started to matter for men as well. I'm not sure why — probably some combination of the increasing power of women, the increasing influence of actors as models, and the fact that so many people work in offices now: you can't show off by wearing clothes too fancy to wear in a factory, so you have to show off with your body instead.

Hipness is another thing you wouldn't have seen on the list 100 years ago. Or wouldn't you? What it means is to know what's what. So maybe it has simply replaced the component of social class that consisted of being "au fait." That could explain why hipness seems particularly admired in London: it's version 2 of the traditional English delight in obscure codes that only insiders understand.

Economic power would have been on the list 100 years ago, but what we mean by it is changing. It used to mean the control of vast human and material resources. But increasingly it means the ability to direct the course of technology, and some of the people in a position to do that are not even rich — leaders of important open source projects, for example. The Captains of Industry of times past had laboratories full of clever people cooking up new technologies for them. The new breed are themselves those people.

As this force gets more attention, another is dropping off the list: social class. I think the two changes are related. Economic power, wealth, and social class are just names for the same thing at different stages in its life: economic power converts to wealth, and wealth to social class. So the focus of admiration is simply shifting upstream.

Does anyone who wants to do great work have to live in a great city? No; all great cities inspire some sort of ambition, but they aren't the only places that do. For some kinds of work, all you need is a handful of talented colleagues.

What cities provide is an audience, and a funnel for peers. These aren't so critical in something like math or physics, where no audience matters except your peers, and judging ability is sufficiently straightforward that hiring and admissions committees can do it reliably. In a field like math or physics all you need is a department with the right colleagues in it. It could be anywhere — in Los Alamos, New Mexico, for example.

It's in fields like the arts or writing or technology that the larger environment matters. In these the best practitioners aren't conveniently collected in a few top university departments and research labs — partly because talent is harder to judge, and partly because people pay for these things, so one doesn't need to rely on teaching or research funding to support oneself. It's in these more chaotic

fields that it helps most to be in a great city: you need the encouragement of feeling that people around you care about the kind of work you do, and since you have to find peers for yourself, you need the much larger intake mechanism of a great city.

You don't have to live in a great city your whole life to benefit from it. The critical years seem to be the early and middle ones of your career. Clearly you don't have to grow up in a great city. Nor does it seem to matter if you go to college in one. To most college students a world of a few thousand people seems big enough. Plus in college you don't yet have to face the hardest kind of work — discovering new problems to solve.

It's when you move on to the next and much harder step that it helps most to be in a place where you can find peers and encouragement. You seem to be able to leave, if you want, once you've found both. The Impressionists show the typical pattern: they were born all over France (Pissarro was born in the Carriibbean) and died all over France, but what defined them were the years they spent together in Paris.

Unless you're sure what you want to do and where the leading center for it is, your best bet is probably to try living in several places when you're young. You can never tell what message a city sends till you live there, or even whether it still sends one. Often your information will be wrong: I tried living in Florence when I was 25, thinking it would be an art center, but it turned out I was 450 years too late.

Even when a city is still a live center of ambition, you won't know for sure whether its message will resonate with you till you hear it. When I moved to New York, I was very excited at first. It's an exciting place. So it took me quite a while to realize I just wasn't like the people there. I kept searching for the Cambridge of New York. It turned out it was way, way uptown: an hour uptown by air.

Some people know at 16 what sort of work they're going to do, but in most ambitious kids, ambition seems to precede anything specific to be ambitious about. They know they want to do something great. They just haven't decided yet whether they're going to be a rock star or a brain surgeon. There's nothing wrong with that. But it means if you have this most common type of ambition, you'll probably have to figure out where to live by trial and error. You'll probably have to find the city where you feel at home to know what sort of ambition you have.

Notes

[1] This is one of the advantages of not having the universities in your country controlled by the government. When governments decide how to allocate resources, political deal-making causes things to be spread out geographically. No central government would put its two best universities in the same town, unless it was the capital (which would cause other problems). But scholars seem to like to cluster together as much as people in any other field, and when given the freedom to they derive the same advantages from it.

[2] There are still a few old professors in Palo Alto, but one by one they die and their houses are transformed by developers into McMansions and sold to VPs of Bus Dev.

[3] How many times have you read about startup founders who continued to live inexpensively as their companies took off? Who continued to dress in jeans and t-shirts, to drive the old car they had in grad school, and so on? If you did that in New York, people would treat you like shit. If you walk into a fancy restaurant in San Francisco wearing a jeans and a t-shirt, they're nice to you; who knows who you might be? Not in New York.

One sign of a city's potential as a technology center is the number of restaurants that still require jackets for men. According to Zagat's there are none in San Francisco, LA, Boston, or Seattle, 4 in DC, 6 in Chicago, 8 in London, 13 in New York, and 20 in Paris.

(Zagat's lists the Ritz Carlton Dining Room in SF as requiring jackets but I couldn't believe it, so I called to check and in fact they don't. Apparently there's only one restaurant left on the entire West Coast that still requires jackets: The French Laundry in Napa Valley.)

[4] Ideas are one step upstream from economic power, so it's conceivable that intellectual centers like Cambridge will one day have an edge over Silicon Valley like the one the Valley has over New York.

This seems unlikely at the moment; if anything Boston is falling further and further behind. The only reason I even mention the possibility is that the path from ideas to startups has recently been getting smoother. It's a lot easier now for a couple of hackers with no business experience to start a startup than it was 10 years ago. If you extrapolate another 20 years, maybe the balance of power will start to shift back. I wouldn't bet on it, but I wouldn't bet against it either.

[5] If Paris is where people care most about art, why is New York the center of gravity of the art business? Because in the twentieth century, art as brand split apart from art as stuff. New York is where the richest buyers are, but all they demand from art is brand, and since you can base brand on anything with a sufficiently identifiable style, you may as well use the local stuff.

Thanks to Trevor Blackwell, Sarah Harlin, Jessica Livingston, Jackie McDonough, Robert Morris, and David Sloo for reading drafts of this.

Lies We Tell Kids

May 2008

Adults lie constantly to kids. I'm not saying we should stop, but I think we should at least examine which lies we tell and why.

There may also be a benefit to us. We were all lied to as kids, and some of the lies we were told still affect us. So by studying the ways adults lie to kids, we may be able to clear our heads of lies we were told.

I'm using the word "lie" in a very general sense: not just overt falsehoods, but also all the more subtle ways we mislead kids. Though "lie" has negative connotations, I don't mean to suggest we should never do this—just that we should pay attention when we do. [1]

One of the most remarkable things about the way we lie to kids is how broad the conspiracy is. All adults know what their culture lies to kids about: they're the questions you answer "Ask your parents." If a kid asked who won the World Series in 1982 or what the atomic weight of carbon was, you could just tell him. But if a kid asks you "Is there a God?" or "What's a prostitute?" you'll probably say "Ask your parents."

Since we all agree, kids see few cracks in the view of the world presented to them. The biggest disagreements are between parents and schools, but even those are small. Schools are careful what they say about controversial topics, and if they do contradict what parents want their kids to believe, parents either pressure the school into keeping quiet or move their kids to a new school.

The conspiracy is so thorough that most kids who discover it do so only by discovering internal contradictions in what they're told. It can be traumatic for the ones who wake up during the operation. Here's what happened to Einstein:

Through the reading of popular scientific books I soon reached the conviction that much in the stories of the Bible could not be true. The consequence was a positively fanatic freethinking coupled with

the impression that youth is intentionally being deceived by the state through lies: it was a crushing impression. [2]

I remember that feeling. By 15 I was convinced the world was corrupt from end to end. That's why movies like *The Matrix* have such resonance. Every kid grows up in a fake world. In a way it would be easier if the forces behind it were as clearly differentiated as a bunch of evil machines, and one could make a clean break just by taking a pill.

Protection

If you ask adults why they lie to kids, the most common reason they give is to protect them. And kids do need protecting. The environment you want to create for a newborn child will be quite unlike the streets of a big city.

That seems so obvious it seems wrong to call it a lie. It's certainly not a bad lie to tell, to give a baby the impression the world is quiet and warm and safe. But this harmless type of lie can turn sour if left unexamined.

Imagine if you tried to keep someone in as protected an environment as a newborn till age 18. To mislead someone so grossly about the world would seem not protection but abuse. That's an extreme example, of course; when parents do that sort of thing it becomes national news. But you see the same problem on a smaller scale in the malaise teenagers feel in suburbia.

The main purpose of suburbia is to provide a protected environment for children to grow up in. And it seems great for 10 year olds. I liked living in suburbia when I was 10. I didn't notice how sterile it was. My whole world was no bigger than a few friends' houses I bicycled to and some woods I ran around in. On a log scale I was midway between crib and globe. A suburban street was just the right size. But as I grew older, suburbia started to feel suffocatingly fake.

Life can be pretty good at 10 or 20, but it's often frustrating at 15. This is too big a problem to solve here, but certainly one reason life sucks at 15 is that kids are trapped in a world designed for 10 year olds.

What do parents hope to protect their children from by raising them in suburbia? A friend who moved out of Manhattan said merely that her 3 year old daughter "saw too much." Off the top of my head, that might include: people who are high or drunk, poverty, madness, gruesome medical conditions, sexual behavior of various degrees of oddness, and violent anger.

I think it's the anger that would worry me most if I had a 3 year old. I was 29 when I moved to New York and I was surprised even then. I wouldn't want a 3 year old to see some of the disputes I saw. It would be too frightening. A lot of the things adults conceal from smaller children, they conceal because they'd

be frightening, not because they want to conceal the existence of such things. Misleading the child is just a byproduct.

This seems one of the most justifiable types of lying adults do to kids. But because the lies are indirect we don't keep a very strict accounting of them. Parents know they've concealed the facts about sex, and many at some point sit their kids down and explain more. But few tell their kids about the differences between the real world and the cocoon they grew up in. Combine this with the confidence parents try to instill in their kids, and every year you get a new crop of 18 year olds who think they know how to run the world.

Don't all 18 year olds think they know how to run the world? Actually this seems to be a recent innovation, no more than about 100 years old. In preindustrial times teenage kids were junior members of the adult world and comparatively well aware of their shortcomings. They could see they weren't as strong or skillful as the village smith. In past times people lied to kids about some things more than we do now, but the lies implicit in an artificial, protected environment are a recent invention. Like a lot of new inventions, the rich got this first. Children of kings and great magnates were the first to grow up out of touch with the world. Suburbia means half the population can live like kings in that respect.

Sex (and Drugs)

I'd have different worries about raising teenage kids in New York. I'd worry less about what they'd see, and more about what they'd do. I went to college with a lot of kids who grew up in Manhattan, and as a rule they seemed pretty jaded. They seemed to have lost their virginity at an average of about 14 and by college had tried more drugs than I'd even heard of.

The reasons parents don't want their teenage kids having sex are complex. There are some obvious dangers: pregnancy and sexually transmitted diseases. But those aren't the only reasons parents don't want their kids having sex. The average parents of a 14 year old girl would hate the idea of her having sex even if there were zero risk of pregnancy or sexually transmitted diseases.

Kids can probably sense they aren't being told the whole story. After all, pregnancy and sexually transmitted diseases are just as much a problem for adults, and they have sex.

What really bothers parents about their teenage kids having sex? Their dislike of the idea is so visceral it's probably inborn. But if it's inborn it should be universal, and there are plenty of societies where parents don't mind if their teenage kids have sex—indeed, where it's normal for 14 year olds to become mothers. So what's going on? There does seem to be a universal taboo against sex with prepubescent children. One can imagine evolutionary reasons for that.

And I think this is the main reason parents in industrialized societies dislike teenage kids having sex. They still think of them as children, even though biologically they're not, so the taboo against child sex still has force.

One thing adults conceal about sex they also conceal about drugs: that it can cause great pleasure. That's what makes sex and drugs so dangerous. The desire for them can cloud one's judgement—which is especially frightening when the judgement being clouded is the already wretched judgement of a teenage kid.

Here parents' desires conflict. Older societies told kids they had bad judgement, but modern parents want their children to be confident. This may well be a better plan than the old one of putting them in their place, but it has the side effect that after having implicitly lied to kids about how good their judgement is, we then have to lie again about all the things they might get into trouble with if they believed us.

If parents told their kids the truth about sex and drugs, it would be: the reason you should avoid these things is that you have lousy judgement. People with twice your experience still get burned by them. But this may be one of those cases where the truth wouldn't be convincing, because one of the symptoms of bad judgement is believing you have good judgement. When you're too weak to lift something, you can tell, but when you're making a decision impetuously, you're all the more sure of it.

Innocence

Another reason parents don't want their kids having sex is that they want to keep them innocent. Adults have a certain model of how kids are supposed to behave, and it's different from what they expect of other adults.

One of the most obvious differences is the words kids are allowed to use. Most parents use words when talking to other adults that they wouldn't want their kids using. They try to hide even the existence of these words for as long as they can. And this is another of those conspiracies everyone participates in: everyone knows you're not supposed to swear in front of kids.

I've never heard more different explanations for anything parents tell kids than why they shouldn't swear. Every parent I know forbids their children to swear, and yet no two of them have the same justification. It's clear most start with not wanting kids to swear, then make up the reason afterward.

So my theory about what's going on is that the *function* of swearwords is to mark the speaker as an adult. There's no difference in the meaning of "shit" and "poopoo." So why should one be ok for kids to say and one forbidden? The only explanation is: by definition. [3]

Why does it bother adults so much when kids do things reserved for adults? The idea of a foul-mouthed, cynical 10 year old leaning against a lamppost with a cigarette hanging out of the corner of his mouth is very disconcerting. But why?

One reason we want kids to be innocent is that we're programmed to like certain kinds of helplessness. I've several times heard mothers say they deliberately refrained from correcting their young children's mispronunciations because they were so cute. And if you think about it, cuteness is helplessness. Toys and cartoon characters meant to be cute always have clueless expressions and stubby, ineffectual limbs.

It's not surprising we'd have an inborn desire to love and protect helpless creatures, considering human offspring are so helpless for so long. Without the helplessness that makes kids cute, they'd be very annoying. They'd merely seem like incompetent adults. But there's more to it than that. The reason our hypothetical jaded 10 year old bothers me so much is not just that he'd be annoying, but that he'd have cut off his prospects for growth so early. To be jaded you have to think you know how the world works, and any theory a 10 year old had about that would probably be a pretty narrow one.

Innocence is also open-mindedness. We want kids to be innocent so they can continue to learn. Paradoxical as it sounds, there are some kinds of knowledge that get in the way of other kinds of knowledge. If you're going to learn that the world is a brutal place full of people trying to take advantage of one another, you're better off learning it last. Otherwise you won't bother learning much more.

Very smart adults often seem unusually innocent, and I don't think this is a coincidence. I think they've deliberately avoided learning about certain things. Certainly I do. I used to think I wanted to know everything. Now I know I don't.

Death

After sex, death is the topic adults lie most conspicuously about to kids. Sex I believe they conceal because of deep taboos. But why do we conceal death from kids? Probably because small children are particularly horrified by it. They want to feel safe, and death is the ultimate threat.

One of the most spectacular lies our parents told us was about the death of our first cat. Over the years, as we asked for more details, they were compelled to invent more, so the story grew quite elaborate. The cat had died at the vet's office. Of what? Of the anaesthesia itself. Why was the cat at the vet's office? To be fixed. And why had such a routine operation killed it? It wasn't the vet's fault; the cat had a congenitally weak heart; the anaesthesia was too much for it; but there was no way anyone could have known this in advance. It was not till

we were in our twenties that the truth came out: my sister, then about three, had accidentally stepped on the cat and broken its back.

They didn't feel the need to tell us the cat was now happily in cat heaven. My parents never claimed that people or animals who died had "gone to a better place," or that we'd meet them again. It didn't seem to harm us.

My grandmother told us an edited version of the death of my grandfather. She said they'd been sitting reading one day, and when she said something to him, he didn't answer. He seemed to be asleep, but when she tried to rouse him, she couldn't. "He was gone." Having a heart attack sounded like falling asleep. Later I learned it hadn't been so neat, and the heart attack had taken most of a day to kill him.

Along with such outright lies, there must have been a lot of changing the subject when death came up. I can't remember that, of course, but I can infer it from the fact that I didn't really grasp I was going to die till I was about 19. How could I have missed something so obvious for so long? Now that I've seen parents managing the subject, I can see how: questions about death are gently but firmly turned aside.

On this topic, especially, they're met half-way by kids. Kids often want to be lied to. They want to believe they're living in a comfortable, safe world as much as their parents want them to believe it. [4]

Identity

Some parents feel a strong adherence to an ethnic or religious group and want their kids to feel it too. This usually requires two different kinds of lying: the first is to tell the child that he or she is an X, and the second is whatever specific lies Xes differentiate themselves by believing. [5]

Telling a child they have a particular ethnic or religious identity is one of the stickiest things you can tell them. Almost anything else you tell a kid, they can change their mind about later when they start to think for themselves. But if you tell a kid they're a member of a certain group, that seems nearly impossible to shake.

This despite the fact that it can be one of the most premeditated lies parents tell. When parents are of different religions, they'll often agree between themselves that their children will be "raised as Xes." And it works. The kids obligingly grow up considering themselves as Xes, despite the fact that if their parents had chosen the other way, they'd have grown up considering themselves as Ys.

One reason this works so well is the second kind of lie involved. The truth is common property. You can't distinguish your group by doing things that are

rational, and believing things that are true. If you want to set yourself apart from other people, you have to do things that are arbitrary, and believe things that are false. And after having spent their whole lives doing things that are arbitrary and believing things that are false, and being regarded as odd by “outsiders” on that account, the cognitive dissonance pushing children to regard themselves as Xes must be enormous. If they aren’t an X, why are they attached to all these arbitrary beliefs and customs? If they aren’t an X, why do all the non-Xes call them one?

This form of lie is not without its uses. You can use it to carry a payload of beneficial beliefs, and they will also become part of the child’s identity. You can tell the child that in addition to never wearing the color yellow, believing the world was created by a giant rabbit, and always snapping their fingers before eating fish, Xes are also particularly honest and industrious. Then X children will grow up feeling it’s part of their identity to be honest and industrious.

This probably accounts for a lot of the spread of modern religions, and explains why their doctrines are a combination of the useful and the bizarre. The bizarre half is what makes the religion stick, and the useful half is the payload. [6]

Authority

One of the least excusable reasons adults lie to kids is to maintain power over them. Sometimes these lies are truly sinister, like a child molester telling his victims they’ll get in trouble if they tell anyone what happened to them. Others seem more innocent; it depends how badly adults lie to maintain their power, and what they use it for.

Most adults make some effort to conceal their flaws from children. Usually their motives are mixed. For example, a father who has an affair generally conceals it from his children. His motive is partly that it would worry them, partly that this would introduce the topic of sex, and partly (a larger part than he would admit) that he doesn’t want to tarnish himself in their eyes.

If you want to learn what lies are told to kids, read almost any book written to teach them about “issues.” [7] Peter Mayle wrote one called *Why Are We Getting a Divorce?* It begins with the three most important things to remember about divorce, one of which is:

You shouldn’t put the blame on one parent, because divorce is never only one person’s fault. [8]

Really? When a man runs off with his secretary, is it always partly his wife’s fault? But I can see why Mayle might have said this. Maybe it’s more important for kids to respect their parents than to know the truth about them.

But because adults conceal their flaws, and at the same time insist on high standards of behavior for kids, a lot of kids grow up feeling they fall hopelessly short. They walk around feeling horribly evil for having used a swearword, while in fact most of the adults around them are doing much worse things.

This happens in intellectual as well as moral questions. The more confident people are, the more willing they seem to be to answer a question “I don’t know.” Less confident people feel they have to have an answer or they’ll look bad. My parents were pretty good about admitting when they didn’t know things, but I must have been told a lot of lies of this type by teachers, because I rarely heard a teacher say “I don’t know” till I got to college. I remember because it was so surprising to hear someone say that in front of a class.

The first hint I had that teachers weren’t omniscient came in sixth grade, after my father contradicted something I’d learned in school. When I protested that the teacher had said the opposite, my father replied that the guy had no idea what he was talking about—that he was just an elementary school teacher, after all.

Just a teacher? The phrase seemed almost grammatically ill-formed. Didn’t teachers know everything about the subjects they taught? And if not, why were they the ones teaching us?

The sad fact is, US public school teachers don’t generally understand the stuff they’re teaching very well. There are some sterling exceptions, but as a rule people planning to go into teaching rank academically near the bottom of the college population. So the fact that I still thought at age 11 that teachers were infallible shows what a job the system must have done on my brain.

School

What kids get taught in school is a complex mix of lies. The most excusable are those told to simplify ideas to make them easy to learn. The problem is, a lot of propaganda gets slipped into the curriculum in the name of simplification.

Public school textbooks represent a compromise between what various powerful groups want kids to be told. The lies are rarely overt. Usually they consist either of omissions or of over-emphasizing certain topics at the expense of others. The view of history we got in elementary school was a crude hagiography, with at least one representative of each powerful group.

The famous scientists I remember were Einstein, Marie Curie, and George Washington Carver. Einstein was a big deal because his work led to the atom bomb. Marie Curie was involved with X-rays. But I was mystified about Carver. He seemed to have done stuff with peanuts.

It's obvious now that he was on the list because he was black (and for that matter that Marie Curie was on it because she was a woman), but as a kid I was confused for years about him. I wonder if it wouldn't have been better just to tell us the truth: that there weren't any famous black scientists. Ranking George Washington Carver with Einstein misled us not only about science, but about the obstacles blacks faced in his time.

As subjects got softer, the lies got more frequent. By the time you got to politics and recent history, what we were taught was pretty much pure propaganda. For example, we were taught to regard political leaders as saints—especially the recently martyred Kennedy and King. It was astonishing to learn later that they'd both been serial womanizers, and that Kennedy was a speed freak to boot. (By the time King's plagiarism emerged, I'd lost the ability to be surprised by the misdeeds of famous people.)

I doubt you could teach kids recent history without teaching them lies, because practically everyone who has anything to say about it has some kind of spin to put on it. Much recent history *consists* of spin. It would probably be better just to teach them metafacts like that.

Probably the biggest lie told in schools, though, is that the way to succeed is through following "the rules." In fact most such rules are just hacks to manage large groups efficiently.

Peace

Of all the reasons we lie to kids, the most powerful is probably the same mundane reason they lie to us.

Often when we lie to people it's not part of any conscious strategy, but because they'd react violently to the truth. Kids, almost by definition, lack self-control. They react violently to things—and so they get lied to a lot. [9]

A few Thanksgivings ago, a friend of mine found himself in a situation that perfectly illustrates the complex motives we have when we lie to kids. As the roast turkey appeared on the table, his alarmingly perceptive 5 year old son suddenly asked if the turkey had wanted to die. Foreseeing disaster, my friend and his wife rapidly improvised: yes, the turkey had wanted to die, and in fact had lived its whole life with the aim of being their Thanksgiving dinner. And that (phew) was the end of that.

Whenever we lie to kids to protect them, we're usually also lying to keep the peace.

One consequence of this sort of calming lie is that we grow up thinking horrible things are normal. It's hard for us to feel a sense of urgency as adults over

something we've literally been trained not to worry about. When I was about 10 I saw a documentary on pollution that put me into a panic. It seemed the planet was being irretrievably ruined. I went to my mother afterward to ask if this was so. I don't remember what she said, but she made me feel better, so I stopped worrying about it.

That was probably the best way to handle a frightened 10 year old. But we should understand the price. This sort of lie is one of the main reasons bad things persist: we're all trained to ignore them.

Detox

A sprinter in a race almost immediately enters a state called "oxygen debt." His body switches to an emergency source of energy that's faster than regular aerobic respiration. But this process builds up waste products that ultimately require extra oxygen to break down, so at the end of the race he has to stop and pant for a while to recover.

We arrive at adulthood with a kind of truth debt. We were told a lot of lies to get us (and our parents) through our childhood. Some may have been necessary. Some probably weren't. But we all arrive at adulthood with heads full of lies.

There's never a point where the adults sit you down and explain all the lies they told you. They've forgotten most of them. So if you're going to clear these lies out of your head, you're going to have to do it yourself.

Few do. Most people go through life with bits of packing material adhering to their minds and never know it. You probably never can completely undo the effects of lies you were told as a kid, but it's worth trying. I've found that whenever I've been able to undo a lie I was told, a lot of other things fell into place.

Fortunately, once you arrive at adulthood you get a valuable new resource you can use to figure out what lies you were told. You're now one of the liars. You get to watch behind the scenes as adults spin the world for the next generation of kids.

The first step in clearing your head is to realize how far you are from a neutral observer. When I left high school I was, I thought, a complete skeptic. I'd realized high school was crap. I thought I was ready to question everything I knew. But among the many other things I was ignorant of was how much debris there already was in my head. It's not enough to consider your mind a blank slate. You have to consciously erase it.

Notes

[1] One reason I stuck with such a brutally simple word is that the lies we tell kids are probably not quite as harmless as we think. If you look at what adults told children in the past, it's shocking how much they lied to them. Like us, they did it with the best intentions. So if we think we're as open as one could reasonably be with children, we're probably fooling ourselves. Odds are people in 100 years will be as shocked at some of the lies we tell as we are at some of the lies people told 100 years ago.

I can't predict which these will be, and I don't want to write an essay that will seem dumb in 100 years. So instead of using special euphemisms for lies that seem excusable according to present fashions, I'm just going to call all our lies lies.

(I have omitted one type: lies told to play games with kids' credulity. These range from "make-believe," which is not really a lie because it's told with a wink, to the frightening lies told by older siblings. There's not much to say about these: I wouldn't want the first type to go away, and wouldn't expect the second type to.)

[2] Calaprice, Alice (ed.), *The Quotable Einstein*, Princeton University Press, 1996.

[3] If you ask parents why kids shouldn't swear, the less educated ones usually reply with some question-begging answer like "it's inappropriate," while the more educated ones come up with elaborate rationalizations. In fact the less educated parents seem closer to the truth.

[4] As a friend with small children pointed out, it's easy for small children to consider themselves immortal, because time seems to pass so slowly for them. To a 3 year old, a day feels like a month might to an adult. So 80 years sounds to him like 2400 years would to us.

[5] I realize I'm going to get endless grief for classifying religion as a type of lie. Usually people skirt that issue with some equivocation implying that lies believed for a sufficiently long time by sufficiently large numbers of people are immune to the usual standards for truth. But because I can't predict which lies future generations will consider inexcusable, I can't safely omit any type we tell. Yes, it seems unlikely that religion will be out of fashion in 100 years, but no more unlikely than it would have seemed to someone in 1880 that schoolchildren in 1980 would be taught that masturbation was perfectly normal and not to feel guilty about it.

[6] Unfortunately the payload can consist of bad customs as well as good ones. For example, there are certain qualities that some groups in America consider "acting white." In fact most of them could as accurately be called "acting Japanese."

There's nothing specifically white about such customs. They're common to all cultures with long traditions of living in cities. So it is probably a losing bet for a group to consider behaving the opposite way as part of its identity.

[7] In this context, "issues" basically means "things we're going to lie to them about." That's why there's a special name for these topics.

[8] Mayle, Peter, *Why Are We Getting a Divorce?*, Harmony, 1988.

[9] The ironic thing is, this is also the main reason kids lie to adults. If you freak out when people tell you alarming things, they won't tell you them. Teenagers don't tell their parents what happened that night they were supposed to be staying at a friend's house for the same reason parents don't tell 5 year olds the truth about the Thanksgiving turkey. They'd freak if they knew.

Thanks to Sam Altman, Marc Andreessen, Trevor Blackwell, Patrick Collison, Jessica Livingston, Jackie McDonough, Robert Morris, and David Sloo for reading drafts of this. And since there are some controversial ideas here, I should add that none of them agreed with everything in it.

Part Three — Ideas and Determination

2009–2013

Startups take off because the founders make them take off.

— Paul Graham, “Do Things that Don’t Scale”

Keep Your Identity Small

February 2009

I finally realized today why politics and religion yield such uniquely useless discussions.

As a rule, any mention of religion on an online forum degenerates into a religious argument. Why? Why does this happen with religion and not with Javascript or baking or other topics people talk about on forums?

What's different about religion is that people don't feel they need to have any particular expertise to have opinions about it. All they need is strongly held beliefs, and anyone can have those. No thread about Javascript will grow as fast as one about religion, because people feel they have to be over some threshold of expertise to post comments about that. But on religion everyone's an expert.

Then it struck me: this is the problem with politics too. Politics, like religion, is a topic where there's no threshold of expertise for expressing an opinion. All you need is strong convictions.

Do religion and politics have something in common that explains this similarity? One possible explanation is that they deal with questions that have no definite answers, so there's no back pressure on people's opinions. Since no one can be proven wrong, every opinion is equally valid, and sensing this, everyone lets fly with theirs.

But this isn't true. There are certainly some political questions that have definite answers, like how much a new government policy will cost. But the more precise political questions suffer the same fate as the vaguer ones.

I think what religion and politics have in common is that they become part of people's identity, and people can never have a fruitful argument about something that's part of their identity. By definition they're partisan.

Which topics engage people's identity depends on the people, not the topic. For

example, a discussion about a battle that included citizens of one or more of the countries involved would probably degenerate into a political argument. But a discussion today about a battle that took place in the Bronze Age probably wouldn't. No one would know what side to be on. So it's not politics that's the source of the trouble, but identity. When people say a discussion has degenerated into a religious war, what they really mean is that it has started to be driven mostly by people's identities. [1]

Because the point at which this happens depends on the people rather than the topic, it's a mistake to conclude that because a question tends to provoke religious wars, it must have no answer. For example, the question of the relative merits of programming languages often degenerates into a religious war, because so many programmers identify as X programmers or Y programmers. This sometimes leads people to conclude the question must be unanswerable—that all languages are equally good. Obviously that's false: anything else people make can be well or badly designed; why should this be uniquely impossible for programming languages? And indeed, you can have a fruitful discussion about the relative merits of programming languages, so long as you exclude people who respond from identity.

More generally, you can have a fruitful discussion about a topic only if it doesn't engage the identities of any of the participants. What makes politics and religion such minefields is that they engage so many people's identities. But you could in principle have a useful conversation about them with some people. And there are other topics that might seem harmless, like the relative merits of Ford and Chevy pickup trucks, that you couldn't safely talk about with others.

The most intriguing thing about this theory, if it's right, is that it explains not merely which kinds of discussions to avoid, but how to have better ideas. If people can't think clearly about anything that has become part of their identity, then all other things being equal, the best plan is to let as few things into your identity as possible. [2]

Most people reading this will already be fairly tolerant. But there is a step beyond thinking of yourself as x but tolerating y: not even to consider yourself an x. The more labels you have for yourself, the dumber they make you.

Notes

[1] When that happens, it tends to happen fast, like a core going critical. The threshold for participating goes down to zero, which brings in more people. And they tend to say incendiary things, which draw more and angrier counterarguments.

[2] There may be some things it's a net win to include in your identity. For exam-

ple, being a scientist. But arguably that is more of a placeholder than an actual label—like putting NMI on a form that asks for your middle initial—because it doesn't commit you to believing anything in particular. A scientist isn't committed to believing in natural selection in the same way a biblical literalist is committed to rejecting it. All he's committed to is following the evidence wherever it leads.

Considering yourself a scientist is equivalent to putting a sign in a cupboard saying "this cupboard must be kept empty." Yes, strictly speaking, you're putting something in the cupboard, but not in the ordinary sense.

Thanks to Sam Altman, Trevor Blackwell, Paul Buchheit, and Robert Morris for reading drafts of this.

Startups in 13 Sentences

February 2009

— Watch how this essay was written.

One of the things I always tell startups is a principle I learned from Paul Buchheit: it's better to make a few people really happy than to make a lot of people semi-happy. I was saying recently to a reporter that if I could only tell startups 10 things, this would be one of them. Then I thought: what would the other 9 be?

When I made the list there turned out to be 13:

1. Pick good cofounders.

Cofounders are for a startup what location is for real estate. You can change anything about a house except where it is. In a startup you can change your idea easily, but changing your cofounders is hard. [1] And the success of a startup is almost always a function of its founders.

2. Launch fast.

The reason to launch fast is not so much that it's critical to get your product to market early, but that you haven't really started working on it till you've launched. Launching teaches you what you should have been building. Till you know that you're wasting your time. So the main value of whatever you launch with is as a pretext for engaging users.

3. Let your idea evolve.

This is the second half of launching fast. Launch fast and iterate. It's a big mistake to treat a startup as if it were merely a matter of implementing some brilliant initial idea. As in an essay, most of the ideas appear in the implementing.

4. Understand your users.

You can envision the wealth created by a startup as a rectangle, where one side is the number of users and the other is how much you improve their lives. [2]

The second dimension is the one you have most control over. And indeed, the growth in the first will be driven by how well you do in the second. As in science, the hard part is not answering questions but asking them: the hard part is seeing something new that users lack. The better you understand them the better the odds of doing that. That's why so many successful startups make something the founders needed.

5. Better to make a few users love you than a lot ambivalent.

Ideally you want to make large numbers of users love you, but you can't expect to hit that right away. Initially you have to choose between satisfying all the needs of a subset of potential users, or satisfying a subset of the needs of all potential users. Take the first. It's easier to expand userwise than satisfactionwise. And perhaps more importantly, it's harder to lie to yourself. If you think you're 85% of the way to a great product, how do you know it's not 70%? Or 10%? Whereas it's easy to know how many users you have.

6. Offer surprisingly good customer service.

Customers are used to being maltreated. Most of the companies they deal with are quasi-monopolies that get away with atrocious customer service. Your own ideas about what's possible have been unconsciously lowered by such experiences. Try making your customer service not merely good, but surprisingly good. Go out of your way to make people happy. They'll be overwhelmed; you'll see. In the earliest stages of a startup, it pays to offer customer service on a level that wouldn't scale, because it's a way of learning about your users.

7. You make what you measure.

I learned this one from Joe Kraus. [3] Merely measuring something has an uncanny tendency to improve it. If you want to make your user numbers go up, put a big piece of paper on your wall and every day plot the number of users. You'll be delighted when it goes up and disappointed when it goes down. Pretty soon you'll start noticing what makes the number go up, and you'll start to do more of that. Corollary: be careful what you measure.

8. Spend little.

I can't emphasize enough how important it is for a startup to be cheap. Most startups fail before they make something people want, and the most common form of failure is running out of money. So being cheap is (almost) interchangeable with iterating rapidly. [4] But it's more than that. A culture of cheapness keeps companies young in something like the way exercise keeps people young.

9. Get ramen profitable.

“Ramen profitable” means a startup makes just enough to pay the founders’ living expenses. It’s not rapid prototyping for business models (though it can be), but more a way of hacking the investment process. Once you cross over into ramen profitable, it completely changes your relationship with investors. It’s also great for morale.

10. Avoid distractions.

Nothing kills startups like distractions. The worst type are those that pay money: day jobs, consulting, profitable side-projects. The startup may have more long-term potential, but you’ll always interrupt working on it to answer calls from people paying you now. Paradoxically, fundraising is this type of distraction, so try to minimize that too.

11. Don’t get demoralized.

Though the immediate cause of death in a startup tends to be running out of money, the underlying cause is usually lack of focus. Either the company is run by stupid people (which can’t be fixed with advice) or the people are smart but got demoralized. Starting a startup is a huge moral weight. Understand this and make a conscious effort not to be ground down by it, just as you’d be careful to bend at the knees when picking up a heavy box.

12. Don’t give up.

Even if you get demoralized, don’t give up. You can get surprisingly far by just not giving up. This isn’t true in all fields. There are a lot of people who couldn’t become good mathematicians no matter how long they persisted. But startups aren’t like that. Sheer effort is usually enough, so long as you keep morphing your idea.

13. Deals fall through.

One of the most useful skills we learned from Viaweb was not getting our hopes up. We probably had 20 deals of various types fall through. After the first 10 or so we learned to treat deals as background processes that we should ignore till they terminated. It’s very dangerous to morale to start to depend on deals closing, not just because they so often don’t, but because it makes them less likely to.

Having gotten it down to 13 sentences, I asked myself which I’d choose if I could only keep one.

Understand your users. That’s the key. The essential task in a startup is to create wealth; the dimension of wealth you have most control over is how much you improve users’ lives; and the hardest part of that is knowing what to make

for them. Once you know what to make, it's mere effort to make it, and most decent hackers are capable of that.

Understanding your users is part of half the principles in this list. That's the reason to launch early, to understand your users. Evolving your idea is the embodiment of understanding your users. Understanding your users well will tend to push you toward making something that makes a few people deeply happy. The most important reason for having surprisingly good customer service is that it helps you understand your users. And understanding your users will even ensure your morale, because when everything else is collapsing around you, having just ten users who love you will keep you going.

Notes

[1] Strictly speaking it's impossible without a time machine.

[2] In practice it's more like a ragged comb.

[3] Joe thinks one of the founders of Hewlett Packard said it first, but he doesn't remember which.

[4] They'd be interchangeable if markets stood still. Since they don't, working twice as fast is better than having twice as much time.

Relentlessly Resourceful

March 2009

A couple days ago I finally got being a good startup founder down to two words: relentlessly resourceful.

Till then the best I'd managed was to get the opposite quality down to one: hapless. Most dictionaries say hapless means unlucky. But the dictionaries are not doing a very good job. A team that outplays its opponents but loses because of a bad decision by the referee could be called unlucky, but not hapless. Hapless implies passivity. To be hapless is to be battered by circumstances — to let the world have its way with you, instead of having your way with the world. [1]

Unfortunately there's no antonym of hapless, which makes it difficult to tell founders what to aim for. "Don't be hapless" is not much of a rallying cry.

It's not hard to express the quality we're looking for in metaphors. The best is probably a running back. A good running back is not merely determined, but flexible as well. They want to get downfield, but they adapt their plans on the fly.

Unfortunately this is just a metaphor, and not a useful one to most people outside the US. "Be like a running back" is no better than "Don't be hapless."

But finally I've figured out how to express this quality directly. I was writing a talk for investors, and I had to explain what to look for in founders. What would someone who was the opposite of hapless be like? They'd be relentlessly resourceful. Not merely relentless. That's not enough to make things go your way except in a few mostly uninteresting domains. In any interesting domain, the difficulties will be novel. Which means you can't simply plow through them, because you don't know initially how hard they are; you don't know whether you're about to plow through a block of foam or granite. So you have to be resourceful. You have to keep trying new things.

Be relentlessly resourceful.

That sounds right, but is it simply a description of how to be successful in general? I don't think so. This isn't the recipe for success in writing or painting, for example. In that kind of work the recipe is more to be actively curious. Resourceful implies the obstacles are external, which they generally are in startups. But in writing and painting they're mostly internal; the obstacle is your own obtuseness. [2]

There probably are other fields where "relentlessly resourceful" is the recipe for success. But though other fields may share it, I think this is the best short description we'll find of what makes a good startup founder. I doubt it could be made more precise.

Now that we know what we're looking for, that leads to other questions. For example, can this quality be taught? After four years of trying to teach it to people, I'd say that yes, surprisingly often it can. Not to everyone, but to many people. [3] Some people are just constitutionally passive, but others have a latent ability to be relentlessly resourceful that only needs to be brought out.

This is particularly true of young people who have till now always been under the thumb of some kind of authority. Being relentlessly resourceful is definitely not the recipe for success in big companies, or in most schools. I don't even want to think what the recipe is in big companies, but it is certainly longer and messier, involving some combination of resourcefulness, obedience, and building alliances.

Identifying this quality also brings us closer to answering a question people often wonder about: how many startups there could be. There is not, as some people seem to think, any economic upper bound on this number. There's no reason to believe there is any limit on the amount of newly created wealth consumers can absorb, any more than there is a limit on the number of theorems that can be proven. So probably the limiting factor on the number of startups is the pool of potential founders. Some people would make good founders, and others wouldn't. And now that we can say what makes a good founder, we know how to put an upper bound on the size of the pool.

This test is also useful to individuals. If you want to know whether you're the right sort of person to start a startup, ask yourself whether you're relentlessly resourceful. And if you want to know whether to recruit someone as a cofounder, ask if they are.

You can even use it tactically. If I were running a startup, this would be the phrase I'd tape to the mirror. "Make something people want" is the destination, but "Be relentlessly resourceful" is how you get there.

Notes

[1] I think the reason the dictionaries are wrong is that the meaning of the word

has shifted. No one writing a dictionary from scratch today would say that hapless meant unlucky. But a couple hundred years ago they might have. People were more at the mercy of circumstances in the past, and as a result a lot of the words we use for good and bad outcomes have origins in words about luck.

When I was living in Italy, I was once trying to tell someone that I hadn't had much success in doing something, but I couldn't think of the Italian word for success. I spent some time trying to describe the word I meant. Finally she said "Ah! Fortuna!"

[2] There are aspects of startups where the recipe is to be actively curious. There can be times when what you're doing is almost pure discovery. Unfortunately these times are a small proportion of the whole. On the other hand, they are in research too.

[3] I'd almost say to most people, but I realize (a) I have no idea what most people are like, and (b) I'm pathologically optimistic about people's ability to change.

Thanks to Trevor Blackwell and Jessica Livingston for reading drafts of this.

Maker's Schedule, Manager's Schedule

July 2009

— Charles Dickens —

One reason programmers dislike meetings so much is that they're on a different type of schedule from other people. Meetings cost them more.

There are two types of schedule, which I'll call the manager's schedule and the maker's schedule. The manager's schedule is for bosses. It's embodied in the traditional appointment book, with each day cut into one hour intervals. You can block off several hours for a single task if you need to, but by default you change what you're doing every hour.

When you use time that way, it's merely a practical problem to meet with someone. Find an open slot in your schedule, book them, and you're done.

Most powerful people are on the manager's schedule. It's the schedule of command. But there's another way of using time that's common among people who make things, like programmers and writers. They generally prefer to use time in units of half a day at least. You can't write or program well in units of an hour. That's barely enough time to get started.

When you're operating on the maker's schedule, meetings are a disaster. A single meeting can blow a whole afternoon, by breaking it into two pieces each too small to do anything hard in. Plus you have to remember to go to the meeting. That's no problem for someone on the manager's schedule. There's always something coming on the next hour; the only question is what. But when someone on the maker's schedule has a meeting, they have to think about it.

For someone on the maker's schedule, having a meeting is like throwing an exception. It doesn't merely cause you to switch from one task to another; it

changes the mode in which you work.

I find one meeting can sometimes affect a whole day. A meeting commonly blows at least half a day, by breaking up a morning or afternoon. But in addition there's sometimes a cascading effect. If I know the afternoon is going to be broken up, I'm slightly less likely to start something ambitious in the morning. I know this may sound oversensitive, but if you're a maker, think of your own case. Don't your spirits rise at the thought of having an entire day free to work, with no appointments at all? Well, that means your spirits are correspondingly depressed when you don't. And ambitious projects are by definition close to the limits of your capacity. A small decrease in morale is enough to kill them off.

Each type of schedule works fine by itself. Problems arise when they meet. Since most powerful people operate on the manager's schedule, they're in a position to make everyone resonate at their frequency if they want to. But the smarter ones restrain themselves, if they know that some of the people working for them need long chunks of time to work in.

Our case is an unusual one. Nearly all investors, including all VCs I know, operate on the manager's schedule. But Y Combinator runs on the maker's schedule. Rtm and Trevor and I do because we always have, and Jessica does too, mostly, because she's gotten into sync with us.

I wouldn't be surprised if there start to be more companies like us. I suspect founders may increasingly be able to resist, or at least postpone, turning into managers, just as a few decades ago they started to be able to resist switching from jeans to suits.

How do we manage to advise so many startups on the maker's schedule? By using the classic device for simulating the manager's schedule within the maker's: office hours. Several times a week I set aside a chunk of time to meet founders we've funded. These chunks of time are at the end of my working day, and I wrote a signup program that ensures all the appointments within a given set of office hours are clustered at the end. Because they come at the end of my day these meetings are never an interruption. (Unless their working day ends at the same time as mine, the meeting presumably interrupts theirs, but since they made the appointment it must be worth it to them.) During busy periods, office hours sometimes get long enough that they compress the day, but they never interrupt it.

When we were working on our own startup, back in the 90s, I evolved another trick for partitioning the day. I used to program from dinner till about 3 am every day, because at night no one could interrupt me. Then I'd sleep till about 11 am, and come in and work until dinner on what I called "business stuff." I

never thought of it in these terms, but in effect I had two workdays each day, one on the manager's schedule and one on the maker's.

When you're operating on the manager's schedule you can do something you'd never want to do on the maker's: you can have speculative meetings. You can meet someone just to get to know one another. If you have an empty slot in your schedule, why not? Maybe it will turn out you can help one another in some way.

Business people in Silicon Valley (and the whole world, for that matter) have speculative meetings all the time. They're effectively free if you're on the manager's schedule. They're so common that there's distinctive language for proposing them: saying that you want to "grab coffee," for example.

Speculative meetings are terribly costly if you're on the maker's schedule, though. Which puts us in something of a bind. Everyone assumes that, like other investors, we run on the manager's schedule. So they introduce us to someone they think we ought to meet, or send us an email proposing we grab coffee. At this point we have two options, neither of them good: we can meet with them, and lose half a day's work; or we can try to avoid meeting them, and probably offend them.

Till recently we weren't clear in our own minds about the source of the problem. We just took it for granted that we had to either blow our schedules or offend people. But now that I've realized what's going on, perhaps there's a third option: to write something explaining the two types of schedule. Maybe eventually, if the conflict between the manager's schedule and the maker's schedule starts to be more widely understood, it will become less of a problem.

Those of us on the maker's schedule are willing to compromise. We know we have to have some number of meetings. All we ask from those on the manager's schedule is that they understand the cost.

Thanks to Sam Altman, Trevor Blackwell, Paul Buchheit, Jessica Livingston, and Robert Morris for reading drafts of this.

Related:

— How to Do What You Love | | Good and Bad Procrastination

The Acceleration of Addictiveness

July 2010

What hard liquor, cigarettes, heroin, and crack have in common is that they're all more concentrated forms of less addictive predecessors. Most if not all the things we describe as addictive are. And the scary thing is, the process that created them is accelerating.

We wouldn't want to stop it. It's the same process that cures diseases: technological progress. Technological progress means making things do more of what we want. When the thing we want is something we want to want, we consider technological progress good. If some new technique makes solar cells $x\%$ more efficient, that seems strictly better. When progress concentrates something we don't want to want — when it transforms opium into heroin — it seems bad. But it's the same process at work. [1]

No one doubts this process is accelerating, which means increasing numbers of things we like will be transformed into things we like too much. [2]

As far as I know there's no word for something we like too much. The closest is the colloquial sense of "addictive." That usage has become increasingly common during my lifetime. And it's clear why: there are an increasing number of things we need it for. At the extreme end of the spectrum are crack and meth. Food has been transformed by a combination of factory farming and innovations in food processing into something with way more immediate bang for the buck, and you can see the results in any town in America. Checkers and solitaire have been replaced by World of Warcraft and FarmVille. TV has become much more engaging, and even so it can't compete with Facebook.

The world is more addictive than it was 40 years ago. And unless the forms of technological progress that produced these things are subject to different laws

than technological progress in general, the world will get more addictive in the next 40 years than it did in the last 40.

The next 40 years will bring us some wonderful things. I don't mean to imply they're all to be avoided. Alcohol is a dangerous drug, but I'd rather live in a world with wine than one without. Most people can coexist with alcohol; but you have to be careful. More things we like will mean more things we have to be careful about.

Most people won't, unfortunately. Which means that as the world becomes more addictive, the two senses in which one can live a normal life will be driven ever further apart. One sense of "normal" is statistically normal: what everyone else does. The other is the sense we mean when we talk about the normal operating range of a piece of machinery: what works best.

These two senses are already quite far apart. Already someone trying to live well would seem eccentrically abstemious in most of the US. That phenomenon is only going to become more pronounced. You can probably take it as a rule of thumb from now on that if people don't think you're weird, you're living badly.

Societies eventually develop antibodies to addictive new things. I've seen that happen with cigarettes. When cigarettes first appeared, they spread the way an infectious disease spreads through a previously isolated population. Smoking rapidly became a (statistically) normal thing. There were ashtrays everywhere. We had ashtrays in our house when I was a kid, even though neither of my parents smoked. You had to for guests.

As knowledge spread about the dangers of smoking, customs changed. In the last 20 years, smoking has been transformed from something that seemed totally normal into a rather seedy habit: from something movie stars did in publicity shots to something small huddles of addicts do outside the doors of office buildings. A lot of the change was due to legislation, of course, but the legislation couldn't have happened if customs hadn't already changed.

It took a while though—on the order of 100 years. And unless the rate at which social antibodies evolve can increase to match the accelerating rate at which technological progress throws off new addictions, we'll be increasingly unable to rely on customs to protect us. [3] Unless we want to be canaries in the coal mine of each new addiction—the people whose sad example becomes a lesson to future generations—we'll have to figure out for ourselves what to avoid and how. It will actually become a reasonable strategy (or a more reasonable strategy) to suspect everything new.

In fact, even that won't be enough. We'll have to worry not just about new things, but also about existing things becoming more addictive. That's what bit

me. I've avoided most addictions, but the Internet got me because it became addictive while I was using it. [4]

Most people I know have problems with Internet addiction. We're all trying to figure out our own customs for getting free of it. That's why I don't have an iPhone, for example; the last thing I want is for the Internet to follow me out into the world. [5] My latest trick is taking long hikes. I used to think running was a better form of exercise than hiking because it took less time. Now the slowness of hiking seems an advantage, because the longer I spend on the trail, the longer I have to think without interruption.

Sounds pretty eccentric, doesn't it? It always will when you're trying to solve problems where there are no customs yet to guide you. Maybe I can't plead Occam's razor; maybe I'm simply eccentric. But if I'm right about the acceleration of addictiveness, then this kind of lonely squirming to avoid it will increasingly be the fate of anyone who wants to get things done. We'll increasingly be defined by what we say no to.

Notes

[1] Could you restrict technological progress to areas where you wanted it? Only in a limited way, without becoming a police state. And even then your restrictions would have undesirable side effects. "Good" and "bad" technological progress aren't sharply differentiated, so you'd find you couldn't slow the latter without also slowing the former. And in any case, as Prohibition and the "war on drugs" show, bans often do more harm than good.

[2] Technology has always been accelerating. By Paleolithic standards, technology evolved at a blistering pace in the Neolithic period.

[3] Unless we mass produce social customs. I suspect the recent resurgence of evangelical Christianity in the US is partly a reaction to drugs. In desperation people reach for the sledgehammer; if their kids won't listen to them, maybe they'll listen to God. But that solution has broader consequences than just getting kids to say no to drugs. You end up saying no to science as well.

I worry we may be heading for a future in which only a few people plot their own itinerary through no-land, while everyone else books a package tour. Or worse still, has one booked for them by the government.

[4] People commonly use the word "procrastination" to describe what they do on the Internet. It seems to me too mild to describe what's happening as merely not-doing-work. We don't call it procrastination when someone gets drunk instead of working.

[5] Several people have told me they like the iPad because it lets them bring the

Internet into situations where a laptop would be too conspicuous. In other words, it's a hip flask. (This is true of the iPhone too, of course, but this advantage isn't as obvious because it reads as a phone, and everyone's used to those.)

Thanks to Sam Altman, Patrick Collison, Jessica Livingston, and Robert Morris for reading drafts of this.

The Top Idea in Your Mind

July 2010

I realized recently that what one thinks about in the shower in the morning is more important than I'd thought. I knew it was a good time to have ideas. Now I'd go further: now I'd say it's hard to do a really good job on anything you don't think about in the shower.

Everyone who's worked on difficult problems is probably familiar with the phenomenon of working hard to figure something out, failing, and then suddenly seeing the answer a bit later while doing something else. There's a kind of thinking you do without trying to. I'm increasingly convinced this type of thinking is not merely helpful in solving hard problems, but necessary. The tricky part is, you can only control it indirectly. [1]

I think most people have one top idea in their mind at any given time. That's the idea their thoughts will drift toward when they're allowed to drift freely. And this idea will thus tend to get all the benefit of that type of thinking, while others are starved of it. Which means it's a disaster to let the wrong idea become the top one in your mind.

What made this clear to me was having an idea I didn't want as the top one in my mind for two long stretches.

I'd noticed startups got way less done when they started raising money, but it was not till we ourselves raised money that I understood why. The problem is not the actual time it takes to meet with investors. The problem is that once you start raising money, raising money becomes the top idea in your mind. That becomes what you think about when you take a shower in the morning. And that means other questions aren't.

I'd hated raising money when I was running Viaweb, but I'd forgotten why I hated it so much. When we raised money for Y Combinator, I remembered. Money matters are particularly likely to become the top idea in your mind. The

reason is that they have to be. It's hard to get money. It's not the sort of thing that happens by default. It's not going to happen unless you let it become the thing you think about in the shower. And then you'll make little progress on anything else you'd rather be working on. [2]

(I hear similar complaints from friends who are professors. Professors nowadays seem to have become professional fundraisers who do a little research on the side. It may be time to fix that.)

The reason this struck me so forcibly is that for most of the preceding 10 years I'd been able to think about what I wanted. So the contrast when I couldn't was sharp. But I don't think this problem is unique to me, because just about every startup I've seen grinds to a halt when they start raising money — or talking to acquirers.

You can't directly control where your thoughts drift. If you're controlling them, they're not drifting. But you can control them indirectly, by controlling what situations you let yourself get into. That has been the lesson for me: be careful what you let become critical to you. Try to get yourself into situations where the most urgent problems are ones you want to think about.

You don't have complete control, of course. An emergency could push other thoughts out of your head. But barring emergencies you have a good deal of indirect control over what becomes the top idea in your mind.

I've found there are two types of thoughts especially worth avoiding — thoughts like the Nile Perch in the way they push out more interesting ideas. One I've already mentioned: thoughts about money. Getting money is almost by definition an attention sink. The other is disputes. These too are engaging in the wrong way: they have the same velcro-like shape as genuinely interesting ideas, but without the substance. So avoid disputes if you want to get real work done. [3]

Even Newton fell into this trap. After publishing his theory of colors in 1672 he found himself distracted by disputes for years, finally concluding that the only solution was to stop publishing:

I see I have made myself a slave to Philosophy, but if I get free of Mr Linus's business I will resolutely bid adieu to it eternally, excepting what I do for my privat satisfaction or leave to come out after me. For I see a man must either resolve to put out nothing new or become a slave to defend it. [4]

Linus and his students at Liege were among the more tenacious critics. Newton's biographer Westfall seems to feel he was overreacting:

Recall that at the time he wrote, Newton's "slavery" consisted of five

replies to Liege, totalling fourteen printed pages, over the course of a year.

I'm more sympathetic to Newton. The problem was not the 14 pages, but the pain of having this stupid controversy constantly reintroduced as the top idea in a mind that wanted so eagerly to think about other things.

Turning the other cheek turns out to have selfish advantages. Someone who does you an injury hurts you twice: first by the injury itself, and second by taking up your time afterward thinking about it. If you learn to ignore injuries you can at least avoid the second half. I've found I can to some extent avoid thinking about nasty things people have done to me by telling myself: this doesn't deserve space in my head. I'm always delighted to find I've forgotten the details of disputes, because that means I hadn't been thinking about them. My wife thinks I'm more forgiving than she is, but my motives are purely selfish.

I suspect a lot of people aren't sure what's the top idea in their mind at any given time. I'm often mistaken about it. I tend to think it's the idea I'd want to be the top one, rather than the one that is. But it's easy to figure this out: just take a shower. What topic do your thoughts keep returning to? If it's not what you want to be thinking about, you may want to change something.

Notes

[1] No doubt there are already names for this type of thinking, but I call it "ambient thought."

[2] This was made particularly clear in our case, because neither of the funds we raised was difficult, and yet in both cases the process dragged on for months. Moving large amounts of money around is never something people treat casually. The attention required increases with the amount—maybe not linearly, but definitely monotonically.

[3] Corollary: Avoid becoming an administrator, or your job will consist of dealing with money and disputes.

[4] Letter to Oldenburg, quoted in Westfall, Richard, *Life of Isaac Newton*, p. 107.

Thanks to Sam Altman, Patrick Collison, Jessica Livingston, and Robert Morris for reading drafts of this.

Schlep Blindness

January 2012

There are great startup ideas lying around unexploited right under our noses. One reason we don't see them is a phenomenon I call *schlep blindness*. Schlep was originally a Yiddish word but has passed into general use in the US. It means a tedious, unpleasant task.

No one likes schleps, but hackers especially dislike them. Most hackers who start startups wish they could do it by just writing some clever software, putting it on a server somewhere, and watching the money roll in—without ever having to talk to users, or negotiate with other companies, or deal with other people's broken code. Maybe that's possible, but I haven't seen it.

One of the many things we do at Y Combinator is teach hackers about the inevitability of schleps. No, you can't start a startup by just writing code. I remember going through this realization myself. There was a point in 1995 when I was still trying to convince myself I could start a company by just writing code. But I soon learned from experience that schleps are not merely inevitable, but pretty much what business consists of. A company is defined by the schleps it will undertake. And schleps should be dealt with the same way you'd deal with a cold swimming pool: just jump in. Which is not to say you should seek out unpleasant work per se, but that you should never shrink from it if it's on the path to something great.

The most dangerous thing about our dislike of schleps is that much of it is unconscious. Your unconscious won't even let you see ideas that involve painful schleps. That's *schlep blindness*.

The phenomenon isn't limited to startups. Most people don't consciously decide not to be in as good physical shape as Olympic athletes, for example. Their unconscious mind decides for them, shrinking from the work involved.

The most striking example I know of *schlep blindness* is Stripe, or rather Stripe's

idea. For over a decade, every hacker who'd ever had to process payments online knew how painful the experience was. Thousands of people must have known about this problem. And yet when they started startups, they decided to build recipe sites, or aggregators for local events. Why? Why work on problems few care much about and no one will pay for, when you could fix one of the most important components of the world's infrastructure? Because schlep blindness prevented people from even considering the idea of fixing payments.

Probably no one who applied to Y Combinator to work on a recipe site began by asking "should we fix payments, or build a recipe site?" and chose the recipe site. Though the idea of fixing payments was right there in plain sight, they never saw it, because their unconscious mind shrank from the complications involved. You'd have to make deals with banks. How do you do that? Plus you're moving money, so you're going to have to deal with fraud, and people trying to break into your servers. Plus there are probably all sorts of regulations to comply with. It's a lot more intimidating to start a startup like this than a recipe site.

That scariness makes ambitious ideas doubly valuable. In addition to their intrinsic value, they're like undervalued stocks in the sense that there's less demand for them among founders. If you pick an ambitious idea, you'll have less competition, because everyone else will have been frightened off by the challenges involved. (This is also true of starting a startup generally.)

How do you overcome schlep blindness? Frankly, the most valuable antidote to schlep blindness is probably ignorance. Most successful founders would probably say that if they'd known when they were starting their company about the obstacles they'd have to overcome, they might never have started it. Maybe that's one reason the most successful startups of all so often have young founders.

In practice the founders grow with the problems. But no one seems able to foresee that, not even older, more experienced founders. So the reason younger founders have an advantage is that they make two mistakes that cancel each other out. They don't know how much they can grow, but they also don't know how much they'll need to. Older founders only make the first mistake.

Ignorance can't solve everything though. Some ideas so obviously entail alarming schleps that anyone can see them. How do you see ideas like that? The trick I recommend is to take yourself out of the picture. Instead of asking "what problem should I solve?" ask "what problem do I wish someone else would solve for me?" If someone who had to process payments before Stripe had tried asking that, Stripe would have been one of the first things they wished for.

It's too late now to be Stripe, but there's plenty still broken in the world, if you know how to see it.

Thanks to Sam Altman, Paul Buchheit, Patrick Collison, Aaron Iba, Jessica Livingston, Emmett Shear, and Harj Taggar for reading drafts of this.

Startup = Growth

September 2012

A startup is a company designed to grow fast. Being newly founded does not in itself make a company a startup. Nor is it necessary for a startup to work on technology, or take venture funding, or have some sort of “exit.” The only essential thing is growth. Everything else we associate with startups follows from growth.

If you want to start one it’s important to understand that. Startups are so hard that you can’t be pointed off to the side and hope to succeed. You have to know that growth is what you’re after. The good news is, if you get growth, everything else tends to fall into place. Which means you can use growth like a compass to make almost every decision you face.

Redwoods

Let’s start with a distinction that should be obvious but is often overlooked: not every newly founded company is a startup. Millions of companies are started every year in the US. Only a tiny fraction are startups. Most are service businesses — restaurants, barbershops, plumbers, and so on. These are not startups, except in a few unusual cases. A barbershop isn’t designed to grow fast. Whereas a search engine, for example, is.

When I say startups are designed to grow fast, I mean it in two senses. Partly I mean designed in the sense of intended, because most startups fail. But I also mean startups are different by nature, in the same way a redwood seedling has a different destiny from a bean sprout.

That difference is why there’s a distinct word, “startup,” for companies designed to grow fast. If all companies were essentially similar, but some through luck or the efforts of their founders ended up growing very fast, we wouldn’t need a separate word. We could just talk about super-successful companies and less successful ones. But in fact startups do have a different sort of DNA from other

businesses. Google is not just a barbershop whose founders were unusually lucky and hard-working. Google was different from the beginning.

To grow rapidly, you need to make something you can sell to a big market. That's the difference between Google and a barbershop. A barbershop doesn't scale.

For a company to grow really big, it must (a) make something lots of people want, and (b) reach and serve all those people. Barbershops are doing fine in the (a) department. Almost everyone needs their hair cut. The problem for a barbershop, as for any retail establishment, is (b). A barbershop serves customers in person, and few will travel far for a haircut. And even if they did, the barbershop couldn't accomodate them. [1]

Writing software is a great way to solve (b), but you can still end up constrained in (a). If you write software to teach Tibetan to Hungarian speakers, you'll be able to reach most of the people who want it, but there won't be many of them. If you make software to teach English to Chinese speakers, however, you're in startup territory.

Most businesses are tightly constrained in (a) or (b). The distinctive feature of successful startups is that they're not.

Ideas

It might seem that it would always be better to start a startup than an ordinary business. If you're going to start a company, why not start the type with the most potential? The catch is that this is a (fairly) efficient market. If you write software to teach Tibetan to Hungarians, you won't have much competition. If you write software to teach English to Chinese speakers, you'll face ferocious competition, precisely because that's such a larger prize. [2]

The constraints that limit ordinary companies also protect them. That's the tradeoff. If you start a barbershop, you only have to compete with other local barbers. If you start a search engine you have to compete with the whole world.

The most important thing that the constraints on a normal business protect it from is not competition, however, but the difficulty of coming up with new ideas. If you open a bar in a particular neighborhood, as well as limiting your potential and protecting you from competitors, that geographic constraint also helps define your company. Bar + neighborhood is a sufficient idea for a small business. Similarly for companies constrained in (a). Your niche both protects and defines you.

Whereas if you want to start a startup, you're probably going to have to think of something fairly novel. A startup has to make something it can deliver to a

large market, and ideas of that type are so valuable that all the obvious ones are already taken.

That space of ideas has been so thoroughly picked over that a startup generally has to work on something everyone else has overlooked. I was going to write that one has to make a conscious effort to find ideas everyone else has overlooked. But that's not how most startups get started. Usually successful startups happen because the founders are sufficiently different from other people that ideas few others can see seem obvious to them. Perhaps later they step back and notice they've found an idea in everyone else's blind spot, and from that point make a deliberate effort to stay there. [3] But at the moment when successful startups get started, much of the innovation is unconscious.

What's different about successful founders is that they can see different problems. It's a particularly good combination both to be good at technology and to face problems that can be solved by it, because technology changes so rapidly that formerly bad ideas often become good without anyone noticing. Steve Wozniak's problem was that he wanted his own computer. That was an unusual problem to have in 1975. But technological change was about to make it a much more common one. Because he not only wanted a computer but knew how to build them, Wozniak was able to make himself one. And the problem he solved for himself became one that Apple solved for millions of people in the coming years. But by the time it was obvious to ordinary people that this was a big market, Apple was already established.

Google has similar origins. Larry Page and Sergey Brin wanted to search the web. But unlike most people they had the technical expertise both to notice that existing search engines were not as good as they could be, and to know how to improve them. Over the next few years their problem became everyone's problem, as the web grew to a size where you didn't have to be a picky search expert to notice the old algorithms weren't good enough. But as happened with Apple, by the time everyone else realized how important search was, Google was entrenched.

That's one connection between startup ideas and technology. Rapid change in one area uncovers big, soluble problems in other areas. Sometimes the changes are advances, and what they change is solubility. That was the kind of change that yielded Apple; advances in chip technology finally let Steve Wozniak design a computer he could afford. But in Google's case the most important change was the growth of the web. What changed there was not solubility but bigness.

The other connection between startups and technology is that startups create new ways of doing things, and new ways of doing things are, in the broader sense

of the word, new technology. When a startup both begins with an idea exposed by technological change and makes a product consisting of technology in the narrower sense (what used to be called “high technology”), it’s easy to conflate the two. But the two connections are distinct and in principle one could start a startup that was neither driven by technological change, nor whose product consisted of technology except in the broader sense. [4]

Rate

How fast does a company have to grow to be considered a startup? There’s no precise answer to that. “Startup” is a pole, not a threshold. Starting one is at first no more than a declaration of one’s ambitions. You’re committing not just to starting a company, but to starting a fast growing one, and you’re thus committing to search for one of the rare ideas of that type. But at first you have no more than commitment. Starting a startup is like being an actor in that respect. “Actor” too is a pole rather than a threshold. At the beginning of his career, an actor is a waiter who goes to auditions. Getting work makes him a successful actor, but he doesn’t only become an actor when he’s successful.

So the real question is not what growth rate makes a company a startup, but what growth rate successful startups tend to have. For founders that’s more than a theoretical question, because it’s equivalent to asking if they’re on the right path.

The growth of a successful startup usually has three phases:

1. There’s an initial period of slow or no growth while the startup tries to figure out what it’s doing.
2. As the startup figures out how to make something lots of people want and how to reach those people, there’s a period of rapid growth.
3. Eventually a successful startup will grow into a big company. Growth will slow, partly due to internal limits and partly because the company is starting to bump up against the limits of the markets it serves. [5]

Together these three phases produce an S-curve. The phase whose growth defines the startup is the second one, the ascent. Its length and slope determine how big the company will be.

The slope is the company’s growth rate. If there’s one number every founder should always know, it’s the company’s growth rate. That’s the measure of a startup. If you don’t know that number, you don’t even know if you’re doing well or badly.

When I first meet founders and ask what their growth rate is, sometimes they

tell me “we get about a hundred new customers a month.” That’s not a rate. What matters is not the absolute number of new customers, but the ratio of new customers to existing ones. If you’re really getting a constant number of new customers every month, you’re in trouble, because that means your growth rate is decreasing.

During Y Combinator we measure growth rate per week, partly because there is so little time before Demo Day, and partly because startups early on need frequent feedback from their users to tweak what they’re doing. [6]

A good growth rate during YC is 5-7% a week. If you can hit 10% a week you’re doing exceptionally well. If you can only manage 1%, it’s a sign you haven’t yet figured out what you’re doing.

The best thing to measure the growth rate of is revenue. The next best, for startups that aren’t charging initially, is active users. That’s a reasonable proxy for revenue growth because whenever the startup does start trying to make money, their revenues will probably be a constant multiple of active users. [7]

Compass

We usually advise startups to pick a growth rate they think they can hit, and then just try to hit it every week. The key word here is “just.” If they decide to grow at 7% a week and they hit that number, they’re successful for that week. There’s nothing more they need to do. But if they don’t hit it, they’ve failed in the only thing that mattered, and should be correspondingly alarmed.

Programmers will recognize what we’re doing here. We’re turning starting a startup into an optimization problem. And anyone who has tried optimizing code knows how wonderfully effective that sort of narrow focus can be. Optimizing code means taking an existing program and changing it to use less of something, usually time or memory. You don’t have to think about what the program should do, just make it faster. For most programmers this is very satisfying work. The narrow focus makes it a sort of puzzle, and you’re generally surprised how fast you can solve it.

Focusing on hitting a growth rate reduces the otherwise bewilderingly multifarious problem of starting a startup to a single problem. You can use that target growth rate to make all your decisions for you; anything that gets you the growth you need is ipso facto right. Should you spend two days at a conference? Should you hire another programmer? Should you focus more on marketing? Should you spend time courting some big customer? Should you add x feature? Whatever gets you your target growth rate. [8]

Judging yourself by weekly growth doesn’t mean you can look no more than a

week ahead. Once you experience the pain of missing your target one week (it was the only thing that mattered, and you failed at it), you become interested in anything that could spare you such pain in the future. So you'll be willing for example to hire another programmer, who won't contribute to this week's growth but perhaps in a month will have implemented some new feature that will get you more users. But only if (a) the distraction of hiring someone won't make you miss your numbers in the short term, and (b) you're sufficiently worried about whether you can keep hitting your numbers without hiring someone new.

It's not that you don't think about the future, just that you think about it no more than necessary.

In theory this sort of hill-climbing could get a startup into trouble. They could end up on a local maximum. But in practice that never happens. Having to hit a growth number every week forces founders to act, and acting versus not acting is the high bit of succeeding. Nine times out of ten, sitting around strategizing is just a form of procrastination. Whereas founders' intuitions about which hill to climb are usually better than they realize. Plus the maxima in the space of startup ideas are not spiky and isolated. Most fairly good ideas are adjacent to even better ones.

The fascinating thing about optimizing for growth is that it can actually discover startup ideas. You can use the need for growth as a form of evolutionary pressure. If you start out with some initial plan and modify it as necessary to keep hitting, say, 10% weekly growth, you may end up with a quite different company than you meant to start. But anything that grows consistently at 10% a week is almost certainly a better idea than you started with.

There's a parallel here to small businesses. Just as the constraint of being located in a particular neighborhood helps define a bar, the constraint of growing at a certain rate can help define a startup.

You'll generally do best to follow that constraint wherever it leads rather than being influenced by some initial vision, just as a scientist is better off following the truth wherever it leads rather than being influenced by what he wishes were the case. When Richard Feynman said that the imagination of nature was greater than the imagination of man, he meant that if you just keep following the truth you'll discover cooler things than you could ever have made up. For startups, growth is a constraint much like truth. Every successful startup is at least partly a product of the imagination of growth. [9]

Value

It's hard to find something that grows consistently at several percent a week, but if you do you may have found something surprisingly valuable. If we project

forward we see why.

weekly| yearly —|— 1%| 1.7x 2%| 2.8x 5%| 12.6x 7%| 33.7x 10%| 142.0x

A company that grows at 1% a week will grow 1.7x a year, whereas a company that grows at 5% a week will grow 12.6x. A company making \$1000 a month (a typical number early in YC) and growing at 1% a week will 4 years later be making \$7900 a month, which is less than a good programmer makes in salary in Silicon Valley. A startup that grows at 5% a week will in 4 years be making \$25 million a month. [10]

Our ancestors must rarely have encountered cases of exponential growth, because our intuitions are no guide here. What happens to fast growing startups tends to surprise even the founders.

Small variations in growth rate produce qualitatively different outcomes. That's why there's a separate word for startups, and why startups do things that ordinary companies don't, like raising money and getting acquired. And, strangely enough, it's also why they fail so frequently.

Considering how valuable a successful startup can become, anyone familiar with the concept of expected value would be surprised if the failure rate weren't high. If a successful startup could make a founder \$100 million, then even if the chance of succeeding were only 1%, the expected value of starting one would be \$1 million. And the probability of a group of sufficiently smart and determined founders succeeding on that scale might be significantly over 1%. For the right people — e.g. the young Bill Gates — the probability might be 20% or even 50%. So it's not surprising that so many want to take a shot at it. In an efficient market, the number of failed startups should be proportionate to the size of the successes. And since the latter is huge the former should be too. [11]

What this means is that at any given time, the great majority of startups will be working on something that's never going to go anywhere, and yet glorifying their doomed efforts with the grandiose title of “startup.”

This doesn't bother me. It's the same with other high-beta vocations, like being an actor or a novelist. I've long since gotten used to it. But it seems to bother a lot of people, particularly those who've started ordinary businesses. Many are annoyed that these so-called startups get all the attention, when hardly any of them will amount to anything.

If they stepped back and looked at the whole picture they might be less indignant. The mistake they're making is that by basing their opinions on anecdotal evidence they're implicitly judging by the median rather than the average. If you judge by the median startup, the whole concept of a startup seems like a fraud. You

have to invent a bubble to explain why founders want to start them or investors want to fund them. But it's a mistake to use the median in a domain with so much variation. If you look at the average outcome rather than the median, you can understand why investors like them, and why, if they aren't median people, it's a rational choice for founders to start them.

Deals

Why do investors like startups so much? Why are they so hot to invest in photo-sharing apps, rather than solid money-making businesses? Not only for the obvious reason.

The test of any investment is the ratio of return to risk. Startups pass that test because although they're appallingly risky, the returns when they do succeed are so high. But that's not the only reason investors like startups. An ordinary slower-growing business might have just as good a ratio of return to risk, if both were lower. So why are VCs interested only in high-growth companies? The reason is that they get paid by getting their capital back, ideally after the startup IPOs, or failing that when it's acquired.

The other way to get returns from an investment is in the form of dividends. Why isn't there a parallel VC industry that invests in ordinary companies in return for a percentage of their profits? Because it's too easy for people who control a private company to funnel its revenues to themselves (e.g. by buying overpriced components from a supplier they control) while making it look like the company is making little profit. Anyone who invested in private companies in return for dividends would have to pay close attention to their books.

The reason VCs like to invest in startups is not simply the returns, but also because such investments are so easy to oversee. The founders can't enrich themselves without also enriching the investors. [12]

Why do founders want to take the VCs' money? Growth, again. The constraint between good ideas and growth operates in both directions. It's not merely that you need a scalable idea to grow. If you have such an idea and don't grow fast enough, competitors will. Growing too slowly is particularly dangerous in a business with network effects, which the best startups usually have to some degree.

Almost every company needs some amount of funding to get started. But startups often raise money even when they are or could be profitable. It might seem foolish to sell stock in a profitable company for less than you think it will later be worth, but it's no more foolish than buying insurance. Fundamentally that's how the most successful startups view fundraising. They could grow the company on

its own revenues, but the extra money and help supplied by VCs will let them grow even faster. Raising money lets you *choose* your growth rate.

Money to grow faster is always at the command of the most successful startups, because the VCs need them more than they need the VCs. A profitable startup could if it wanted just grow on its own revenues. Growing slower might be slightly dangerous, but chances are it wouldn't kill them. Whereas VCs need to invest in startups, and in particular the most successful startups, or they'll be out of business. Which means that any sufficiently promising startup will be offered money on terms they'd be crazy to refuse. And yet because of the scale of the successes in the startup business, VCs can still make money from such investments. You'd have to be crazy to believe your company was going to become as valuable as a high growth rate can make it, but some do.

Pretty much every successful startup will get acquisition offers too. Why? What is it about startups that makes other companies want to buy them? [13]

Fundamentally the same thing that makes everyone else want the stock of successful startups: a rapidly growing company is valuable. It's a good thing eBay bought Paypal, for example, because Paypal is now responsible for 43% of their sales and probably more of their growth.

But acquirers have an additional reason to want startups. A rapidly growing company is not merely valuable, but dangerous. If it keeps expanding, it might expand into the acquirer's own territory. Most product acquisitions have some component of fear. Even if an acquirer isn't threatened by the startup itself, they might be alarmed at the thought of what a competitor could do with it. And because startups are in this sense doubly valuable to acquirers, acquirers will often pay more than an ordinary investor would. [14]

Understand

The combination of founders, investors, and acquirers forms a natural ecosystem. It works so well that those who don't understand it are driven to invent conspiracy theories to explain how neatly things sometimes turn out. Just as our ancestors did to explain the apparently too neat workings of the natural world. But there is no secret cabal making it all work.

If you start from the mistaken assumption that Instagram was worthless, you have to invent a secret boss to force Mark Zuckerberg to buy it. To anyone who knows Mark Zuckerberg, that is the *reductio ad absurdum* of the initial assumption. The reason he bought Instagram was that it was valuable and dangerous, and what made it so was growth.

If you want to understand startups, understand growth. Growth drives everything

in this world. Growth is why startups usually work on technology — because ideas for fast growing companies are so rare that the best way to find new ones is to discover those recently made viable by change, and technology is the best source of rapid change. Growth is why it's a rational choice economically for so many founders to try starting a startup: growth makes the successful companies so valuable that the expected value is high even though the risk is too. Growth is why VCs want to invest in startups: not just because the returns are high but also because generating returns from capital gains is easier to manage than generating returns from dividends. Growth explains why the most successful startups take VC money even if they don't need to: it lets them choose their growth rate. And growth explains why successful startups almost invariably get acquisition offers. To acquirers a fast-growing company is not merely valuable but dangerous too.

It's not just that if you want to succeed in some domain, you have to understand the forces driving it. Understanding growth is what starting a startup *consists* of. What you're really doing (and to the dismay of some observers, all you're really doing) when you start a startup is committing to solve a harder type of problem than ordinary businesses do. You're committing to search for one of the rare ideas that generates rapid growth. Because these ideas are so valuable, finding one is hard. The startup is the embodiment of your discoveries so far. Starting a startup is thus very much like deciding to be a research scientist: you're not committing to solve any specific problem; you don't know for sure which problems are soluble; but you're committing to try to discover something no one knew before. A startup founder is in effect an economic research scientist. Most don't discover anything that remarkable, but some discover relativity.

Notes

[1] Strictly speaking it's not lots of customers you need but a big market, meaning a high product of number of customers times how much they'll pay. But it's dangerous to have too few customers even if they pay a lot, or the power that individual customers have over you could turn you into a de facto consulting firm. So whatever market you're in, you'll usually do best to err on the side of making the broadest type of product for it.

[2] One year at Startup School David Heinemeier Hansson encouraged programmers who wanted to start businesses to use a restaurant as a model. What he meant, I believe, is that it's fine to start software companies constrained in (a) in the same way a restaurant is constrained in (b). I agree. Most people should not try to start startups.

[3] That sort of stepping back is one of the things we focus on at Y Combinator.

It's common for founders to have discovered something intuitively without understanding all its implications. That's probably true of the biggest discoveries in any field.

[4] I got it wrong in “How to Make Wealth” when I said that a startup was a small company that takes on a hard technical problem. That is the most common recipe but not the only one.

[5] In principle companies aren't limited by the size of the markets they serve, because they could just expand into new markets. But there seem to be limits on the ability of big companies to do that. Which means the slowdown that comes from bumping up against the limits of one's markets is ultimately just another way in which internal limits are expressed.

It may be that some of these limits could be overcome by changing the shape of the organization — specifically by sharding it.

[6] This is, obviously, only for startups that have already launched or can launch during YC. A startup building a new database will probably not do that. On the other hand, launching something small and then using growth rate as evolutionary pressure is such a valuable technique that any company that could start this way probably should.

[7] If the startup is taking the Facebook/Twitter route and building something they hope will be very popular but from which they don't yet have a definite plan to make money, the growth rate has to be higher, even though it's a proxy for revenue growth, because such companies need huge numbers of users to succeed at all.

Beware too of the edge case where something spreads rapidly but the churn is high as well, so that you have good net growth till you run through all the potential users, at which point it suddenly stops.

[8] Within YC when we say it's ipso facto right to do whatever gets you growth, it's implicit that this excludes trickery like buying users for more than their lifetime value, counting users as active when they're really not, bleeding out invites at a regularly increasing rate to manufacture a perfect growth curve, etc. Even if you were able to fool investors with such tricks, you'd ultimately be hurting yourself, because you're throwing off your own compass.

[9] Which is why it's such a dangerous mistake to believe that successful startups are simply the embodiment of some brilliant initial idea. What you're looking for initially is not so much a great idea as an idea that could evolve into a great one. The danger is that promising ideas are not merely blurry versions of great ones. They're often different in kind, because the early adopters you evolve the

idea upon have different needs from the rest of the market. For example, the idea that evolves into Facebook isn't merely a subset of Facebook; the idea that evolves into Facebook is a site for Harvard undergrads.

[10] What if a company grew at 1.7x a year for a really long time? Could it not grow just as big as any successful startup? In principle yes, of course. If our hypothetical company making \$1000 a month grew at 1% a week for 19 years, it would grow as big as a company growing at 5% a week for 4 years. But while such trajectories may be common in, say, real estate development, you don't see them much in the technology business. In technology, companies that grow slowly tend not to grow as big.

[11] Any expected value calculation varies from person to person depending on their utility function for money. I.e. the first million is worth more to most people than subsequent millions. How much more depends on the person. For founders who are younger or more ambitious the utility function is flatter. Which is probably part of the reason the founders of the most successful startups of all tend to be on the young side.

[12] More precisely, this is the case in the biggest winners, which is where all the returns come from. A startup founder could pull the same trick of enriching himself at the company's expense by selling them overpriced components. But it wouldn't be worth it for the founders of Google to do that. Only founders of failing startups would even be tempted, but those are writeoffs from the VCs' point of view anyway.

[13] Acquisitions fall into two categories: those where the acquirer wants the business, and those where the acquirer just wants the employees. The latter type is sometimes called an HR acquisition. Though nominally acquisitions and sometimes on a scale that has a significant effect on the expected value calculation for potential founders, HR acquisitions are viewed by acquirers as more akin to hiring bonuses.

[14] I once explained this to some founders who had recently arrived from Russia. They found it novel that if you threatened a company they'd pay a premium for you. "In Russia they just kill you," they said, and they were only partly joking. Economically, the fact that established companies can't simply eliminate new competitors may be one of the most valuable aspects of the rule of law. And so to the extent we see incumbents suppressing competitors via regulations or patent suits, we should worry, not because it's a departure from the rule of law per se but from what the rule of law is aiming at.

Thanks to Sam Altman, Marc Andreessen, Paul Buchheit, Patrick Collison, Jessica Livingston, Geoff Ralston, and Harj Taggar for reading drafts of this.

How to Get Startup Ideas

November 2012

The way to get startup ideas is not to try to think of startup ideas. It's to look for problems, preferably problems you have yourself.

The very best startup ideas tend to have three things in common: they're something the founders themselves want, that they themselves can build, and that few others realize are worth doing. Microsoft, Apple, Yahoo, Google, and Facebook all began this way.

Problems

Why is it so important to work on a problem you have? Among other things, it ensures the problem really exists. It sounds obvious to say you should only work on problems that exist. And yet by far the most common mistake startups make is to solve problems no one has.

I made it myself. In 1995 I started a company to put art galleries online. But galleries didn't want to be online. It's not how the art business works. So why did I spend 6 months working on this stupid idea? Because I didn't pay attention to users. I invented a model of the world that didn't correspond to reality, and worked from that. I didn't notice my model was wrong until I tried to convince users to pay for what we'd built. Even then I took embarrassingly long to catch on. I was attached to my model of the world, and I'd spent a lot of time on the software. They had to want it!

Why do so many founders build things no one wants? Because they begin by trying to think of startup ideas. That m.o. is doubly dangerous: it doesn't merely yield few good ideas; it yields bad ideas that sound plausible enough to fool you into working on them.

At YC we call these "made-up" or "sitcom" startup ideas. Imagine one of the characters on a TV show was starting a startup. The writers would have to invent something for it to do. But coming up with good startup ideas is hard.

It's not something you can do for the asking. So (unless they got amazingly lucky) the writers would come up with an idea that sounded plausible, but was actually bad.

For example, a social network for pet owners. It doesn't sound obviously mistaken. Millions of people have pets. Often they care a lot about their pets and spend a lot of money on them. Surely many of these people would like a site where they could talk to other pet owners. Not all of them perhaps, but if just 2 or 3 percent were regular visitors, you could have millions of users. You could serve them targeted offers, and maybe charge for premium features. [1]

The danger of an idea like this is that when you run it by your friends with pets, they don't say "I would *never* use this." They say "Yeah, maybe I could see using something like that." Even when the startup launches, it will sound plausible to a lot of people. They don't want to use it themselves, at least not right now, but they could imagine other people wanting it. Sum that reaction across the entire population, and you have zero users. [2]

Well

When a startup launches, there have to be at least some users who really need what they're making — not just people who could see themselves using it one day, but who want it urgently. Usually this initial group of users is small, for the simple reason that if there were something that large numbers of people urgently needed and that could be built with the amount of effort a startup usually puts into a version one, it would probably already exist. Which means you have to compromise on one dimension: you can either build something a large number of people want a small amount, or something a small number of people want a large amount. Choose the latter. Not all ideas of that type are good startup ideas, but nearly all good startup ideas are of that type.

Imagine a graph whose x axis represents all the people who might want what you're making and whose y axis represents how much they want it. If you invert the scale on the y axis, you can envision companies as holes. Google is an immense crater: hundreds of millions of people use it, and they need it a lot. A startup just starting out can't expect to excavate that much volume. So you have two choices about the shape of hole you start with. You can either dig a hole that's broad but shallow, or one that's narrow and deep, like a well.

Made-up startup ideas are usually of the first type. Lots of people are mildly interested in a social network for pet owners.

Nearly all good startup ideas are of the second type. Microsoft was a well when they made Altair Basic. There were only a couple thousand Altair owners, but without this software they were programming in machine language. Thirty years

later Facebook had the same shape. Their first site was exclusively for Harvard students, of which there are only a few thousand, but those few thousand users wanted it a lot.

When you have an idea for a startup, ask yourself: who wants this right now? Who wants this so much that they'll use it even when it's a crappy version one made by a two-person startup they've never heard of? If you can't answer that, the idea is probably bad. [3]

You don't need the narrowness of the well per se. It's depth you need; you get narrowness as a byproduct of optimizing for depth (and speed). But you almost always do get it. In practice the link between depth and narrowness is so strong that it's a good sign when you know that an idea will appeal strongly to a specific group or type of user.

But while demand shaped like a well is almost a necessary condition for a good startup idea, it's not a sufficient one. If Mark Zuckerberg had built something that could only ever have appealed to Harvard students, it would not have been a good startup idea. Facebook was a good idea because it started with a small market there was a fast path out of. Colleges are similar enough that if you build a facebook that works at Harvard, it will work at any college. So you spread rapidly through all the colleges. Once you have all the college students, you get everyone else simply by letting them in.

Similarly for Microsoft: Basic for the Altair; Basic for other machines; other languages besides Basic; operating systems; applications; IPO.

Self

How do you tell whether there's a path out of an idea? How do you tell whether something is the germ of a giant company, or just a niche product? Often you can't. The founders of Airbnb didn't realize at first how big a market they were tapping. Initially they had a much narrower idea. They were going to let hosts rent out space on their floors during conventions. They didn't foresee the expansion of this idea; it forced itself upon them gradually. All they knew at first is that they were onto something. That's probably as much as Bill Gates or Mark Zuckerberg knew at first.

Occasionally it's obvious from the beginning when there's a path out of the initial niche. And sometimes I can see a path that's not immediately obvious; that's one of our specialties at YC. But there are limits to how well this can be done, no matter how much experience you have. The most important thing to understand about paths out of the initial idea is the meta-fact that these are hard to see.

So if you can't predict whether there's a path out of an idea, how do you choose

between ideas? The truth is disappointing but interesting: if you're the right sort of person, you have the right sort of hunches. If you're at the leading edge of a field that's changing fast, when you have a hunch that something is worth doing, you're more likely to be right.

In *Zen and the Art of Motorcycle Maintenance*, Robert Pirsig says:

You want to know how to paint a perfect painting? It's easy. Make yourself perfect and then just paint naturally.

I've wondered about that passage since I read it in high school. I'm not sure how useful his advice is for painting specifically, but it fits this situation well. Empirically, the way to have good startup ideas is to become the sort of person who has them.

Being at the leading edge of a field doesn't mean you have to be one of the people pushing it forward. You can also be at the leading edge as a user. It was not so much because he was a programmer that Facebook seemed a good idea to Mark Zuckerberg as because he used computers so much. If you'd asked most 40 year olds in 2004 whether they'd like to publish their lives semi-publicly on the Internet, they'd have been horrified at the idea. But Mark already lived online; to him it seemed natural.

Paul Buchheit says that people at the leading edge of a rapidly changing field "live in the future." Combine that with Pirsig and you get:

Live in the future, then build what's missing.

That describes the way many if not most of the biggest startups got started. Neither Apple nor Yahoo nor Google nor Facebook were even supposed to be companies at first. They grew out of things their founders built because there seemed a gap in the world.

If you look at the way successful founders have had their ideas, it's generally the result of some external stimulus hitting a prepared mind. Bill Gates and Paul Allen hear about the Altair and think "I bet we could write a Basic interpreter for it." Drew Houston realizes he's forgotten his USB stick and thinks "I really need to make my files live online." Lots of people heard about the Altair. Lots forgot USB sticks. The reason those stimuli caused those founders to start companies was that their experiences had prepared them to notice the opportunities they represented.

The verb you want to be using with respect to startup ideas is not "think up" but "notice." At YC we call ideas that grow naturally out of the founders' own experiences "organic" startup ideas. The most successful startups almost all begin this way.

That may not have been what you wanted to hear. You may have expected recipes for coming up with startup ideas, and instead I'm telling you that the key is to have a mind that's prepared in the right way. But disappointing though it may be, this is the truth. And it is a recipe of a sort, just one that in the worst case takes a year rather than a weekend.

If you're not at the leading edge of some rapidly changing field, you can get to one. For example, anyone reasonably smart can probably get to an edge of programming (e.g. building mobile apps) in a year. Since a successful startup will consume at least 3-5 years of your life, a year's preparation would be a reasonable investment. Especially if you're also looking for a cofounder. [4]

You don't have to learn programming to be at the leading edge of a domain that's changing fast. Other domains change fast. But while learning to hack is not necessary, it is for the foreseeable future sufficient. As Marc Andreessen put it, software is eating the world, and this trend has decades left to run.

Knowing how to hack also means that when you have ideas, you'll be able to implement them. That's not absolutely necessary (Jeff Bezos couldn't) but it's an advantage. It's a big advantage, when you're considering an idea like putting a college facebook online, if instead of merely thinking "That's an interesting idea," you can think instead "That's an interesting idea. I'll try building an initial version tonight." It's even better when you're both a programmer and the target user, because then the cycle of generating new versions and testing them on users can happen inside one head.

Noticing

Once you're living in the future in some respect, the way to notice startup ideas is to look for things that seem to be missing. If you're really at the leading edge of a rapidly changing field, there will be things that are obviously missing. What won't be obvious is that they're startup ideas. So if you want to find startup ideas, don't merely turn on the filter "What's missing?" Also turn off every other filter, particularly "Could this be a big company?" There's plenty of time to apply that test later. But if you're thinking about that initially, it may not only filter out lots of good ideas, but also cause you to focus on bad ones.

Most things that are missing will take some time to see. You almost have to trick yourself into seeing the ideas around you.

But you *know* the ideas are out there. This is not one of those problems where there might not be an answer. It's impossibly unlikely that this is the exact moment when technological progress stops. You can be sure people are going to build things in the next few years that will make you think "What did I do before x?"

And when these problems get solved, they will probably seem flamingly obvious in retrospect. What you need to do is turn off the filters that usually prevent you from seeing them. The most powerful is simply taking the current state of the world for granted. Even the most radically open-minded of us mostly do that. You couldn't get from your bed to the front door if you stopped to question everything.

But if you're looking for startup ideas you can sacrifice some of the efficiency of taking the status quo for granted and start to question things. Why is your inbox overflowing? Because you get a lot of email, or because it's hard to get email out of your inbox? Why do you get so much email? What problems are people trying to solve by sending you email? Are there better ways to solve them? And why is it hard to get emails out of your inbox? Why do you keep emails around after you've read them? Is an inbox the optimal tool for that?

Pay particular attention to things that chafe you. The advantage of taking the status quo for granted is not just that it makes life (locally) more efficient, but also that it makes life more tolerable. If you knew about all the things we'll get in the next 50 years but don't have yet, you'd find present day life pretty constraining, just as someone from the present would if they were sent back 50 years in a time machine. When something annoys you, it could be because you're living in the future.

When you find the right sort of problem, you should probably be able to describe it as *obvious*, at least to you. When we started Viaweb, all the online stores were built by hand, by web designers making individual HTML pages. It was obvious to us as programmers that these sites would have to be generated by software. [5]

Which means, strangely enough, that coming up with startup ideas is a question of seeing the obvious. That suggests how weird this process is: you're trying to see things that are obvious, and yet that you hadn't seen.

Since what you need to do here is loosen up your own mind, it may be best not to make too much of a direct frontal attack on the problem — i.e. to sit down and try to think of ideas. The best plan may be just to keep a background process running, looking for things that seem to be missing. Work on hard problems, driven mainly by curiosity, but have a second self watching over your shoulder, taking note of gaps and anomalies. [6]

Give yourself some time. You have a lot of control over the rate at which you turn yours into a prepared mind, but you have less control over the stimuli that spark ideas when they hit it. If Bill Gates and Paul Allen had constrained themselves to come up with a startup idea in one month, what if they'd chosen a

month before the Altair appeared? They probably would have worked on a less promising idea. Drew Houston did work on a less promising idea before Dropbox: an SAT prep startup. But Dropbox was a much better idea, both in the absolute sense and also as a match for his skills. [7]

A good way to trick yourself into noticing ideas is to work on projects that seem like they'd be cool. If you do that, you'll naturally tend to build things that are missing. It wouldn't seem as interesting to build something that already existed.

Just as trying to think up startup ideas tends to produce bad ones, working on things that could be dismissed as "toys" often produces good ones. When something is described as a toy, that means it has everything an idea needs except being important. It's cool; users love it; it just doesn't matter. But if you're living in the future and you build something cool that users love, it may matter more than outsiders think. Microcomputers seemed like toys when Apple and Microsoft started working on them. I'm old enough to remember that era; the usual term for people with their own microcomputers was "hobbyists." BackRub seemed like an inconsequential science project. The Facebook was just a way for undergrads to stalk one another.

At YC we're excited when we meet startups working on things that we could imagine know-it-alls on forums dismissing as toys. To us that's positive evidence an idea is good.

If you can afford to take a long view (and arguably you can't afford not to), you can turn "Live in the future and build what's missing" into something even better:

Live in the future and build what seems interesting.

School

That's what I'd advise college students to do, rather than trying to learn about "entrepreneurship." "Entrepreneurship" is something you learn best by doing it. The examples of the most successful founders make that clear. What you should be spending your time on in college is ratcheting yourself into the future. College is an incomparable opportunity to do that. What a waste to sacrifice an opportunity to solve the hard part of starting a startup — becoming the sort of person who can have organic startup ideas — by spending time learning about the easy part. Especially since you won't even really learn about it, any more than you'd learn about sex in a class. All you'll learn is the words for things.

The clash of domains is a particularly fruitful source of ideas. If you know a lot about programming and you start learning about some other field, you'll probably see problems that software could solve. In fact, you're doubly likely

to find good problems in another domain: (a) the inhabitants of that domain are not as likely as software people to have already solved their problems with software, and (b) since you come into the new domain totally ignorant, you don't even know what the status quo is to take it for granted.

So if you're a CS major and you want to start a startup, instead of taking a class on entrepreneurship you're better off taking a class on, say, genetics. Or better still, go work for a biotech company. CS majors normally get summer jobs at computer hardware or software companies. But if you want to find startup ideas, you might do better to get a summer job in some unrelated field. [8]

Or don't take any extra classes, and just build things. It's no coincidence that Microsoft and Facebook both got started in January. At Harvard that is (or was) Reading Period, when students have no classes to attend because they're supposed to be studying for finals. [9]

But don't feel like you have to build things that will become startups. That's premature optimization. Just build things. Preferably with other students. It's not just the classes that make a university such a good place to crank oneself into the future. You're also surrounded by other people trying to do the same thing. If you work together with them on projects, you'll end up producing not just organic ideas, but organic ideas with organic founding teams — and that, empirically, is the best combination.

Beware of research. If an undergrad writes something all his friends start using, it's quite likely to represent a good startup idea. Whereas a PhD dissertation is extremely unlikely to. For some reason, the more a project has to count as research, the less likely it is to be something that could be turned into a startup. [10] I think the reason is that the subset of ideas that count as research is so narrow that it's unlikely that a project that satisfied that constraint would also satisfy the orthogonal constraint of solving users' problems. Whereas when students (or professors) build something as a side-project, they automatically gravitate toward solving users' problems — perhaps even with an additional energy that comes from being freed from the constraints of research.

Competition

Because a good idea should seem obvious, when you have one you'll tend to feel that you're late. Don't let that deter you. Worrying that you're late is one of the signs of a good idea. Ten minutes of searching the web will usually settle the question. Even if you find someone else working on the same thing, you're probably not too late. It's exceptionally rare for startups to be killed by competitors — so rare that you can almost discount the possibility. So unless you discover a competitor with the sort of lock-in that would prevent users from

choosing you, don't discard the idea.

If you're uncertain, ask users. The question of whether you're too late is subsumed by the question of whether anyone urgently needs what you plan to make. If you have something that no competitor does and that some subset of users urgently need, you have a beachhead. [11]

The question then is whether that beachhead is big enough. Or more importantly, who's in it: if the beachhead consists of people doing something lots more people will be doing in the future, then it's probably big enough no matter how small it is. For example, if you're building something differentiated from competitors by the fact that it works on phones, but it only works on the newest phones, that's probably a big enough beachhead.

Err on the side of doing things where you'll face competitors. Inexperienced founders usually give competitors more credit than they deserve. Whether you succeed depends far more on you than on your competitors. So better a good idea with competitors than a bad one without.

You don't need to worry about entering a "crowded market" so long as you have a thesis about what everyone else in it is overlooking. In fact that's a very promising starting point. Google was that type of idea. Your thesis has to be more precise than "we're going to make an x that doesn't suck" though. You have to be able to phrase it in terms of something the incumbents are overlooking. Best of all is when you can say that they didn't have the courage of their convictions, and that your plan is what they'd have done if they'd followed through on their own insights. Google was that type of idea too. The search engines that preceded them shied away from the most radical implications of what they were doing — particularly that the better a job they did, the faster users would leave.

A crowded market is actually a good sign, because it means both that there's demand and that none of the existing solutions are good enough. A startup can't hope to enter a market that's obviously big and yet in which they have no competitors. So any startup that succeeds is either going to be entering a market with existing competitors, but armed with some secret weapon that will get them all the users (like Google), or entering a market that looks small but which will turn out to be big (like Microsoft). [12]

Filters

There are two more filters you'll need to turn off if you want to notice startup ideas: the unsexy filter and the schlep filter.

Most programmers wish they could start a startup by just writing some brilliant

code, pushing it to a server, and having users pay them lots of money. They'd prefer not to deal with tedious problems or get involved in messy ways with the real world. Which is a reasonable preference, because such things slow you down. But this preference is so widespread that the space of convenient startup ideas has been stripped pretty clean. If you let your mind wander a few blocks down the street to the messy, tedious ideas, you'll find valuable ones just sitting there waiting to be implemented.

The schlep filter is so dangerous that I wrote a separate essay about the condition it induces, which I called schlep blindness. I gave Stripe as an example of a startup that benefited from turning off this filter, and a pretty striking example it is. Thousands of programmers were in a position to see this idea; thousands of programmers knew how painful it was to process payments before Stripe. But when they looked for startup ideas they didn't see this one, because unconsciously they shrank from having to deal with payments. And dealing with payments is a schlep for Stripe, but not an intolerable one. In fact they might have had net less pain; because the fear of dealing with payments kept most people away from this idea, Stripe has had comparatively smooth sailing in other areas that are sometimes painful, like user acquisition. They didn't have to try very hard to make themselves heard by users, because users were desperately waiting for what they were building.

The unsexy filter is similar to the schlep filter, except it keeps you from working on problems you despise rather than ones you fear. We overcame this one to work on Viaweb. There were interesting things about the architecture of our software, but we weren't interested in ecommerce per se. We could see the problem was one that needed to be solved though.

Turning off the schlep filter is more important than turning off the unsexy filter, because the schlep filter is more likely to be an illusion. And even to the degree it isn't, it's a worse form of self-indulgence. Starting a successful startup is going to be fairly laborious no matter what. Even if the product doesn't entail a lot of schleps, you'll still have plenty dealing with investors, hiring and firing people, and so on. So if there's some idea you think would be cool but you're kept away from by fear of the schleps involved, don't worry: any sufficiently good idea will have as many.

The unsexy filter, while still a source of error, is not as entirely useless as the schlep filter. If you're at the leading edge of a field that's changing rapidly, your ideas about what's sexy will be somewhat correlated with what's valuable in practice. Particularly as you get older and more experienced. Plus if you find an idea sexy, you'll work on it more enthusiastically. [13]

Recipes

While the best way to discover startup ideas is to become the sort of person who has them and then build whatever interests you, sometimes you don't have that luxury. Sometimes you need an idea now. For example, if you're working on a startup and your initial idea turns out to be bad.

For the rest of this essay I'll talk about tricks for coming up with startup ideas on demand. Although empirically you're better off using the organic strategy, you could succeed this way. You just have to be more disciplined. When you use the organic method, you don't even notice an idea unless it's evidence that something is truly missing. But when you make a conscious effort to think of startup ideas, you have to replace this natural constraint with self-discipline. You'll see a lot more ideas, most of them bad, so you need to be able to filter them.

One of the biggest dangers of not using the organic method is the example of the organic method. Organic ideas feel like inspirations. There are a lot of stories about successful startups that began when the founders had what seemed a crazy idea but "just knew" it was promising. When you feel that about an idea you've had while trying to come up with startup ideas, you're probably mistaken.

When searching for ideas, look in areas where you have some expertise. If you're a database expert, don't build a chat app for teenagers (unless you're also a teenager). Maybe it's a good idea, but you can't trust your judgment about that, so ignore it. There have to be other ideas that involve databases, and whose quality you can judge. Do you find it hard to come up with good ideas involving databases? That's because your expertise raises your standards. Your ideas about chat apps are just as bad, but you're giving yourself a Dunning-Kruger pass in that domain.

The place to start looking for ideas is things you need. There *must* be things you need. [14]

One good trick is to ask yourself whether in your previous job you ever found yourself saying "Why doesn't someone make x? If someone made x we'd buy it in a second." If you can think of any x people said that about, you probably have an idea. You know there's demand, and people don't say that about things that are impossible to build.

More generally, try asking yourself whether there's something unusual about you that makes your needs different from most other people's. You're probably not the only one. It's especially good if you're different in a way people will increasingly be.

If you're changing ideas, one unusual thing about you is the idea you'd previously been working on. Did you discover any needs while working on it? Several well-known startups began this way. Hotmail began as something its founders wrote to talk about their previous startup idea while they were working at their day jobs. [15]

A particularly promising way to be unusual is to be young. Some of the most valuable new ideas take root first among people in their teens and early twenties. And while young founders are at a disadvantage in some respects, they're the only ones who really understand their peers. It would have been very hard for someone who wasn't a college student to start Facebook. So if you're a young founder (under 23 say), are there things you and your friends would like to do that current technology won't let you?

The next best thing to an unmet need of your own is an unmet need of someone else. Try talking to everyone you can about the gaps they find in the world. What's missing? What would they like to do that they can't? What's tedious or annoying, particularly in their work? Let the conversation get general; don't be trying too hard to find startup ideas. You're just looking for something to spark a thought. Maybe you'll notice a problem they didn't consciously realize they had, because you know how to solve it.

When you find an unmet need that isn't your own, it may be somewhat blurry at first. The person who needs something may not know exactly what they need. In that case I often recommend that founders act like consultants — that they do what they'd do if they'd been retained to solve the problems of this one user. People's problems are similar enough that nearly all the code you write this way will be reusable, and whatever isn't will be a small price to start out certain that you've reached the bottom of the well. [16]

One way to ensure you do a good job solving other people's problems is to make them your own. When Rajat Suri of E la Carte decided to write software for restaurants, he got a job as a waiter to learn how restaurants worked. That may seem like taking things to extremes, but startups are extreme. We love it when founders do such things.

In fact, one strategy I recommend to people who need a new idea is not merely to turn off their schlep and unsexy filters, but to seek out ideas that are unsexy or involve schleps. Don't try to start Twitter. Those ideas are so rare that you can't find them by looking for them. Make something unsexy that people will pay you for.

A good trick for bypassing the schlep and to some extent the unsexy filter is to ask what you wish someone else would build, so that you could use it. What

would you pay for right now?

Since startups often garbage-collect broken companies and industries, it can be a good trick to look for those that are dying, or deserve to, and try to imagine what kind of company would profit from their demise. For example, journalism is in free fall at the moment. But there may still be money to be made from something like journalism. What sort of company might cause people in the future to say “this replaced journalism” on some axis?

But imagine asking that in the future, not now. When one company or industry replaces another, it usually comes in from the side. So don’t look for a replacement for x; look for something that people will later say turned out to be a replacement for x. And be imaginative about the axis along which the replacement occurs. Traditional journalism, for example, is a way for readers to get information and to kill time, a way for writers to make money and to get attention, and a vehicle for several different types of advertising. It could be replaced on any of these axes (it has already started to be on most).

When startups consume incumbents, they usually start by serving some small but important market that the big players ignore. It’s particularly good if there’s an admixture of disdain in the big players’ attitude, because that often misleads them. For example, after Steve Wozniak built the computer that became the Apple I, he felt obliged to give his then-employer Hewlett-Packard the option to produce it. Fortunately for him, they turned it down, and one of the reasons they did was that it used a TV for a monitor, which seemed intolerably *déclassé* to a high-end hardware company like HP was at the time. [17]

Are there groups of scruffy but sophisticated users like the early microcomputer “hobbyists” that are currently being ignored by the big players? A startup with its sights set on bigger things can often capture a small market easily by expending an effort that wouldn’t be justified by that market alone.

Similarly, since the most successful startups generally ride some wave bigger than themselves, it could be a good trick to look for waves and ask how one could benefit from them. The prices of gene sequencing and 3D printing are both experiencing Moore’s Law-like declines. What new things will we be able to do in the new world we’ll have in a few years? What are we unconsciously ruling out as impossible that will soon be possible?

Organic

But talking about looking explicitly for waves makes it clear that such recipes are plan B for getting startup ideas. Looking for waves is essentially a way to simulate the organic method. If you’re at the leading edge of some rapidly changing field, you don’t have to look for waves; you are the wave.

Finding startup ideas is a subtle business, and that's why most people who try fail so miserably. It doesn't work well simply to try to think of startup ideas. If you do that, you get bad ones that sound dangerously plausible. The best approach is more indirect: if you have the right sort of background, good startup ideas will seem obvious to you. But even then, not immediately. It takes time to come across situations where you notice something missing. And often these gaps won't seem to be ideas for companies, just things that would be interesting to build. Which is why it's good to have the time and the inclination to build things just because they're interesting.

Live in the future and build what seems interesting. Strange as it sounds, that's the real recipe.

Notes

[1] This form of bad idea has been around as long as the web. It was common in the 1990s, except then people who had it used to say they were going to create a portal for x instead of a social network for x. Structurally the idea is stone soup: you post a sign saying "this is the place for people interested in x," and all those people show up and you make money from them. What lures founders into this sort of idea are statistics about the millions of people who might be interested in each type of x. What they forget is that any given person might have 20 affinities by this standard, and no one is going to visit 20 different communities regularly.

[2] I'm not saying, incidentally, that I know for sure a social network for pet owners is a bad idea. I know it's a bad idea the way I know randomly generated DNA would not produce a viable organism. The set of plausible sounding startup ideas is many times larger than the set of good ones, and many of the good ones don't even sound that plausible. So if all you know about a startup idea is that it sounds plausible, you have to assume it's bad.

[3] More precisely, the users' need has to give them sufficient activation energy to start using whatever you make, which can vary a lot. For example, the activation energy for enterprise software sold through traditional channels is very high, so you'd have to be a *lot* better to get users to switch. Whereas the activation energy required to switch to a new search engine is low. Which in turn is why search engines are so much better than enterprise software.

[4] This gets harder as you get older. While the space of ideas doesn't have dangerous local maxima, the space of careers does. There are fairly high walls between most of the paths people take through life, and the older you get, the higher the walls become.

[5] It was also obvious to us that the web was going to be a big deal. Few non-programmers grasped that in 1995, but the programmers had seen what

GUIs had done for desktop computers.

[6] Maybe it would work to have this second self keep a journal, and each night to make a brief entry listing the gaps and anomalies you'd noticed that day. Not startup ideas, just the raw gaps and anomalies.

[7] Sam Altman points out that taking time to come up with an idea is not merely a better strategy in an absolute sense, but also like an undervalued stock in that so few founders do it.

There's comparatively little competition for the best ideas, because few founders are willing to put in the time required to notice them. Whereas there is a great deal of competition for mediocre ideas, because when people make up startup ideas, they tend to make up the same ones.

[8] For the computer hardware and software companies, summer jobs are the first phase of the recruiting funnel. But if you're good you can skip the first phase. If you're good you'll have no trouble getting hired by these companies when you graduate, regardless of how you spent your summers.

[9] The empirical evidence suggests that if colleges want to help their students start startups, the best thing they can do is leave them alone in the right way.

[10] I'm speaking here of IT startups; in biotech things are different.

[11] This is an instance of a more general rule: focus on users, not competitors. The most important information about competitors is what you learn via users anyway.

[12] In practice most successful startups have elements of both. And you can describe each strategy in terms of the other by adjusting the boundaries of what you call the market. But it's useful to consider these two ideas separately.

[13] I almost hesitate to raise that point though. Startups are businesses; the point of a business is to make money; and with that additional constraint, you can't expect you'll be able to spend all your time working on what interests you most.

[14] The need has to be a strong one. You can retroactively describe any made-up idea as something you need. But do you really need that recipe site or local event aggregator as much as Drew Houston needed Dropbox, or Brian Chesky and Joe Gebbia needed Airbnb?

Quite often at YC I find myself asking founders "Would you use this thing yourself, if you hadn't written it?" and you'd be surprised how often the answer is no.

[15] Paul Buchheit points out that trying to sell something bad can be a source of better ideas:

“The best technique I’ve found for dealing with YC companies that have bad ideas is to tell them to go sell the product ASAP (before wasting time building it). Not only do they learn that nobody wants what they are building, they very often come back with a real idea that they discovered in the process of trying to sell the bad idea.”

[16] Here’s a recipe that might produce the next Facebook, if you’re college students. If you have a connection to one of the more powerful sororities at your school, approach the queen bees thereof and offer to be their personal IT consultants, building anything they could imagine needing in their social lives that didn’t already exist. Anything that got built this way would be very promising, because such users are not just the most demanding but also the perfect point to spread from.

I have no idea whether this would work.

[17] And the reason it used a TV for a monitor is that Steve Wozniak started out by solving his own problems. He, like most of his peers, couldn’t afford a monitor.

Thanks to Sam Altman, Mike Arrington, Paul Buchheit, John Collison, Patrick Collison, Garry Tan, and Harj Taggar for reading drafts of this, and Marc Andreessen, Joe Gebbia, Reid Hoffman, Shel Kaphan, Mike Moritz and Kevin Systrom for answering my questions about startup history.

Do Things that Don't Scale

July 2013

One of the most common types of advice we give at Y Combinator is to do things that don't scale. A lot of would-be founders believe that startups either take off or don't. You build something, make it available, and if you've made a better mousetrap, people beat a path to your door as promised. Or they don't, in which case the market must not exist. [1]

Actually startups take off because the founders make them take off. There may be a handful that just grew by themselves, but usually it takes some sort of push to get them going. A good metaphor would be the cranks that car engines had before they got electric starters. Once the engine was going, it would keep going, but there was a separate and laborious process to get it going.

Recruit

The most common unscalable thing founders have to do at the start is to recruit users manually. Nearly all startups have to. You can't wait for users to come to you. You have to go out and get them.

Stripe is one of the most successful startups we've funded, and the problem they solved was an urgent one. If anyone could have sat back and waited for users, it was Stripe. But in fact they're famous within YC for aggressive early user acquisition.

Startups building things for other startups have a big pool of potential users in the other companies we've funded, and none took better advantage of it than Stripe. At YC we use the term "Collison installation" for the technique they invented. More diffident founders ask "Will you try our beta?" and if the answer is yes, they say "Great, we'll send you a link." But the Collison brothers weren't going to wait. When anyone agreed to try Stripe they'd say "Right then, give me your laptop" and set them up on the spot.

There are two reasons founders resist going out and recruiting users individually.

One is a combination of shyness and laziness. They'd rather sit at home writing code than go out and talk to a bunch of strangers and probably be rejected by most of them. But for a startup to succeed, at least one founder (usually the CEO) will have to spend a lot of time on sales and marketing. [2]

The other reason founders ignore this path is that the absolute numbers seem so small at first. This can't be how the big, famous startups got started, they think. The mistake they make is to underestimate the power of compound growth. We encourage every startup to measure their progress by weekly growth rate. If you have 100 users, you need to get 10 more next week to grow 10% a week. And while 110 may not seem much better than 100, if you keep growing at 10% a week you'll be surprised how big the numbers get. After a year you'll have 14,000 users, and after 2 years you'll have 2 million.

You'll be doing different things when you're acquiring users a thousand at a time, and growth has to slow down eventually. But if the market exists you can usually start by recruiting users manually and then gradually switch to less manual methods. [3]

Airbnb is a classic example of this technique. Marketplaces are so hard to get rolling that you should expect to take heroic measures at first. In Airbnb's case, these consisted of going door to door in New York, recruiting new users and helping existing ones improve their listings. When I remember the Airbnbs during YC, I picture them with rolly bags, because when they showed up for tuesday dinners they'd always just flown back from somewhere.

Fragile

Airbnb now seems like an unstoppable juggernaut, but early on it was so fragile that about 30 days of going out and engaging in person with users made the difference between success and failure.

That initial fragility was not a unique feature of Airbnb. Almost all startups are fragile initially. And that's one of the biggest things inexperienced founders and investors (and reporters and know-it-alls on forums) get wrong about them. They unconsciously judge larval startups by the standards of established ones. They're like someone looking at a newborn baby and concluding "there's no way this tiny creature could ever accomplish anything."

It's harmless if reporters and know-it-alls dismiss your startup. They always get things wrong. It's even ok if investors dismiss your startup; they'll change their minds when they see growth. The big danger is that you'll dismiss your startup yourself. I've seen it happen. I often have to encourage founders who don't see the full potential of what they're building. Even Bill Gates made that mistake. He returned to Harvard for the fall semester after starting Microsoft. He didn't

stay long, but he wouldn't have returned at all if he'd realized Microsoft was going to be even a fraction of the size it turned out to be. [4]

The question to ask about an early stage startup is not “is this company taking over the world?” but “how big could this company get if the founders did the right things?” And the right things often seem both laborious and inconsequential at the time. Microsoft can't have seemed very impressive when it was just a couple guys in Albuquerque writing Basic interpreters for a market of a few thousand hobbyists (as they were then called), but in retrospect that was the optimal path to dominating microcomputer software. And I know Brian Chesky and Joe Gebbia didn't feel like they were en route to the big time as they were taking “professional” photos of their first hosts' apartments. They were just trying to survive. But in retrospect that too was the optimal path to dominating a big market.

How do you find users to recruit manually? If you build something to solve your own problems, then you only have to find your peers, which is usually straightforward. Otherwise you'll have to make a more deliberate effort to locate the most promising vein of users. The usual way to do that is to get some initial set of users by doing a comparatively untargeted launch, and then to observe which kind seem most enthusiastic, and seek out more like them. For example, Ben Silbermann noticed that a lot of the earliest Pinterest users were interested in design, so he went to a conference of design bloggers to recruit users, and that worked well. [5]

Delight

You should take extraordinary measures not just to acquire users, but also to make them happy. For as long as they could (which turned out to be surprisingly long), Wufoo sent each new user a hand-written thank you note. Your first users should feel that signing up with you was one of the best choices they ever made. And you in turn should be racking your brains to think of new ways to delight them.

Why do we have to teach startups this? Why is it counterintuitive for founders? Three reasons, I think.

One is that a lot of startup founders are trained as engineers, and customer service is not part of the training of engineers. You're supposed to build things that are robust and elegant, not be slavishly attentive to individual users like some kind of salesperson. Ironically, part of the reason engineering is traditionally averse to handholding is that its traditions date from a time when engineers were less powerful — when they were only in charge of their narrow domain of building things, rather than running the whole show. You can be ornery when

you're Scotty, but not when you're Kirk.

Another reason founders don't focus enough on individual customers is that they worry it won't scale. But when founders of larval startups worry about this, I point out that in their current state they have nothing to lose. Maybe if they go out of their way to make existing users super happy, they'll one day have too many to do so much for. That would be a great problem to have. See if you can make it happen. And incidentally, when it does, you'll find that delighting customers scales better than you expected. Partly because you can usually find ways to make anything scale more than you would have predicted, and partly because delighting customers will by then have permeated your culture.

I have never once seen a startup lured down a blind alley by trying too hard to make their initial users happy.

But perhaps the biggest thing preventing founders from realizing how attentive they could be to their users is that they've never experienced such attention themselves. Their standards for customer service have been set by the companies they've been customers of, which are mostly big ones. Tim Cook doesn't send you a hand-written note after you buy a laptop. He can't. But you can. That's one advantage of being small: you can provide a level of service no big company can. [6]

Once you realize that existing conventions are not the upper bound on user experience, it's interesting in a very pleasant way to think about how far you could go to delight your users.

Experience

I was trying to think of a phrase to convey how extreme your attention to users should be, and I realized Steve Jobs had already done it: insanely great. Steve wasn't just using "insanely" as a synonym for "very." He meant it more literally — that one should focus on quality of execution to a degree that in everyday life would be considered pathological.

All the most successful startups we've funded have, and that probably doesn't surprise would-be founders. What novice founders don't get is what insanely great translates to in a larval startup. When Steve Jobs started using that phrase, Apple was already an established company. He meant the Mac (and its documentation and even packaging — such is the nature of obsession) should be insanely well designed and manufactured. That's not hard for engineers to grasp. It's just a more extreme version of designing a robust and elegant product.

What founders have a hard time grasping (and Steve himself might have had a hard time grasping) is what insanely great morphs into as you roll the time

slider back to the first couple months of a startup's life. It's not the product that should be insanely great, but the experience of being your user. The product is just one component of that. For a big company it's necessarily the dominant one. But you can and should give users an insanely great experience with an early, incomplete, buggy product, if you make up the difference with attentiveness.

Can, perhaps, but should? Yes. Over-engaging with early users is not just a permissible technique for getting growth rolling. For most successful startups it's a necessary part of the feedback loop that makes the product good. Making a better mousetrap is not an atomic operation. Even if you start the way most successful startups have, by building something you yourself need, the first thing you build is never quite right. And except in domains with big penalties for making mistakes, it's often better not to aim for perfection initially. In software, especially, it usually works best to get something in front of users as soon as it has a quantum of utility, and then see what they do with it. Perfectionism is often an excuse for procrastination, and in any case your initial model of users is always inaccurate, even if you're one of them. [7]

The feedback you get from engaging directly with your earliest users will be the best you ever get. When you're so big you have to resort to focus groups, you'll wish you could go over to your users' homes and offices and watch them use your stuff like you did when there were only a handful of them.

Fire

Sometimes the right unscalable trick is to focus on a deliberately narrow market. It's like keeping a fire contained at first to get it really hot before adding more logs.

That's what Facebook did. At first it was just for Harvard students. In that form it only had a potential market of a few thousand people, but because they felt it was really for them, a critical mass of them signed up. After Facebook stopped being for Harvard students, it remained for students at specific colleges for quite a while. When I interviewed Mark Zuckerberg at Startup School, he said that while it was a lot of work creating course lists for each school, doing that made students feel the site was their natural home.

Any startup that could be described as a marketplace usually has to start in a subset of the market, but this can work for other startups as well. It's always worth asking if there's a subset of the market in which you can get a critical mass of users quickly. [8]

Most startups that use the contained fire strategy do it unconsciously. They build something for themselves and their friends, who happen to be the early adopters, and only realize later that they could offer it to a broader market. The

strategy works just as well if you do it unconsciously. The biggest danger of not being consciously aware of this pattern is for those who naively discard part of it. E.g. if you don't build something for yourself and your friends, or even if you do, but you come from the corporate world and your friends are not early adopters, you'll no longer have a perfect initial market handed to you on a platter.

Among companies, the best early adopters are usually other startups. They're more open to new things both by nature and because, having just been started, they haven't made all their choices yet. Plus when they succeed they grow fast, and you with them. It was one of many unforeseen advantages of the YC model (and specifically of making YC big) that B2B startups now have an instant market of hundreds of other startups ready at hand.

Meraki

For hardware startups there's a variant of doing things that don't scale that we call "pulling a Meraki." Although we didn't fund Meraki, the founders were Robert Morris's grad students, so we know their history. They got started by doing something that really doesn't scale: assembling their routers themselves.

Hardware startups face an obstacle that software startups don't. The minimum order for a factory production run is usually several hundred thousand dollars. Which can put you in a catch-22: without a product you can't generate the growth you need to raise the money to manufacture your product. Back when hardware startups had to rely on investors for money, you had to be pretty convincing to overcome this. The arrival of crowdfunding (or more precisely, preorders) has helped a lot. But even so I'd advise startups to pull a Meraki initially if they can. That's what Pebble did. The Pebbles assembled the first several hundred watches themselves. If they hadn't gone through that phase, they probably wouldn't have sold \$10 million worth of watches when they did go on Kickstarter.

Like paying excessive attention to early customers, fabricating things yourself turns out to be valuable for hardware startups. You can tweak the design faster when you're the factory, and you learn things you'd never have known otherwise. Eric Migicovsky of Pebble said one of the things he learned was "how valuable it was to source good screws." Who knew?

Consult

Sometimes we advise founders of B2B startups to take over-engagement to an extreme, and to pick a single user and act as if they were consultants building something just for that one user. The initial user serves as the form for your mold; keep tweaking till you fit their needs perfectly, and you'll usually find you've made something other users want too. Even if there aren't many of them,

there are probably adjacent territories that have more. As long as you can find just one user who really needs something and can act on that need, you've got a toehold in making something people want, and that's as much as any startup needs initially. [9]

Consulting is the canonical example of work that doesn't scale. But (like other ways of bestowing one's favors liberally) it's safe to do it so long as you're not being paid to. That's where companies cross the line. So long as you're a product company that's merely being extra attentive to a customer, they're very grateful even if you don't solve all their problems. But when they start paying you specifically for that attentiveness — when they start paying you by the hour — they expect you to do everything.

Another consulting-like technique for recruiting initially lukewarm users is to use your software yourselves on their behalf. We did that at Viaweb. When we approached merchants asking if they wanted to use our software to make online stores, some said no, but they'd let us make one for them. Since we would do anything to get users, we did. We felt pretty lame at the time. Instead of organizing big strategic e-commerce partnerships, we were trying to sell luggage and pens and men's shirts. But in retrospect it was exactly the right thing to do, because it taught us how it would feel to merchants to use our software. Sometimes the feedback loop was near instantaneous: in the middle of building some merchant's site I'd find I needed a feature we didn't have, so I'd spend a couple hours implementing it and then resume building the site.

Manual

There's a more extreme variant where you don't just use your software, but are your software. When you only have a small number of users, you can sometimes get away with doing by hand things that you plan to automate later. This lets you launch faster, and when you do finally automate yourself out of the loop, you'll know exactly what to build because you'll have muscle memory from doing it yourself.

When manual components look to the user like software, this technique starts to have aspects of a practical joke. For example, the way Stripe delivered “instant” merchant accounts to its first users was that the founders manually signed them up for traditional merchant accounts behind the scenes.

Some startups could be entirely manual at first. If you can find someone with a problem that needs solving and you can solve it manually, go ahead and do that for as long as you can, and then gradually automate the bottlenecks. It would be a little frightening to be solving users' problems in a way that wasn't yet automatic, but less frightening than the far more common case of having

something automatic that doesn't yet solve anyone's problems.

Big

I should mention one sort of initial tactic that usually doesn't work: the Big Launch. I occasionally meet founders who seem to believe startups are projectiles rather than powered aircraft, and that they'll make it big if and only if they're launched with sufficient initial velocity. They want to launch simultaneously in 8 different publications, with embargoes. And on a tuesday, of course, since they read somewhere that's the optimum day to launch something.

It's easy to see how little launches matter. Think of some successful startups. How many of their launches do you remember? All you need from a launch is some initial core of users. How well you're doing a few months later will depend more on how happy you made those users than how many there were of them. [10]

So why do founders think launches matter? A combination of solipsism and laziness. They think what they're building is so great that everyone who hears about it will immediately sign up. Plus it would be so much less work if you could get users merely by broadcasting your existence, rather than recruiting them one at a time. But even if what you're building really is great, getting users will always be a gradual process — partly because great things are usually also novel, but mainly because users have other things to think about.

Partnerships too usually don't work. They don't work for startups in general, but they especially don't work as a way to get growth started. It's a common mistake among inexperienced founders to believe that a partnership with a big company will be their big break. Six months later they're all saying the same thing: that was way more work than we expected, and we ended up getting practically nothing out of it. [11]

It's not enough just to do something extraordinary initially. You have to make an extraordinary *effort* initially. Any strategy that omits the effort — whether it's expecting a big launch to get you users, or a big partner — is ipso facto suspect.

Vector

The need to do something unscalably laborious to get started is so nearly universal that it might be a good idea to stop thinking of startup ideas as scalars. Instead we should try thinking of them as pairs of what you're going to build, plus the unscalable thing(s) you're going to do initially to get the company going.

It could be interesting to start viewing startup ideas this way, because now that there are two components you can try to be imaginative about the second as well

as the first. But in most cases the second component will be what it usually is — recruit users manually and give them an overwhelmingly good experience — and the main benefit of treating startups as vectors will be to remind founders they need to work hard in two dimensions. [12]

In the best case, both components of the vector contribute to your company's DNA: the unscalable things you have to do to get started are not merely a necessary evil, but change the company permanently for the better. If you have to be aggressive about user acquisition when you're small, you'll probably still be aggressive when you're big. If you have to manufacture your own hardware, or use your software on users's behalf, you'll learn things you couldn't have learned otherwise. And most importantly, if you have to work hard to delight users when you only have a handful of them, you'll keep doing it when you have a lot.

Notes

[1] Actually Emerson never mentioned mousetraps specifically. He wrote “If a man has good corn or wood, or boards, or pigs, to sell, or can make better chairs or knives, crucibles or church organs, than anybody else, you will find a broad hard-beaten road to his house, though it be in the woods.”

[2] Thanks to Sam Altman for suggesting I make this explicit. And no, you can't avoid doing sales by hiring someone to do it for you. You have to do sales yourself initially. Later you can hire a real salesperson to replace you.

[3] The reason this works is that as you get bigger, your size helps you grow. Patrick Collison wrote “At some point, there was a very noticeable change in how Stripe felt. It tipped from being this boulder we had to push to being a train car that in fact had its own momentum.”

[4] One of the more subtle ways in which YC can help founders is by calibrating their ambitions, because we know exactly how a lot of successful startups looked when they were just getting started.

[5] If you're building something for which you can't easily get a small set of users to observe — e.g. enterprise software — and in a domain where you have no connections, you'll have to rely on cold calls and introductions. But should you even be working on such an idea?

[6] Garry Tan pointed out an interesting trap founders fall into in the beginning. They want so much to seem big that they imitate even the flaws of big companies, like indifference to individual users. This seems to them more “professional.” Actually it's better to embrace the fact that you're small and use whatever advantages that brings.

[7] Your user model almost couldn't be perfectly accurate, because users' needs

often change in response to what you build for them. Build them a microcomputer, and suddenly they need to run spreadsheets on it, because the arrival of your new microcomputer causes someone to invent the spreadsheet.

[8] If you have to choose between the subset that will sign up quickest and those that will pay the most, it's usually best to pick the former, because those are probably the early adopters. They'll have a better influence on your product, and they won't make you expend as much effort on sales. And though they have less money, you don't need that much to maintain your target growth rate early on.

[9] Yes, I can imagine cases where you could end up making something that was really only useful for one user. But those are usually obvious, even to inexperienced founders. So if it's not obvious you'd be making something for a market of one, don't worry about that danger.

[10] There may even be an inverse correlation between launch magnitude and success. The only launches I remember are famous flops like the Segway and Google Wave. Wave is a particularly alarming example, because I think it was actually a great idea that was killed partly by its overdone launch.

[11] Google grew big on the back of Yahoo, but that wasn't a partnership. Yahoo was their customer.

[12] It will also remind founders that an idea where the second component is empty — an idea where there is nothing you can do to get going, e.g. because you have no way to find users to recruit manually — is probably a bad idea, at least for those founders.

Thanks to Sam Altman, Paul Buchheit, Patrick Collison, Kevin Hale, Steven Levy, Jessica Livingston, Geoff Ralston, and Garry Tan for reading drafts of this.

How to Raise Money

September 2013

Most startups that raise money do it more than once. A typical trajectory might be (1) to get started with a few tens of thousands from something like Y Combinator or individual angels, then (2) raise a few hundred thousand to a few million to build the company, and then (3) once the company is clearly succeeding, raise one or more later rounds to accelerate growth.

Reality can be messier. Some companies raise money twice in phase 2. Others skip phase 1 and go straight to phase 2. And at Y Combinator we get an increasing number of companies that have already raised amounts in the hundreds of thousands. But the three phase path is at least the one about which individual startups' paths oscillate.

This essay focuses on phase 2 fundraising. That's the type the startups we fund are doing on Demo Day, and this essay is the advice we give them.

Forces

Fundraising is hard in both senses: hard like lifting a heavy weight, and hard like solving a puzzle. It's hard like lifting a weight because it's intrinsically hard to convince people to part with large sums of money. That problem is irreducible; it should be hard. But much of the other kind of difficulty can be eliminated. Fundraising only seems a puzzle because it's an alien world to most founders, and I hope to fix that by supplying a map through it.

To founders, the behavior of investors is often opaque — partly because their motivations are obscure, but partly because they deliberately mislead you. And the misleading ways of investors combine horribly with the wishful thinking of inexperienced founders. At YC we're always warning founders about this danger, and investors are probably more circumspect with YC startups than with other companies they talk to, and even so we witness a constant series of explosions as these two volatile components combine. [1]

If you're an inexperienced founder, the only way to survive is by imposing external constraints on yourself. You can't trust your intuitions. I'm going to give you a set of rules here that will get you through this process if anything will. At certain moments you'll be tempted to ignore them. So rule number zero is: these rules exist for a reason. You wouldn't need a rule to keep you going in one direction if there weren't powerful forces pushing you in another.

The ultimate source of the forces acting on you are the forces acting on investors. Investors are pinched between two kinds of fear: fear of investing in startups that fizzle, and fear of missing out on startups that take off. The cause of all this fear is the very thing that makes startups such attractive investments: the successful ones grow very fast. But that fast growth means investors can't wait around. If you wait till a startup is obviously a success, it's too late. To get the really high returns, you have to invest in startups when it's still unclear how they'll do. But that in turn makes investors nervous they're about to invest in a flop. As indeed they often are.

What investors would like to do, if they could, is wait. When a startup is only a few months old, every week that passes gives you significantly more information about them. But if you wait too long, other investors might take the deal away from you. And of course the other investors are all subject to the same forces. So what tends to happen is that they all wait as long as they can, then when some act the rest have to.

Don't raise money unless you want it and it wants you.

Such a high proportion of successful startups raise money that it might seem fundraising is one of the defining qualities of a startup. Actually it isn't. Rapid growth is what makes a company a startup. Most companies in a position to grow rapidly find that (a) taking outside money helps them grow faster, and (b) their growth potential makes it easy to attract such money. It's so common for both (a) and (b) to be true of a successful startup that practically all do raise outside money. But there may be cases where a startup either wouldn't want to grow faster, or outside money wouldn't help them to, and if you're one of them, don't raise money.

The other time not to raise money is when you won't be able to. If you try to raise money before you can convince investors, you'll not only waste your time, but also burn your reputation with those investors.

Be in fundraising mode or not.

One of the things that surprises founders most about fundraising is how distracting it is. When you start fundraising, everything else grinds to a halt. The problem is not the time fundraising consumes but that it becomes the top idea in your

mind. A startup can't endure that level of distraction for long. An early stage startup grows mostly because the founders make it grow, and if the founders look away, growth usually drops sharply.

Because fundraising is so distracting, a startup should either be in fundraising mode or not. And when you do decide to raise money, you should focus your whole attention on it so you can get it done quickly and get back to work. [2]

You can take money from investors when you're not in fundraising mode. You just can't expend any attention on it. There are two things that take attention: convincing investors, and negotiating with them. So when you're not in fundraising mode, you should take money from investors only if they require no convincing, and are willing to invest on terms you'll take without negotiation. For example, if a reputable investor is willing to invest on a convertible note, using standard paperwork, that is either uncapped or capped at a good valuation, you can take that without having to think. [3] The terms will be whatever they turn out to be in your next equity round. And "no convincing" means just that: zero time spent meeting with investors or preparing materials for them. If an investor says they're ready to invest, but they need you to come in for one meeting to meet some of the partners, tell them no, if you're not in fundraising mode, because that's fundraising. [4] Tell them politely; tell them you're focusing on the company right now, and that you'll get back to them when you're fundraising; but do not get sucked down the slippery slope.

Investors will try to lure you into fundraising when you're not. It's great for them if they can, because they can thereby get a shot at you before everyone else. They'll send you emails saying they want to meet to learn more about you. If you get cold-emailed by an associate at a VC firm, you shouldn't meet even if you are in fundraising mode. Deals don't happen that way. [5] But even if you get an email from a partner you should try to delay meeting till you're in fundraising mode. They may say they just want to meet and chat, but investors never just want to meet and chat. What if they like you? What if they start to talk about giving you money? Will you be able to resist having that conversation? Unless you're experienced enough at fundraising to have a casual conversation with investors that stays casual, it's safer to tell them that you'd be happy to later, when you're fundraising, but that right now you need to focus on the company. [6]

Companies that are successful at raising money in phase 2 sometimes tack on a few investors after leaving fundraising mode. This is fine; if fundraising went well, you'll be able to do it without spending time convincing them or negotiating about terms.

Get introductions to investors.

Before you can talk to investors, you have to be introduced to them. If you're presenting at a Demo Day, you'll be introduced to a whole bunch simultaneously. But even if you are, you should supplement these with intros you collect yourself.

Do you have to be introduced? In phase 2, yes. Some investors will let you email them a business plan, but you can tell from the way their sites are organized that they don't really want startups to approach them directly.

Intros vary greatly in effectiveness. The best type of intro is from a well-known investor who has just invested in you. So when you get an investor to commit, ask them to introduce you to other investors they respect. [7] The next best type of intro is from a founder of a company they've funded. You can also get intros from other people in the startup community, like lawyers and reporters.

There are now sites like AngelList, FundersClub, and WeFunder that can introduce you to investors. We recommend startups treat them as auxiliary sources of money. Raise money first from leads you get yourself. Those will on average be better investors. Plus you'll have an easier time raising money on these sites once you can say you've already raised some from well-known investors.

Hear no till you hear yes.

Treat investors as saying no till they unequivocally say yes, in the form of a definite offer with no contingencies.

I mentioned earlier that investors prefer to wait if they can. What's particularly dangerous for founders is the way they wait. Essentially, they lead you on. They seem like they're about to invest right up till the moment they say no. If they even say no. Some of the worse ones never actually do say no; they just stop replying to your emails. They hope that way to get a free option on investing. If they decide later that they want to invest — usually because they've heard you're a hot deal — they can pretend they just got distracted and then restart the conversation as if they'd been about to. [8]

That's not the worst thing investors will do. Some will use language that makes it sound as if they're committing, but which doesn't actually commit them. And wishful thinking founders are happy to meet them half way. [9]

Fortunately, the next rule is a tactic for neutralizing this behavior. But to work it depends on you not being tricked by the no that sounds like yes. It's so common for founders to be misled/mistaken about this that we designed a protocol to fix the problem. If you believe an investor has committed, get them to confirm it. If you and they have different views of reality, whether the source of the discrepancy is their sketchiness or your wishful thinking, the prospect

of confirming a commitment in writing will flush it out. And till they confirm, regard them as saying no.

Do breadth-first search weighted by expected value.

When you talk to investors your m.o. should be breadth-first search, weighted by expected value. You should always talk to investors in parallel rather than serially. You can't afford the time it takes to talk to investors serially, plus if you only talk to one investor at a time, they don't have the pressure of other investors to make them act. But you shouldn't pay the same attention to every investor, because some are more promising prospects than others. The optimal solution is to talk to all potential investors in parallel, but give higher priority to the more promising ones. [10]

Expected value = how likely an investor is to say yes, multiplied by how good it would be if they did. So for example, an eminent investor who would invest a lot, but will be hard to convince, might have the same expected value as an obscure angel who won't invest much, but will be easy to convince. Whereas an obscure angel who will only invest a small amount, and yet needs to meet multiple times before making up his mind, has very low expected value. Meet such investors last, if at all. [11]

Doing breadth-first search weighted by expected value will save you from investors who never explicitly say no but merely drift away, because you'll drift away from them at the same rate. It protects you from investors who flake in much the same way that a distributed algorithm protects you from processors that fail. If some investor isn't returning your emails, or wants to have lots of meetings but isn't progressing toward making you an offer, you automatically focus less on them. But you have to be disciplined about assigning probabilities. You can't let how much you want an investor influence your estimate of how much they want you.

Know where you stand.

How do you judge how well you're doing with an investor, when investors habitually seem more positive than they are? By looking at their actions rather than their words. Every investor has some track they need to move along from the first conversation to wiring the money, and you should always know what that track consists of, where you are on it, and how fast you're moving forward.

Never leave a meeting with an investor without asking what happens next. What more do they need in order to decide? Do they need another meeting with you? To talk about what? And how soon? Do they need to do something internally, like talk to their partners, or investigate some issue? How long do they expect it to take? Don't be too pushy, but know where you stand. If investors are vague

or resist answering such questions, assume the worst; investors who are seriously interested in you will usually be happy to talk about what has to happen between now and wiring the money, because they're already running through that in their heads. [12]

If you're experienced at negotiations, you already know how to ask such questions. [13] If you're not, there's a trick you can use in this situation. Investors know you're inexperienced at raising money. Inexperience there doesn't make you unattractive. Being a noob at technology would, if you're starting a technology startup, but not being a noob at fundraising. Larry and Sergey were noobs at fundraising. So you can just confess that you're inexperienced at this and ask how their process works and where you are in it. [14]

Get the first commitment.

The biggest factor in most investors' opinions of you is the opinion of other investors. Once you start getting investors to commit, it becomes increasingly easy to get more to. But the other side of this coin is that it's often hard to get the first commitment.

Getting the first substantial offer can be half the total difficulty of fundraising. What counts as a substantial offer depends on who it's from and how much it is. Money from friends and family doesn't usually count, no matter how much. But if you get \$50k from a well known VC firm or angel investor, that will usually be enough to set things rolling. [15]

Close committed money.

It's not a deal till the money's in the bank. I often hear inexperienced founders say things like "We've raised \$800,000," only to discover that zero of it is in the bank so far. Remember the twin fears that torment investors? The fear of missing out that makes them jump early, and the fear of jumping onto a turd that results? This is a market where people are exceptionally prone to buyer's remorse. And it's also one that furnishes them plenty of excuses to gratify it. The public markets snap startup investing around like a whip. If the Chinese economy blows up tomorrow, all bets are off. But there are lots of surprises for individual startups too, and they tend to be concentrated around fundraising. Tomorrow a big competitor could appear, or you could get C&Ded, or your cofounder could quit. [16]

Even a day's delay can bring news that causes an investor to change their mind. So when someone commits, get the money. Knowing where you stand doesn't end when they say they'll invest. After they say yes, know what the timetable is for getting the money, and then babysit that process till it happens. Institutional

investors have people in charge of wiring money, but you may have to hunt angels down in person to collect a check.

Inexperienced investors are the ones most likely to get buyer's remorse. Established ones have learned to treat saying yes as like diving off a diving board, and they also have more brand to preserve. But I've heard of cases of even top-tier VC firms welching on deals.

Avoid investors who don't "lead."

Since getting the first offer is most of the difficulty of fundraising, that should be part of your calculation of expected value when you start. You have to estimate not just the probability that an investor will say yes, but the probability that they'd be the *first* to say yes, and the latter is not simply a constant fraction of the former. Some investors are known for deciding quickly, and those are extra valuable early on.

Conversely, an investor who will only invest once other investors have is worthless initially. And while most investors are influenced by how interested other investors are in you, there are some who have an explicit policy of only investing after other investors have. You can recognize this contemptible subspecies of investor because they often talk about "leads." They say that they don't lead, or that they'll invest once you have a lead. Sometimes they even claim to be willing to lead themselves, by which they mean they won't invest till you get \$x from other investors. (It's great if by "lead" they mean they'll invest unilaterally, and in addition will help you raise more. What's lame is when they use the term to mean they won't invest unless you can raise more elsewhere.) [17]

Where does this term "lead" come from? Up till a few years ago, startups raising money in phase 2 would usually raise equity rounds in which several investors invested at the same time using the same paperwork. You'd negotiate the terms with one "lead" investor, and then all the others would sign the same documents and all the money change hands at the closing.

Series A rounds still work that way, but things now work differently for most fundraising prior to the series A. Now there are rarely actual rounds before the A round, or leads for them. Now startups simply raise money from investors one at a time till they feel they have enough.

Since there are no longer leads, why do investors use that term? Because it's a more legitimate-sounding way of saying what they really mean. All they really mean is that their interest in you is a function of other investors' interest in you. I.e. the spectral signature of all mediocre investors. But when phrased in terms of leads, it sounds like there is something structural and therefore legitimate about their behavior.

When an investor tells you “I want to invest in you, but I don’t lead,” translate that in your mind to “No, except yes if you turn out to be a hot deal.” And since that’s the default opinion of any investor about any startup, they’ve essentially just told you nothing.

When you first start fundraising, the expected value of an investor who won’t “lead” is zero, so talk to such investors last if at all.

Have multiple plans.

Many investors will ask how much you’re planning to raise. This question makes founders feel they should be planning to raise a specific amount. But in fact you shouldn’t. It’s a mistake to have fixed plans in an undertaking as unpredictable as fundraising.

So why do investors ask how much you plan to raise? For much the same reasons a salesperson in a store will ask “How much were you planning to spend?” if you walk in looking for a gift for a friend. You probably didn’t have a precise amount in mind; you just want to find something good, and if it’s inexpensive, so much the better. The salesperson asks you this not because you’re supposed to have a plan to spend a specific amount, but so they can show you only things that cost the most you’ll pay.

Similarly, when investors ask how much you plan to raise, it’s not because you’re supposed to have a plan. It’s to see whether you’d be a suitable recipient for the size of investment they like to make, and also to judge your ambition, reasonableness, and how far you are along with fundraising.

If you’re a wizard at fundraising, you can say “We plan to raise a \$7 million series A round, and we’ll be accepting termsheets next tuesday.” I’ve known a handful of founders who could pull that off without having VCs laugh in their faces. But if you’re in the inexperienced but earnest majority, the solution is analogous to the solution I recommend for pitching your startup: do the right thing and then just tell investors what you’re doing.

And the right strategy, in fundraising, is to have multiple plans depending on how much you can raise. Ideally you should be able to tell investors something like: we can make it to profitability without raising any more money, but if we raise a few hundred thousand we can hire one or two smart friends, and if we raise a couple million, we can hire a whole engineering team, etc.

Different plans match different investors. If you’re talking to a VC firm that only does series A rounds (though there are few of those left), it would be a waste of time talking about any but your most expensive plan. Whereas if you’re talking to an angel who invests \$20k at a time and you haven’t raised any money yet,

you probably want to focus on your least expensive plan.

If you're so fortunate as to have to think about the upper limit on what you should raise, a good rule of thumb is to multiply the number of people you want to hire times \$15k times 18 months. In most startups, nearly all the costs are a function of the number of people, and \$15k per month is the conventional total cost (including benefits and even office space) per person. \$15k per month is high, so don't actually spend that much. But it's ok to use a high estimate when fundraising to add a margin for error. If you have additional expenses, like manufacturing, add in those at the end. Assuming you have none and you think you might hire 20 people, the most you'd want to raise is $20 \times \$15k \times 18 = \5.4 million. [18]

Underestimate how much you want.

Though you can focus on different plans when talking to different types of investors, you should on the whole err on the side of underestimating the amount you hope to raise.

For example, if you'd like to raise \$500k, it's better to say initially that you're trying to raise \$250k. Then when you reach \$150k you're more than half done. That sends two useful signals to investors: that you're doing well, and that they have to decide quickly because you're running out of room. Whereas if you'd said you were raising \$500k, you'd be less than a third done at \$150k. If fundraising stalled there for an appreciable time, you'd start to read as a failure.

Saying initially that you're raising \$250k doesn't limit you to raising that much. When you reach your initial target and you still have investor interest, you can just decide to raise more. Startups do that all the time. In fact, most startups that are very successful at fundraising end up raising more than they originally intended.

I'm not saying you should lie, but that you should lower your expectations initially. There is almost no downside in starting with a low number. It not only won't cap the amount you raise, but will on the whole tend to increase it.

A good metaphor here is angle of attack. If you try to fly at too steep an angle of attack, you just stall. If you say right out of the gate that you want to raise a \$5 million series A round, unless you're in a very strong position, you not only won't get that but won't get anything. Better to start at a low angle of attack, build up speed, and then gradually increase the angle if you want.

Be profitable if you can.

You will be in a much stronger position if your collection of plans includes one for raising zero dollars — i.e. if you can make it to profitability without raising

any additional money. Ideally you want to be able to say to investors “We’ll succeed no matter what, but raising money will help us do it faster.”

There are many analogies between fundraising and dating, and this is one of the strongest. No one wants you if you seem desperate. And the best way not to seem desperate is not to *be* desperate. That’s one reason we urge startups during YC to keep expenses low and to try to make it to ramen profitability before Demo Day. Though it sounds slightly paradoxical, if you want to raise money, the best thing you can do is get yourself to the point where you don’t need to.

There are almost two distinct modes of fundraising: one in which founders who need money knock on doors seeking it, knowing that otherwise the company will die or at the very least people will have to be fired, and one in which founders who don’t need money take some to grow faster than they could merely on their own revenues. To emphasize the distinction I’m going to name them: type A fundraising is when you don’t need money, and type B fundraising is when you do.

Inexperienced founders read about famous startups doing what was type A fundraising, and decide they should raise money too, since that seems to be how startups work. Except when they raise money they don’t have a clear path to profitability and are thus doing type B fundraising. And they are then surprised how difficult and unpleasant it is.

Of course not all startups can make it to ramen profitability in a few months. And some that don’t still manage to have the upper hand over investors, if they have some other advantage like extraordinary growth numbers or exceptionally formidable founders. But as time passes it gets increasingly difficult to fundraise from a position of strength without being profitable. [19]

Don’t optimize for valuation.

When you raise money, what should your valuation be? The most important thing to understand about valuation is that it’s not that important.

Founders who raise money at high valuations tend to be unduly proud of it. Founders are often competitive people, and since valuation is usually the only visible number attached to a startup, they end up competing to raise money at the highest valuation. This is stupid, because fundraising is not the test that matters. The real test is revenue. Fundraising is just a means to that end. Being proud of how well you did at fundraising is like being proud of your college grades.

Not only is fundraising not the test that matters, valuation is not even the thing to optimize about fundraising. The number one thing you want from phase 2

fundraising is to get the money you need, so you can get back to focusing on the real test, the success of your company. Number two is good investors. Valuation is at best third.

The empirical evidence shows just how unimportant it is. Dropbox and Airbnb are the most successful companies we've funded so far, and they raised money after Y Combinator at premoney valuations of \$4 million and \$2.6 million respectively. Prices are so much higher now that if you can raise money at all you'll probably raise it at higher valuations than Dropbox and Airbnb. So let that satisfy your competitiveness. You're doing better than Dropbox and Airbnb! At a test that doesn't matter.

When you start fundraising, your initial valuation (or valuation cap) will be set by the deal you make with the first investor who commits. You can increase the price for later investors, if you get a lot of interest, but by default the valuation you got from the first investor becomes your asking price.

So if you're raising money from multiple investors, as most companies do in phase 2, you have to be careful to avoid raising the first from an over-eager investor at a price you won't be able to sustain. You can of course lower your price if you need to (in which case you should give the same terms to investors who invested earlier at a higher price), but you may lose a bunch of leads in the process of realizing you need to do this.

What you can do if you have eager first investors is raise money from them on an uncapped convertible note with an MFN clause. This is essentially a way of saying that the valuation cap of the note will be determined by the next investors you raise money from.

It will be easier to raise money at a lower valuation. It shouldn't be, but it is. Since phase 2 prices vary at most 10x and the big successes generate returns of at least 100x, investors should pick startups entirely based on their estimate of the probability that the company will be a big success and hardly at all on price. But although it's a mistake for investors to care about price, a significant number do. A startup that investors seem to like but won't invest in at a cap of \$x will have an easier time at \$x/2. [20]

Yes/no before valuation.

Some investors want to know what your valuation is before they even talk to you about investing. If your valuation has already been set by a prior investment at a specific valuation or cap, you can tell them that number. But if it isn't set because you haven't closed anyone yet, and they try to push you to name a price, resist doing so. If this would be the first investor you've closed, then this could be the tipping point of fundraising. That means closing this investor is the first

priority, and you need to get the conversation onto that instead of being dragged sideways into a discussion of price.

Fortunately there is a way to avoid naming a price in this situation. And it is not just a negotiating trick; it's how you (both) should be operating. Tell them that valuation is not the most important thing to you and that you haven't thought much about it, that you are looking for investors you want to partner with and who want to partner with you, and that you should talk first about whether they want to invest at all. Then if they decide they do want to invest, you can figure out a price. But first things first.

Since valuation isn't that important and getting fundraising rolling is, we usually tell founders to give the first investor who commits as low a price as they need to. This is a safe technique so long as you combine it with the next one. [21]

Beware “valuation sensitive” investors.

Occasionally you'll encounter investors who describe themselves as “valuation sensitive.” What this means in practice is that they are compulsive negotiators who will suck up a lot of your time trying to push your price down. You should therefore never approach such investors first. While you shouldn't chase high valuations, you also don't want your valuation to be set artificially low because the first investor who committed happened to be a compulsive negotiator. Some such investors have value, but the time to approach them is near the end of fundraising, when you're in a position to say “this is the price everyone else has paid; take it or leave it” and not mind if they leave it. This way, you'll not only get market price, but it will also take less time.

Ideally you know which investors have a reputation for being “valuation sensitive” and can postpone dealing with them till last, but occasionally one you didn't know about will pop up early on. The rule of doing breadth first search weighted by expected value already tells you what to do in this case: slow down your interactions with them.

There are a handful of investors who will try to invest at a lower valuation even when your price has already been set. Lowering your price is a backup plan you resort to when you discover you've let the price get set too high to close all the money you need. So you'd only want to talk to this sort of investor if you were about to do that anyway. But since investor meetings have to be arranged at least a few days in advance and you can't predict when you'll need to resort to lowering your price, this means in practice that you should approach this type of investor last if at all.

If you're surprised by a lowball offer, treat it as a backup offer and delay responding to it. When someone makes an offer in good faith, you have a moral

obligation to respond in a reasonable time. But lowballing you is a dick move that should be met with the corresponding countermove.

Accept offers greedily.

I'm a little leery of using the term “greedily” when writing about fundraising lest non-programmers misunderstand me, but a greedy algorithm is simply one that doesn't try to look into the future. A greedy algorithm takes the best of the options in front of it right now. And that is how startups should approach fundraising in phases 2 and later. Don't try to look into the future because (a) the future is unpredictable, and indeed in this business you're often being deliberately misled about it and (b) your first priority in fundraising should be to get it finished and get back to work anyway.

If someone makes you an acceptable offer, take it. If you have multiple incompatible offers, take the best. Don't reject an acceptable offer in the hope of getting a better one in the future.

These simple rules cover a wide variety of cases. If you're raising money from many investors, roll them up as they say yes. As you start to feel you've raised enough, the threshold for acceptable will start to get higher.

In practice offers exist for stretches of time, not points. So when you get an acceptable offer that would be incompatible with others (e.g. an offer to invest most of the money you need), you can tell the other investors you're talking to that you have an offer good enough to accept, and give them a few days to make their own. This could lose you some that might have made an offer if they had more time. But by definition you don't care; the initial offer was acceptable.

Some investors will try to prevent others from having time to decide by giving you an “exploding” offer, meaning one that's only valid for a few days. Offers from the very best investors explode less frequently and less rapidly — Fred Wilson never gives exploding offers, for example — because they're confident you'll pick them. But lower-tier investors sometimes give offers with very short fuses, because they believe no one who had other options would choose them. A deadline of three working days is acceptable. You shouldn't need more than that if you've been talking to investors in parallel. But a deadline any shorter is a sign you're dealing with a sketchy investor. You can usually call their bluff, and you may need to. [22]

It might seem that instead of accepting offers greedily, your goal should be to get the best investors as partners. That is certainly a good goal, but in phase 2 “get the best investors” only rarely conflicts with “accept offers greedily,” because the best investors don't usually take any longer to decide than the others. The only case where the two strategies give conflicting advice is when you have to forgo

an offer from an acceptable investor to see if you'll get an offer from a better one. If you talk to investors in parallel and push back on exploding offers with excessively short deadlines, that will almost never happen. But if it does, "get the best investors" is in the average case bad advice. The best investors are also the most selective, because they get their pick of all the startups. They reject nearly everyone they talk to, which means in the average case it's a bad trade to exchange a definite offer from an acceptable investor for a potential offer from a better one.

(The situation is different in phase 1. You can't apply to all the incubators in parallel, because some offset their schedules to prevent this. In phase 1, "accept offers greedily" and "get the best investors" do conflict, so if you want to apply to multiple incubators, you should do it in such a way that the ones you want most decide first.)

Sometimes when you're raising money from multiple investors, a series A will emerge out of those conversations, and these rules even cover what to do in that case. When an investor starts to talk to you about a series A, keep taking smaller investments till they actually give you a termsheet. There's no practical difficulty. If the smaller investments are on convertible notes, they'll just convert into the series A round. The series A investor won't like having all these other random investors as bedfellows, but if it bothers them so much they should get on with giving you a termsheet. Till they do, you don't know for sure they will, and the greedy algorithm tells you what to do. [23]

Don't sell more than 25% in phase 2.

If you do well, you will probably raise a series A round eventually. I say probably because things are changing with series A rounds. Startups may start to skip them. But only one company we've funded has so far, so tentatively assume the path to huge passes through an A round. [24]

Which means you should avoid doing things in earlier rounds that will mess up raising an A round. For example, if you've sold more than about 40% of your company total, it starts to get harder to raise an A round, because VCs worry there will not be enough stock left to keep the founders motivated.

Our rule of thumb is not to sell more than 25% in phase 2, on top of whatever you sold in phase 1, which should be less than 15%. If you're raising money on uncapped notes, you'll have to guess what the eventual equity round valuation might be. Guess conservatively.

(Since the goal of this rule is to avoid messing up the series A, there's obviously an exception if you end up raising a series A in phase 2, as a handful of startups do.)

Have one person handle fundraising.

If you have multiple founders, pick one to handle fundraising so the other(s) can keep working on the company. And since the danger of fundraising is not the time taken up by the actual meetings but that it becomes the top idea in your mind, the founder who handles fundraising should make a conscious effort to insulate the other founder(s) from the details of the process. [25]

(If the founders mistrust one another, this could cause some friction. But if the founders mistrust one another, you have worse problems to worry about than how to organize fundraising.)

The founder who handles fundraising should be the CEO, who should in turn be the most formidable of the founders. Even if the CEO is a programmer and another founder is a salesperson? Yes. If you happen to be that type of founding team, you're effectively a single founder when it comes to fundraising.

It's ok to bring all the founders to meet an investor who will invest a lot, and who needs this meeting as the final step before deciding. But wait till that point. Introducing an investor to your cofounder(s) should be like introducing a girl/boyfriend to your parents — something you do only when things reach a certain stage of seriousness.

Even if there are still one or more founders focusing on the company during fundraising, growth will slow. But try to get as much growth as you can, because fundraising is a segment of time, not a point, and what happens to the company during that time affects the outcome. If your numbers grow significantly between two investor meetings, investors will be hot to close, and if your numbers are flat or down they'll start to get cold feet.

You'll need an executive summary and (maybe) a deck.

Traditionally phase 2 fundraising consists of presenting a slide deck in person to investors. Sequoia describes what such a deck should contain, and since they're the customer you can take their word for it.

I say "traditionally" because I'm ambivalent about decks, and (though perhaps this is wishful thinking) they seem to be on the way out. A lot of the most successful startups we fund never make decks in phase 2. They just talk to investors and explain what they plan to do. Fundraising usually takes off fast for the startups that are most successful at it, and they're thus able to excuse themselves by saying that they haven't had time to make a deck.

You'll also want an executive summary, which should be no more than a page long and describe in the most matter of fact language what you plan to do, why it's a good idea, and what progress you've made so far. The point of the summary

is to remind the investor (who may have met many startups that day) what you talked about.

Assume that if you give someone a copy of your deck or executive summary, it will be passed on to whoever you'd least like to have it. But don't refuse on that account to give copies to investors you meet. You just have to treat such leaks as a cost of doing business. In practice it's not that high a cost. Though founders are rightly indignant when their plans get leaked to competitors, I can't think of a startup whose outcome has been affected by it.

Sometimes an investor will ask you to send them your deck and/or executive summary before they decide whether to meet with you. I wouldn't do that. It's a sign they're not really interested.

Stop fundraising when it stops working.

When do you stop fundraising? Ideally when you've raised enough. But what if you haven't raised as much as you'd like? When do you give up?

It's hard to give general advice about this, because there have been cases of startups that kept trying to raise money even when it seemed hopeless, and miraculously succeeded. But what I usually tell founders is to stop fundraising when you start to get a lot of air in the straw. When you're drinking through a straw, you can tell when you get to the end of the liquid because you start to get a lot of air in the straw. When your fundraising options run out, they usually run out in the same way. Don't keep sucking on the straw if you're just getting air. It's not going to get better.

Don't get addicted to fundraising.

Fundraising is a chore for most founders, but some find it more interesting than working on their startup. The work at an early stage startup often consists of unglamorous schleps. Whereas fundraising, when it's going well, can be quite the opposite. Instead of sitting in your grubby apartment listening to users complain about bugs in your software, you're being offered millions of dollars by famous investors over lunch at a nice restaurant. [26]

The danger of fundraising is particularly acute for people who are good at it. It's always fun to work on something you're good at. If you're one of these people, beware. Fundraising is not what will make your company successful. Listening to users complain about bugs in your software is what will make you successful. And the big danger of getting addicted to fundraising is not merely that you'll spend too long on it or raise too much money. It's that you'll start to think of yourself as being already successful, and lose your taste for the schleps you need to undertake to actually be successful. Startups can be destroyed by this.

When I see a startup with young founders that is fabulously successful at fundraising, I mentally decrease my estimate of the probability that they'll succeed. The press may be writing about them as if they'd been anointed as the next Google, but I'm thinking "this is going to end badly."

Don't raise too much.

Though only a handful of startups have to worry about this, it is possible to raise too much. The dangers of raising too much are subtle but insidious. One is that it will set impossibly high expectations. If you raise an excessive amount of money, it will be at a high valuation, and the danger of raising money at too high a valuation is that you won't be able to increase it sufficiently the next time you raise money.

A company's valuation is expected to rise each time it raises money. If not it's a sign of a company in trouble, which makes you unattractive to investors. So if you raise money in phase 2 at a post-money valuation of \$30 million, the pre-money valuation of your next round, if you want to raise one, is going to have to be at least \$50 million. And you have to be doing really, really well to raise money at \$50 million.

It's very dangerous to let the competitiveness of your current round set the performance threshold you have to meet to raise your next one, because the two are only loosely coupled.

But the money itself may be more dangerous than the valuation. The more you raise, the more you spend, and spending a lot of money can be disastrous for an early stage startup. Spending a lot makes it harder to become profitable, and perhaps even worse, it makes you more rigid, because the main way to spend money is people, and the more people you have, the harder it is to change directions. So if you do raise a huge amount of money, don't spend it. (You will find that advice almost impossible to follow, so hot will be the money burning a hole in your pocket, but I feel obliged at least to try.)

Be nice.

Startups raising money occasionally alienate investors by seeming arrogant. Sometimes because they are arrogant, and sometimes because they're noobs clumsily attempting to mimic the toughness they've observed in experienced founders.

It's a mistake to behave arrogantly to investors. While there are certain situations in which certain investors like certain kinds of arrogance, investors vary greatly in this respect, and a flick of the whip that will bring one to heel will make another roar with indignation. The only safe strategy is never to seem arrogant at all.

That will require some diplomacy if you follow the advice I've given here, because the advice I've given is essentially how to play hardball back. When you refuse to meet an investor because you're not in fundraising mode, or slow down your interactions with an investor who moves too slow, or treat a contingent offer as the no it actually is and then, by accepting offers greedily, end up leaving that investor out, you're going to be doing things investors don't like. So you must cushion the blow with soft words. At YC we tell startups they can blame us. And now that I've written this, everyone else can blame me if they want. That plus the inexperience card should work in most situations: sorry, we think you're great, but PG said startups shouldn't ____, and since we're new to fundraising, we feel like we have to play it safe.

The danger of behaving arrogantly is greatest when you're doing well. When everyone wants you, it's hard not to let it go to your head. Especially if till recently no one wanted you. But restrain yourself. The startup world is a small place, and startups have lots of ups and downs. This is a domain where it's more true than usual that pride goeth before a fall. [27]

Be nice when investors reject you as well. The best investors are not wedded to their initial opinion of you. If they reject you in phase 2 and you end up doing well, they'll often invest in phase 3. In fact investors who reject you are some of your warmest leads for future fundraising. Any investor who spent significant time deciding probably came close to saying yes. Often you have some internal champion who only needs a little more evidence to convince the skeptics. So it's wise not merely to be nice to investors who reject you, but (unless they behaved badly) to treat it as the beginning of a relationship.

The bar will be higher next time.

Assume the money you raise in phase 2 will be the last you ever raise. You must make it to profitability on this money if you can.

Over the past several years, the investment community has evolved from a strategy of anointing a small number of winners early and then supporting them for years to a strategy of spraying money at early stage startups and then ruthlessly culling them at the next stage. This is probably the optimal strategy for investors. It's too hard to pick winners early on. Better to let the market do it for you. But it often comes as a surprise to startups how much harder it is to raise money in phase 3.

When your company is only a couple months old, all it has to be is a promising experiment that's worth funding to see how it turns out. The next time you raise money, the experiment has to have worked. You have to be on a trajectory that leads to going public. And while there are some ideas where the proof that the

experiment worked might consist of e.g. query response times, usually the proof is profitability. Usually phase 3 fundraising has to be type A fundraising.

In practice there are two ways startups hose themselves between phases 2 and 3. Some are just too slow to become profitable. They raise enough money to last for two years. There doesn't seem any particular urgency to be profitable. So they don't make any effort to make money for a year. But by that time, not making money has become habitual. When they finally decide to try, they find they can't.

The other way companies hose themselves is by letting their expenses grow too fast. Which almost always means hiring too many people. You usually shouldn't go out and hire 8 people as soon as you raise money at phase 2. Usually you want to wait till you have growth (and thus usually revenues) to justify them. A lot of VCs will encourage you to hire aggressively. VCs generally tell you to spend too much, partly because as money people they err on the side of solving problems by spending money, and partly because they want you to sell them more of your company in subsequent rounds. Don't listen to them.

Don't make things complicated.

I realize it may seem odd to sum up this huge treatise by saying that my overall advice is not to make fundraising too complicated, but if you go back and look at this list you'll see it's basically a simple recipe with a lot of implications and edge cases. Avoid investors till you decide to raise money, and then when you do, talk to them all in parallel, prioritized by expected value, and accept offers greedily. That's fundraising in one sentence. Don't introduce complicated optimizations, and don't let investors introduce complications either.

Fundraising is not what will make you successful. It's just a means to an end. Your primary goal should be to get it over with and get back to what will make you successful — making things and talking to users — and the path I've described will for most startups be the surest way to that destination.

Be good, take care of yourselves, and *don't leave the path*.

Notes

[1] The worst explosions happen when unpromising-seeming startups encounter mediocre investors. Good investors don't lead startups on; their reputations are too valuable. And startups that seem promising can usually get enough money from good investors that they don't have to talk to mediocre ones. It is the unpromising-seeming startups that have to resort to raising money from mediocre investors. And it's particularly damaging when these investors flake, because unpromising-seeming startups are usually more desperate for money.

(Not all unpromising-seeming startups do badly. Some are merely ugly ducklings in the sense that they violate current startup fashions.)

[2] One YC founder told me:

I think in general we've done ok at fundraising, but I managed to screw up twice at the exact same thing — trying to focus on building the company and fundraising at the same time.

[3] There is one subtle danger you have to watch out for here, which I warn about later: beware of getting too high a valuation from an eager investor, lest that set an impossibly high target when raising additional money.

[4] If they really need a meeting, then they're not ready to invest, regardless of what they say. They're still deciding, which means you're being asked to come in and convince them. Which is fundraising.

[5] Associates at VC firms regularly cold email startups. Naive founders think "Wow, a VC is interested in us!" But an associate is not a VC. They have no decision-making power. And while they may introduce startups they like to partners at their firm, the partners discriminate against deals that come to them this way. I don't know of a single VC investment that began with an associate cold-emailing a startup. If you want to approach a specific firm, get an intro to a partner from someone they respect.

It's ok to talk to an associate if you get an intro to a VC firm or they see you at a Demo Day and they begin by having an associate vet you. That's not a promising lead and should therefore get low priority, but it's not as completely worthless as a cold email.

Because the title "associate" has gotten a bad reputation, a few VC firms have started to give their associates the title "partner," which can make things very confusing. If you're a YC startup you can ask us who's who; otherwise you may have to do some research online. There may be a special title for actual partners. If someone speaks for the firm in the press or a blog on the firm's site, they're probably a real partner. If they're on boards of directors they're probably a real partner.

There are titles between "associate" and "partner," including "principal" and "venture partner." The meanings of these titles vary too much to generalize.

[6] For similar reasons, avoid casual conversations with potential acquirers. They can lead to distractions even more dangerous than fundraising. Don't even take a meeting with a potential acquirer unless you want to sell your company right now.

[7] Joshua Reeves specifically suggests asking each investor to intro you to two more investors.

Don't ask investors who say no for introductions to other investors. That will in many cases be an anti-recommendation.

[8] This is not always as deliberate as it sounds. A lot of the delays and disconnects between founders and investors are induced by the customs of the venture business, which have evolved the way they have because they suit investors' interests.

[9] One YC founder who read a draft of this essay wrote:

This is the most important section. I think it might bear stating even more clearly. "Investors will deliberately affect more interest than they have to preserve optionality. If an investor seems very interested in you, they still probably won't invest. The solution for this is to assume the worst — that an investor is just feigning interest — until you get a definite commitment."

[10] Though you should probably pack investor meetings as closely as you can, Jeff Byun mentions one reason not to: if you pack investor meetings too closely, you'll have less time for your pitch to evolve.

Some founders deliberately schedule a handful of lame investors first, to get the bugs out of their pitch.

[11] There is not an efficient market in this respect. Some of the most useless investors are also the highest maintenance.

[12] Incidentally, this paragraph is sales 101. If you want to see it in action, go talk to a car dealer.

[13] I know one very smooth founder who used to end investor meetings with "So, can I count you in?" delivered as if it were "Can you pass the salt?" Unless you're very smooth (if you're not sure...), do not do this yourself. There is nothing more unconvincing, for an investor, than a nerdy founder trying to deliver the lines meant for a smooth one.

Investors are fine with funding nerds. So if you're a nerd, just try to be a good nerd, rather than doing a bad imitation of a smooth salesman.

[14] Ian Hogarth suggests a good way to tell how serious potential investors are: the resources they expend on you after the first meeting. An investor who's seriously interested will already be working to help you even before they've committed.

[15] In principle you might have to think about so-called “signalling risk.” If a prestigious VC makes a small seed investment in you, what if they don’t want to invest the next time you raise money? Other investors might assume that the VC knows you well, since they’re an existing investor, and if they don’t want to invest in your next round, that must mean you suck. The reason I say “in principle” is that in practice signalling hasn’t been much of a problem so far. It rarely arises, and in the few cases where it does, the startup in question usually is doing badly and is doomed anyway.

If you have the luxury of choosing among seed investors, you can play it safe by excluding VC firms. But it isn’t critical to.

[16] Sometimes a competitor will deliberately threaten you with a lawsuit just as you start fundraising, because they know you’ll have to disclose the threat to potential investors and they hope this will make it harder for you to raise money. If this happens it will probably frighten you more than investors. Experienced investors know about this trick, and know the actual lawsuits rarely happen. So if you’re attacked in this way, be forthright with investors. They’ll be more alarmed if you seem evasive than if you tell them everything.

[17] A related trick is to claim that they’ll only invest contingently on other investors doing so because otherwise you’d be “undercapitalized.” This is almost always bullshit. They can’t estimate your minimum capital needs that precisely.

[18] You won’t hire all those 20 people at once, and you’ll probably have some revenues before 18 months are out. But those too are acceptable or at least accepted additions to the margin for error.

[19] Type A fundraising is so much better that it might even be worth doing something different if it gets you there sooner. One YC founder told me that if he were a first-time founder again he’d “leave ideas that are up-front capital intensive to founders with established reputations.”

[20] I don’t know whether this happens because they’re innumerate, or because they believe they have zero ability to predict startup outcomes (in which case this behavior at least wouldn’t be irrational). In either case the implications are similar.

[21] If you’re a YC startup and you have an investor who for some reason insists that you decide the price, any YC partner can estimate a market price for you.

[22] You should respond in kind when investors behave upstandingly too. When an investor makes you a clean offer with no deadline, you have a moral obligation to respond promptly.

[23] Tell the investors talking to you about an A round about the smaller

investments you raise as you raise them. You owe them such updates on your cap table, and this is also a good way to pressure them to act. They won't like you raising other money and may pressure you to stop, but they can't legitimately ask you to commit to them till they also commit to you. If they want you to stop raising money, the way to do it is to give you a series A termsheet with a no-shop clause.

You can relent a little if the potential series A investor has a great reputation and they're clearly working fast to get you a termsheet, particularly if a third party like YC is involved to ensure there are no misunderstandings. But be careful.

[24] The company is Weebly, which made it to profitability on a seed investment of \$650k. They did try to raise a series A in the fall of 2008 but (no doubt partly because it was the fall of 2008) the terms they were offered were so bad that they decided to skip raising an A round.

[25] Another advantage of having one founder take fundraising meetings is that you never have to negotiate in real time, which is something inexperienced founders should avoid. One YC founder told me:

Investors are professional negotiators and can negotiate on the spot very easily. If only one founder is in the room, you can say "I need to circle back with my co-founder" before making any commitments. I used to do this all the time.

[26] You'll be lucky if fundraising feels pleasant enough to become addictive. More often you have to worry about the other extreme — becoming demoralized when investors reject you. As one (very successful) YC founder wrote after reading a draft of this:

It's hard to mentally deal with the sheer scale of rejection in fundraising and if you are not in the right mindset you will fail. Users may love you but these supposedly smart investors may not understand you at all. At this point for me, rejection still rankles but I've come to accept that investors are just not super thoughtful for the most part and you need to play the game according to certain somewhat depressing rules (many of which you are listing) in order to win.

[27] The actual sentence in the King James Bible is "Pride goeth before destruction, and an haughty spirit before a fall."

Thanks to Slava Akhmechet, Sam Altman, Nate Blecharczyk, Adora Cheung, Bill Clerico, John Collison, Patrick Collison, Parker Conrad, Ron Conway, Travis Deyle, Jason Freedman, Joe Gebbia, Mattan Griffel, Kevin Hale, Jacob Heller, Ian Hogarth, Justin Kan, Professor Moriarty, Nikhil Nirmel, David Petersen,

Geoff Ralston, Joshua Reeves, Yuri Sagalov, Emmett Shear, Rajat Suri, Garry Tan, and Nick Tomarelli for reading drafts of this.

Part Four — Work, Wealth, and Life

2014-2022

Life is short, as everyone knows.

— Paul Graham, “Life is Short”

Before the Startup

October 2014

(This essay is derived from a guest lecture in Sam Altman's startup class at Stanford. It's intended for college students, but much of it is applicable to potential founders at other ages.)

One of the advantages of having kids is that when you have to give advice, you can ask yourself “what would I tell my own kids?” My kids are little, but I can imagine what I'd tell them about startups if they were in college, and that's what I'm going to tell you.

Startups are very counterintuitive. I'm not sure why. Maybe it's just because knowledge about them hasn't permeated our culture yet. But whatever the reason, starting a startup is a task where you can't always trust your instincts.

It's like skiing in that way. When you first try skiing and you want to slow down, your instinct is to lean back. But if you lean back on skis you fly down the hill out of control. So part of learning to ski is learning to suppress that impulse. Eventually you get new habits, but at first it takes a conscious effort. At first there's a list of things you're trying to remember as you start down the hill.

Startups are as unnatural as skiing, so there's a similar list for startups. Here I'm going to give you the first part of it — the things to remember if you want to prepare yourself to start a startup.

Counterintuitive

The first item on it is the fact I already mentioned: that startups are so weird that if you trust your instincts, you'll make a lot of mistakes. If you know nothing more than this, you may at least pause before making them.

When I was running Y Combinator I used to joke that our function was to tell founders things they would ignore. It's really true. Batch after batch, the YC partners warn founders about mistakes they're about to make, and the founders

ignore them, and then come back a year later and say “I wish we’d listened.”

Why do the founders ignore the partners’ advice? Well, that’s the thing about counterintuitive ideas: they contradict your intuitions. They seem wrong. So of course your first impulse is to disregard them. And in fact my joking description is not merely the curse of Y Combinator but part of its *raison d’être*. If founders’ instincts already gave them the right answers, they wouldn’t need us. You only need other people to give you advice that surprises you. That’s why there are a lot of ski instructors and not many running instructors. [1]

You can, however, trust your instincts about people. And in fact one of the most common mistakes young founders make is not to do that enough. They get involved with people who seem impressive, but about whom they feel some misgivings personally. Later when things blow up they say “I knew there was something off about him, but I ignored it because he seemed so impressive.”

If you’re thinking about getting involved with someone — as a cofounder, an employee, an investor, or an acquirer — and you have misgivings about them, trust your gut. If someone seems slippery, or bogus, or a jerk, don’t ignore it.

This is one case where it pays to be self-indulgent. Work with people you genuinely like, and you’ve known long enough to be sure.

Expertise

The second counterintuitive point is that it’s not that important to know a lot about startups. The way to succeed in a startup is not to be an expert on startups, but to be an expert on your users and the problem you’re solving for them. Mark Zuckerberg didn’t succeed because he was an expert on startups. He succeeded despite being a complete noob at startups, because he understood his users really well.

If you don’t know anything about, say, how to raise an angel round, don’t feel bad on that account. That sort of thing you can learn when you need to, and forget after you’ve done it.

In fact, I worry it’s not merely unnecessary to learn in great detail about the mechanics of startups, but possibly somewhat dangerous. If I met an undergrad who knew all about convertible notes and employee agreements and (God forbid) class FF stock, I wouldn’t think “here is someone who is way ahead of their peers.” It would set off alarms. Because another of the characteristic mistakes of young founders is to go through the motions of starting a startup. They make up some plausible-sounding idea, raise money at a good valuation, rent a cool office, hire a bunch of people. From the outside that seems like what startups do. But the next step after rent a cool office and hire a bunch of people is: gradually

realize how completely fucked they are, because while imitating all the outward forms of a startup they have neglected the one thing that's actually essential: making something people want.

Game

We saw this happen so often that we made up a name for it: playing house. Eventually I realized why it was happening. The reason young founders go through the motions of starting a startup is because that's what they've been trained to do for their whole lives up to that point. Think about what you have to do to get into college, for example. Extracurricular activities, check. Even in college classes most of the work is as artificial as running laps.

I'm not attacking the educational system for being this way. There will always be a certain amount of fakeness in the work you do when you're being taught something, and if you measure their performance it's inevitable that people will exploit the difference to the point where much of what you're measuring is artifacts of the fakeness.

I confess I did it myself in college. I found that in a lot of classes there might only be 20 or 30 ideas that were the right shape to make good exam questions. The way I studied for exams in these classes was not (except incidentally) to master the material taught in the class, but to make a list of potential exam questions and work out the answers in advance. When I walked into the final, the main thing I'd be feeling was curiosity about which of my questions would turn up on the exam. It was like a game.

It's not surprising that after being trained for their whole lives to play such games, young founders' first impulse on starting a startup is to try to figure out the tricks for winning at this new game. Since fundraising appears to be the measure of success for startups (another classic noob mistake), they always want to know what the tricks are for convincing investors. We tell them the best way to convince investors is to make a startup that's actually doing well, meaning growing fast, and then simply tell investors so. Then they want to know what the tricks are for growing fast. And we have to tell them the best way to do that is simply to make something people want.

So many of the conversations YC partners have with young founders begin with the founder asking "How do we..." and the partner replying "Just..."

Why do the founders always make things so complicated? The reason, I realized, is that they're looking for the trick.

So this is the third counterintuitive thing to remember about startups: starting a startup is where gaming the system stops working. Gaming the system may

continue to work if you go to work for a big company. Depending on how broken the company is, you can succeed by sucking up to the right people, giving the impression of productivity, and so on. [2] But that doesn't work with startups. There is no boss to trick, only users, and all users care about is whether your product does what they want. Startups are as impersonal as physics. You have to make something people want, and you prosper only to the extent you do.

The dangerous thing is, faking does work to some degree on investors. If you're super good at sounding like you know what you're talking about, you can fool investors for at least one and perhaps even two rounds of funding. But it's not in your interest to. The company is ultimately doomed. All you're doing is wasting your own time riding it down.

So stop looking for the trick. There are tricks in startups, as there are in any domain, but they are an order of magnitude less important than solving the real problem. A founder who knows nothing about fundraising but has made something users love will have an easier time raising money than one who knows every trick in the book but has a flat usage graph. And more importantly, the founder who has made something users love is the one who will go on to succeed after raising the money.

Though in a sense it's bad news in that you're deprived of one of your most powerful weapons, I think it's exciting that gaming the system stops working when you start a startup. It's exciting that there even exist parts of the world where you win by doing good work. Imagine how depressing the world would be if it were all like school and big companies, where you either have to spend a lot of time on bullshit things or lose to people who do. [3] I would have been delighted if I'd realized in college that there were parts of the real world where gaming the system mattered less than others, and a few where it hardly mattered at all. But there are, and this variation is one of the most important things to consider when you're thinking about your future. How do you win in each type of work, and what would you like to win by doing? [4]

All-Consuming

That brings us to our fourth counterintuitive point: startups are all-consuming. If you start a startup, it will take over your life to a degree you cannot imagine. And if your startup succeeds, it will take over your life for a long time: for several years at the very least, maybe for a decade, maybe for the rest of your working life. So there is a real opportunity cost here.

Larry Page may seem to have an enviable life, but there are aspects of it that are unenviable. Basically at 25 he started running as fast as he could and it must seem to him that he hasn't stopped to catch his breath since. Every day new

shit happens in the Google empire that only the CEO can deal with, and he, as CEO, has to deal with it. If he goes on vacation for even a week, a whole week's backlog of shit accumulates. And he has to bear this uncomplainingly, partly because as the company's daddy he can never show fear or weakness, and partly because billionaires get less than zero sympathy if they talk about having difficult lives. Which has the strange side effect that the difficulty of being a successful startup founder is concealed from almost everyone except those who've done it.

Y Combinator has now funded several companies that can be called big successes, and in every single case the founders say the same thing. It never gets any easier. The nature of the problems change. You're worrying about construction delays at your London office instead of the broken air conditioner in your studio apartment. But the total volume of worry never decreases; if anything it increases.

Starting a successful startup is similar to having kids in that it's like a button you push that changes your life irrevocably. And while it's truly wonderful having kids, there are a lot of things that are easier to do before you have them than after. Many of which will make you a better parent when you do have kids. And since you can delay pushing the button for a while, most people in rich countries do.

Yet when it comes to startups, a lot of people seem to think they're supposed to start them while they're still in college. Are you crazy? And what are the universities thinking? They go out of their way to ensure their students are well supplied with contraceptives, and yet they're setting up entrepreneurship programs and startup incubators left and right.

To be fair, the universities have their hand forced here. A lot of incoming students are interested in startups. Universities are, at least de facto, expected to prepare them for their careers. So students who want to start startups hope universities can teach them about startups. And whether universities can do this or not, there's some pressure to claim they can, lest they lose applicants to other universities that do.

Can universities teach students about startups? Yes and no. They can teach students about startups, but as I explained before, this is not what you need to know. What you need to learn about are the needs of your own users, and you can't do that until you actually start the company. [5] So starting a startup is intrinsically something you can only really learn by doing it. And it's impossible to do that in college, for the reason I just explained: startups take over your life. You can't start a startup for real as a student, because if you start a startup for real you're not a student anymore. You may be nominally a student for a bit,

but you won't even be that for long. [6]

Given this dichotomy, which of the two paths should you take? Be a real student and not start a startup, or start a real startup and not be a student? I can answer that one for you. Do not start a startup in college. How to start a startup is just a subset of a bigger problem you're trying to solve: how to have a good life. And though starting a startup can be part of a good life for a lot of ambitious people, age 20 is not the optimal time to do it. Starting a startup is like a brutally fast depth-first search. Most people should still be searching breadth-first at 20.

You can do things in your early 20s that you can't do as well before or after, like plunge deeply into projects on a whim and travel super cheaply with no sense of a deadline. For unambitious people, this sort of thing is the dreaded "failure to launch," but for the ambitious ones it can be an incomparably valuable sort of exploration. If you start a startup at 20 and you're sufficiently successful, you'll never get to do it. [7]

Mark Zuckerberg will never get to bum around a foreign country. He can do other things most people can't, like charter jets to fly him to foreign countries. But success has taken a lot of the serendipity out of his life. Facebook is running him as much as he's running Facebook. And while it can be very cool to be in the grip of a project you consider your life's work, there are advantages to serendipity too, especially early in life. Among other things it gives you more options to choose your life's work from.

There's not even a tradeoff here. You're not sacrificing anything if you forgo starting a startup at 20, because you're more likely to succeed if you wait. In the unlikely case that you're 20 and one of your side projects takes off like Facebook did, you'll face a choice of running with it or not, and it may be reasonable to run with it. But the usual way startups take off is for the founders to make them take off, and it's gratuitously stupid to do that at 20.

Try

Should you do it at any age? I realize I've made startups sound pretty hard. If I haven't, let me try again: starting a startup is really hard. What if it's too hard? How can you tell if you're up to this challenge?

The answer is the fifth counterintuitive point: you can't tell. Your life so far may have given you some idea what your prospects might be if you tried to become a mathematician, or a professional football player. But unless you've had a very strange life you haven't done much that was like being a startup founder. Starting a startup will change you a lot. So what you're trying to estimate is not just what you are, but what you could grow into, and who can do that?

For the past 9 years it was my job to predict whether people would have what it took to start successful startups. It was easy to tell how smart they were, and most people reading this will be over that threshold. The hard part was predicting how tough and ambitious they would become. There may be no one who has more experience at trying to predict that, so I can tell you how much an expert can know about it, and the answer is: not much. I learned to keep a completely open mind about which of the startups in each batch would turn out to be the stars.

The founders sometimes think they know. Some arrive feeling sure they will ace Y Combinator just as they've aced every one of the (few, artificial, easy) tests they've faced in life so far. Others arrive wondering how they got in, and hoping YC doesn't discover whatever mistake caused it to accept them. But there is little correlation between founders' initial attitudes and how well their companies do.

I've read that the same is true in the military — that the swaggering recruits are no more likely to turn out to be really tough than the quiet ones. And probably for the same reason: that the tests involved are so different from the ones in their previous lives.

If you're absolutely terrified of starting a startup, you probably shouldn't do it. But if you're merely unsure whether you're up to it, the only way to find out is to try. Just not now.

Ideas

So if you want to start a startup one day, what should you do in college? There are only two things you need initially: an idea and cofounders. And the m.o. for getting both is the same. Which leads to our sixth and last counterintuitive point: that the way to get startup ideas is not to try to think of startup ideas.

I've written a whole essay on this, so I won't repeat it all here. But the short version is that if you make a conscious effort to think of startup ideas, the ideas you come up with will not merely be bad, but bad and plausible-sounding, meaning you'll waste a lot of time on them before realizing they're bad.

The way to come up with good startup ideas is to take a step back. Instead of making a conscious effort to think of startup ideas, turn your mind into the type that startup ideas form in without any conscious effort. In fact, so unconsciously that you don't even realize at first that they're startup ideas.

This is not only possible, it's how Apple, Yahoo, Google, and Facebook all got started. None of these companies were even meant to be companies at first. They were all just side projects. The best startups almost have to start as side

projects, because great ideas tend to be such outliers that your conscious mind would reject them as ideas for companies.

Ok, so how do you turn your mind into the type that startup ideas form in unconsciously? (1) Learn a lot about things that matter, then (2) work on problems that interest you (3) with people you like and respect. The third part, incidentally, is how you get cofounders at the same time as the idea.

The first time I wrote that paragraph, instead of “learn a lot about things that matter,” I wrote “become good at some technology.” But that prescription, though sufficient, is too narrow. What was special about Brian Chesky and Joe Gebbia was not that they were experts in technology. They were good at design, and perhaps even more importantly, they were good at organizing groups and making projects happen. So you don’t have to work on technology per se, so long as you work on problems demanding enough to stretch you.

What kind of problems are those? That is very hard to answer in the general case. History is full of examples of young people who were working on important problems that no one else at the time thought were important, and in particular that their parents didn’t think were important. On the other hand, history is even fuller of examples of parents who thought their kids were wasting their time and who were right. So how do you know when you’re working on real stuff? [8]

I know how *I* know. Real problems are interesting, and I am self-indulgent in the sense that I always want to work on interesting things, even if no one else cares about them (in fact, especially if no one else cares about them), and find it very hard to make myself work on boring things, even if they’re supposed to be important.

My life is full of case after case where I worked on something just because it seemed interesting, and it turned out later to be useful in some worldly way. Y Combinator itself was something I only did because it seemed interesting. So I seem to have some sort of internal compass that helps me out. But I don’t know what other people have in their heads. Maybe if I think more about this I can come up with heuristics for recognizing genuinely interesting problems, but for the moment the best I can offer is the hopelessly question-begging advice that if you have a taste for genuinely interesting problems, indulging it energetically is the best way to prepare yourself for a startup. And indeed, probably also the best way to live. [9]

But although I can’t explain in the general case what counts as an interesting problem, I can tell you about a large subset of them. If you think of technology as something that’s spreading like a sort of fractal stain, every moving point on the edge represents an interesting problem. So one guaranteed way to turn

your mind into the type that has good startup ideas is to get yourself to the leading edge of some technology — to cause yourself, as Paul Buchheit put it, to “live in the future.” When you reach that point, ideas that will seem to other people uncannily prescient will seem obvious to you. You may not realize they’re startup ideas, but you’ll know they’re something that ought to exist.

For example, back at Harvard in the mid 90s a fellow grad student of my friends Robert and Trevor wrote his own voice over IP software. He didn’t mean it to be a startup, and he never tried to turn it into one. He just wanted to talk to his girlfriend in Taiwan without paying for long distance calls, and since he was an expert on networks it seemed obvious to him that the way to do it was turn the sound into packets and ship it over the Internet. He never did any more with his software than talk to his girlfriend, but this is exactly the way the best startups get started.

So strangely enough the optimal thing to do in college if you want to be a successful startup founder is not some sort of new, vocational version of college focused on “entrepreneurship.” It’s the classic version of college as education for its own sake. If you want to start a startup after college, what you should do in college is learn powerful things. And if you have genuine intellectual curiosity, that’s what you’ll naturally tend to do if you just follow your own inclinations. [10]

The component of entrepreneurship that really matters is domain expertise. The way to become Larry Page was to become an expert on search. And the way to become an expert on search was to be driven by genuine curiosity, not some ulterior motive.

At its best, starting a startup is merely an ulterior motive for curiosity. And you’ll do it best if you introduce the ulterior motive toward the end of the process.

So here is the ultimate advice for young would-be startup founders, boiled down to two words: just learn.

Notes

[1] Some founders listen more than others, and this tends to be a predictor of success. One of the things I remember about the Airbnbs during YC is how intently they listened.

[2] In fact, this is one of the reasons startups are possible. If big companies weren’t plagued by internal inefficiencies, they’d be proportionately more effective, leaving less room for startups.

[3] In a startup you have to spend a lot of time on schleps, but this sort of work is merely unglamorous, not bogus.

[4] What should you do if your true calling is gaming the system? Management consulting.

[5] The company may not be incorporated, but if you start to get significant numbers of users, you've started it, whether you realize it yet or not.

[6] It shouldn't be that surprising that colleges can't teach students how to be good startup founders, because they can't teach them how to be good employees either.

The way universities "teach" students how to be employees is to hand off the task to companies via internship programs. But you couldn't do the equivalent thing for startups, because by definition if the students did well they would never come back.

[7] Charles Darwin was 22 when he received an invitation to travel aboard the HMS Beagle as a naturalist. It was only because he was otherwise unoccupied, to a degree that alarmed his family, that he could accept it. And yet if he hadn't we probably would not know his name.

[8] Parents can sometimes be especially conservative in this department. There are some whose definition of important problems includes only those on the critical path to med school.

[9] I did manage to think of a heuristic for detecting whether you have a taste for interesting ideas: whether you find known boring ideas intolerable. Could you endure studying literary theory, or working in middle management at a large company?

[10] In fact, if your goal is to start a startup, you can stick even more closely to the ideal of a liberal education than past generations have. Back when students focused mainly on getting a job after college, they thought at least a little about how the courses they took might look to an employer. And perhaps even worse, they might shy away from taking a difficult class lest they get a low grade, which would harm their all-important GPA. Good news: users don't care what your GPA was. And I've never heard of investors caring either. Y Combinator certainly never asks what classes you took in college or what grades you got in them.

Thanks to Sam Altman, Paul Buchheit, John Collison, Patrick Collison, Jessica Livingston, Robert Morris, Geoff Ralston, and Fred Wilson for reading drafts of this.

Default Alive or Default Dead?

October 2015

When I talk to a startup that's been operating for more than 8 or 9 months, the first thing I want to know is almost always the same. Assuming their expenses remain constant and their revenue growth is what it has been over the last several months, do they make it to profitability on the money they have left? Or to put it more dramatically, by default do they live or die?

The startling thing is how often the founders themselves don't know. Half the founders I talk to don't know whether they're default alive or default dead.

If you're among that number, Trevor Blackwell has made a handy *calculator* you can use to find out.

The reason I want to know first whether a startup is default alive or default dead is that the rest of the conversation depends on the answer. If the company is default alive, we can talk about ambitious new things they could do. If it's default dead, we probably need to talk about how to save it. We know the current trajectory ends badly. How can they get off that trajectory?

Why do so few founders know whether they're default alive or default dead? Mainly, I think, because they're not used to asking that. It's not a question that makes sense to ask early on, any more than it makes sense to ask a 3 year old how he plans to support himself. But as the company grows older, the question switches from meaningless to critical. That kind of switch often takes people by surprise.

I propose the following solution: instead of starting to ask too late whether you're default alive or default dead, start asking too early. It's hard to say precisely when the question switches polarity. But it's probably not that dangerous to

start worrying too early that you're default dead, whereas it's very dangerous to start worrying too late.

The reason is a phenomenon I wrote about earlier: the fatal pinch. The fatal pinch is default dead + slow growth + not enough time to fix it. And the way founders end up in it is by not realizing that's where they're headed.

There is another reason founders don't ask themselves whether they're default alive or default dead: they assume it will be easy to raise more money. But that assumption is often false, and worse still, the more you depend on it, the falser it becomes.

Maybe it will help to separate facts from hopes. Instead of thinking of the future with vague optimism, explicitly separate the components. Say "We're default dead, but we're counting on investors to save us." Maybe as you say that, it will set off the same alarms in your head that it does in mine. And if you set off the alarms sufficiently early, you may be able to avoid the fatal pinch.

It would be safe to be default dead if you could count on investors saving you. As a rule their interest is a function of growth. If you have steep revenue growth, say over 5x a year, you can start to count on investors being interested even if you're not profitable. [1] But investors are so fickle that you can never do more than start to count on them. Sometimes something about your business will spook investors even if your growth is great. So no matter how good your growth is, you can never safely treat fundraising as more than a plan A. You should always have a plan B as well: you should know (as in write down) precisely what you'll need to do to survive if you can't raise more money, and precisely when you'll have to switch to plan B if plan A isn't working.

In any case, growing fast versus operating cheaply is far from the sharp dichotomy many founders assume it to be. In practice there is surprisingly little connection between how much a startup spends and how fast it grows. When a startup grows fast, it's usually because the product hits a nerve, in the sense of hitting some big need straight on. When a startup spends a lot, it's usually because the product is expensive to develop or sell, or simply because they're wasteful.

If you're paying attention, you'll be asking at this point not just how to avoid the fatal pinch, but how to avoid being default dead. That one is easy: don't hire too fast. Hiring too fast is by far the biggest killer of startups that raise money. [2]

Founders tell themselves they need to hire in order to grow. But most err on the side of overestimating this need rather than underestimating it. Why? Partly because there's so much work to do. Naive founders think that if they can just hire enough people, it will all get done. Partly because successful startups have

lots of employees, so it seems like that's what one does in order to be successful. In fact the large staffs of successful startups are probably more the effect of growth than the cause. And partly because when founders have slow growth they don't want to face what is usually the real reason: the product is not appealing enough.

Plus founders who've just raised money are often encouraged to overhire by the VCs who funded them. Kill-or-cure strategies are optimal for VCs because they're protected by the portfolio effect. VCs want to blow you up, in one sense of the phrase or the other. But as a founder your incentives are different. You want above all to survive. [3]

Here's a common way startups die. They make something moderately appealing and have decent initial growth. They raise their first round fairly easily, because the founders seem smart and the idea sounds plausible. But because the product is only moderately appealing, growth is ok but not great. The founders convince themselves that hiring a bunch of people is the way to boost growth. Their investors agree. But (because the product is only moderately appealing) the growth never comes. Now they're rapidly running out of runway. They hope further investment will save them. But because they have high expenses and slow growth, they're now unappealing to investors. They're unable to raise more, and the company dies.

What the company should have done is address the fundamental problem: that the product is only moderately appealing. Hiring people is rarely the way to fix that. More often than not it makes it harder. At this early stage, the product needs to evolve more than to be "built out," and that's usually easier with fewer people. [4]

Asking whether you're default alive or default dead may save you from this. Maybe the alarm bells it sets off will counteract the forces that push you to overhire. Instead you'll be compelled to seek growth in other ways. For example, by *doing things that don't scale*, or by redesigning the product in the way only founders can. And for many if not most startups, these paths to growth will be the ones that actually work.

Airbnb waited 4 months after raising money at the end of Y Combinator before they hired their first employee. In the meantime the founders were terribly overworked. But they were overworked evolving Airbnb into the astonishingly successful organism it is now.

Notes

[1] Steep usage growth will also interest investors. Revenue will ultimately be a constant multiple of usage, so $x\%$ usage growth predicts $x\%$ revenue growth. But

in practice investors discount merely predicted revenue, so if you're measuring usage you need a higher growth rate to impress investors.

[2] Startups that don't raise money are saved from hiring too fast because they can't afford to. But that doesn't mean you should avoid raising money in order to avoid this problem, any more than that total abstinence is the only way to avoid becoming an alcoholic.

[3] I would not be surprised if VCs' tendency to push founders to overhire is not even in their own interest. They don't know how many of the companies that get killed by overspending might have done well if they'd survived. My guess is a significant number.

[4] After reading a draft, Sam Altman wrote:

"I think you should make the hiring point more strongly. I think it's roughly correct to say that YC's most successful companies have never been the fastest to hire, and one of the marks of a great founder is being able to resist this urge."

Paul Buchheit adds:

"A related problem that I see a lot is premature scaling—founders take a small business that isn't really working (bad unit economics, typically) and then scale it up because they want impressive growth numbers. This is similar to over-hiring in that it makes the business much harder to fix once it's big, plus they are bleeding cash really fast."

Thanks to Sam Altman, Paul Buchheit, Joe Gebbia, Jessica Livingston, and Geoff Ralston for reading drafts of this.

Economic Inequality

January 2016

Since the 1970s, economic inequality in the US has increased dramatically. And in particular, the rich have gotten a lot richer. Nearly everyone who writes about the topic says that economic inequality should be decreased.

I'm interested in this question because I was one of the founders of a company called Y Combinator that helps people start startups. Almost by definition, if a startup succeeds, its founders become rich. Which means by helping startup founders I've been helping to increase economic inequality. If economic inequality should be decreased, I shouldn't be helping founders. No one should be.

But that doesn't sound right. What's going on here? What's going on is that while economic inequality is a single measure (or more precisely, two: variation in income, and variation in wealth), it has multiple causes. Many of these causes are bad, like tax loopholes and drug addiction. But some are good, like Larry Page and Sergey Brin starting the company you use to find things online.

If you want to understand economic inequality — and more importantly, if you actually want to fix the bad aspects of it — you have to tease apart the components. And yet the trend in nearly everything written about the subject is to do the opposite: to squash together all the aspects of economic inequality as if it were a single phenomenon.

Sometimes this is done for ideological reasons. Sometimes it's because the writer only has very high-level data and so draws conclusions from that, like the proverbial drunk who looks for his keys under the lamppost, instead of where he dropped them, because the light is better there. Sometimes it's because the writer doesn't understand critical aspects of inequality, like the role of technology in wealth creation. Much of the time, perhaps most of the time, writing about economic inequality combines all three.

The most common mistake people make about economic inequality is to treat it as a single phenomenon. The most naive version of which is the one based on the pie fallacy: that the rich get rich by taking money from the poor.

Usually this is an assumption people start from rather than a conclusion they arrive at by examining the evidence. Sometimes the pie fallacy is stated explicitly:

... those at the top are grabbing an increasing fraction of the nation's income — so much of a larger share that what's left over for the rest is diminished. ... [1]

Other times it's more unconscious. But the unconscious form is very widespread. I think because we grow up in a world where the pie fallacy is actually true. To kids, wealth *is* a fixed pie that's shared out, and if one person gets more, it's at the expense of another. It takes a conscious effort to remind oneself that the real world doesn't work that way.

In the real world you can create wealth as well as taking it from others. A woodworker creates wealth. He makes a chair, and you willingly give him money in return for it. A high-frequency trader does not. He makes a dollar only when someone on the other end of a trade loses a dollar.

If the rich people in a society got that way by taking wealth from the poor, then you have the degenerate case of economic inequality, where the cause of poverty is the same as the cause of wealth. But instances of inequality don't have to be instances of the degenerate case. If one woodworker makes 5 chairs and another makes none, the second woodworker will have less money, but not because anyone took anything from him.

Even people sophisticated enough to know about the pie fallacy are led toward it by the custom of describing economic inequality as a ratio of one quantile's income or wealth to another's. It's so easy to slip from talking about income shifting from one quantile to another, as a figure of speech, into believing that is literally what's happening.

Except in the degenerate case, economic inequality can't be described by a ratio or even a curve. In the general case it consists of multiple ways people become poor, and multiple ways people become rich. Which means to understand economic inequality in a country, you have to go find individual people who are poor or rich and figure out why. [2]

If you want to understand *change* in economic inequality, you should ask what those people would have done when it was different. This is one way I know the rich aren't all getting richer simply from some new system for transferring wealth to them from everyone else. When you use the would-have method with startup

founders, you find what most would have done *back in 1960*, when economic inequality was lower, was to join big companies or become professors. Before Mark Zuckerberg started Facebook, his default expectation was that he'd end up working at Microsoft. The reason he and most other startup founders are richer than they would have been in the mid 20th century is not because of some right turn the country took during the Reagan administration, but because progress in technology has made it much easier to start a new company that *grows fast*.

Traditional economists seem strangely averse to studying individual humans. It seems to be a rule with them that everything has to start with statistics. So they give you very precise numbers about variation in wealth and income, then follow it with the most naive speculation about the underlying causes.

But while there are a lot of people who get rich through rent-seeking of various forms, and a lot who get rich by playing zero-sum games, there are also a significant number who get rich by creating wealth. And creating wealth, as a source of economic inequality, is different from taking it — not just morally, but also practically, in the sense that it is harder to eradicate. One reason is that variation in productivity is accelerating. The rate at which individuals can create wealth depends on the technology available to them, and that grows exponentially. The other reason creating wealth is such a tenacious source of inequality is that it can expand to accommodate a lot of people.

I'm all for shutting down the crooked ways to get rich. But that won't eliminate great variations in wealth, because as long as you leave open the option of getting rich by creating wealth, people who want to get rich will do that instead.

Most people who get rich tend to be fairly driven. Whatever their other flaws, laziness is usually not one of them. Suppose new policies make it hard to make a fortune in finance. Does it seem plausible that the people who currently go into finance to make their fortunes will continue to do so, but be content to work for ordinary salaries? The reason they go into finance is not because they love finance but because they want to get rich. If the only way left to get rich is to start startups, they'll start startups. They'll do well at it too, because determination is the main factor in the success of a startup. [3] And while it would probably be a good thing for the world if people who wanted to get rich switched from playing zero-sum games to creating wealth, that would not only not eliminate great variations in wealth, but might even exacerbate them. In a zero-sum game there is at least a limit to the upside. Plus a lot of the new startups would create new technology that further accelerated variation in productivity.

Variation in productivity is far from the only source of economic inequality, but

it is the irreducible core of it, in the sense that you'll have that left when you eliminate all other sources. And if you do, that core will be big, because it will have expanded to include the efforts of all the refugees. Plus it will have a large Baumol penumbra around it: anyone who could get rich by creating wealth on their own account will have to be paid enough to prevent them from doing it.

You can't prevent great variations in wealth without preventing people from getting rich, and you can't do that without preventing them from starting startups.

So let's be clear about that. Eliminating great variations in wealth would mean eliminating startups. And that doesn't seem a wise move. Especially since it would only mean you eliminated startups in your own country. Ambitious people already move halfway around the world to further their careers, and startups can operate from anywhere nowadays. So if you made it impossible to get rich by creating wealth in your country, people who wanted to do that would just leave and do it somewhere else. Which would certainly get you a lower Gini coefficient, along with a lesson in being careful what you ask for. [4]

I think rising economic inequality is the inevitable fate of countries that don't choose something worse. We had a 40 year stretch in the middle of the 20th century that convinced some people otherwise. But as I explained in *The Refragmentation*, that was an anomaly — a unique combination of circumstances that compressed American society not just economically but culturally too. [5]

And while some of the growth in economic inequality we've seen since then has been due to bad behavior of various kinds, there has simultaneously been a huge increase in individuals' ability to create wealth. Startups are almost entirely a product of this period. And even within the startup world, there has been a qualitative change in the last 10 years. Technology has decreased the cost of starting a startup so much that founders now have the upper hand over investors. Founders get less diluted, and it is now common for them to retain *board control* as well. Both further increase economic inequality, the former because founders own more stock, and the latter because, as investors have learned, founders tend to be better at running their companies than investors.

While the surface manifestations change, the underlying forces are very, very old. The acceleration of productivity we see in Silicon Valley has been happening for thousands of years. If you look at the history of stone tools, technology was already accelerating in the Mesolithic. The acceleration would have been too slow to perceive in one lifetime. Such is the nature of the leftmost part of an exponential curve. But it was the same curve.

You do not want to design your society in a way that's incompatible with this

curve. The evolution of technology is one of the most powerful forces in history.

Louis Brandeis said “We may have democracy, or we may have wealth concentrated in the hands of a few, but we can’t have both.” That sounds plausible. But if I have to choose between ignoring him and ignoring an exponential curve that has been operating for thousands of years, I’ll bet on the curve. Ignoring any trend that has been operating for thousands of years is dangerous. But exponential growth, especially, tends to bite you.

If accelerating variation in productivity is always going to produce some baseline growth in economic inequality, it would be a good idea to spend some time thinking about that future. Can you have a healthy society with great variation in wealth? What would it look like?

Notice how novel it feels to think about that. The public conversation so far has been exclusively about the need to decrease economic inequality. We’ve barely given a thought to how to live with it.

I’m hopeful we’ll be able to. Brandeis was a product of the Gilded Age, and things have changed since then. It’s harder to hide wrongdoing now. And to get rich now you don’t have to buy politicians the way railroad or oil magnates did. [6] The great concentrations of wealth I see around me in Silicon Valley don’t seem to be destroying democracy.

There are lots of things wrong with the US that have economic inequality as a symptom. We should fix those things. In the process we may decrease economic inequality. But we can’t start from the symptom and hope to fix the underlying causes. [7]

The most obvious is poverty. I’m sure most of those who want to decrease economic inequality want to do it mainly to help the poor, not to hurt the rich. [8] Indeed, a good number are merely being sloppy by speaking of decreasing economic inequality when what they mean is decreasing poverty. But this is a situation where it would be good to be precise about what we want. Poverty and economic inequality are not identical. When the city is turning off your *water* because you can’t pay the bill, it doesn’t make any difference what Larry Page’s net worth is compared to yours. He might only be a few times richer than you, and it would still be just as much of a problem that your water was getting turned off.

Closely related to poverty is lack of social mobility. I’ve seen this myself: you don’t have to grow up rich or even upper middle class to get rich as a startup founder, but few successful founders grew up desperately poor. But again, the

problem here is not simply economic inequality. There is an enormous difference in wealth between the household Larry Page grew up in and that of a successful startup founder, but that didn't prevent him from joining their ranks. It's not economic inequality per se that's blocking social mobility, but some specific combination of things that go wrong when kids grow up sufficiently poor.

One of the most important principles in Silicon Valley is that "you make what you measure." It means that if you pick some number to focus on, it will tend to improve, but that you have to choose the right number, because only the one you choose will improve; another that seems conceptually adjacent might not. For example, if you're a university president and you decide to focus on graduation rates, then you'll improve graduation rates. But only graduation rates, not how much students learn. Students could learn less, if to improve graduation rates you made classes easier.

Economic inequality is sufficiently far from identical with the various problems that have it as a symptom that we'll probably only hit whichever of the two we aim at. If we aim at economic inequality, we won't fix these problems. So I say let's aim at the problems.

For example, let's attack poverty, and if necessary damage wealth in the process. That's much more likely to work than attacking wealth in the hope that you will thereby fix poverty. [9] And if there are people getting rich by tricking consumers or lobbying the government for anti-competitive regulations or tax loopholes, then let's stop them. Not because it's causing economic inequality, but because it's stealing. [10]

If all you have is statistics, it seems like that's what you need to fix. But behind a broad statistical measure like economic inequality there are some things that are good and some that are bad, some that are historical trends with immense momentum and others that are random accidents. If we want to fix the world behind the statistics, we have to understand it, and focus our efforts where they'll do the most good.

Notes

[1] Stiglitz, Joseph. *The Price of Inequality*. Norton, 2012. p. 32.

[2] Particularly since economic inequality is a matter of outliers, and outliers are disproportionately likely to have gotten where they are by ways that have little to do with the sort of things economists usually think about, like wages and productivity, but rather by, say, ending up on the wrong side of the "War on Drugs."

[3] Determination is the most important factor in deciding between success and

failure, which in startups tend to be sharply differentiated. But it takes more than determination to create one of the hugely successful startups. Though most founders start out excited about the idea of getting rich, purely mercenary founders will usually take one of the big acquisition offers most successful startups get on the way up. The founders who go on to the next stage tend to be driven by a sense of mission. They have the same attachment to their companies that an artist or writer has to their work. But it is very hard to predict at the outset which founders will do that. It's not simply a function of their initial attitude. Starting a company changes people.

[4] After reading a draft of this essay, Richard Florida told me how he had once talked to a group of Europeans “who said they wanted to make Europe more entrepreneurial and more like Silicon Valley. I said by definition this will give you more inequality. They thought I was insane — they could not process it.”

[5] Economic inequality has been decreasing globally. But this is mainly due to the erosion of the kleptocracies that formerly dominated all the poorer countries. Once the playing field is leveler politically, we'll see economic inequality start to rise again. The US is the bellwether. The situation we face here, the rest of the world will sooner or later.

[6] Some people still get rich by buying politicians. My point is that it's no longer a precondition.

[7] As well as problems that have economic inequality as a symptom, there are those that have it as a cause. But in most if not all, economic inequality is not the primary cause. There is usually some injustice that is allowing economic inequality to turn into other forms of inequality, and that injustice is what we need to fix. For example, the police in the US treat the poor worse than the rich. But the solution is not to make people richer. It's to make the police treat people more equitably. Otherwise they'll continue to maltreat people who are weak in other ways.

[8] Some who read this essay will say that I'm clueless or even being deliberately misleading by focusing so much on the richer end of economic inequality — that economic inequality is really about poverty. But that is exactly the point I'm making, though sloppier language than I'd use to make it. The real problem is poverty, not economic inequality. And if you conflate them you're aiming at the wrong target.

Others will say I'm clueless or being misleading by focusing on people who get rich by creating wealth — that startups aren't the problem, but corrupt practices in finance, healthcare, and so on. Once again, that is exactly my point. The problem is not economic inequality, but those specific abuses.

It's a strange task to write an essay about why something isn't the problem, but that's the situation you find yourself in when so many people mistakenly think it is.

[9] Particularly since many causes of poverty are only partially driven by people trying to make money from them. For example, America's abnormally high incarceration rate is a major cause of poverty. But although *for-profit prison companies* and *prison guard unions* both spend a lot lobbying for harsh sentencing laws, they are not the original source of them.

[10] Incidentally, tax loopholes are definitely not a product of some power shift due to recent increases in economic inequality. The golden age of economic equality in the mid 20th century was also the golden age of tax avoidance. Indeed, it was so widespread and so effective that I'm skeptical whether economic inequality was really so low then as we think. In a period when people are trying to hide wealth from the government, it will tend to be hidden from statistics too. One sign of the potential magnitude of the problem is the discrepancy between government receipts as a percentage of GDP, which have remained more or less constant during the entire period from the end of World War II to the present, and tax rates, which have varied dramatically.

Thanks to Sam Altman, Tiffani Ashley Bell, Patrick Collison, Ron Conway, Richard Florida, Ben Horowitz, Jessica Livingston, Robert Morris, Tim O'Reilly, Max Roser, and Alexia Tsotsis for reading drafts of this.

Note: This is a new version from which I removed a pair of metaphors that made a lot of people mad, essentially by macroexpanding them. If anyone wants to see the old version, I put it *here*.

Related:

— The Short Version | | A Reply to Ezra Klein

Life is Short

January 2016

Life is short, as everyone knows. When I was a kid I used to wonder about this. Is life actually short, or are we really complaining about its finiteness? Would we be just as likely to feel life was short if we lived 10 times as long?

Since there didn't seem any way to answer this question, I stopped wondering about it. Then I had kids. That gave me a way to answer the question, and the answer is that life actually is short.

Having kids showed me how to convert a continuous quantity, time, into discrete quantities. You only get 52 weekends with your 2 year old. If Christmas-as-magic lasts from say ages 3 to 10, you only get to watch your child experience it 8 times. And while it's impossible to say what is a lot or a little of a continuous quantity like time, 8 is not a lot of something. If you had a handful of 8 peanuts, or a shelf of 8 books to choose from, the quantity would definitely seem limited, no matter what your lifespan was.

Ok, so life actually is short. Does it make any difference to know that?

It has for me. It means arguments of the form "Life is too short for x" have great force. It's not just a figure of speech to say that life is too short for something. It's not just a synonym for annoying. If you find yourself thinking that life is too short for something, you should try to eliminate it if you can.

When I ask myself what I've found life is too short for, the word that pops into my head is "bullshit." I realize that answer is somewhat tautological. It's almost the definition of bullshit that it's the stuff that life is too short for. And yet bullshit does have a distinctive character. There's something fake about it. It's the junk food of experience. [1]

If you ask yourself what you spend your time on that's bullshit, you probably already know the answer. Unnecessary meetings, pointless disputes, bureaucracy,

posturing, dealing with other people's mistakes, traffic jams, addictive but unrewarding pastimes.

There are two ways this kind of thing gets into your life: it's either forced on you, or it tricks you. To some extent you have to put up with the bullshit forced on you by circumstances. You need to make money, and making money consists mostly of errands. Indeed, the law of supply and demand ensures that: the more rewarding some kind of work is, the cheaper people will do it. It may be that less bullshit is forced on you than you think, though. There has always been a stream of people who opt out of the default grind and go live somewhere where opportunities are fewer in the conventional sense, but life feels more authentic. This could become more common.

You can do it on a smaller scale without moving. The amount of time you have to spend on bullshit varies between employers. Most large organizations (and many small ones) are steeped in it. But if you consciously prioritize bullshit avoidance over other factors like money and prestige, you can probably find employers that will waste less of your time.

If you're a freelancer or a small company, you can do this at the level of individual customers. If you fire or avoid toxic customers, you can decrease the amount of bullshit in your life by more than you decrease your income.

But while some amount of bullshit is inevitably forced on you, the bullshit that sneaks into your life by tricking you is no one's fault but your own. And yet the bullshit you choose may be harder to eliminate than the bullshit that's forced on you. Things that lure you into wasting your time have to be really good at tricking you. An example that will be familiar to a lot of people is arguing online. When someone contradicts you, they're in a sense attacking you. Sometimes pretty overtly. Your instinct when attacked is to defend yourself. But like a lot of instincts, this one wasn't designed for the world we now live in. Counterintuitive as it feels, it's better most of the time not to defend yourself. Otherwise these people are literally taking your life. [2]

Arguing online is only incidentally addictive. There are more dangerous things than that. As I've written before, one byproduct of technical progress is that things we like tend to become *more addictive*. Which means we will increasingly have to make a conscious effort to avoid addictions — to stand outside ourselves and ask “is this how I want to be spending my time?”

As well as avoiding bullshit, one should actively seek out things that matter. But different things matter to different people, and most have to learn what matters to them. A few are lucky and realize early on that they love math or taking care of animals or writing, and then figure out a way to spend a lot of time doing

it. But most people start out with a life that's a mix of things that matter and things that don't, and only gradually learn to distinguish between them.

For the young especially, much of this confusion is induced by the artificial situations they find themselves in. In middle school and high school, what the other kids think of you seems the most important thing in the world. But when you ask adults what they got wrong at that age, nearly all say they cared too much what other kids thought of them.

One heuristic for distinguishing stuff that matters is to ask yourself whether you'll care about it in the future. Fake stuff that matters usually has a sharp peak of seeming to matter. That's how it tricks you. The area under the curve is small, but its shape jabs into your consciousness like a pin.

The things that matter aren't necessarily the ones people would call "important." Having coffee with a friend matters. You won't feel later like that was a waste of time.

One great thing about having small children is that they make you spend time on things that matter: them. They grab your sleeve as you're staring at your phone and say "will you play with me?" And odds are that is in fact the bullshit-minimizing option.

If life is short, we should expect its shortness to take us by surprise. And that is just what tends to happen. You take things for granted, and then they're gone. You think you can always write that book, or climb that mountain, or whatever, and then you realize the window has closed. The saddest windows close when other people die. Their lives are short too. After my mother died, I wished I'd spent more time with her. I lived as if she'd always be there. And in her typical quiet way she encouraged that illusion. But an illusion it was. I think a lot of people make the same mistake I did.

The usual way to avoid being taken by surprise by something is to be consciously aware of it. Back when life was more precarious, people used to be aware of death to a degree that would now seem a bit morbid. I'm not sure why, but it doesn't seem the right answer to be constantly reminding oneself of the grim reaper hovering at everyone's shoulder. Perhaps a better solution is to look at the problem from the other end. Cultivate a habit of impatience about the things you most want to do. Don't wait before climbing that mountain or writing that book or visiting your mother. You don't need to be constantly reminding yourself why you shouldn't wait. Just don't wait.

I can think of two more things one does when one doesn't have much of something: try to get more of it, and savor what one has. Both make sense here.

How you live affects how long you live. Most people could do better. Me among them.

But you can probably get even more effect by paying closer attention to the time you have. It's easy to let the days rush by. The "flow" that imaginative people love so much has a darker cousin that prevents you from pausing to savor life amid the daily slurry of errands and alarms. One of the most striking things I've read was not in a book, but the title of one: James Salter's *Burning the Days*.

It is possible to slow time somewhat. I've gotten better at it. Kids help. When you have small children, there are a lot of moments so perfect that you can't help noticing.

It does help too to feel that you've squeezed everything out of some experience. The reason I'm sad about my mother is not just that I miss her but that I think of all the things we could have done that we didn't. My oldest son will be 7 soon. And while I miss the 3 year old version of him, I at least don't have any regrets over what might have been. We had the best time a daddy and a 3 year old ever had.

Relentlessly prune bullshit, don't wait to do things that matter, and savor the time you have. That's what you do when life is short.

Notes

[1] At first I didn't like it that the word that came to mind was one that had other meanings. But then I realized the other meanings are fairly closely related. Bullshit in the sense of things you waste your time on is a lot like intellectual bullshit.

[2] I chose this example deliberately as a note to self. I get attacked a lot online. People tell the craziest lies about me. And I have so far done a pretty mediocre job of suppressing the natural human inclination to say "Hey, that's not true!"

Thanks to Jessica Livingston and Geoff Ralston for reading drafts of this.

How to Write Usefully

February 2020

What should an essay be? Many people would say persuasive. That's what a lot of us were taught essays should be. But I think we can aim for something more ambitious: that an essay should be useful.

To start with, that means it should be correct. But it's not enough merely to be correct. It's easy to make a statement correct by making it vague. That's a common flaw in academic writing, for example. If you know nothing at all about an issue, you can't go wrong by saying that the issue is a complex one, that there are many factors to be considered, that it's a mistake to take too simplistic a view of it, and so on.

Though no doubt correct, such statements tell the reader nothing. Useful writing makes claims that are as strong as they can be made without becoming false.

For example, it's more useful to say that Pike's Peak is near the middle of Colorado than merely somewhere in Colorado. But if I say it's in the exact middle of Colorado, I've now gone too far, because it's a bit east of the middle.

Precision and correctness are like opposing forces. It's easy to satisfy one if you ignore the other. The converse of vaporous academic writing is the bold, but false, rhetoric of demagogues. Useful writing is bold, but true.

It's also two other things: it tells people something important, and that at least some of them didn't already know.

Telling people something they didn't know doesn't always mean surprising them. Sometimes it means telling them something they knew unconsciously but had never put into words. In fact those may be the more valuable insights, because they tend to be more fundamental.

Let's put them all together. Useful writing tells people something true and important that they didn't already know, and tells them as unequivocally as

possible.

Notice these are all a matter of degree. For example, you can't expect an idea to be novel to everyone. Any insight that you have will probably have already been had by at least one of the world's 7 billion people. But it's sufficient if an idea is novel to a lot of readers.

Ditto for correctness, importance, and strength. In effect the four components are like numbers you can multiply together to get a score for usefulness. Which I realize is almost awkwardly reductive, but nonetheless true.

How can you ensure that the things you say are true and novel and important? Believe it or not, there is a trick for doing this. I learned it from my friend Robert Morris, who has a horror of saying anything dumb. His trick is not to say anything unless he's sure it's worth hearing. This makes it hard to get opinions out of him, but when you do, they're usually right.

Translated into essay writing, what this means is that if you write a bad sentence, you don't publish it. You delete it and try again. Often you abandon whole branches of four or five paragraphs. Sometimes a whole essay.

You can't ensure that every idea you have is good, but you can ensure that every one you publish is, by simply not publishing the ones that aren't.

In the sciences, this is called publication bias, and is considered bad. When some hypothesis you're exploring gets inconclusive results, you're supposed to tell people about that too. But with essay writing, publication bias is the way to go.

My strategy is loose, then tight. I write the first draft of an essay fast, trying out all kinds of ideas. Then I spend days rewriting it very carefully.

I've never tried to count how many times I proofread essays, but I'm sure there are sentences I've read 100 times before publishing them. When I proofread an essay, there are usually passages that stick out in an annoying way, sometimes because they're clumsily written, and sometimes because I'm not sure they're true. The annoyance starts out unconscious, but after the tenth reading or so I'm saying "Ugh, that part" each time I hit it. They become like briars that catch your sleeve as you walk past. Usually I won't publish an essay till they're all gone — till I can read through the whole thing without the feeling of anything catching.

I'll sometimes let through a sentence that seems clumsy, if I can't think of a way to rephrase it, but I will never knowingly let through one that doesn't seem

correct. You never have to. If a sentence doesn't seem right, all you have to do is ask why it doesn't, and you've usually got the replacement right there in your head.

This is where essayists have an advantage over journalists. You don't have a deadline. You can work for as long on an essay as you need to get it right. You don't have to publish the essay at all, if you can't get it right. Mistakes seem to lose courage in the face of an enemy with unlimited resources. Or that's what it feels like. What's really going on is that you have different expectations for yourself. You're like a parent saying to a child "we can sit here all night till you eat your vegetables." Except you're the child too.

I'm not saying no mistake gets through. For example, I added condition (c) in "*A Way to Detect Bias*" after readers pointed out that I'd omitted it. But in practice you can catch nearly all of them.

There's a trick for getting importance too. It's like the trick I suggest to young founders for getting startup ideas: to make something you yourself want. You can use yourself as a proxy for the reader. The reader is not completely unlike you, so if you write about topics that seem important to you, they'll probably seem important to a significant number of readers as well.

Importance has two factors. It's the number of people something matters to, times how much it matters to them. Which means of course that it's not a rectangle, but a sort of ragged comb, like a Riemann sum.

The way to get novelty is to write about topics you've thought about a lot. Then you can use yourself as a proxy for the reader in this department too. Anything you notice that surprises you, who've thought about the topic a lot, will probably also surprise a significant number of readers. And here, as with correctness and importance, you can use the Morris technique to ensure that you will. If you don't learn anything from writing an essay, don't publish it.

You need humility to measure novelty, because acknowledging the novelty of an idea means acknowledging your previous ignorance of it. Confidence and humility are often seen as opposites, but in this case, as in many others, confidence helps you to be humble. If you know you're an expert on some topic, you can freely admit when you learn something you didn't know, because you can be confident that most other people wouldn't know it either.

The fourth component of useful writing, strength, comes from two things: thinking well, and the skillful use of qualification. These two counterbalance each other, like the accelerator and clutch in a car with a manual transmission. As you try to refine the expression of an idea, you adjust the qualification accordingly. Something you're sure of, you can state baldly with no qualification at all, as I

did the four components of useful writing. Whereas points that seem dubious have to be held at arm's length with perhapses.

As you refine an idea, you're pushing in the direction of less qualification. But you can rarely get it down to zero. Sometimes you don't even want to, if it's a side point and a fully refined version would be too long.

Some say that qualifications weaken writing. For example, that you should never begin a sentence in an essay with "I think," because if you're saying it, then of course you think it. And it's true that "I think x" is a weaker statement than simply "x." Which is exactly why you need "I think." You need it to express your degree of certainty.

But qualifications are not scalars. They're not just experimental error. There must be 50 things they can express: how broadly something applies, how you know it, how happy you are it's so, even how it could be falsified. I'm not going to try to explore the structure of qualification here. It's probably more complex than the whole topic of writing usefully. Instead I'll just give you a practical tip: Don't underestimate qualification. It's an important skill in its own right, not just a sort of tax you have to pay in order to avoid saying things that are false. So learn and use its full range. It may not be fully half of having good ideas, but it's part of having them.

There's one other quality I aim for in essays: to say things as simply as possible. But I don't think this is a component of usefulness. It's more a matter of consideration for the reader. And it's a practical aid in getting things right; a mistake is more obvious when expressed in simple language. But I'll admit that the main reason I write simply is not for the reader's sake or because it helps get things right, but because it bothers me to use more or fancier words than I need to. It seems inelegant, like a program that's too long.

I realize florid writing works for some people. But unless you're sure you're one of them, the best advice is to write as simply as you can.

I believe the formula I've given you, importance + novelty + correctness + strength, is the recipe for a good essay. But I should warn you that it's also a recipe for making people mad.

The root of the problem is novelty. When you tell people something they didn't know, they don't always thank you for it. Sometimes the reason people don't know something is because they don't want to know it. Usually because it contradicts some cherished belief. And indeed, if you're looking for novel ideas, popular but mistaken beliefs are a good place to find them. Every popular mistaken belief

creates a *dead zone* of ideas around it that are relatively unexplored because they contradict it.

The strength component just makes things worse. If there's anything that annoys people more than having their cherished assumptions contradicted, it's having them flatly contradicted.

Plus if you've used the Morris technique, your writing will seem quite confident. Perhaps offensively confident, to people who disagree with you. The reason you'll seem confident is that you are confident: you've cheated, by only publishing the things you're sure of. It will seem to people who try to disagree with you that you never admit you're wrong. In fact you constantly admit you're wrong. You just do it before publishing instead of after.

And if your writing is as simple as possible, that just makes things worse. Brevity is the diction of command. If you watch someone delivering unwelcome news from a position of inferiority, you'll notice they tend to use lots of words, to soften the blow. Whereas to be short with someone is more or less to be rude to them.

It can sometimes work to deliberately phrase statements more weakly than you mean. To put "perhaps" in front of something you're actually quite sure of. But you'll notice that when writers do this, they usually do it with a wink.

I don't like to do this too much. It's cheesy to adopt an ironic tone for a whole essay. I think we just have to face the fact that elegance and curtness are two names for the same thing.

You might think that if you work sufficiently hard to ensure that an essay is correct, it will be invulnerable to attack. That's sort of true. It will be invulnerable to valid attacks. But in practice that's little consolation.

In fact, the strength component of useful writing will make you particularly vulnerable to misrepresentation. If you've stated an idea as strongly as you could without making it false, all anyone has to do is to exaggerate slightly what you said, and now it is false.

Much of the time they're not even doing it deliberately. One of the most surprising things you'll discover, if you start writing essays, is that people who disagree with you rarely disagree with what you've actually written. Instead they make up something you said and disagree with that.

For what it's worth, the countermove is to ask someone who does this to quote a specific sentence or passage you wrote that they believe is false, and explain why. I say "for what it's worth" because they never do. So although it might seem

that this could get a broken discussion back on track, the truth is that it was never on track in the first place.

Should you explicitly forestall likely misinterpretations? Yes, if they're misinterpretations a reasonably smart and well-intentioned person might make. In fact it's sometimes better to say something slightly misleading and then add the correction than to try to get an idea right in one shot. That can be more efficient, and can also model the way such an idea would be discovered.

But I don't think you should explicitly forestall intentional misinterpretations in the body of an essay. An essay is a place to meet honest readers. You don't want to spoil your house by putting bars on the windows to protect against dishonest ones. The place to protect against intentional misinterpretations is in end-notes. But don't think you can predict them all. People are as ingenious at misrepresenting you when you say something they don't want to hear as they are at coming up with rationalizations for things they want to do but know they shouldn't. I suspect it's the same skill.

As with most other things, the way to get better at writing essays is to practice. But how do you start? Now that we've examined the structure of useful writing, we can rephrase that question more precisely. Which constraint do you relax initially? The answer is, the first component of importance: the number of people who care about what you write.

If you narrow the topic sufficiently, you can probably find something you're an expert on. Write about that to start with. If you only have ten readers who care, that's fine. You're helping them, and you're writing. Later you can expand the breadth of topics you write about.

The other constraint you can relax is a little surprising: publication. Writing essays doesn't have to mean publishing them. That may seem strange now that the trend is to publish every random thought, but it worked for me. I wrote what amounted to essays in notebooks for about 15 years. I never published any of them and never expected to. I wrote them as a way of figuring things out. But when the web came along I'd had a lot of practice.

Incidentally, *Steve Wozniak* did the same thing. In high school he designed computers on paper for fun. He couldn't build them because he couldn't afford the components. But when Intel launched 4K DRAMs in 1975, he was ready.

How many essays are there left to write though? The answer to that question is probably the most exciting thing I've learned about essay writing. Nearly all of

them are left to write.

Although *the essay* is an old form, it hasn't been assiduously cultivated. In the print era, publication was expensive, and there wasn't enough demand for essays to publish that many. You could publish essays if you were already well known for writing something else, like novels. Or you could write book reviews that you took over to express your own ideas. But there was not really a direct path to becoming an essayist. Which meant few essays got written, and those that did tended to be about a narrow range of subjects.

Now, thanks to the internet, there's a path. Anyone can publish essays online. You start in obscurity, perhaps, but at least you can start. You don't need anyone's permission.

It sometimes happens that an area of knowledge sits quietly for years, till some change makes it explode. Cryptography did this to number theory. The internet is doing it to the essay.

The exciting thing is not that there's a lot left to write, but that there's a lot left to discover. There's a certain kind of idea that's best discovered by writing essays. If most essays are still unwritten, most such ideas are still undiscovered.

Notes

[1] Put railings on the balconies, but don't put bars on the windows.

[2] Even now I sometimes write essays that are not meant for publication. I wrote several to figure out what Y Combinator should do, and they were really helpful.

Thanks to Trevor Blackwell, Daniel Gackle, Jessica Livingston, and Robert Morris for reading drafts of this.

The Four Quadrants of Conformism

July 2020

One of the most revealing ways to classify people is by the degree and aggressiveness of their conformism. Imagine a Cartesian coordinate system whose horizontal axis runs from conventional-minded on the left to independent-minded on the right, and whose vertical axis runs from passive at the bottom to aggressive at the top. The resulting four quadrants define four types of people. Starting in the upper left and going counter-clockwise: aggressively conventional-minded, passively conventional-minded, passively independent-minded, and aggressively independent-minded.

I think that you'll find all four types in most societies, and that which quadrant people fall into depends more on their own personality than the beliefs prevalent in their society. [1]

Young children offer some of the best evidence for both points. Anyone who's been to primary school has seen the four types, and the fact that school rules are so arbitrary is strong evidence that which quadrant people fall into depends more on them than the rules.

The kids in the upper left quadrant, the aggressively conventional-minded ones, are the tattletales. They believe not only that rules must be obeyed, but that those who disobey them must be punished.

The kids in the lower left quadrant, the passively conventional-minded, are the sheep. They're careful to obey the rules, but when other kids break them, their impulse is to worry that those kids will be punished, not to ensure that they will.

The kids in the lower right quadrant, the passively independent-minded, are the dreamy ones. They don't care much about rules and probably aren't 100% sure what the rules even are.

And the kids in the upper right quadrant, the aggressively independent-minded, are the naughty ones. When they see a rule, their first impulse is to question it. Merely being told what to do makes them inclined to do the opposite.

When measuring conformism, of course, you have to say with respect to what, and this changes as kids get older. For younger kids it's the rules set by adults. But as kids get older, the source of rules becomes their peers. So a pack of teenagers who all flout school rules in the same way are not independent-minded; rather the opposite.

In adulthood we can recognize the four types by their distinctive calls, much as you could recognize four species of birds. The call of the aggressively conventional-minded is "Crush !" (It's rather alarming to see an exclamation point after a variable, but that's the whole problem with the aggressively conventional-minded.) The call of the passively conventional-minded is "What will the neighbors think?" The call of the passively independent-minded is "To each his own." And the call of the aggressively independent-minded is "Eppur si muove."

The four types are not equally common. There are more passive people than aggressive ones, and far more conventional-minded people than independent-minded ones. So the passively conventional-minded are the largest group, and the aggressively independent-minded the smallest.

Since one's quadrant depends more on one's personality than the nature of the rules, most people would occupy the same quadrant even if they'd grown up in a quite different society.

Princeton professor Robert George recently wrote:

I sometimes ask students what their position on slavery would have been had they been white and living in the South before abolition. Guess what? They all would have been abolitionists! They all would have bravely spoken out against slavery, and worked tirelessly against it.

He's too polite to say so, but of course they wouldn't. And indeed, our default assumption should not merely be that his students would, on average, have behaved the same way people did at the time, but that the ones who are aggressively conventional-minded today would have been aggressively conventional-minded then too. In other words, that they'd not only not have fought against slavery, but that they'd have been among its staunchest defenders.

I'm biased, I admit, but it seems to me that aggressively conventional-minded people are responsible for a disproportionate amount of the trouble in the world, and that a lot of the customs we've evolved since the Enlightenment have been

designed to protect the rest of us from them. In particular, the retirement of the concept of heresy and its replacement by the principle of freely debating all sorts of different ideas, even ones that are currently considered unacceptable, without any punishment for those who try them out to see if they work. [2]

Why do the independent-minded need to be protected, though? Because they have all the new ideas. To be a successful scientist, for example, it's not enough just to be right. You have to be right when everyone else is wrong. Conventional-minded people can't do that. For similar reasons, all successful startup CEOs are not merely independent-minded, but aggressively so. So it's no coincidence that societies prosper only to the extent that they have customs for keeping the conventional-minded at bay. [3]

In the last few years, many of us have noticed that the customs protecting free inquiry have been weakened. Some say we're overreacting — that they haven't been weakened very much, or that they've been weakened in the service of a greater good. The latter I'll dispose of immediately. When the conventional-minded get the upper hand, they always say it's in the service of a greater good. It just happens to be a different, incompatible greater good each time.

As for the former worry, that the independent-minded are being oversensitive, and that free inquiry hasn't been shut down that much, you can't judge that unless you are yourself independent-minded. You can't know how much of the space of ideas is being lopped off unless you have them, and only the independent-minded have the ones at the edges. Precisely because of this, they tend to be very sensitive to changes in how freely one can explore ideas. They're the canaries in this coalmine.

The conventional-minded say, as they always do, that they don't want to shut down the discussion of all ideas, just the bad ones.

You'd think it would be obvious just from that sentence what a dangerous game they're playing. But I'll spell it out. There are two reasons why we need to be able to discuss even "bad" ideas.

The first is that any process for deciding which ideas to ban is bound to make mistakes. All the more so because no one intelligent wants to undertake that kind of work, so it ends up being done by the stupid. And when a process makes a lot of mistakes, you need to leave a margin for error. Which in this case means you need to ban fewer ideas than you'd like to. But that's hard for the aggressively conventional-minded to do, partly because they enjoy seeing people punished, as they have since they were children, and partly because they compete with one another. Enforcers of orthodoxy can't allow a borderline idea to exist, because that gives other enforcers an opportunity to one-up them in the moral purity

department, and perhaps even to turn enforcer upon them. So instead of getting the margin for error we need, we get the opposite: a race to the bottom in which any idea that seems at all bannable ends up being banned. [4]

The second reason it's dangerous to ban the discussion of ideas is that ideas are more closely related than they look. Which means if you restrict the discussion of some topics, it doesn't only affect those topics. The restrictions propagate back into any topic that yields implications in the forbidden ones. And that is not an edge case. The best ideas do exactly that: they have consequences in fields far removed from their origins. Having ideas in a world where some ideas are banned is like playing soccer on a pitch that has a minefield in one corner. You don't just play the same game you would have, but on a different shaped pitch. You play a much more subdued game even on the ground that's safe.

In the past, the way the independent-minded protected themselves was to congregate in a handful of places — first in courts, and later in universities — where they could to some extent make their own rules. Places where people work with ideas tend to have customs protecting free inquiry, for the same reason wafer fabs have powerful air filters, or recording studios good sound insulation. For the last couple centuries at least, when the aggressively conventional-minded were on the rampage for whatever reason, universities were the safest places to be.

That may not work this time though, due to the unfortunate fact that the latest wave of intolerance began in universities. It began in the mid 1980s, and by 2000 seemed to have died down, but it has recently flared up again with the arrival of social media. This seems, unfortunately, to have been an own goal by Silicon Valley. Though the people who run Silicon Valley are almost all independent-minded, they've handed the aggressively conventional-minded a tool such as they could only have dreamed of.

On the other hand, perhaps the decline in the spirit of free inquiry within universities is as much the symptom of the departure of the independent-minded as the cause. People who would have become professors 50 years ago have other options now. Now they can become quants or start startups. You have to be independent-minded to succeed at either of those. If these people had been professors, they'd have put up a stiffer resistance on behalf of academic freedom. So perhaps the picture of the independent-minded fleeing declining universities is too gloomy. Perhaps the universities are declining because so many have already left. [5]

Though I've spent a lot of time thinking about this situation, I can't predict how it plays out. Could some universities reverse the current trend and remain places where the independent-minded want to congregate? Or will the independent-

minded gradually abandon them? I worry a lot about what we might lose if that happened.

But I'm hopeful long term. The independent-minded are good at protecting themselves. If existing institutions are compromised, they'll create new ones. That may require some imagination. But imagination is, after all, their specialty.

Notes

[1] I realize of course that if people's personalities vary in any two ways, you can use them as axes and call the resulting four quadrants personality types. So what I'm really claiming is that the axes are orthogonal and that there's significant variation in both.

[2] The aggressively conventional-minded aren't responsible for all the trouble in the world. Another big source of trouble is the sort of charismatic leader who gains power by appealing to them. They become much more dangerous when such leaders emerge.

[3] I never worried about writing things that offended the conventional-minded when I was running Y Combinator. If YC were a cookie company, I'd have faced a difficult moral choice. Conventional-minded people eat cookies too. But they don't start successful startups. So if I deterred them from applying to YC, the only effect was to save us work reading applications.

[4] There has been progress in one area: the punishments for talking about banned ideas are less severe than in the past. There's little danger of being killed, at least in richer countries. The aggressively conventional-minded are mostly satisfied with getting people fired.

[5] Many professors are independent-minded — especially in math, the hard sciences, and engineering, where you have to be to succeed. But students are more representative of the general population, and thus mostly conventional-minded. So when professors and students are in conflict, it's not just a conflict between generations but also between different types of people.

Thanks to Sam Altman, Trevor Blackwell, Nicholas Christakis, Patrick Collison, Sam Gichuru, Jessica Livingston, Patrick McKenzie, Geoff Ralston, and Harj Taggar for reading drafts of this.

How to Think for Yourself

November 2020

There are some kinds of work that you can't do well without thinking differently from your peers. To be a successful scientist, for example, it's not enough just to be correct. Your ideas have to be both correct and novel. You can't publish papers saying things other people already know. You need to say things no one else has realized yet.

The same is true for investors. It's not enough for a public market investor to predict correctly how a company will do. If a lot of other people make the same prediction, the stock price will already reflect it, and there's no room to make money. The only valuable insights are the ones most other investors don't share.

You see this pattern with startup founders too. You don't want to start a startup to do something that everyone agrees is a good idea, or there will already be other companies doing it. You have to do something that sounds to most other people like a bad idea, but that you know isn't — like writing software for a tiny computer used by a few thousand hobbyists, or starting a site to let people rent airbeds on strangers' floors.

Ditto for essayists. An essay that told people things they already knew would be boring. You have to tell them something *new*.

But this pattern isn't universal. In fact, it doesn't hold for most kinds of work. In most kinds of work — to be an administrator, for example — all you need is the first half. All you need is to be right. It's not essential that everyone else be wrong.

There's room for a little novelty in most kinds of work, but in practice there's a fairly sharp distinction between the kinds of work where it's essential to be independent-minded, and the kinds where it's not.

I wish someone had told me about this distinction when I was a kid, because it's one of the most important things to think about when you're deciding what

kind of work you want to do. Do you want to do the kind of work where you can only win by thinking differently from everyone else? I suspect most people's unconscious mind will answer that question before their conscious mind has a chance to. I know mine does.

Independent-mindedness seems to be more a matter of nature than nurture. Which means if you pick the wrong type of work, you're going to be unhappy. If you're naturally independent-minded, you're going to find it frustrating to be a middle manager. And if you're naturally conventional-minded, you're going to be sailing into a headwind if you try to do original research.

One difficulty here, though, is that people are often mistaken about where they fall on the spectrum from conventional- to independent-minded. Conventional-minded people don't like to think of themselves as conventional-minded. And in any case, it genuinely feels to them as if they make up their own minds about everything. It's just a coincidence that their beliefs are identical to their peers'. And the independent-minded, meanwhile, are often unaware how different their ideas are from conventional ones, at least till they state them publicly. [1]

By the time they reach adulthood, most people know roughly how smart they are (in the narrow sense of ability to solve pre-set problems), because they're constantly being tested and ranked according to it. But schools generally ignore independent-mindedness, except to the extent they try to suppress it. So we don't get anything like the same kind of feedback about how independent-minded we are.

There may even be a phenomenon like Dunning-Kruger at work, where the most conventional-minded people are confident that they're independent-minded, while the genuinely independent-minded worry they might not be independent-minded enough.

Can you make yourself more independent-minded? I think so. This quality may be largely inborn, but there seem to be ways to magnify it, or at least not to suppress it.

One of the most effective techniques is one practiced unintentionally by most nerds: simply to be less aware what conventional beliefs are. It's hard to be a conformist if you don't know what you're supposed to conform to. Though again, it may be that such people already are independent-minded. A conventional-minded person would probably feel anxious not knowing what other people thought, and make more effort to find out.

It matters a lot who you surround yourself with. If you're surrounded by

conventional-minded people, it will constrain which ideas you can express, and that in turn will constrain which ideas you have. But if you surround yourself with independent-minded people, you'll have the opposite experience: hearing other people say surprising things will encourage you to, and to think of more.

Because the independent-minded find it uncomfortable to be surrounded by conventional-minded people, they tend to self-segregate once they have a chance to. The problem with high school is that they haven't yet had a chance to. Plus high school tends to be an inward-looking little world whose inhabitants lack confidence, both of which magnify the forces of conformism. So high school is often a *bad time* for the independent-minded. But there is some advantage even here: it teaches you what to avoid. If you later find yourself in a situation that makes you think "this is like high school," you know you should get out. [2]

Another place where the independent- and conventional-minded are thrown together is in successful startups. The founders and early employees are almost always independent-minded; otherwise the startup wouldn't be successful. But conventional-minded people greatly outnumber independent-minded ones, so as the company grows, the original spirit of independent-mindedness is inevitably diluted. This causes all kinds of problems besides the obvious one that the company starts to suck. One of the strangest is that the founders find themselves able to speak more freely with founders of other companies than with their own employees. [3]

Fortunately you don't have to spend all your time with independent-minded people. It's enough to have one or two you can talk to regularly. And once you find them, they're usually as eager to talk as you are; they need you too. Although universities no longer have the kind of monopoly they used to have on education, good universities are still an excellent way to meet independent-minded people. Most students will still be conventional-minded, but you'll at least find clumps of independent-minded ones, rather than the near zero you may have found in high school.

It also works to go in the other direction: as well as cultivating a small collection of independent-minded friends, to try to meet as many different types of people as you can. It will decrease the influence of your immediate peers if you have several other groups of peers. Plus if you're part of several different worlds, you can often import ideas from one to another.

But by different types of people, I don't mean demographically different. For this technique to work, they have to think differently. So while it's an excellent idea to go and visit other countries, you can probably find people who think differently right around the corner. When I meet someone who knows a lot about

something unusual (which includes practically everyone, if you dig deep enough), I try to learn what they know that other people don't. There are almost always surprises here. It's a good way to make conversation when you meet strangers, but I don't do it to make conversation. I really want to know.

You can expand the source of influences in time as well as space, by reading history. When I read history I do it not just to learn what happened, but to try to get inside the heads of people who lived in the past. How did things look to them? This is hard to do, but worth the effort for the same reason it's worth travelling far to triangulate a point.

You can also take more explicit measures to prevent yourself from automatically adopting conventional opinions. The most general is to cultivate an attitude of skepticism. When you hear someone say something, stop and ask yourself "Is that true?" Don't say it out loud. I'm not suggesting that you impose on everyone who talks to you the burden of proving what they say, but rather that you take upon yourself the burden of evaluating what they say.

Treat it as a puzzle. You know that some accepted ideas will later turn out to be wrong. See if you can guess which. The end goal is not to find flaws in the things you're told, but to find the new ideas that had been concealed by the broken ones. So this game should be an exciting quest for novelty, not a boring protocol for intellectual hygiene. And you'll be surprised, when you start asking "Is this true?", how often the answer is not an immediate yes. If you have any imagination, you're more likely to have too many leads to follow than too few.

More generally your goal should be not to let anything into your head unexamined, and things don't always enter your head in the form of statements. Some of the most powerful influences are implicit. How do you even notice these? By standing back and watching how other people get their ideas.

When you stand back at a sufficient distance, you can see ideas spreading through groups of people like waves. The most obvious are in fashion: you notice a few people wearing a certain kind of shirt, and then more and more, until half the people around you are wearing the same shirt. You may not care much what you wear, but there are intellectual fashions too, and you definitely don't want to participate in those. Not just because you want sovereignty over your own thoughts, but because *unfashionable* ideas are disproportionately likely to lead somewhere interesting. The best place to find undiscovered ideas is where no one else is looking. [4]

To go beyond this general advice, we need to look at the internal structure of independent-mindedness — at the individual muscles we need to exercise, as it

were. It seems to me that it has three components: fastidiousness about truth, resistance to being told what to think, and curiosity.

Fastidiousness about truth means more than just not believing things that are false. It means being careful about degree of belief. For most people, degree of belief rushes unexamined toward the extremes: the unlikely becomes impossible, and the probable becomes certain. [5] To the independent-minded, this seems unpardonably sloppy. They're willing to have anything in their heads, from highly speculative hypotheses to (apparent) tautologies, but on subjects they care about, everything has to be labelled with a carefully considered degree of belief. [6]

The independent-minded thus have a horror of ideologies, which require one to accept a whole collection of beliefs at once, and to treat them as articles of faith. To an independent-minded person that would seem revolting, just as it would seem to someone fastidious about food to take a bite of a submarine sandwich filled with a large variety of ingredients of indeterminate age and provenance.

Without this fastidiousness about truth, you can't be truly independent-minded. It's not enough just to have resistance to being told what to think. Those kind of people reject conventional ideas only to replace them with the most random conspiracy theories. And since these conspiracy theories have often been manufactured to capture them, they end up being less independent-minded than ordinary people, because they're subject to a much more exacting master than mere convention. [7]

Can you increase your fastidiousness about truth? I would think so. In my experience, merely thinking about something you're fastidious about causes that fastidiousness to grow. If so, this is one of those rare virtues we can have more of merely by wanting it. And if it's like other forms of fastidiousness, it should also be possible to encourage in children. I certainly got a strong dose of it from my father. [8]

The second component of independent-mindedness, resistance to being told what to think, is the most visible of the three. But even this is often misunderstood. The big mistake people make about it is to think of it as a merely negative quality. The language we use reinforces that idea. You're *un* conventional. You *don't* care what other people think. But it's not just a kind of immunity. In the most independent-minded people, the desire not to be told what to think is a positive force. It's not mere skepticism, but an active *delight* in ideas that subvert the conventional wisdom, the more counterintuitive the better.

Some of the most novel ideas seemed at the time almost like practical jokes. Think how often your reaction to a novel idea is to laugh. I don't think it's

because novel ideas are funny per se, but because novelty and humor share a certain kind of surprisingness. But while not identical, the two are close enough that there is a definite correlation between having a sense of humor and being independent-minded — just as there is between being humorless and being conventional-minded. [9]

I don't think we can significantly increase our resistance to being told what to think. It seems the most innate of the three components of independent-mindedness; people who have this quality as adults usually showed all too visible signs of it as children. But if we can't increase our resistance to being told what to think, we can at least shore it up, by surrounding ourselves with other independent-minded people.

The third component of independent-mindedness, curiosity, may be the most interesting. To the extent that we can give a brief answer to the question of where novel ideas come from, it's curiosity. That's what people are usually feeling before having them.

In my experience, independent-mindedness and curiosity predict one another perfectly. Everyone I know who's independent-minded is deeply curious, and everyone I know who's conventional-minded isn't. Except, curiously, children. All small children are curious. Perhaps the reason is that even the conventional-minded have to be curious in the beginning, in order to learn what the conventions are. Whereas the independent-minded are the gluttons of curiosity, who keep eating even after they're full. [10]

The three components of independent-mindedness work in concert: fastidiousness about truth and resistance to being told what to think leave space in your brain, and curiosity finds new ideas to fill it.

Interestingly, the three components can substitute for one another in much the same way muscles can. If you're sufficiently fastidious about truth, you don't need to be as resistant to being told what to think, because fastidiousness alone will create sufficient gaps in your knowledge. And either one can compensate for curiosity, because if you create enough space in your brain, your discomfort at the resulting vacuum will add force to your curiosity. Or curiosity can compensate for them: if you're sufficiently curious, you don't need to clear space in your brain, because the new ideas you discover will push out the conventional ones you acquired by default.

Because the components of independent-mindedness are so interchangeable, you can have them to varying degrees and still get the same result. So there is not just a single model of independent-mindedness. Some independent-minded people are openly subversive, and others are quietly curious. They all know the secret

handshake though.

Is there a way to cultivate curiosity? To start with, you want to avoid situations that suppress it. How much does the work you're currently doing engage your curiosity? If the answer is "not much," maybe you should change something.

The most important active step you can take to cultivate your curiosity is probably to seek out the topics that engage it. Few adults are equally curious about everything, and it doesn't seem as if you can choose which topics interest you. So it's up to you to *find* them. Or invent them, if necessary.

Another way to increase your curiosity is to indulge it, by investigating things you're interested in. Curiosity is unlike most other appetites in this respect: indulging it tends to increase rather than to sate it. Questions lead to more questions.

Curiosity seems to be more individual than fastidiousness about truth or resistance to being told what to think. To the degree people have the latter two, they're usually pretty general, whereas different people can be curious about very different things. So perhaps curiosity is the compass here. Perhaps, if your goal is to discover novel ideas, your motto should not be "do what you love" so much as "do what you're curious about."

Notes

[1] One convenient consequence of the fact that no one identifies as conventional-minded is that you can say what you like about conventional-minded people without getting in too much trouble. When I wrote "*The Four Quadrants of Conformism*" I expected a firestorm of rage from the aggressively conventional-minded, but in fact it was quite muted. They sensed that there was something about the essay that they disliked intensely, but they had a hard time finding a specific passage to pin it on.

[2] When I ask myself what in my life is like high school, the answer is Twitter. It's not just full of conventional-minded people, as anything its size will inevitably be, but subject to violent storms of conventional-mindedness that remind me of descriptions of Jupiter. But while it probably is a net loss to spend time there, it has at least made me think more about the distinction between independent- and conventional-mindedness, which I probably wouldn't have done otherwise.

[3] The decrease in independent-mindedness in growing startups is still an open problem, but there may be solutions.

Founders can delay the problem by making a conscious effort only to hire independent-minded people. Which of course also has the ancillary benefit that they have better ideas.

Another possible solution is to create policies that somehow disrupt the force of conformism, much as control rods slow chain reactions, so that the conventional-minded aren't as dangerous. The physical separation of Lockheed's Skunk Works may have had this as a side benefit. Recent examples suggest employee forums like Slack may not be an unmitigated good.

The most radical solution would be to grow revenues without growing the company. You think hiring that junior PR person will be cheap, compared to a programmer, but what will be the effect on the average level of independent-mindedness in your company? (The growth in staff relative to faculty seems to have had a similar effect on universities.) Perhaps the rule about outsourcing work that's not your "core competency" should be augmented by one about outsourcing work done by people who'd ruin your culture as employees.

Some investment firms already seem to be able to grow revenues without growing the number of employees. Automation plus the ever increasing articulation of the "tech stack" suggest this may one day be possible for product companies.

[4] There are intellectual fashions in every field, but their influence varies. One of the reasons politics, for example, tends to be boring is that it's so extremely subject to them. The threshold for having opinions about politics is much *lower* than the one for having opinions about set theory. So while there are some ideas in politics, in practice they tend to be swamped by waves of intellectual fashion.

[5] The conventional-minded are often fooled by the strength of their opinions into believing that they're independent-minded. But strong convictions are not a sign of independent-mindedness. Rather the opposite.

[6] Fastidiousness about truth doesn't imply that an independent-minded person won't be dishonest, but that he won't be deluded. It's sort of like the definition of a gentleman as someone who is never unintentionally rude.

[7] You see this especially among political extremists. They think themselves nonconformists, but actually they're niche conformists. Their opinions may be different from the average person's, but they are often more influenced by their peers' opinions than the average person's are.

[8] If we broaden the concept of fastidiousness about truth so that it excludes pandering, bogusness, and pomposity as well as falsehood in the strict sense, our model of independent-mindedness can expand further into the arts.

[9] This correlation is far from perfect, though. Gödel and Dirac don't seem to have been very strong in the humor department. But someone who is both "neurotypical" and humorless is very likely to be conventional-minded.

[10] Exception: gossip. Almost everyone is curious about gossip.

Thanks to Trevor Blackwell, Paul Buchheit, Patrick Collison, Jessica Livingston, Robert Morris, Harj Taggar, and Peter Thiel for reading drafts of this.

What I Worked On

February 2021

Before college the two main things I worked on, outside of school, were writing and programming. I didn't write essays. I wrote what beginning writers were supposed to write then, and probably still are: short stories. My stories were awful. They had hardly any plot, just characters with strong feelings, which I imagined made them deep.

The first programs I tried writing were on the IBM 1401 that our school district used for what was then called “data processing.” This was in 9th grade, so I was 13 or 14. The school district's 1401 happened to be in the basement of our junior high school, and my friend Rich Draves and I got permission to use it. It was like a mini Bond villain's lair down there, with all these alien-looking machines — CPU, disk drives, printer, card reader — sitting up on a raised floor under bright fluorescent lights.

The language we used was an early version of Fortran. You had to type programs on punch cards, then stack them in the card reader and press a button to load the program into memory and run it. The result would ordinarily be to print something on the spectacularly loud printer.

I was puzzled by the 1401. I couldn't figure out what to do with it. And in retrospect there's not much I could have done with it. The only form of input to programs was data stored on punched cards, and I didn't have any data stored on punched cards. The only other option was to do things that didn't rely on any input, like calculate approximations of pi, but I didn't know enough math to do anything interesting of that type. So I'm not surprised I can't remember any programs I wrote, because they can't have done much. My clearest memory is of the moment I learned it was possible for programs not to terminate, when one of mine didn't. On a machine without time-sharing, this was a social as well as a technical error, as the data center manager's expression made clear.

With microcomputers, everything changed. Now you could have a computer

sitting right in front of you, on a desk, that could respond to your keystrokes as it was running instead of just churning through a stack of punch cards and then stopping. [1]

The first of my friends to get a microcomputer built it himself. It was sold as a kit by Heathkit. I remember vividly how impressed and envious I felt watching him sitting in front of it, typing programs right into the computer.

Computers were expensive in those days and it took me years of nagging before I convinced my father to buy one, a TRS-80, in about 1980. The gold standard then was the Apple II, but a TRS-80 was good enough. This was when I really started programming. I wrote simple games, a program to predict how high my model rockets would fly, and a word processor that my father used to write at least one book. There was only room in memory for about 2 pages of text, so he'd write 2 pages at a time and then print them out, but it was a lot better than a typewriter.

Though I liked programming, I didn't plan to study it in college. In college I was going to study philosophy, which sounded much more powerful. It seemed, to my naive high school self, to be the study of the ultimate truths, compared to which the things studied in other fields would be mere domain knowledge. What I discovered when I got to college was that the other fields took up so much of the space of ideas that there wasn't much left for these supposed ultimate truths. All that seemed left for philosophy were edge cases that people in other fields felt could safely be ignored.

I couldn't have put this into words when I was 18. All I knew at the time was that I kept taking philosophy courses and they kept being boring. So I decided to switch to AI.

AI was in the air in the mid 1980s, but there were two things especially that made me want to work on it: a novel by Heinlein called *The Moon is a Harsh Mistress*, which featured an intelligent computer called Mike, and a PBS documentary that showed Terry Winograd using SHRDLU. I haven't tried rereading *The Moon is a Harsh Mistress*, so I don't know how well it has aged, but when I read it I was drawn entirely into its world. It seemed only a matter of time before we'd have Mike, and when I saw Winograd using SHRDLU, it seemed like that time would be a few years at most. All you had to do was teach SHRDLU more words.

There weren't any classes in AI at Cornell then, not even graduate classes, so I started trying to teach myself. Which meant learning Lisp, since in those days Lisp was regarded as the language of AI. The commonly used programming languages then were pretty primitive, and programmers' ideas correspondingly so. The default language at Cornell was a Pascal-like language called PL/I, and

the situation was similar elsewhere. Learning Lisp expanded my concept of a program so fast that it was years before I started to have a sense of where the new limits were. This was more like it; this was what I had expected college to do. It wasn't happening in a class, like it was supposed to, but that was ok. For the next couple years I was on a roll. I knew what I was going to do.

For my undergraduate thesis, I reverse-engineered SHRDLU. My God did I love working on that program. It was a pleasing bit of code, but what made it even more exciting was my belief — hard to imagine now, but not unique in 1985 — that it was already climbing the lower slopes of intelligence.

I had gotten into a program at Cornell that didn't make you choose a major. You could take whatever classes you liked, and choose whatever you liked to put on your degree. I of course chose "Artificial Intelligence." When I got the actual physical diploma, I was dismayed to find that the quotes had been included, which made them read as scare-quotes. At the time this bothered me, but now it seems amusingly accurate, for reasons I was about to discover.

I applied to 3 grad schools: MIT and Yale, which were renowned for AI at the time, and Harvard, which I'd visited because Rich Draves went there, and was also home to Bill Woods, who'd invented the type of parser I used in my SHRDLU clone. Only Harvard accepted me, so that was where I went.

I don't remember the moment it happened, or if there even was a specific moment, but during the first year of grad school I realized that AI, as practiced at the time, was a hoax. By which I mean the sort of AI in which a program that's told "the dog is sitting on the chair" translates this into some formal representation and adds it to the list of things it knows.

What these programs really showed was that there's a subset of natural language that's a formal language. But a very proper subset. It was clear that there was an unbridgeable gap between what they could do and actually understanding natural language. It was not, in fact, simply a matter of teaching SHRDLU more words. That whole way of doing AI, with explicit data structures representing concepts, was not going to work. Its brokenness did, as so often happens, generate a lot of opportunities to write papers about various band-aids that could be applied to it, but it was never going to get us Mike.

So I looked around to see what I could salvage from the wreckage of my plans, and there was Lisp. I knew from experience that Lisp was interesting for its own sake and not just for its association with AI, even though that was the main reason people cared about it at the time. So I decided to focus on Lisp. In fact, I decided to write a book about Lisp hacking. It's scary to think how little I knew about Lisp hacking when I started writing that book. But there's nothing like

writing a book about something to help you learn it. The book, *On Lisp*, wasn't published till 1993, but I wrote much of it in grad school.

Computer Science is an uneasy alliance between two halves, theory and systems. The theory people prove things, and the systems people build things. I wanted to build things. I had plenty of respect for theory — indeed, a sneaking suspicion that it was the more admirable of the two halves — but building things seemed so much more exciting.

The problem with systems work, though, was that it didn't last. Any program you wrote today, no matter how good, would be obsolete in a couple decades at best. People might mention your software in footnotes, but no one would actually use it. And indeed, it would seem very feeble work. Only people with a sense of the history of the field would even realize that, in its time, it had been good.

There were some surplus Xerox Dandelions floating around the computer lab at one point. Anyone who wanted one to play around with could have one. I was briefly tempted, but they were so slow by present standards; what was the point? No one else wanted one either, so off they went. That was what happened to systems work.

I wanted not just to build things, but to build things that would last.

In this dissatisfied state I went in 1988 to visit Rich Draves at CMU, where he was in grad school. One day I went to visit the Carnegie Institute, where I'd spent a lot of time as a kid. While looking at a painting there I realized something that might seem obvious, but was a big surprise to me. There, right on the wall, was something you could make that would last. Paintings didn't become obsolete. Some of the best ones were hundreds of years old.

And moreover this was something you could make a living doing. Not as easily as you could by writing software, of course, but I thought if you were really industrious and lived really cheaply, it had to be possible to make enough to survive. And as an artist you could be truly independent. You wouldn't have a boss, or even need to get research funding.

I had always liked looking at paintings. Could I make them? I had no idea. I'd never imagined it was even possible. I knew intellectually that people made art — that it didn't just appear spontaneously — but it was as if the people who made it were a different species. They either lived long ago or were mysterious geniuses doing strange things in profiles in *Life* magazine. The idea of actually being able to make art, to put that verb before that noun, seemed almost miraculous.

That fall I started taking art classes at Harvard. Grad students could take classes

in any department, and my advisor, Tom Cheatham, was very easy going. If he even knew about the strange classes I was taking, he never said anything.

So now I was in a PhD program in computer science, yet planning to be an artist, yet also genuinely in love with Lisp hacking and working away at *On Lisp*. In other words, like many a grad student, I was working energetically on multiple projects that were not my thesis.

I didn't see a way out of this situation. I didn't want to drop out of grad school, but how else was I going to get out? I remember when my friend Robert Morris got kicked out of Cornell for writing the internet worm of 1988, I was envious that he'd found such a spectacular way to get out of grad school.

Then one day in April 1990 a crack appeared in the wall. I ran into professor Cheatham and he asked if I was far enough along to graduate that June. I didn't have a word of my dissertation written, but in what must have been the quickest bit of thinking in my life, I decided to take a shot at writing one in the 5 weeks or so that remained before the deadline, reusing parts of *On Lisp* where I could, and I was able to respond, with no perceptible delay "Yes, I think so. I'll give you something to read in a few days."

I picked applications of continuations as the topic. In retrospect I should have written about macros and embedded languages. There's a whole world there that's barely been explored. But all I wanted was to get out of grad school, and my rapidly written dissertation sufficed, just barely.

Meanwhile I was applying to art schools. I applied to two: RISD in the US, and the Accademia di Belli Arti in Florence, which, because it was the oldest art school, I imagined would be good. RISD accepted me, and I never heard back from the Accademia, so off to Providence I went.

I'd applied for the BFA program at RISD, which meant in effect that I had to go to college again. This was not as strange as it sounds, because I was only 25, and art schools are full of people of different ages. RISD counted me as a transfer sophomore and said I had to do the foundation that summer. The foundation means the classes that everyone has to take in fundamental subjects like drawing, color, and design.

Toward the end of the summer I got a big surprise: a letter from the Accademia, which had been delayed because they'd sent it to Cambridge England instead of Cambridge Massachusetts, inviting me to take the entrance exam in Florence that fall. This was now only weeks away. My nice landlady let me leave my stuff in her attic. I had some money saved from consulting work I'd done in grad school; there was probably enough to last a year if I lived cheaply. Now all I had to do was learn Italian.

Only *stranieri* (foreigners) had to take this entrance exam. In retrospect it may well have been a way of excluding them, because there were so many *stranieri* attracted by the idea of studying art in Florence that the Italian students would otherwise have been outnumbered. I was in decent shape at painting and drawing from the RISD foundation that summer, but I still don't know how I managed to pass the written exam. I remember that I answered the essay question by writing about Cezanne, and that I cranked up the intellectual level as high as I could to make the most of my limited vocabulary. [2]

I'm only up to age 25 and already there are such conspicuous patterns. Here I was, yet again about to attend some august institution in the hopes of learning about some prestigious subject, and yet again about to be disappointed. The students and faculty in the painting department at the Accademia were the nicest people you could imagine, but they had long since arrived at an arrangement whereby the students wouldn't require the faculty to teach anything, and in return the faculty wouldn't require the students to learn anything. And at the same time all involved would adhere outwardly to the conventions of a 19th century atelier. We actually had one of those little stoves, fed with kindling, that you see in 19th century studio paintings, and a nude model sitting as close to it as possible without getting burned. Except hardly anyone else painted her besides me. The rest of the students spent their time chatting or occasionally trying to imitate things they'd seen in American art magazines.

Our model turned out to live just down the street from me. She made a living from a combination of modelling and making fakes for a local antique dealer. She'd copy an obscure old painting out of a book, and then he'd take the copy and maltreat it to make it look old. [3]

While I was a student at the Accademia I started painting still lifes in my bedroom at night. These paintings were tiny, because the room was, and because I painted them on leftover scraps of canvas, which was all I could afford at the time. Painting still lifes is different from painting people, because the subject, as its name suggests, can't move. People can't sit for more than about 15 minutes at a time, and when they do they don't sit very still. So the traditional m.o. for painting people is to know how to paint a generic person, which you then modify to match the specific person you're painting. Whereas a still life you can, if you want, copy pixel by pixel from what you're seeing. You don't want to stop there, of course, or you get merely photographic accuracy, and what makes a still life interesting is that it's been through a head. You want to emphasize the visual cues that tell you, for example, that the reason the color changes suddenly at a certain point is that it's the edge of an object. By subtly emphasizing such things you can make paintings that are more realistic than photographs not just

in some metaphorical sense, but in the strict information-theoretic sense. [4]

I liked painting still lifes because I was curious about what I was seeing. In everyday life, we aren't consciously aware of much we're seeing. Most visual perception is handled by low-level processes that merely tell your brain "that's a water droplet" without telling you details like where the lightest and darkest points are, or "that's a bush" without telling you the shape and position of every leaf. This is a feature of brains, not a bug. In everyday life it would be distracting to notice every leaf on every bush. But when you have to paint something, you have to look more closely, and when you do there's a lot to see. You can still be noticing new things after days of trying to paint something people usually take for granted, just as you can after days of trying to write an essay about something people usually take for granted.

This is not the only way to paint. I'm not 100% sure it's even a good way to paint. But it seemed a good enough bet to be worth trying.

Our teacher, professor Ulivi, was a nice guy. He could see I worked hard, and gave me a good grade, which he wrote down in a sort of passport each student had. But the Accademia wasn't teaching me anything except Italian, and my money was running out, so at the end of the first year I went back to the US.

I wanted to go back to RISD, but I was now broke and RISD was very expensive, so I decided to get a job for a year and then return to RISD the next fall. I got one at a company called Interleaf, which made software for creating documents. You mean like Microsoft Word? Exactly. That was how I learned that low end software tends to eat high end software. But Interleaf still had a few years to live yet. [5]

Interleaf had done something pretty bold. Inspired by Emacs, they'd added a scripting language, and even made the scripting language a dialect of Lisp. Now they wanted a Lisp hacker to write things in it. This was the closest thing I've had to a normal job, and I hereby apologize to my boss and coworkers, because I was a bad employee. Their Lisp was the thinnest icing on a giant C cake, and since I didn't know C and didn't want to learn it, I never understood most of the software. Plus I was terribly irresponsible. This was back when a programming job meant showing up every day during certain working hours. That seemed unnatural to me, and on this point the rest of the world is coming around to my way of thinking, but at the time it caused a lot of friction. Toward the end of the year I spent much of my time surreptitiously working on *On Lisp*, which I had by this time gotten a contract to publish.

The good part was that I got paid huge amounts of money, especially by art student standards. In Florence, after paying my part of the rent, my budget for

everything else had been \$7 a day. Now I was getting paid more than 4 times that every hour, even when I was just sitting in a meeting. By living cheaply I not only managed to save enough to go back to RISD, but also paid off my college loans.

I learned some useful things at Interleaf, though they were mostly about what not to do. I learned that it's better for technology companies to be run by product people than sales people (though sales is a real skill and people who are good at it are really good at it), that it leads to bugs when code is edited by too many people, that cheap office space is no bargain if it's depressing, that planned meetings are inferior to corridor conversations, that big, bureaucratic customers are a dangerous source of money, and that there's not much overlap between conventional office hours and the optimal time for hacking, or conventional offices and the optimal place for it.

But the most important thing I learned, and which I used in both Viaweb and Y Combinator, is that the low end eats the high end: that it's good to be the "entry level" option, even though that will be less prestigious, because if you're not, someone else will be, and will squash you against the ceiling. Which in turn means that prestige is a danger sign.

When I left to go back to RISD the next fall, I arranged to do freelance work for the group that did projects for customers, and this was how I survived for the next several years. When I came back to visit for a project later on, someone told me about a new thing called HTML, which was, as he described it, a derivative of SGML. Markup language enthusiasts were an occupational hazard at Interleaf and I ignored him, but this HTML thing later became a big part of my life.

In the fall of 1992 I moved back to Providence to continue at RISD. The foundation had merely been intro stuff, and the Accademia had been a (very civilized) joke. Now I was going to see what real art school was like. But alas it was more like the Accademia than not. Better organized, certainly, and a lot more expensive, but it was now becoming clear that art school did not bear the same relationship to art that medical school bore to medicine. At least not the painting department. The textile department, which my next door neighbor belonged to, seemed to be pretty rigorous. No doubt illustration and architecture were too. But painting was post-rigorous. Painting students were supposed to express themselves, which to the more worldly ones meant to try to cook up some sort of distinctive signature style.

A signature style is the visual equivalent of what in show business is known as a "schtick": something that immediately identifies the work as yours and no one else's. For example, when you see a painting that looks like a certain kind

of cartoon, you know it's by Roy Lichtenstein. So if you see a big painting of this type hanging in the apartment of a hedge fund manager, you know he paid millions of dollars for it. That's not always why artists have a signature style, but it's usually why buyers pay a lot for such work. [6]

There were plenty of earnest students too: kids who "could draw" in high school, and now had come to what was supposed to be the best art school in the country, to learn to draw even better. They tended to be confused and demoralized by what they found at RISD, but they kept going, because painting was what they did. I was not one of the kids who could draw in high school, but at RISD I was definitely closer to their tribe than the tribe of signature style seekers.

I learned a lot in the color class I took at RISD, but otherwise I was basically teaching myself to paint, and I could do that for free. So in 1993 I dropped out. I hung around Providence for a bit, and then my college friend Nancy Parmet did me a big favor. A rent-controlled apartment in a building her mother owned in New York was becoming vacant. Did I want it? It wasn't much more than my current place, and New York was supposed to be where the artists were. So yes, I wanted it! [7]

Asterix comics begin by zooming in on a tiny corner of Roman Gaul that turns out not to be controlled by the Romans. You can do something similar on a map of New York City: if you zoom in on the Upper East Side, there's a tiny corner that's not rich, or at least wasn't in 1993. It's called Yorkville, and that was my new home. Now I was a New York artist — in the strictly technical sense of making paintings and living in New York.

I was nervous about money, because I could sense that Interleaf was on the way down. Freelance Lisp hacking work was very rare, and I didn't want to have to program in another language, which in those days would have meant C++ if I was lucky. So with my unerring nose for financial opportunity, I decided to write another book on Lisp. This would be a popular book, the sort of book that could be used as a textbook. I imagined myself living frugally off the royalties and spending all my time painting. (The painting on the cover of this book, *ANSI Common Lisp*, is one that I painted around this time.)

The best thing about New York for me was the presence of Idelle and Julian Weber. Idelle Weber was a painter, one of the early photorealists, and I'd taken her painting class at Harvard. I've never known a teacher more beloved by her students. Large numbers of former students kept in touch with her, including me. After I moved to New York I became her de facto studio assistant.

She liked to paint on big, square canvases, 4 to 5 feet on a side. One day in late 1994 as I was stretching one of these monsters there was something on the radio

about a famous fund manager. He wasn't that much older than me, and was super rich. The thought suddenly occurred to me: why don't I become rich? Then I'll be able to work on whatever I want.

Meanwhile I'd been hearing more and more about this new thing called the World Wide Web. Robert Morris showed it to me when I visited him in Cambridge, where he was now in grad school at Harvard. It seemed to me that the web would be a big deal. I'd seen what graphical user interfaces had done for the popularity of microcomputers. It seemed like the web would do the same for the internet.

If I wanted to get rich, here was the next train leaving the station. I was right about that part. What I got wrong was the idea. I decided we should start a company to put art galleries online. I can't honestly say, after reading so many Y Combinator applications, that this was the worst startup idea ever, but it was up there. Art galleries didn't want to be online, and still don't, not the fancy ones. That's not how they sell. I wrote some software to generate web sites for galleries, and Robert wrote some to resize images and set up an http server to serve the pages. Then we tried to sign up galleries. To call this a difficult sale would be an understatement. It was difficult to give away. A few galleries let us make sites for them for free, but none paid us.

Then some online stores started to appear, and I realized that except for the order buttons they were identical to the sites we'd been generating for galleries. This impressive-sounding thing called an "internet storefront" was something we already knew how to build.

So in the summer of 1995, after I submitted the camera-ready copy of *ANSI Common Lisp* to the publishers, we started trying to write software to build online stores. At first this was going to be normal desktop software, which in those days meant Windows software. That was an alarming prospect, because neither of us knew how to write Windows software or wanted to learn. We lived in the Unix world. But we decided we'd at least try writing a prototype store builder on Unix. Robert wrote a shopping cart, and I wrote a new site generator for stores — in Lisp, of course.

We were working out of Robert's apartment in Cambridge. His roommate was away for big chunks of time, during which I got to sleep in his room. For some reason there was no bed frame or sheets, just a mattress on the floor. One morning as I was lying on this mattress I had an idea that made me sit up like a capital L. What if we ran the software on the server, and let users control it by clicking on links? Then we'd never have to write anything to run on users' computers. We could generate the sites on the same server we'd serve them from. Users wouldn't need anything more than a browser.

This kind of software, known as a web app, is common now, but at the time it wasn't clear that it was even possible. To find out, we decided to try making a version of our store builder that you could control through the browser. A couple days later, on August 12, we had one that worked. The UI was horrible, but it proved you could build a whole store through the browser, without any client software or typing anything into the command line on the server.

Now we felt like we were really onto something. I had visions of a whole new generation of software working this way. You wouldn't need versions, or ports, or any of that crap. At Interleaf there had been a whole group called Release Engineering that seemed to be at least as big as the group that actually wrote the software. Now you could just update the software right on the server.

We started a new company we called Viaweb, after the fact that our software worked via the web, and we got \$10,000 in seed funding from Idelle's husband Julian. In return for that and doing the initial legal work and giving us business advice, we gave him 10% of the company. Ten years later this deal became the model for Y Combinator's. We knew founders needed something like this, because we'd needed it ourselves.

At this stage I had a negative net worth, because the thousand dollars or so I had in the bank was more than counterbalanced by what I owed the government in taxes. (Had I diligently set aside the proper proportion of the money I'd made consulting for Interleaf? No, I had not.) So although Robert had his graduate student stipend, I needed that seed funding to live on.

We originally hoped to launch in September, but we got more ambitious about the software as we worked on it. Eventually we managed to build a WYSIWYG site builder, in the sense that as you were creating pages, they looked exactly like the static ones that would be generated later, except that instead of leading to static pages, the links all referred to closures stored in a hash table on the server.

It helped to have studied art, because the main goal of an online store builder is to make users look legit, and the key to looking legit is high production values. If you get page layouts and fonts and colors right, you can make a guy running a store out of his bedroom look more legit than a big company.

(If you're curious why my site looks so old-fashioned, it's because it's still made with this software. It may look clunky today, but in 1996 it was the last word in slick.)

In September, Robert rebelled. "We've been working on this for a month," he said, "and it's still not done." This is funny in retrospect, because he would still be working on it almost 3 years later. But I decided it might be prudent to recruit more programmers, and I asked Robert who else in grad school with him

was really good. He recommended Trevor Blackwell, which surprised me at first, because at that point I knew Trevor mainly for his plan to reduce everything in his life to a stack of notecards, which he carried around with him. But Rtm was right, as usual. Trevor turned out to be a frighteningly effective hacker.

It was a lot of fun working with Robert and Trevor. They're the two most *independent-minded* people I know, and in completely different ways. If you could see inside Rtm's brain it would look like a colonial New England church, and if you could see inside Trevor's it would look like the worst excesses of Austrian Rococo.

We opened for business, with 6 stores, in January 1996. It was just as well we waited a few months, because although we worried we were late, we were actually almost fatally early. There was a lot of talk in the press then about ecommerce, but not many people actually wanted online stores. [8]

There were three main parts to the software: the editor, which people used to build sites and which I wrote, the shopping cart, which Robert wrote, and the manager, which kept track of orders and statistics, and which Trevor wrote. In its time, the editor was one of the best general-purpose site builders. I kept the code tight and didn't have to integrate with any other software except Robert's and Trevor's, so it was quite fun to work on. If all I'd had to do was work on this software, the next 3 years would have been the easiest of my life. Unfortunately I had to do a lot more, all of it stuff I was worse at than programming, and the next 3 years were instead the most stressful.

There were a lot of startups making ecommerce software in the second half of the 90s. We were determined to be the Microsoft Word, not the Interleaf. Which meant being easy to use and inexpensive. It was lucky for us that we were poor, because that caused us to make Viaweb even more inexpensive than we realized. We charged \$100 a month for a small store and \$300 a month for a big one. This low price was a big attraction, and a constant thorn in the sides of competitors, but it wasn't because of some clever insight that we set the price low. We had no idea what businesses paid for things. \$300 a month seemed like a lot of money to us.

We did a lot of things right by accident like that. For example, we did what's now called "doing things that *don't scale*," although at the time we would have described it as "being so lame that we're driven to the most desperate measures to get users." The most common of which was building stores for them. This seemed particularly humiliating, since the whole *raison d'être* of our software was that people could use it to make their own stores. But anything to get users.

We learned a lot more about retail than we wanted to know. For example, that

if you could only have a small image of a man's shirt (and all images were small then by present standards), it was better to have a closeup of the collar than a picture of the whole shirt. The reason I remember learning this was that it meant I had to rescan about 30 images of men's shirts. My first set of scans were so beautiful too.

Though this felt wrong, it was exactly the right thing to be doing. Building stores for users taught us about retail, and about how it felt to use our software. I was initially both mystified and repelled by "business" and thought we needed a "business person" to be in charge of it, but once we started to get users, I was converted, in much the same way I was converted to *fatherhood* once I had kids. Whatever users wanted, I was all theirs. Maybe one day we'd have so many users that I couldn't scan their images for them, but in the meantime there was nothing more important to do.

Another thing I didn't get at the time is that *growth rate* is the ultimate test of a startup. Our growth rate was fine. We had about 70 stores at the end of 1996 and about 500 at the end of 1997. I mistakenly thought the thing that mattered was the absolute number of users. And that is the thing that matters in the sense that that's how much money you're making, and if you're not making enough, you might go out of business. But in the long term the growth rate takes care of the absolute number. If we'd been a startup I was advising at Y Combinator, I would have said: Stop being so stressed out, because you're doing fine. You're growing 7x a year. Just don't hire too many more people and you'll soon be profitable, and then you'll control your own destiny.

Alas I hired lots more people, partly because our investors wanted me to, and partly because that's what startups did during the Internet Bubble. A company with just a handful of employees would have seemed amateurish. So we didn't reach breakeven until about when Yahoo bought us in the summer of 1998. Which in turn meant we were at the mercy of investors for the entire life of the company. And since both we and our investors were noobs at startups, the result was a mess even by startup standards.

It was a huge relief when Yahoo bought us. In principle our Viaweb stock was valuable. It was a share in a business that was profitable and growing rapidly. But it didn't feel very valuable to me; I had no idea how to value a business, but I was all too keenly aware of the near-death experiences we seemed to have every few months. Nor had I changed my grad student lifestyle significantly since we started. So when Yahoo bought us it felt like going from rags to riches. Since we were going to California, I bought a car, a yellow 1998 VW GTI. I remember thinking that its leather seats alone were by far the most luxurious thing I owned.

The next year, from the summer of 1998 to the summer of 1999, must have been the least productive of my life. I didn't realize it at the time, but I was worn out from the effort and stress of running Viaweb. For a while after I got to California I tried to continue my usual m.o. of programming till 3 in the morning, but fatigue combined with Yahoo's prematurely aged *culture* and grim cube farm in Santa Clara gradually dragged me down. After a few months it felt disconcertingly like working at Interleaf.

Yahoo had given us a lot of options when they bought us. At the time I thought Yahoo was so overvalued that they'd never be worth anything, but to my astonishment the stock went up 5x in the next year. I hung on till the first chunk of options vested, then in the summer of 1999 I left. It had been so long since I'd painted anything that I'd half forgotten why I was doing this. My brain had been entirely full of software and men's shirts for 4 years. But I had done this to get rich so I could paint, I reminded myself, and now I was rich, so I should go paint.

When I said I was leaving, my boss at Yahoo had a long conversation with me about my plans. I told him all about the kinds of pictures I wanted to paint. At the time I was touched that he took such an interest in me. Now I realize it was because he thought I was lying. My options at that point were worth about \$2 million a month. If I was leaving that kind of money on the table, it could only be to go and start some new startup, and if I did, I might take people with me. This was the height of the Internet Bubble, and Yahoo was ground zero of it. My boss was at that moment a billionaire. Leaving then to start a new startup must have seemed to him an insanely, and yet also plausibly, ambitious plan.

But I really was quitting to paint, and I started immediately. There was no time to lose. I'd already burned 4 years getting rich. Now when I talk to founders who are leaving after selling their companies, my advice is always the same: take a vacation. That's what I should have done, just gone off somewhere and done nothing for a month or two, but the idea never occurred to me.

So I tried to paint, but I just didn't seem to have any energy or ambition. Part of the problem was that I didn't know many people in California. I'd compounded this problem by buying a house up in the Santa Cruz Mountains, with a beautiful view but miles from anywhere. I stuck it out for a few more months, then in desperation I went back to New York, where unless you understand about rent control you'll be surprised to hear I still had my apartment, sealed up like a tomb of my old life. Idelle was in New York at least, and there were other people trying to paint there, even though I didn't know any of them.

When I got back to New York I resumed my old life, except now I was rich. It

was as weird as it sounds. I resumed all my old patterns, except now there were doors where there hadn't been. Now when I was tired of walking, all I had to do was raise my hand, and (unless it was raining) a taxi would stop to pick me up. Now when I walked past charming little restaurants I could go in and order lunch. It was exciting for a while. Painting started to go better. I experimented with a new kind of still life where I'd paint one painting in the old way, then photograph it and print it, blown up, on canvas, and then use that as the underpainting for a second still life, painted from the same objects (which hopefully hadn't rotted yet).

Meanwhile I looked for an apartment to buy. Now I could actually choose what neighborhood to live in. Where, I asked myself and various real estate agents, is the Cambridge of New York? Aided by occasional visits to actual Cambridge, I gradually realized there wasn't one. Huh.

Around this time, in the spring of 2000, I had an idea. It was clear from our experience with Viaweb that web apps were the future. Why not build a web app for making web apps? Why not let people edit code on our server through the browser, and then host the resulting applications for them? [9] You could run all sorts of services on the servers that these applications could use just by making an API call: making and receiving phone calls, manipulating images, taking credit card payments, etc.

I got so excited about this idea that I couldn't think about anything else. It seemed obvious that this was the future. I didn't particularly want to start another company, but it was clear that this idea would have to be embodied as one, so I decided to move to Cambridge and start it. I hoped to lure Robert into working on it with me, but there I ran into a hitch. Robert was now a postdoc at MIT, and though he'd made a lot of money the last time I'd lured him into working on one of my schemes, it had also been a huge time sink. So while he agreed that it sounded like a plausible idea, he firmly refused to work on it.

Hmph. Well, I'd do it myself then. I recruited Dan Giffin, who had worked for Viaweb, and two undergrads who wanted summer jobs, and we got to work trying to build what it's now clear is about twenty companies and several open source projects worth of software. The language for defining applications would of course be a dialect of Lisp. But I wasn't so naive as to assume I could spring an overt Lisp on a general audience; we'd hide the parentheses, like Dylan did.

By then there was a name for the kind of company Viaweb was, an "application service provider," or ASP. This name didn't last long before it was replaced by "software as a service," but it was current for long enough that I named this new company after it: it was going to be called Aspra.

I started working on the application builder, Dan worked on network infrastructure, and the two undergrads worked on the first two services (images and phone calls). But about halfway through the summer I realized I really didn't want to run a company — especially not a big one, which it was looking like this would have to be. I'd only started Viaweb because I needed the money. Now that I didn't need money anymore, why was I doing this? If this vision had to be realized as a company, then screw the vision. I'd build a subset that could be done as an open source project.

Much to my surprise, the time I spent working on this stuff was not wasted after all. After we started Y Combinator, I would often encounter startups working on parts of this new architecture, and it was very useful to have spent so much time thinking about it and even trying to write some of it.

The subset I would build as an open source project was the new Lisp, whose parentheses I now wouldn't even have to hide. A lot of Lisp hackers dream of building a new Lisp, partly because one of the distinctive features of the language is that it has dialects, and partly, I think, because we have in our minds a Platonic form of Lisp that all existing dialects fall short of. I certainly did. So at the end of the summer Dan and I switched to working on this new dialect of Lisp, which I called Arc, in a house I bought in Cambridge.

The following spring, lightning struck. I was invited to give a talk at a Lisp conference, so I gave one about how we'd used Lisp at Viaweb. Afterward I put a postscript file of this talk online, on paulgraham.com, which I'd created years before using Viaweb but had never used for anything. In one day it got 30,000 page views. What on earth had happened? The referring urls showed that someone had posted it on Slashdot. [10]

Wow, I thought, there's an audience. If I write something and put it on the web, anyone can read it. That may seem obvious now, but it was surprising then. In the print era there was a narrow channel to readers, guarded by fierce monsters known as editors. The only way to get an audience for anything you wrote was to get it published as a book, or in a newspaper or magazine. Now anyone could publish anything.

This had been possible in principle since 1993, but not many people had realized it yet. I had been intimately involved with building the infrastructure of the web for most of that time, and a writer as well, and it had taken me 8 years to realize it. Even then it took me several years to understand the implications. It meant there would be a whole new generation of *essays*. [11]

In the print era, the channel for publishing essays had been vanishingly small. Except for a few officially anointed thinkers who went to the right parties in

New York, the only people allowed to publish essays were specialists writing about their specialties. There were so many essays that had never been written, because there had been no way to publish them. Now they could be, and I was going to write them. [12]

I've worked on several different things, but to the extent there was a turning point where I figured out what to work on, it was when I started publishing essays online. From then on I knew that whatever else I did, I'd always write essays too.

I knew that online essays would be a *marginal* medium at first. Socially they'd seem more like rants posted by nutjobs on their GeoCities sites than the genteel and beautifully typeset compositions published in *The New Yorker*. But by this point I knew enough to find that encouraging instead of discouraging.

One of the most conspicuous patterns I've noticed in my life is how well it has worked, for me at least, to work on things that weren't prestigious. Still life has always been the least prestigious form of painting. Viaweb and Y Combinator both seemed lame when we started them. I still get the glassy eye from strangers when they ask what I'm writing, and I explain that it's an essay I'm going to publish on my web site. Even Lisp, though prestigious intellectually in something like the way Latin is, also seems about as hip.

It's not that unprestigious types of work are good per se. But when you find yourself drawn to some kind of work despite its current lack of prestige, it's a sign both that there's something real to be discovered there, and that you have the right kind of motives. Impure motives are a big danger for the ambitious. If anything is going to lead you astray, it will be the desire to impress people. So while working on things that aren't prestigious doesn't guarantee you're on the right track, it at least guarantees you're not on the most common type of wrong one.

Over the next several years I wrote lots of essays about all kinds of different topics. O'Reilly reprinted a collection of them as a book, called *Hackers & Painters* after one of the essays in it. I also worked on spam filters, and did some more painting. I used to have dinners for a group of friends every thursday night, which taught me how to cook for groups. And I bought another building in Cambridge, a former candy factory (and later, twas said, porn studio), to use as an office.

One night in October 2003 there was a big party at my house. It was a clever idea of my friend Maria Daniels, who was one of the thursday diners. Three separate hosts would all invite their friends to one party. So for every guest, two thirds of the other guests would be people they didn't know but would probably

like. One of the guests was someone I didn't know but would turn out to like a lot: a woman called Jessica Livingston. A couple days later I asked her out.

Jessica was in charge of marketing at a Boston investment bank. This bank thought it understood startups, but over the next year, as she met friends of mine from the startup world, she was surprised how different reality was. And how colorful their stories were. So she decided to compile a book of *interviews* with startup founders.

When the bank had financial problems and she had to fire half her staff, she started looking for a new job. In early 2005 she interviewed for a marketing job at a Boston VC firm. It took them weeks to make up their minds, and during this time I started telling her about all the things that needed to be fixed about venture capital. They should make a larger number of smaller investments instead of a handful of giant ones, they should be funding younger, more technical founders instead of MBAs, they should let the founders remain as CEO, and so on.

One of my tricks for writing essays had always been to give talks. The prospect of having to stand up in front of a group of people and tell them something that won't waste their time is a great spur to the imagination. When the Harvard Computer Society, the undergrad computer club, asked me to give a talk, I decided I would tell them how to start a startup. Maybe they'd be able to avoid the worst of the mistakes we'd made.

So I gave this talk, in the course of which I told them that the best sources of seed funding were successful startup founders, because then they'd be sources of advice too. Whereupon it seemed they were all looking expectantly at me. Horrified at the prospect of having my inbox flooded by business plans (if I'd only known), I blurted out "But not me!" and went on with the talk. But afterward it occurred to me that I should really stop procrastinating about angel investing. I'd been meaning to since Yahoo bought us, and now it was 7 years later and I still hadn't done one angel investment.

Meanwhile I had been scheming with Robert and Trevor about projects we could work on together. I missed working with them, and it seemed like there had to be something we could collaborate on.

As Jessica and I were walking home from dinner on March 11, at the corner of Garden and Walker streets, these three threads converged. Screw the VCs who were taking so long to make up their minds. We'd start our own investment firm and actually implement the ideas we'd been talking about. I'd fund it, and Jessica could quit her job and work for it, and we'd get Robert and Trevor as partners too. [13]

Once again, ignorance worked in our favor. We had no idea how to be angel investors, and in Boston in 2005 there were no Ron Conways to learn from. So we just made what seemed like the obvious choices, and some of the things we did turned out to be novel.

There are multiple components to Y Combinator, and we didn't figure them all out at once. The part we got first was to be an angel firm. In those days, those two words didn't go together. There were VC firms, which were organized companies with people whose job it was to make investments, but they only did big, million dollar investments. And there were angels, who did smaller investments, but these were individuals who were usually focused on other things and made investments on the side. And neither of them helped founders enough in the beginning. We knew how helpless founders were in some respects, because we remembered how helpless we'd been. For example, one thing Julian had done for us that seemed to us like magic was to get us set up as a company. We were fine writing fairly difficult software, but actually getting incorporated, with bylaws and stock and all that stuff, how on earth did you do that? Our plan was not only to make seed investments, but to do for startups everything Julian had done for us.

YC was not organized as a fund. It was cheap enough to run that we funded it with our own money. That went right by 99% of readers, but professional investors are thinking "Wow, that means they got all the returns." But once again, this was not due to any particular insight on our part. We didn't know how VC firms were organized. It never occurred to us to try to raise a fund, and if it had, we wouldn't have known where to start. [14]

The most distinctive thing about YC is the batch model: to fund a bunch of startups all at once, twice a year, and then to spend three months focusing intensively on trying to help them. That part we discovered by accident, not merely implicitly but explicitly due to our ignorance about investing. We needed to get experience as investors. What better way, we thought, than to fund a whole bunch of startups at once? We knew undergrads got temporary jobs at tech companies during the summer. Why not organize a summer program where they'd start startups instead? We wouldn't feel guilty for being in a sense fake investors, because they would in a similar sense be fake founders. So while we probably wouldn't make much money out of it, we'd at least get to practice being investors on them, and they for their part would probably have a more interesting summer than they would working at Microsoft.

We'd use the building I owned in Cambridge as our headquarters. We'd all have dinner there once a week — on tuesdays, since I was already cooking for the thursday diners on thursdays — and after dinner we'd bring in experts on

startups to give talks.

We knew undergrads were deciding then about summer jobs, so in a matter of days we cooked up something we called the Summer Founders Program, and I posted an *announcement* on my site, inviting undergrads to apply. I had never imagined that writing essays would be a way to get “deal flow,” as investors call it, but it turned out to be the perfect source. [15] We got 225 applications for the Summer Founders Program, and we were surprised to find that a lot of them were from people who’d already graduated, or were about to that spring. Already this SFP thing was starting to feel more serious than we’d intended.

We invited about 20 of the 225 groups to interview in person, and from those we picked 8 to fund. They were an impressive group. That first batch included reddit, Justin Kan and Emmett Shear, who went on to found Twitch, Aaron Swartz, who had already helped write the RSS spec and would a few years later become a martyr for open access, and Sam Altman, who would later become the second president of YC. I don’t think it was entirely luck that the first batch was so good. You had to be pretty bold to sign up for a weird thing like the Summer Founders Program instead of a summer job at a legit place like Microsoft or Goldman Sachs.

The deal for startups was based on a combination of the deal we did with Julian (\$10k for 10%) and what Robert said MIT grad students got for the summer (\$6k). We invested \$6k per founder, which in the typical two-founder case was \$12k, in return for 6%. That had to be fair, because it was twice as good as the deal we ourselves had taken. Plus that first summer, which was really hot, Jessica brought the founders free air conditioners. [16]

Fairly quickly I realized that we had stumbled upon the way to scale startup funding. Funding startups in batches was more convenient for us, because it meant we could do things for a lot of startups at once, but being part of a batch was better for the startups too. It solved one of the biggest problems faced by founders: the isolation. Now you not only had colleagues, but colleagues who understood the problems you were facing and could tell you how they were solving them.

As YC grew, we started to notice other advantages of scale. The alumni became a tight community, dedicated to helping one another, and especially the current batch, whose shoes they remembered being in. We also noticed that the startups were becoming one another’s customers. We used to refer jokingly to the “YC GDP,” but as YC grows this becomes less and less of a joke. Now lots of startups get their initial set of customers almost entirely from among their batchmates.

I had not originally intended YC to be a full-time job. I was going to do three

things: hack, write essays, and work on YC. As YC grew, and I grew more excited about it, it started to take up a lot more than a third of my attention. But for the first few years I was still able to work on other things.

In the summer of 2006, Robert and I started working on a new version of Arc. This one was reasonably fast, because it was compiled into Scheme. To test this new Arc, I wrote Hacker News in it. It was originally meant to be a news aggregator for startup founders and was called Startup News, but after a few months I got tired of reading about nothing but startups. Plus it wasn't startup founders we wanted to reach. It was future startup founders. So I changed the name to Hacker News and the topic to whatever engaged one's intellectual curiosity.

HN was no doubt good for YC, but it was also by far the biggest source of stress for me. If all I'd had to do was select and help founders, life would have been so easy. And that implies that HN was a mistake. Surely the biggest source of stress in one's work should at least be something close to the core of the work. Whereas I was like someone who was in pain while running a marathon not from the exertion of running, but because I had a blister from an ill-fitting shoe. When I was dealing with some urgent problem during YC, there was about a 60% chance it had to do with HN, and a 40% chance it had to do with everything else combined. [17]

As well as HN, I wrote all of YC's internal software in Arc. But while I continued to work a good deal *in* Arc, I gradually stopped working *on* Arc, partly because I didn't have time to, and partly because it was a lot less attractive to mess around with the language now that we had all this infrastructure depending on it. So now my three projects were reduced to two: writing essays and working on YC.

YC was different from other kinds of work I've done. Instead of deciding for myself what to work on, the problems came to me. Every 6 months there was a new batch of startups, and their problems, whatever they were, became our problems. It was very engaging work, because their problems were quite varied, and the good founders were very effective. If you were trying to learn the most you could about startups in the shortest possible time, you couldn't have picked a better way to do it.

There were parts of the job I didn't like. Disputes between cofounders, figuring out when people were lying to us, fighting with people who maltreated the startups, and so on. But I worked hard even at the parts I didn't like. I was haunted by something Kevin Hale once said about companies: "No one works harder than the boss." He meant it both descriptively and prescriptively, and it

was the second part that scared me. I wanted YC to be good, so if how hard I worked set the upper bound on how hard everyone else worked, I'd better work very hard.

One day in 2010, when he was visiting California for interviews, Robert Morris did something astonishing: he offered me unsolicited advice. I can only remember him doing that once before. One day at Viaweb, when I was bent over double from a kidney stone, he suggested that it would be a good idea for him to take me to the hospital. That was what it took for Rtm to offer unsolicited advice. So I remember his exact words very clearly. "You know," he said, "you should make sure Y Combinator isn't the last cool thing you do."

At the time I didn't understand what he meant, but gradually it dawned on me that he was saying I should quit. This seemed strange advice, because YC was doing great. But if there was one thing rarer than Rtm offering advice, it was Rtm being wrong. So this set me thinking. It was true that on my current trajectory, YC would be the last thing I did, because it was only taking up more of my attention. It had already eaten Arc, and was in the process of eating essays too. Either YC was my life's work or I'd have to leave eventually. And it wasn't, so I would.

In the summer of 2012 my mother had a stroke, and the cause turned out to be a blood clot caused by colon cancer. The stroke destroyed her balance, and she was put in a nursing home, but she really wanted to get out of it and back to her house, and my sister and I were determined to help her do it. I used to fly up to Oregon to visit her regularly, and I had a lot of time to think on those flights. On one of them I realized I was ready to hand YC over to someone else.

I asked Jessica if she wanted to be president, but she didn't, so we decided we'd try to recruit Sam Altman. We talked to Robert and Trevor and we agreed to make it a complete changing of the guard. Up till that point YC had been controlled by the original LLC we four had started. But we wanted YC to last for a long time, and to do that it couldn't be controlled by the founders. So if Sam said yes, we'd let him reorganize YC. Robert and I would retire, and Jessica and Trevor would become ordinary partners.

When we asked Sam if he wanted to be president of YC, initially he said no. He wanted to start a startup to make nuclear reactors. But I kept at it, and in October 2013 he finally agreed. We decided he'd take over starting with the winter 2014 batch. For the rest of 2013 I left running YC more and more to Sam, partly so he could learn the job, and partly because I was focused on my mother, whose cancer had returned.

She died on January 15, 2014. We knew this was coming, but it was still hard

when it did.

I kept working on YC till March, to help get that batch of startups through Demo Day, then I checked out pretty completely. (I still talk to alumni and to new startups working on things I'm interested in, but that only takes a few hours a week.)

What should I do next? Rtm's advice hadn't included anything about that. I wanted to do something completely different, so I decided I'd paint. I wanted to see how good I could get if I really focused on it. So the day after I stopped working on YC, I started painting. I was rusty and it took a while to get back into shape, but it was at least completely engaging. [18]

I spent most of the rest of 2014 painting. I'd never been able to work so uninterruptedly before, and I got to be better than I had been. Not good enough, but better. Then in November, right in the middle of a painting, I ran out of steam. Up till that point I'd always been curious to see how the painting I was working on would turn out, but suddenly finishing this one seemed like a chore. So I stopped working on it and cleaned my brushes and haven't painted since. So far anyway.

I realize that sounds rather wimpy. But attention is a zero sum game. If you can choose what to work on, and you choose a project that's not the best one (or at least a good one) for you, then it's getting in the way of another project that is. And at 50 there was some opportunity cost to screwing around.

I started writing essays again, and wrote a bunch of new ones over the next few months. I even wrote a couple that *weren't* about startups. Then in March 2015 I started working on Lisp again.

The distinctive thing about Lisp is that its core is a language defined by writing an interpreter in itself. It wasn't originally intended as a programming language in the ordinary sense. It was meant to be a formal model of computation, an alternative to the Turing machine. If you want to write an interpreter for a language in itself, what's the minimum set of predefined operators you need? The Lisp that John McCarthy invented, or more accurately discovered, is an answer to that question. [19]

McCarthy didn't realize this Lisp could even be used to program computers till his grad student Steve Russell suggested it. Russell translated McCarthy's interpreter into IBM 704 machine language, and from that point Lisp started also to be a programming language in the ordinary sense. But its origins as a model of computation gave it a power and elegance that other languages couldn't match. It was this that attracted me in college, though I didn't understand why at the time.

McCarthy's 1960 Lisp did nothing more than interpret Lisp expressions. It was missing a lot of things you'd want in a programming language. So these had to be added, and when they were, they weren't defined using McCarthy's original axiomatic approach. That wouldn't have been feasible at the time. McCarthy tested his interpreter by hand-simulating the execution of programs. But it was already getting close to the limit of interpreters you could test that way — indeed, there was a bug in it that McCarthy had overlooked. To test a more complicated interpreter, you'd have had to run it, and computers then weren't powerful enough.

Now they are, though. Now you could continue using McCarthy's axiomatic approach till you'd defined a complete programming language. And as long as every change you made to McCarthy's Lisp was a discoveredness-preserving transformation, you could, in principle, end up with a complete language that had this quality. Harder to do than to talk about, of course, but if it was possible in principle, why not try? So I decided to take a shot at it. It took 4 years, from March 26, 2015 to October 12, 2019. It was fortunate that I had a precisely defined goal, or it would have been hard to keep at it for so long.

I wrote this new Lisp, called *Bel*, in itself in Arc. That may sound like a contradiction, but it's an indication of the sort of trickery I had to engage in to make this work. By means of an egregious collection of hacks I managed to make something close enough to an interpreter written in itself that could actually run. Not fast, but fast enough to test.

I had to ban myself from writing essays during most of this time, or I'd never have finished. In late 2015 I spent 3 months writing essays, and when I went back to working on *Bel* I could barely understand the code. Not so much because it was badly written as because the problem is so convoluted. When you're working on an interpreter written in itself, it's hard to keep track of what's happening at what level, and errors can be practically encrypted by the time you get them.

So I said no more essays till *Bel* was done. But I told few people about *Bel* while I was working on it. So for years it must have seemed that I was doing nothing, when in fact I was working harder than I'd ever worked on anything. Occasionally after wrestling for hours with some gruesome bug I'd check Twitter or HN and see someone asking "Does Paul Graham still code?"

Working on *Bel* was hard but satisfying. I worked on it so intensively that at any given time I had a decent chunk of the code in my head and could write more there. I remember taking the boys to the coast on a sunny day in 2015 and figuring out how to deal with some problem involving continuations while I watched them play in the tide pools. It felt like I was doing life right. I remember

that because I was slightly dismayed at how novel it felt. The good news is that I had more moments like this over the next few years.

In the summer of 2016 we moved to England. We wanted our kids to see what it was like living in another country, and since I was a British citizen by birth, that seemed the obvious choice. We only meant to stay for a year, but we liked it so much that we still live there. So most of *Bel* was written in England.

In the fall of 2019, *Bel* was finally finished. Like McCarthy's original *Lisp*, it's a spec rather than an implementation, although like McCarthy's *Lisp* it's a spec expressed as code.

Now that I could write essays again, I wrote a bunch about topics I'd had stacked up. I kept writing essays through 2020, but I also started to think about other things I could work on. How should I choose what to do? Well, how had I chosen what to work on in the past? I wrote an essay for myself to answer that question, and I was surprised how long and messy the answer turned out to be. If this surprised me, who'd lived it, then I thought perhaps it would be interesting to other people, and encouraging to those with similarly messy lives. So I wrote a more detailed version for others to read, and this is the last sentence of it.

Notes

[1] My experience skipped a step in the evolution of computers: time-sharing machines with interactive OSes. I went straight from batch processing to microcomputers, which made microcomputers seem all the more exciting.

[2] Italian words for abstract concepts can nearly always be predicted from their English cognates (except for occasional traps like *polluzione*). It's the everyday words that differ. So if you string together a lot of abstract concepts with a few simple verbs, you can make a little Italian go a long way.

[3] I lived at Piazza San Felice 4, so my walk to the Accademia went straight down the spine of old Florence: past the Pitti, across the bridge, past Orsanmichele, between the Duomo and the Baptistery, and then up Via Ricasoli to Piazza San Marco. I saw Florence at street level in every possible condition, from empty dark winter evenings to sweltering summer days when the streets were packed with tourists.

[4] You can of course paint people like still lives if you want to, and they're willing. That sort of portrait is arguably the apex of still life painting, though the long sitting does tend to produce pained expressions in the sitters.

[5] Interleaf was one of many companies that had smart people and built impressive technology, and yet got crushed by Moore's Law. In the 1990s the exponential

growth in the power of commodity (i.e. Intel) processors rolled up high-end, special-purpose hardware and software companies like a bulldozer.

[6] The signature style seekers at RISD weren't specifically mercenary. In the art world, money and coolness are tightly coupled. Anything expensive comes to be seen as cool, and anything seen as cool will soon become equally expensive.

[7] Technically the apartment wasn't rent-controlled but rent-stabilized, but this is a refinement only New Yorkers would know or care about. The point is that it was really cheap, less than half market price.

[8] Most software you can launch as soon as it's done. But when the software is an online store builder and you're hosting the stores, if you don't have any users yet, that fact will be painfully obvious. So before we could launch publicly we had to launch privately, in the sense of recruiting an initial set of users and making sure they had decent-looking stores.

[9] We'd had a code editor in Viaweb for users to define their own page styles. They didn't know it, but they were editing Lisp expressions underneath. But this wasn't an app editor, because the code ran when the merchants' sites were generated, not when shoppers visited them.

[10] This was the first instance of what is now a familiar experience, and so was what happened next, when I read the comments and found they were full of angry people. How could I claim that Lisp was better than other languages? Weren't they all Turing complete? People who see the responses to essays I write sometimes tell me how sorry they feel for me, but I'm not exaggerating when I reply that it has always been like this, since the very beginning. It comes with the territory. An essay must tell readers things they *don't already know*, and some people dislike being told such things.

[11] People put plenty of stuff on the internet in the 90s of course, but putting something online is not the same as publishing it online. Publishing online means you treat the online version as the (or at least a) primary version.

[12] There is a general lesson here that our experience with Y Combinator also teaches: Customs continue to constrain you long after the restrictions that caused them have disappeared. Customary VC practice had once, like the customs about publishing essays, been based on real constraints. Startups had once been much more expensive to start, and proportionally rare. Now they could be cheap and common, but the VCs' customs still reflected the old world, just as customs about writing essays still reflected the constraints of the print era.

Which in turn implies that people who are independent-minded (i.e. less influenced by custom) will have an advantage in fields affected by rapid change (where

customs are more likely to be obsolete).

Here's an interesting point, though: you can't always predict which fields will be affected by rapid change. Obviously software and venture capital will be, but who would have predicted that essay writing would be?

[13] Y Combinator was not the original name. At first we were called Cambridge Seed. But we didn't want a regional name, in case someone copied us in Silicon Valley, so we renamed ourselves after one of the coolest tricks in the lambda calculus, the Y combinator.

I picked orange as our color partly because it's the warmest, and partly because no VC used it. In 2005 all the VCs used staid colors like maroon, navy blue, and forest green, because they were trying to appeal to LPs, not founders. The YC logo itself is an inside joke: the Viaweb logo had been a white V on a red circle, so I made the YC logo a white Y on an orange square.

[14] YC did become a fund for a couple years starting in 2009, because it was getting so big I could no longer afford to fund it personally. But after Heroku got bought we had enough money to go back to being self-funded.

[15] I've never liked the term "deal flow," because it implies that the number of new startups at any given time is fixed. This is not only false, but it's the purpose of YC to falsify it, by causing startups to be founded that would not otherwise have existed.

[16] She reports that they were all different shapes and sizes, because there was a run on air conditioners and she had to get whatever she could, but that they were all heavier than she could carry now.

[17] Another problem with HN was a bizarre edge case that occurs when you both write essays and run a forum. When you run a forum, you're assumed to see if not every conversation, at least every conversation involving you. And when you write essays, people post highly imaginative misinterpretations of them on forums. Individually these two phenomena are tedious but bearable, but the combination is disastrous. You actually have to respond to the misinterpretations, because the assumption that you're present in the conversation means that not responding to any sufficiently upvoted misinterpretation reads as a tacit admission that it's correct. But that in turn encourages more; anyone who wants to pick a fight with you senses that now is their chance.

[18] The worst thing about leaving YC was not working with Jessica anymore. We'd been working on YC almost the whole time we'd known each other, and we'd neither tried nor wanted to separate it from our personal lives, so leaving was like pulling up a deeply rooted tree.

[19] One way to get more precise about the concept of invented vs discovered is to talk about space aliens. Any sufficiently advanced alien civilization would certainly know about the Pythagorean theorem, for example. I believe, though with less certainty, that they would also know about the Lisp in McCarthy's 1960 paper.

But if so there's no reason to suppose that this is the limit of the language that might be known to them. Presumably aliens need numbers and errors and I/O too. So it seems likely there exists at least one path out of McCarthy's Lisp along which discoveredness is preserved.

Thanks to Trevor Blackwell, John Collison, Patrick Collison, Daniel Gackle, Ralph Hazell, Jessica Livingston, Robert Morris, and Harj Taggar for reading drafts of this.

A Project of One's Own

June 2021

A few days ago, on the way home from school, my nine year old son told me he couldn't wait to get home to write more of the story he was working on. This made me as happy as anything I've heard him say — not just because he was excited about his story, but because he'd discovered this way of working. Working on a project of your own is as different from ordinary work as skating is from walking. It's more fun, but also much more productive.

What proportion of great work has been done by people who were skating in this sense? If not all of it, certainly a lot.

There is something special about working on a project of your own. I wouldn't say exactly that you're happier. A better word would be excited, or engaged. You're happy when things are going well, but often they aren't. When I'm writing an essay, most of the time I'm worried and puzzled: worried that the essay will turn out badly, and puzzled because I'm groping for some idea that I can't see clearly enough. Will I be able to pin it down with words? In the end I usually can, if I take long enough, but I'm never sure; the first few attempts often fail.

You have moments of happiness when things work out, but they don't last long, because then you're on to the next problem. So why do it at all? Because to the kind of people who like working this way, nothing else feels as right. You feel as if you're an animal in its natural habitat, doing what you were meant to do — not always happy, maybe, but awake and alive.

Many kids experience the excitement of working on projects of their own. The hard part is making this converge with the work you do as an adult. And our customs make it harder. We treat “playing” and “hobbies” as qualitatively different from “work”. It's not clear to a kid building a treehouse that there's a direct (though long) route from that to architecture or engineering. And instead of pointing out the route, we conceal it, by implicitly treating the stuff kids do as different from real work. [1]

Instead of telling kids that their treehouses could be on the path to the work they do as adults, we tell them the path goes through school. And unfortunately schoolwork tends to be very different from working on projects of one's own. It's usually neither a project, nor one's own. So as school gets more serious, working on projects of one's own is something that survives, if at all, as a thin thread off to the side.

It's a bit sad to think of all the high school kids turning their backs on building treehouses and sitting in class dutifully learning about Darwin or Newton to pass some exam, when the work that made Darwin and Newton famous was actually closer in spirit to building treehouses than studying for exams.

If I had to choose between my kids getting good grades and working on ambitious projects of their own, I'd pick the projects. And not because I'm an indulgent parent, but because I've been on the other end and I know which has more predictive value. When I was picking startups for Y Combinator, I didn't care about applicants' grades. But if they'd worked on projects of their own, I wanted to hear all about those. [2]

It may be inevitable that school is the way it is. I'm not saying we have to redesign it (though I'm not saying we don't), just that we should understand what it does to our attitudes to work — that it steers us toward the dutiful plodding kind of work, often using competition as bait, and away from skating.

There are occasionally times when schoolwork becomes a project of one's own. Whenever I had to write a paper, that would become a project of my own — except in English classes, ironically, because the things one has to write in English classes are so *bogus*. And when I got to college and started taking CS classes, the programs I had to write became projects of my own. Whenever I was writing or programming, I was usually skating, and that has been true ever since.

So where exactly is the edge of projects of one's own? That's an interesting question, partly because the answer is so complicated, and partly because there's so much at stake. There turn out to be two senses in which work can be one's own: 1) that you're doing it voluntarily, rather than merely because someone told you to, and 2) that you're doing it by yourself.

The edge of the former is quite sharp. People who care a lot about their work are usually very sensitive to the difference between pulling, and being pushed, and work tends to fall into one category or the other. But the test isn't simply whether you're told to do something. You can choose to do something you're told to do. Indeed, you can own it far more thoroughly than the person who told you to do it.

For example, math homework is for most people something they're told to do.

But for my father, who was a mathematician, it wasn't. Most of us think of the problems in a math book as a way to test or develop our knowledge of the material explained in each section. But to my father the problems were the part that mattered, and the text was merely a sort of annotation. Whenever he got a new math book it was to him like being given a puzzle: here was a new set of problems to solve, and he'd immediately set about solving all of them.

The other sense of a project being one's own — working on it by oneself — has a much softer edge. It shades gradually into collaboration. And interestingly, it shades into collaboration in two different ways. One way to collaborate is to share a single project. For example, when two mathematicians collaborate on a proof that takes shape in the course of a conversation between them. The other way is when multiple people work on separate projects of their own that fit together like a jigsaw puzzle. For example, when one person writes the text of a book and another does the graphic design. [3]

These two paths into collaboration can of course be combined. But under the right conditions, the excitement of working on a project of one's own can be preserved for quite a while before disintegrating into the turbulent flow of work in a large organization. Indeed, the history of successful organizations is partly the history of techniques for preserving that excitement. [4]

The team that made the original Macintosh were a great example of this phenomenon. People like Burrell Smith and Andy Hertzfeld and Bill Atkinson and Susan Kare were not just following orders. They were not tennis balls hit by Steve Jobs, but rockets let loose by Steve Jobs. There was a lot of collaboration between them, but they all seem to have individually felt the excitement of working on a project of one's own.

In Andy Hertzfeld's book on the Macintosh, he describes how they'd come back into the office after dinner and work late into the night. People who've never experienced the thrill of working on a project they're excited about can't distinguish this kind of working long hours from the kind that happens in sweatshops and boiler rooms, but they're at opposite ends of the spectrum. That's why it's a mistake to insist dogmatically on "work/life balance." Indeed, the mere expression "work/life" embodies a mistake: it assumes work and life are distinct. For those to whom the word "work" automatically implies the dutiful plodding kind, they are. But for the skaters, the relationship between work and life would be better represented by a dash than a slash. I wouldn't want to work on anything that I didn't want to take over my life.

Of course, it's easier to achieve this level of motivation when you're making something like the Macintosh. It's easy for something new to feel like a project

of your own. That's one of the reasons for the tendency programmers have to rewrite things that don't need rewriting, and to write their own versions of things that already exist. This sometimes alarms managers, and measured by total number of characters typed, it's rarely the optimal solution. But it's not always driven simply by arrogance or cluelessness. Writing code from scratch is also much more rewarding — so much more rewarding that a good programmer can end up net ahead, despite the shocking waste of characters. Indeed, it may be one of the advantages of capitalism that it encourages such rewriting. A company that needs software to do something can't use the software already written to do it at another company, and thus has to write their own, which often turns out better. [5]

The natural alignment between skating and solving new problems is one of the reasons the payoffs from startups are so high. Not only is the market price of unsolved problems higher, you also get a discount on productivity when you work on them. In fact, you get a double increase in productivity: when you're doing a clean-sheet design, it's easier to recruit skaters, and they get to spend all their time skating.

Steve Jobs knew a thing or two about skaters from having watched Steve Wozniak. If you can find the right people, you only have to tell them what to do at the highest level. They'll handle the details. Indeed, they insist on it. For a project to feel like your own, you must have sufficient autonomy. You can't be working to order, or *slowed down* by bureaucracy.

One way to ensure autonomy is not to have a boss at all. There are two ways to do that: to be the boss yourself, and to work on projects outside of work. Though they're at opposite ends of the scale financially, startups and open source projects have a lot in common, including the fact that they're often run by skaters. And indeed, there's a wormhole from one end of the scale to the other: one of the best ways to discover *startup ideas* is to work on a project just for fun.

If your projects are the kind that make money, it's easy to work on them. It's harder when they're not. And the hardest part, usually, is morale. That's where adults have it harder than kids. Kids just plunge in and build their treehouse without worrying about whether they're wasting their time, or how it compares to other treehouses. And frankly we could learn a lot from kids here. The high standards most grownups have for "real" work do not always serve us well.

The most important phase in a project of one's own is at the beginning: when you go from thinking it might be cool to do x to actually doing x. And at that point high standards are not merely useless but positively harmful. There are a few people who start too many new projects, but far more, I suspect, who are

deterred by fear of failure from starting projects that would have succeeded if they had.

But if we couldn't benefit as kids from the knowledge that our treehouses were on the path to grownup projects, we can at least benefit as grownups from knowing that our projects are on a path that stretches back to treehouses. Remember that careless confidence you had as a kid when starting something new? That would be a powerful thing to recapture.

If it's harder as adults to retain that kind of confidence, we at least tend to be more aware of what we're doing. Kids bounce, or are herded, from one kind of work to the next, barely realizing what's happening to them. Whereas we know more about different types of work and have more control over which we do. Ideally we can have the best of both worlds: to be deliberate in choosing to work on projects of our own, and carelessly confident in starting new ones.

Notes

[1] "Hobby" is a curious word. Now it means work that isn't *real* work — work that one is not to be judged by — but originally it just meant an obsession in a fairly general sense (even a political opinion, for example) that one metaphorically rode as a child rides a hobby-horse. It's hard to say if its recent, narrower meaning is a change for the better or the worse. For sure there are lots of false positives — lots of projects that end up being important but are dismissed initially as mere hobbies. But on the other hand, the concept provides valuable cover for projects in the early, ugly duckling phase.

[2] Tiger parents, as parents so often do, are fighting the last war. Grades mattered more in the old days when the route to success was to acquire *credentials* while ascending some predefined ladder. But it's just as well that their tactics are focused on grades. How awful it would be if they invaded the territory of projects, and thereby gave their kids a distaste for this kind of work by forcing them to do it. Grades are already a grim, fake world, and aren't harmed much by parental interference, but working on one's own projects is a more delicate, private thing that could be damaged very easily.

[3] The complicated, gradual edge between working on one's own projects and collaborating with others is one reason there is so much disagreement about the idea of the "lone genius." In practice people collaborate (or not) in all kinds of different ways, but the idea of the lone genius is definitely not a myth. There's a core of truth to it that goes with a certain way of working.

[4] Collaboration is powerful too. The optimal organization would combine collaboration and ownership in such a way as to do the least damage to each. Interestingly, companies and university departments approach this ideal from

opposite directions: companies insist on collaboration, and occasionally also manage both to recruit skaters and allow them to skate, and university departments insist on the ability to do independent research (which is by custom treated as skating, whether it is or not), and the people they hire collaborate as much as they choose.

[5] If a company could design its software in such a way that the best newly arrived programmers always got a clean sheet, it could have a kind of eternal youth. That might not be impossible. If you had a software backbone defining a game with sufficiently clear rules, individual programmers could write their own players.

Thanks to Trevor Blackwell, Paul Buchheit, Andy Hertzfeld, Jessica Livingston, and Peter Norvig for reading drafts of this.

How to Work Hard

June 2021

It might not seem there's much to learn about how to work hard. Anyone who's been to school knows what it entails, even if they chose not to do it. There are 12 year olds who work amazingly hard. And yet when I ask if I know more about working hard now than when I was in school, the answer is definitely yes.

One thing I know is that if you want to do great things, you'll have to work very hard. I wasn't sure of that as a kid. Schoolwork varied in difficulty; one didn't always have to work super hard to do well. And some of the things famous adults did, they seemed to do almost effortlessly. Was there, perhaps, some way to evade hard work through sheer brilliance? Now I know the answer to that question. There isn't.

The reason some subjects seemed easy was that my school had low standards. And the reason famous adults seemed to do things effortlessly was years of practice; they made it look easy.

Of course, those famous adults usually had a lot of natural ability too. There are three ingredients in great work: natural ability, practice, and effort. You can do pretty well with just two, but to do the best work you need all three: you need great natural ability *and* to have practiced a lot *and* to be trying very hard. [1]

Bill Gates, for example, was among the smartest people in business in his era, but he was also among the hardest working. "I never took a day off in my twenties," he said. "Not one." It was similar with Lionel Messi. He had great natural ability, but when his youth coaches talk about him, what they remember is not his talent but his dedication and his desire to win. P. G. Wodehouse would probably get my vote for best English writer of the 20th century, if I had to choose. Certainly no one ever made it look easier. But no one ever worked harder. At 74, he wrote

with each new book of mine I have, as I say, the feeling that this time I have picked a lemon in the garden of literature. A good thing,

really, I suppose. Keeps one up on one's toes and makes one rewrite every sentence ten times. Or in many cases twenty times.

Sounds a bit extreme, you think. And yet Bill Gates sounds even more extreme. Not one day off in ten years? These two had about as much natural ability as anyone could have, and yet they also worked about as hard as anyone could work. You need both.

That seems so obvious, and yet in practice we find it slightly hard to grasp. There's a faint xor between talent and hard work. It comes partly from popular culture, where it seems to run very deep, and partly from the fact that the outliers are so rare. If great talent and great drive are both rare, then people with both are rare squared. Most people you meet who have a lot of one will have less of the other. But you'll need both if you want to be an outlier yourself. And since you can't really change how much natural talent you have, in practice doing great work, insofar as you can, reduces to working very hard.

It's straightforward to work hard if you have clearly defined, externally imposed goals, as you do in school. There is some technique to it: you have to learn not to lie to yourself, not to procrastinate (which is a form of lying to yourself), not to get distracted, and not to give up when things go wrong. But this level of discipline seems to be within the reach of quite young children, if they want it.

What I've learned since I was a kid is how to work toward goals that are neither clearly defined nor externally imposed. You'll probably have to learn both if you want to do really great things.

The most basic level of which is simply to feel you should be working without anyone telling you to. Now, when I'm not working hard, alarm bells go off. I can't be sure I'm getting anywhere when I'm working hard, but I can be sure I'm getting nowhere when I'm not, and it feels awful. [2]

There wasn't a single point when I learned this. Like most little kids, I enjoyed the feeling of achievement when I learned or did something new. As I grew older, this morphed into a feeling of disgust when I wasn't achieving anything. The one precisely dateable landmark I have is when I stopped watching TV, at age 13.

Several people I've talked to remember getting serious about work around this age. When I asked Patrick Collison when he started to find idleness distasteful, he said

I think around age 13 or 14. I have a clear memory from around then of sitting in the sitting room, staring outside, and wondering why I was wasting my summer holiday.

Perhaps something changes at adolescence. That would make sense.

Strangely enough, the biggest obstacle to getting serious about work was probably school, which made work (what they called work) seem boring and pointless. I had to learn what real work was before I could wholeheartedly desire to do it. That took a while, because even in college a lot of the work is pointless; there are entire departments that are pointless. But as I learned the shape of real work, I found that my desire to do it slotted into it as if they'd been made for each other.

I suspect most people have to learn what work is before they can love it. Hardy wrote eloquently about this in *A Mathematician's Apology*:

I do not remember having felt, as a boy, any *passion* for mathematics, and such notions as I may have had of the career of a mathematician were far from noble. I thought of mathematics in terms of examinations and scholarships: I wanted to beat other boys, and this seemed to be the way in which I could do so most decisively.

He didn't learn what math was really about till part way through college, when he read Jordan's *Cours d'analyse*.

I shall never forget the astonishment with which I read that remarkable work, the first inspiration for so many mathematicians of my generation, and learnt for the first time as I read it what mathematics really meant.

There are two separate kinds of fakeness you need to learn to discount in order to understand what real work is. One is the kind Hardy encountered in school. Subjects get distorted when they're adapted to be taught to kids — often so distorted that they're nothing like the work done by actual practitioners. [3] The other kind of fakeness is intrinsic to certain types of work. Some types of work are inherently bogus, or at best mere busywork.

There's a kind of solidity to real work. It's not all writing the *Principia*, but it all feels necessary. That's a vague criterion, but it's deliberately vague, because it has to cover a lot of different types. [4]

Once you know the shape of real work, you have to learn how many hours a day to spend on it. You can't solve this problem by simply working every waking hour, because in many kinds of work there's a point beyond which the quality of the result will start to decline.

That limit varies depending on the type of work and the person. I've done several different kinds of work, and the limits were different for each. My limit for the harder types of writing or programming is about five hours a day. Whereas when I was running a startup, I could work all the time. At least for the three years I

did it; if I'd kept going much longer, I'd probably have needed to take occasional vacations. [5]

The only way to find the limit is by crossing it. Cultivate a sensitivity to the quality of the work you're doing, and then you'll notice if it decreases because you're working too hard. Honesty is critical here, in both directions: you have to notice when you're being lazy, but also when you're working too hard. And if you think there's something admirable about working too hard, get that idea out of your head. You're not merely getting worse results, but getting them because you're showing off — if not to other people, then to yourself. [6]

Finding the limit of working hard is a constant, ongoing process, not something you do just once. Both the difficulty of the work and your ability to do it can vary hour to hour, so you need to be constantly judging both how hard you're trying and how well you're doing.

Trying hard doesn't mean constantly pushing yourself to work, though. There may be some people who do, but I think my experience is fairly typical, and I only have to push myself occasionally when I'm starting a project or when I encounter some sort of check. That's when I'm in danger of procrastinating. But once I get rolling, I tend to keep going.

What keeps me going depends on the type of work. When I was working on Viaweb, I was driven by fear of failure. I barely procrastinated at all then, because there was always something that needed doing, and if I could put more distance between me and the pursuing beast by doing it, why wait? [7] Whereas what drives me now, writing essays, is the flaws in them. Between essays I fuss for a few days, like a dog circling while it decides exactly where to lie down. But once I get started on one, I don't have to push myself to work, because there's always some error or omission already pushing me.

I do make some amount of effort to focus on important topics. Many problems have a hard core at the center, surrounded by easier stuff at the edges. Working hard means aiming toward the center to the extent you can. Some days you may not be able to; some days you'll only be able to work on the easier, peripheral stuff. But you should always be aiming as close to the center as you can without stalling.

The bigger question of what to do with your life is one of these problems with a hard core. There are important problems at the center, which tend to be hard, and less important, easier ones at the edges. So as well as the small, daily adjustments involved in working on a specific problem, you'll occasionally have to make big, lifetime-scale adjustments about which type of work to do. And the rule is the same: working hard means aiming toward the center — toward the

most ambitious problems.

By center, though, I mean the actual center, not merely the current consensus about the center. The consensus about which problems are most important is often mistaken, both in general and within specific fields. If you disagree with it, and you're right, that could represent a valuable opportunity to do something new.

The more ambitious types of work will usually be harder, but although you should not be in denial about this, neither should you treat difficulty as an infallible guide in deciding what to do. If you discover some ambitious type of work that's a bargain in the sense of being easier for you than other people, either because of the abilities you happen to have, or because of some new way you've found to approach it, or simply because you're more excited about it, by all means work on that. Some of the best work is done by people who find an easy way to do something hard.

As well as learning the shape of real work, you need to figure out which kind you're suited for. And that doesn't just mean figuring out which kind your natural abilities match the best; it doesn't mean that if you're 7 feet tall, you have to play basketball. What you're suited for depends not just on your talents but perhaps even more on your interests. A *deep interest* in a topic makes people work harder than any amount of discipline can.

It can be harder to discover your interests than your talents. There are fewer types of talent than interest, and they start to be judged early in childhood, whereas interest in a topic is a subtle thing that may not mature till your twenties, or even later. The topic may not even exist earlier. Plus there are some powerful sources of error you need to learn to discount. Are you really interested in x , or do you want to work on it because you'll make a lot of money, or because other people will be impressed with you, or because your parents want you to? [8]

The difficulty of figuring out what to work on varies enormously from one person to another. That's one of the most important things I've learned about work since I was a kid. As a kid, you get the impression that everyone has a calling, and all they have to do is figure out what it is. That's how it works in movies, and in the streamlined biographies fed to kids. Sometimes it works that way in real life. Some people figure out what to do as children and just do it, like Mozart. But others, like Newton, turn restlessly from one kind of work to another. Maybe in retrospect we can identify one as their calling — we can wish Newton spent more time on math and physics and less on alchemy and theology — but this is an *illusion* induced by hindsight bias. There was no voice calling to him that he could have heard.

So while some people's lives converge fast, there will be others whose lives never converge. And for these people, figuring out what to work on is not so much a prelude to working hard as an ongoing part of it, like one of a set of simultaneous equations. For these people, the process I described earlier has a third component: along with measuring both how hard you're working and how well you're doing, you have to think about whether you should keep working in this field or switch to another. If you're working hard but not getting good enough results, you should switch. It sounds simple expressed that way, but in practice it's very difficult. You shouldn't give up on the first day just because you work hard and don't get anywhere. You need to give yourself time to get going. But how much time? And what should you do if work that was going well stops going well? How much time do you give yourself then? [9]

What even counts as good results? That can be really hard to decide. If you're exploring an area few others have worked in, you may not even know what good results look like. History is full of examples of people who misjudged the importance of what they were working on.

The best test of whether it's worthwhile to work on something is whether you find it interesting. That may sound like a dangerously subjective measure, but it's probably the most accurate one you're going to get. You're the one working on the stuff. Who's in a better position than you to judge whether it's important, and what's a better predictor of its importance than whether it's interesting?

For this test to work, though, you have to be honest with yourself. Indeed, that's the most striking thing about the whole question of working hard: how at each point it depends on being honest with yourself.

Working hard is not just a dial you turn up to 11. It's a complicated, dynamic system that has to be tuned just right at each point. You have to understand the shape of real work, see clearly what kind you're best suited for, aim as close to the true core of it as you can, accurately judge at each moment both what you're capable of and how you're doing, and put in as many hours each day as you can without harming the quality of the result. This network is too complicated to trick. But if you're consistently honest and clear-sighted, it will automatically assume an optimal shape, and you'll be productive in a way few people are.

Notes

[1] In "The Bus Ticket Theory of Genius" I said the three ingredients in great work were natural ability, determination, and interest. That's the formula in the preceding stage; determination and interest yield practice and effort.

[2] I mean this at a resolution of days, not hours. You'll often get somewhere while not working in the sense that the solution to a problem comes to you while

taking a *shower*, or even in your sleep, but only because you were working hard on it the day before.

It's good to go on vacation occasionally, but when I go on vacation, I like to learn new things. I wouldn't like just sitting on a beach.

[3] The thing kids do in school that's most like the real version is sports. Admittedly because many sports originated as games played in schools. But in this one area, at least, kids are doing exactly what adults do.

In the average American high school, you have a choice of pretending to do something serious, or seriously doing something pretend. Arguably the latter is no worse.

[4] Knowing what you want to work on doesn't mean you'll be able to. Most people have to spend a lot of their time working on things they don't want to, especially early on. But if you know what you want to do, you at least know what direction to nudge your life in.

[5] The lower time limits for intense work suggest a solution to the problem of having less time to work after you have kids: switch to harder problems. In effect I did that, though not deliberately.

[6] Some cultures have a tradition of performative hard work. I don't love this idea, because (a) it makes a parody of something important and (b) it causes people to wear themselves out doing things that don't matter. I don't know enough to say for sure whether it's net good or bad, but my guess is bad.

[7] One of the reasons people work so hard on startups is that startups can fail, and when they do, that failure tends to be both decisive and conspicuous.

[8] It's ok to work on something to make a lot of money. You need to solve the money problem somehow, and there's nothing wrong with doing that efficiently by trying to make a lot at once. I suppose it would even be ok to be interested in money for its own sake; whatever floats your boat. Just so long as you're conscious of your motivations. The thing to avoid is *unconsciously* letting the need for money warp your ideas about what kind of work you find most interesting.

[9] Many people face this question on a smaller scale with individual projects. But it's easier both to recognize and to accept a dead end in a single project than to abandon some type of work entirely. The more determined you are, the harder it gets. Like a Spanish Flu victim, you're fighting your own immune system: Instead of giving up, you tell yourself, I should just try harder. And who can say you're not right?

Thanks to Trevor Blackwell, John Carmack, John Collison, Patrick Collison,

Robert Morris, Geoff Ralston, and Harj Taggar for reading drafts of this.

Putting Ideas into Words

February 2022

Writing about something, even something you know well, usually shows you that you didn't know it as well as you thought. Putting ideas into words is a severe test. The first words you choose are usually wrong; you have to rewrite sentences over and over to get them exactly right. And your ideas won't just be imprecise, but incomplete too. Half the ideas that end up in an essay will be ones you thought of while you were writing it. Indeed, that's why I write them.

Once you publish something, the convention is that whatever you wrote was what you thought before you wrote it. These were your ideas, and now you've expressed them. But you know this isn't true. You know that putting your ideas into words changed them. And not just the ideas you published. Presumably there were others that turned out to be too broken to fix, and those you discarded instead.

It's not just having to commit your ideas to specific words that makes writing so exacting. The real test is reading what you've written. You have to pretend to be a neutral reader who knows nothing of what's in your head, only what you wrote. When he reads what you wrote, does it seem correct? Does it seem complete? If you make an effort, you can read your writing as if you were a complete stranger, and when you do the news is usually bad. It takes me many cycles before I can get an essay past the stranger. But the stranger is rational, so you always can, if you ask him what he needs. If he's not satisfied because you failed to mention *x* or didn't qualify some sentence sufficiently, then you mention *x* or add more qualifications. Happy now? It may cost you some nice sentences, but you have to resign yourself to that. You just have to make them as good as you can and still satisfy the stranger.

This much, I assume, won't be that controversial. I think it will accord with the experience of anyone who has tried to write about anything nontrivial. There may exist people whose thoughts are so perfectly formed that they just flow

straight into words. But I've never known anyone who could do this, and if I met someone who said they could, it would seem evidence of their limitations rather than their ability. Indeed, this is a trope in movies: the guy who claims to have a plan for doing some difficult thing, and who when questioned further, taps his head and says "It's all up here." Everyone watching the movie knows what that means. At best the plan is vague and incomplete. Very likely there's some undiscovered flaw that invalidates it completely. At best it's a plan for a plan.

In precisely defined domains it's possible to form complete ideas in your head. People can play chess in their heads, for example. And mathematicians can do some amount of math in their heads, though they don't seem to feel sure of a proof over a certain length till they write it down. But this only seems possible with ideas you can express in a formal language. [1] Arguably what such people are doing is putting ideas into words in their heads. I can to some extent write essays in my head. I'll sometimes think of a paragraph while walking or lying in bed that survives nearly unchanged in the final version. But really I'm writing when I do this. I'm doing the mental part of writing; my fingers just aren't moving as I do it. [2]

You can know a great deal about something without writing about it. Can you ever know so much that you wouldn't learn more from trying to explain what you know? I don't think so. I've written about at least two subjects I know well — Lisp hacking and startups — and in both cases I learned a lot from writing about them. In both cases there were things I didn't consciously realize till I had to explain them. And I don't think my experience was anomalous. A great deal of knowledge is unconscious, and experts have if anything a higher proportion of unconscious knowledge than beginners.

I'm not saying that writing is the best way to explore all ideas. If you have ideas about architecture, presumably the best way to explore them is to build actual buildings. What I'm saying is that however much you learn from exploring ideas in other ways, you'll still learn new things from writing about them.

Putting ideas into words doesn't have to mean writing, of course. You can also do it the old way, by talking. But in my experience, writing is the stricter test. You have to commit to a single, optimal sequence of words. Less can go unsaid when you don't have tone of voice to carry meaning. And you can focus in a way that would seem excessive in conversation. I'll often spend 2 weeks on an essay and reread drafts 50 times. If you did that in conversation it would seem evidence of some kind of mental disorder. If you're lazy, of course, writing and talking are equally useless. But if you want to push yourself to get things right, writing is the steeper hill. [3]

The reason I've spent so long establishing this rather obvious point is that it leads to another that many people will find shocking. If writing down your ideas always makes them more precise and more complete, then no one who hasn't written about a topic has fully formed ideas about it. And someone who never writes has no fully formed ideas about anything nontrivial.

It feels to them as if they do, especially if they're not in the habit of critically examining their own thinking. Ideas can feel complete. It's only when you try to put them into words that you discover they're not. So if you never subject your ideas to that test, you'll not only never have fully formed ideas, but also never realize it.

Putting ideas into words is certainly no guarantee that they'll be right. Far from it. But though it's not a sufficient condition, it is a necessary one.

Notes

[1] Machinery and circuits are formal languages.

[2] I thought of this sentence as I was walking down the street in Palo Alto.

[3] There are two senses of talking to someone: a strict sense in which the conversation is verbal, and a more general sense in which it can take any form, including writing. In the limit case (e.g. Seneca's letters), conversation in the latter sense becomes essay writing.

It can be very useful to talk (in either sense) with other people as you're writing something. But a verbal conversation will never be more exacting than when you're talking about something you're writing.

Thanks to Trevor Blackwell, Patrick Collison, and Robert Morris for reading drafts of this.